

Algoritmi paraleli

Curs 1

Vlad Olaru

vlad.olaru@fmi.unibuc.ro

Calculatoare si Tehnologia Informatiei

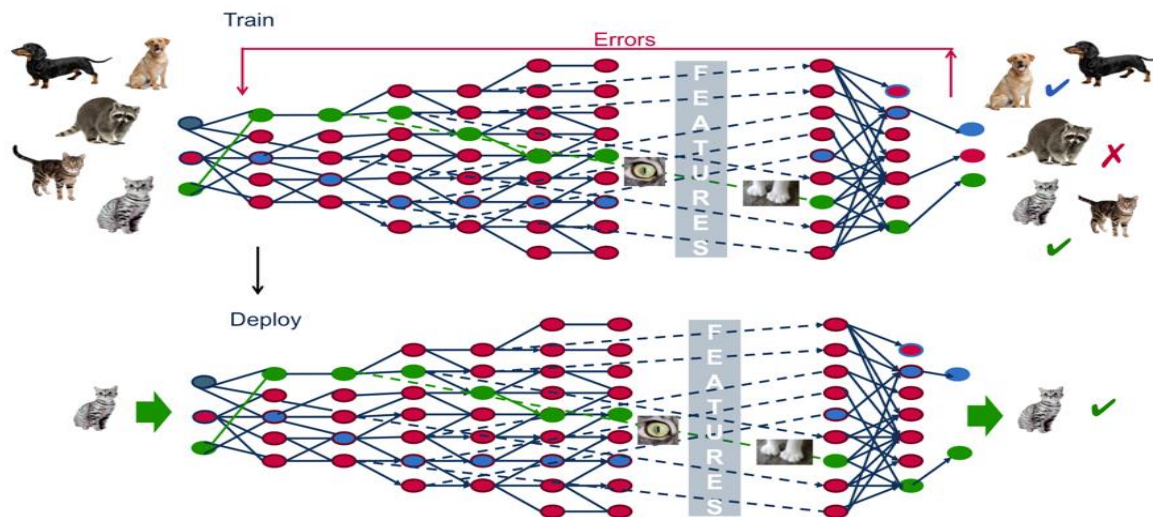
Universitatea din Bucuresti

Organizare

- examinare in doua (chiar trei) etape
 - examen partial scris: 30 pcte
 - examen final: 70 pcte
- examenul partial
 - se promoveaza cu cel putin 15 pcte
 - fara examen partial promovat nu se intra in examenul final
 - saptamana a 6-a la curs (tentativ)
- examen final
 - prezentarea unui proiect de algoritmi paraleli implementat in MPI
 - echipe 2-3 studenti
 - doua etape:
 - prezentarea specificatiei proiectului (cca 10 min / echipa), saptamana a 7-a (tentativ) la laborator
 - sesiune: prezentarea propriu-zisa a proiectului + demo (cca 30 min/echipa)
- laborator: programare in MPI

Motivatie calcul paralel

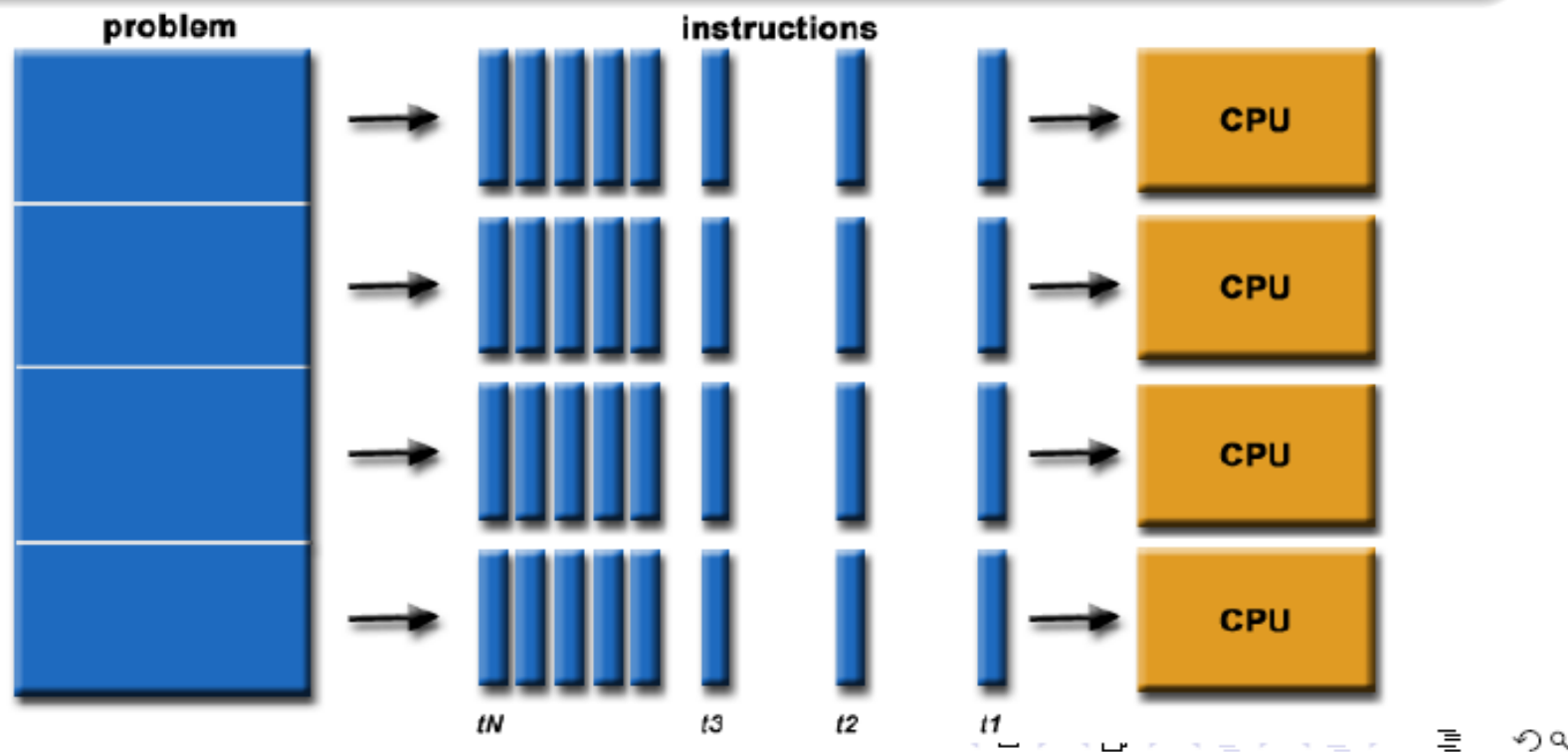
- cerinte sporite de calcul pentru intregi clase de probleme
 - calcul stiintific (simulari, decriptare genom)
 - Web/Internet (retele de socializare, indexare motoare de cautare)
 - deep nets
 - seturi de date pe scara larga (large-scale datasets): calcul paralel + acces paralel la date
 - Large Language Models (LLM) > 600 mil parametri



Calcul paralel

- utilizarea simultana a mai multor unitati de procesare pentru rezolvarea unei singure probleme

Plot obtained from https://computing.llnl.gov/tutorials/parallel_comp/



Arhitecturi paralele

- taxonomia Flynn

1. **SISD** – *Single Instruction stream, Single Data stream*

- arhitectura istorica, secventiala a masinilor uniprocessor

- relevanta actuala

- sisteme de timp real uniprocessor (non multi-core) fara cache, fara executie speculativa a instructiunilor, samd, ceea ce faciliteaza analiza WCET (worst case execution time)
- safety-critical systems (aviatie, radare, centrale nucleare, etc), eg MILS

2. **MISD** – *Multiple Instruction streams, Single Data stream*

- utilizata rar, ex: sisteme tolerante la defecte

Arhitecturi paralele (cont.)

3. **SIMD** – *Single Instruction stream, Multiple Data streams*

- exemplu tipic: arhitecturi vectoriale tip GPU (heartbeat synchronized)
- in particular, pentru arhitecturile vectoriale se foloseste si termenul de **SIMT**: *Single Instruction, Multiple Threads* (NVIDIA)
- *predicated/masked SIMD*: executia respectiv ne-executia (skip) operatiei vectoriale asupra unui element particular din vector (in functie de valoarea sa) poate fi controlata de un predicat

ex:

$c[i] = a[i] / b[i] \quad \text{if } b[i] \neq 0;$

- model computational folosit in framework-uri de lucru din AI (caffe, pytorch, tensorflow, etc)

Arhitecturi paralele (cont.)

4. **MIMD** – *Multiple Instruction streams, Multiple Data streams*

- restul echipamentelor moderne de calcul, de la smartphones la calculatoare paralele/distribuite
- arhitecturi multiprocesor UMA/NUMA, clustere, grid-uri, arhitecturi masiv paralele (CRAY)
- cunoscute si informal sub numele de sisteme multiprocesor (MP)
- subdiviziune:

a. **SPMD** – *Single Program, Multiple Data streams*

- difera de SIMD, instructiunile aceluiasi program se executa independent, nesincronizat, asupra unor date diferite
- cel mai uzual model de programare paralela

b. **MPMD** – *Multiple Programs, Multiple Data streams*

- multiple procesoare independente executa simultan cel putin doua tipuri de programe
- ex: master/slave programs, Intel MPI library MPMD launch

Tipuri de paralelism

- definit de **granularitatea** G a task-ului

$$G = T_{\text{comp}} / T_{\text{comm}}$$

unde T_{comp} = timpul de calcul al unui task

T_{comm} = timpul necesar schimbului de date intre procesoare

1. *paralelism cu granularitate fina (fine-grained parallelism)*

- programul paralel este impartit in multe task-uri mici, fiecare asignate individual unui procesor
- asigura o incarcare echilibrata a sistemului multiprocesor
- in general creste overhead-ul de comunicare si sincronizare
- potrivit pentru arhitecturi paralele cu latenta de comunicare mica, cum sunt sistemele MIMD cu memorie partajata (shared memory / tightly coupled MPs)

Tipuri de paralelism (cont.)

2. *paralelism cu granularitate mare (coarse-grained parallelism)*

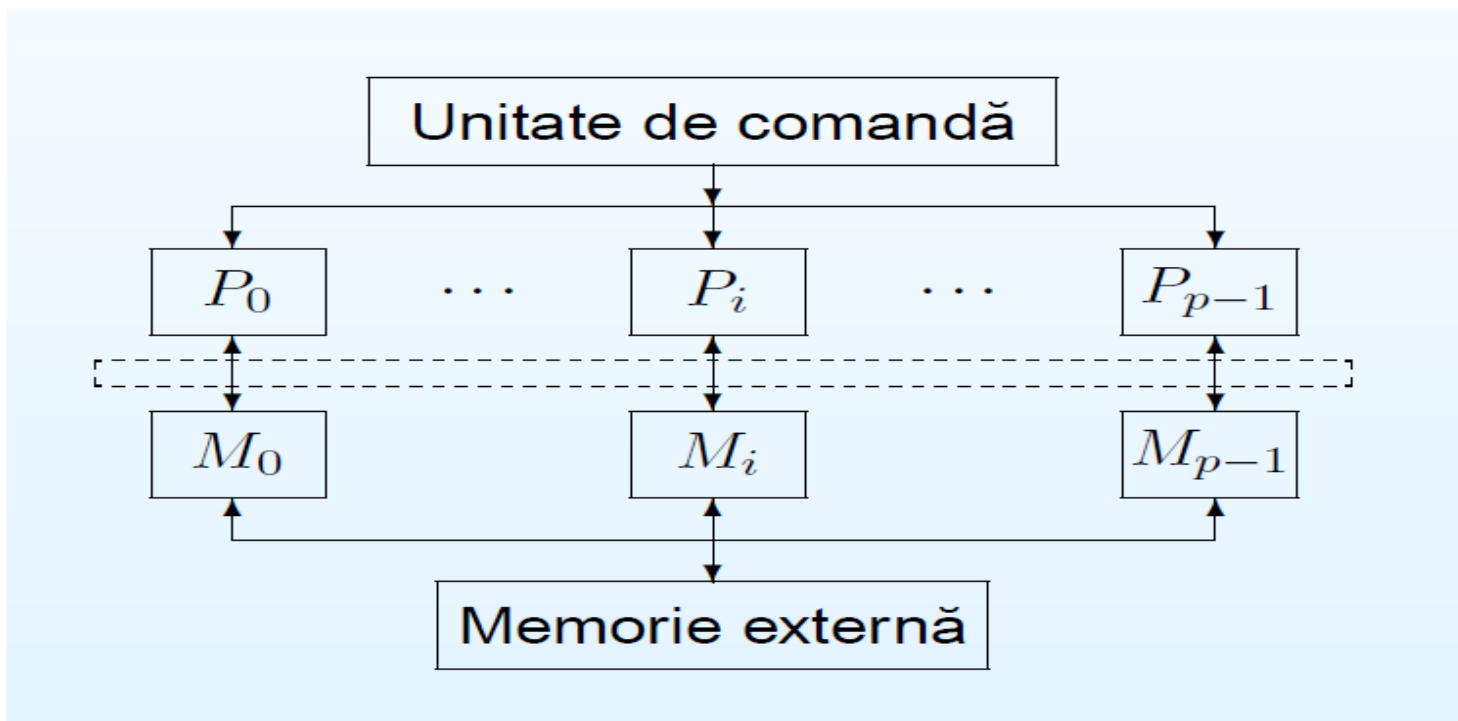
- programul paralel este impartit in task-uri mari
- majoritatea procesarii are loc pe cateva procesoare
- poate induce dezechilibru in incarcarea sistemului multiprocesor
- in general overhead-ul de comunicare si sincronizare este mic, pentru ca task-urile nu prea interactioneaza
- potrivit pentru arhitecturi paralele cu latentă de comunicare mare
 - eg, sistemele MIMD cu memorie distribuita (message-passing / loosely coupled MPs)

Tipuri de paralelism (cont.)

- *concluzie*: performanta unei solutii de paralelizare a unui program depinde de alegerea unei granularitati potrivite pentru arhitectura paralela aflata la dispozitie
- programe paralele cu granularitate fina vor rula suboptimal pe arhitecturi paralele cu memorie distribuita
 - comunicare intensa si scumpa
- programe paralele cu granularitate mare nu vor folosi optimal arhitecturile cu memorie partajata
 - majoritatea calculului se va face pe procesoare putine
 - nu se exploateaza potentialul de paralelism al arhitecturii
- obs: alegerea unei solutii optimale de paralelizare nu e intotdeauna posibila
 - ex: N-body problem pt DSM (Distributed Shared Memory)

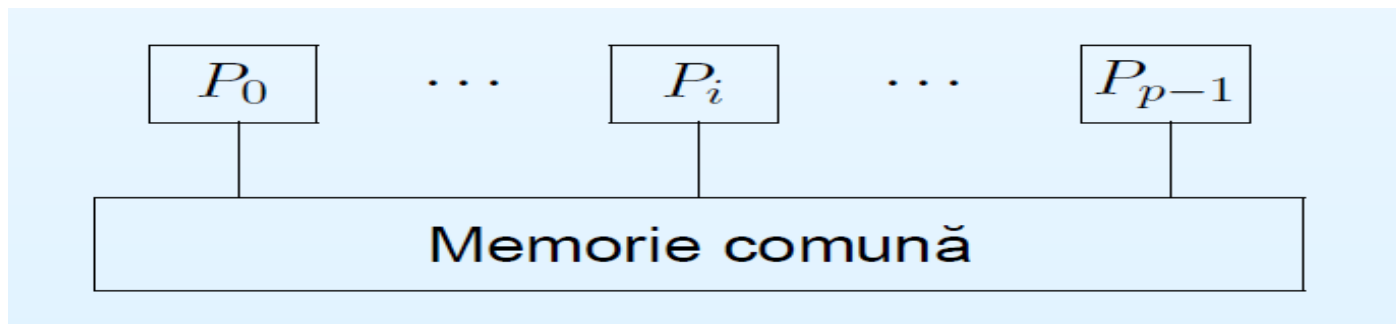
Arhitecturi SIMD

- fiecare procesor P_i executa aceeași instrucțiune asupra unei date din memoria locală M_i

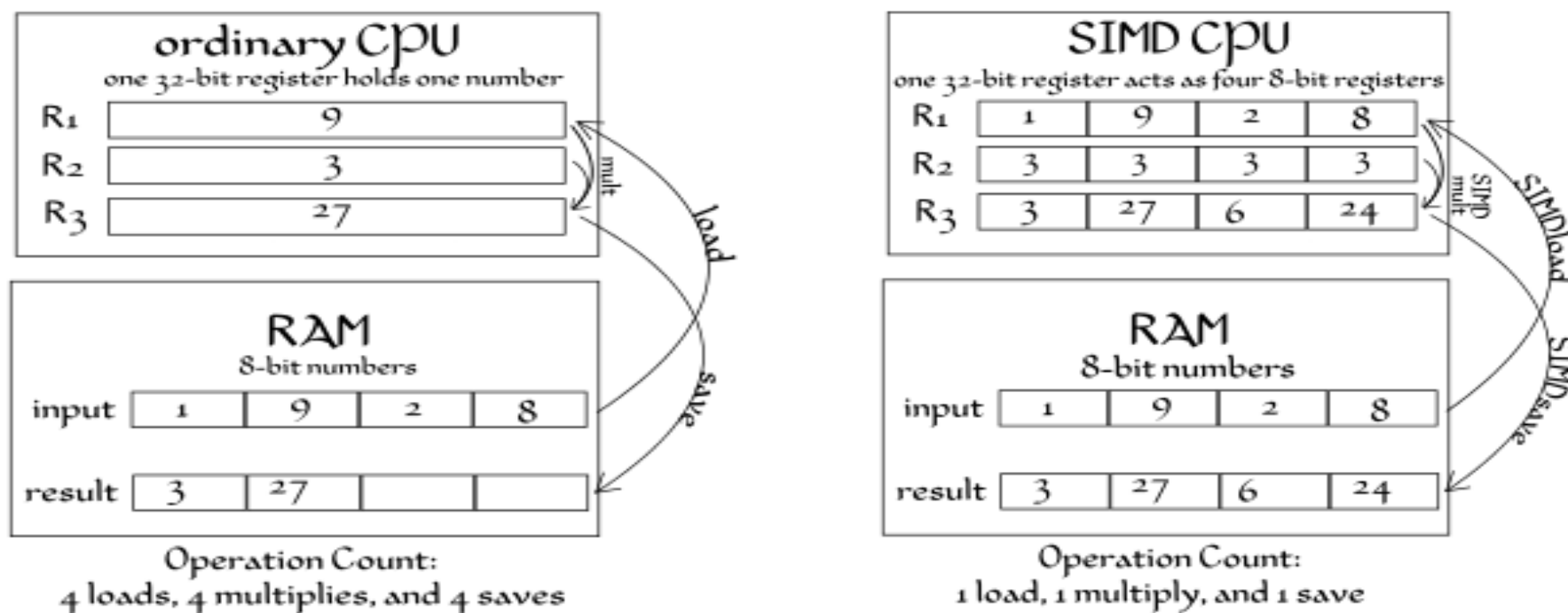


Arhitecturi SIMD (cont.)

- memoriile locale procesoarelor sunt rapide
- procesoarele sunt legate de memoriile locale prin rețele de interconectare configurabile (orice procesor poate accesa orice modul de memorie)
- in general, procesoare simple
- operatii vectoriale
 - vectori de lungime multiplu al numarului de procesoare p
 - se executa intr-un singur tact pe p procesoare (eg, adunare vectori)
- problema critica: accesul la date pentru alimentarea memoriilor locale



Arhitecturi SIMD (cont.)



<https://en.wikipedia.org/wiki/SIMD>

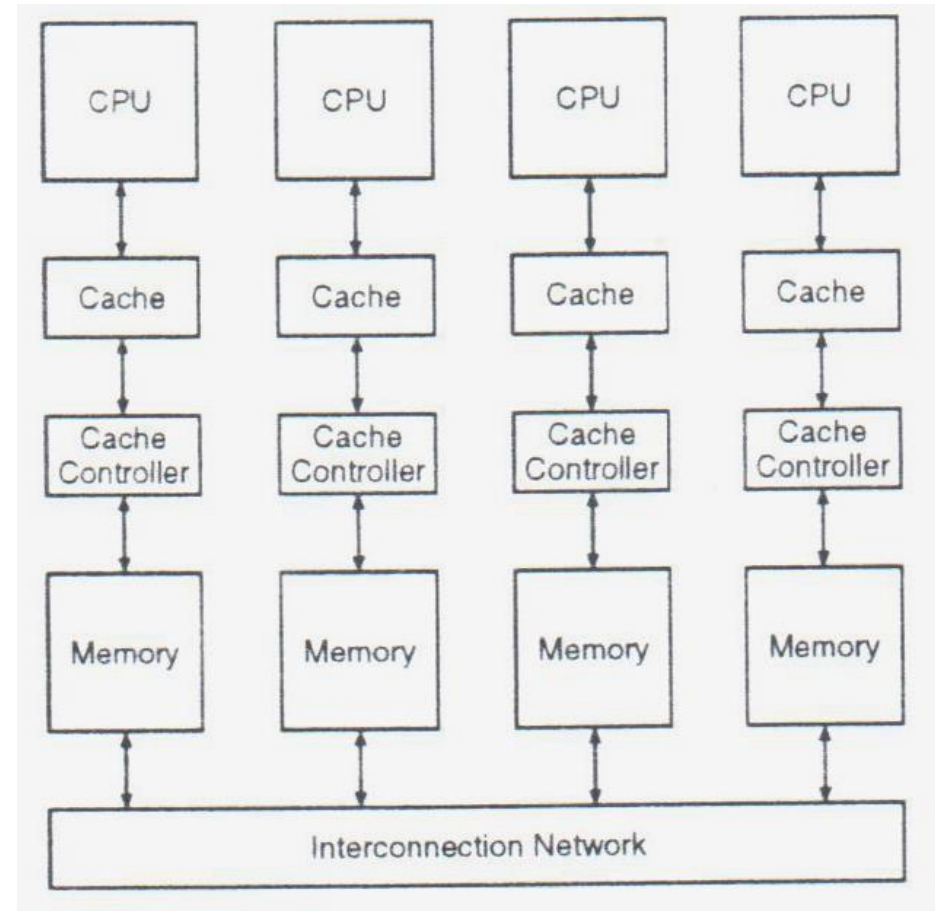
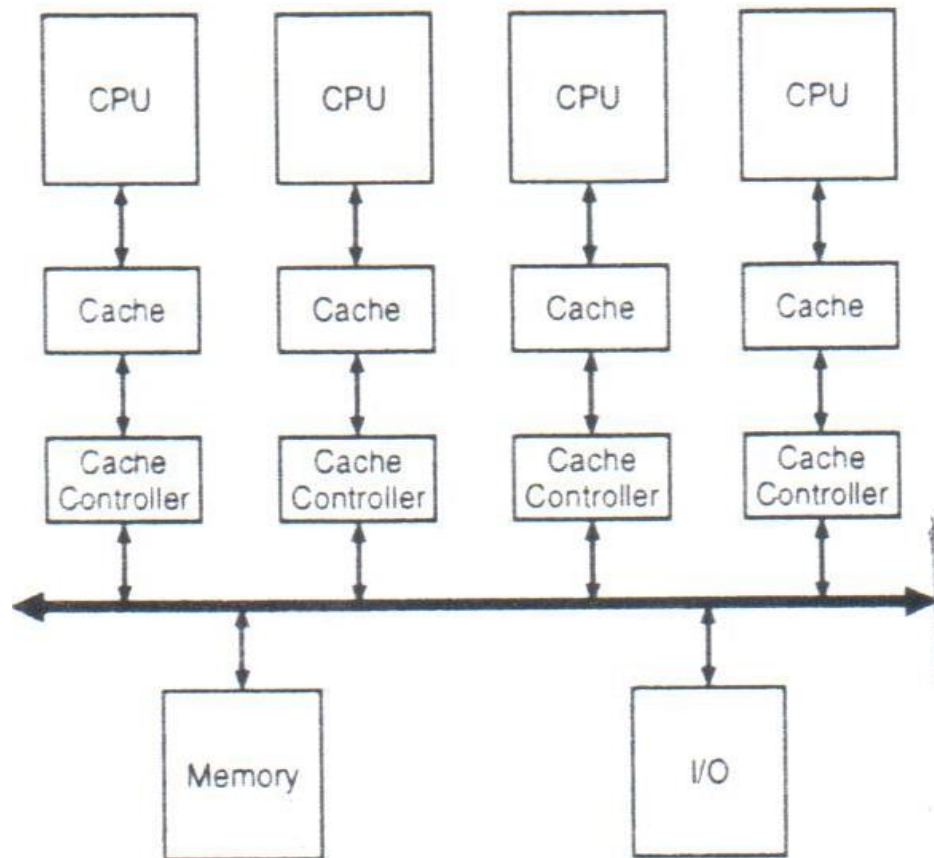
Arhitecturi MIMD

- referite uzual ca sisteme multiprocesor (MP)
- felul in care se interconecteaza procesoarele individuale in sistemele multiprocesor determina in general si tipul de acces la memorie
 - Uniform Memory Access (UMA)
 - Non-Uniform Memory Access (NUMA)
 - No Remote Memory Access (NORMA)
- in sistemele UMA/NUMA procesoarele comunica prin variabile partajate de memorie accesate sincronizat (cu ajutorul lock-urilor, de pilda)
- in sistemele NORMA accesul coordonat la date se face prin schimb de mesaje (message passing)

MP cu memorie partajata

- fiecare procesor executa propriul flux de instructiuni
- datele se afla in memoria comuna
- uzual procesoarele au cache-uri locale
- operatii paralele la nivel de bloc
- avantaj: comuncatie simpla prin memoria comuna
- dezavantaj: scalabilitatea
 - cresterea numarului de procesoare induce cresterea numarului de conflicte la memorie => scaderea vitezei de calcul

MP cu memorie partajata



MP cu o singura magistrala

- magistrala unica reprezinta principala limitare pt. cresterea nr. de CPU
- folosirea cache-urilor reduce traficul pe magistrala si permite cresterea nr de procesoare
- existenta cache-urilor locale => copii multiple ale aceleiasi locatii de memorie => potential de inconsistenta daca un CPU modifica datele (locale)
- accesul uncached la datele partajate nu e indicat
 - scade viteza accesului la date
 - creste traficul pe magistrala ceea ce incetinesc si accesul celorlalte procesoare inclusiv la datele nepartajate
- ideal, cand un procesor citeste date partajate pot exista copii multiple si in alte cache-uri, dar la scriere e nevoie de acces exclusiv la date urmat de invalidarea celorlalte copii existente

MP cu o singura magistrala (cont.)

- solutia: protocoale de mentinere a coerentei cache-urilor locale
- ex de protocol uzual: *snoop coherency*
 - controlerele cache-urilor monitorizeaza magistrala pt traficul care afecteaza datele din cache-urile lor
 - *read miss*: controlerul localizeaza o copie actualizata a datelor (posibil aflata in cache-ul altui CPU)
 - *write*: exista doua protocoale posibile, *write-invalidate* si *write-update*
 - *write-invalidate* invalideaza toate copiile celorlalte procesoare inainte de a scrie datele in cache-ul local (eg. Intel MESI)
 - *write-update* scrie datele modificate pe magistrala printr-o operatie de broadcast
 - controlerele de cache isi pot actualiza copiile locale ale datelor la valoarea modificata

Suport HW pentru sincronizare

- multe procesoare ofera suport HW pt implementarea sectiunilor critice
- uniprocsoare – pot dezactiva intreruperile
 - codul current se executa fara a fi interrupt (preempted)
 - ineficient in sisteme multiprocesor
- instructiuni HW atomice
 - instructiuni procesor speciale executate *in mod atomic (neintrerupt)*
 - fie testeaza si modifica (*test-and-modify*) continutul unui cuvant de memorie, **Test-and-Set (TAS)**
 - fie schimba (*swap*) continutul a doua cuvinte de memorie (sau mai multe), **Compare-and-Swap (CAS, cu varianta CAS2)**

Test-and-set (TAS)

- definitie

```
int tas(int *lock)
{
    int rv = *lock;
    *lock = 1;
    return rv;
}
```

- proprietati
 - executie atomica
 - intoarce valoarea originala a parametrului pasat functiei
 - seteaza noua valoarea a parametrului pe 1

Spinlock-uri cu TAS

- lock (lacat, zavor), concept (abstractie) de nivel inalt pt. protectia sectiunilor critice
- ofera doua operatii: *acquire* (apelata la intrarea in sectiune critica) si respectiv *release* (apelata la iesirea din sectiune critica)
- cand un proces/thread detine lock-ul, celelalte sunt in busy waiting (“spinning in the while loop”) => numele de *spinlocks*
- pt a se reduce busy waiting, sectiunile critice implementate cu spinlocks trebuie sa fie foarte scurte

```
int lock = 0;
void acquire(int *lock)
{
    while(tas(lock))
        ;
}
void release(int *lock)
{
    *lock = 0;
}
```

Instructiuni atomice multiprocesor

- se folosesc instructiuni atomice de tip TAS/CAS
- TAS in sisteme multiprocesor
 - cand un CPU executa TAS, arbitrul de magistrala da drept de folosinta exclusiva a magistralei procesorului respectiv pe durata executiei instructiunii
 - toate celelalte surse generatoare de accese de memorie sunt blocate pe durata TAS
 - se executa ciclul read-modify-write
 - la final, se deblocheaza magistrala pt uzul altor procesoare

Consecinte ciclu read-modify-write

- intr-un astfel de ciclu se citeste si se scrie o valoare in mod atomic, iar datele nu sunt cached

=> operatie mai costisitoare decat operatiile de citire/scriere obisnuite

=> magistrala e ocupata mai mult timp

=> operatiile TAS afecteaza semnificativ rata de transfer pe magistrala

- alte consecinte privesc implementarea spinlock-urilor in sisteme multiprocesor

Spinlock-uri TAS MP

```
void acquire(char *lock_ptr)
{
    disable_interrupts();
    while(tas(lock_ptr))
        ;
}

void release(char *lock_ptr)
{
    *lock_ptr = 0;
    enable_interrupts();
}
```

Observatii

- dezactivarea intreruperilor in *acquire* impiedica pierderea procesorului pe durata detinerii lock-ului
- pierderea procesorului ar insemna ca procese/thread-uri de pe alte CPU nu pot intra in sectiune critica din cauza unui proces/thread care nu ruleaza !
- pe durata detinerii unui lock, celelalte procesoare executa TAS in bucla si consuma inutil largimea de banda a magistralei afectand procesoare care nu sunt implicate in accesul la sectiunea critica !
- la *release*, procesorul care detine lock-ul concureaza cu celelalte procesoare pt accesul la magistrala inainte de a putea ceda lock-ul
- solutie: test-and-test-and-set (TATAS)

TATAS

- traficul pe magistrala se poate reduce daca procesoarele cicleaza in *acquire* pe o copie locala a spinlock-ului
- pt ca *release* e in fond o operatie de scriere a spinlock-ului, protocolul de coerenta snoop detecteaza scrierea si invalideaza copiile locale ale celorlalte procesoare (in cazul protocolului write-invalidate)
- cand celelalte procesoare acceseaza din nou spinlock-ul => cache miss
- la cache miss copia locala se va actualiza si va reflecta starea de lock free
- procesorul incearca din nou sa execute operatia TAS sperand ca acum lock-ul e probabil free

Spinlock-uri TATAS

```
void acquire(char *lock_ptr)
{
    disable_interrupts();
    while(*lock_ptr || tas(lock_ptr))
        ;
}

void release(char *lock_ptr)
{
    *lock_ptr = 0;
    enable_interrupts();
}
```

Obs: codul se bazeaza pe evaluarea scurtcircuitata a conditiilor in C

TATAS vs TAS

- pt sectiuni critice scurte si protocol write-invalidare, costurile sunt asemanatoare
 - dupa ce un CPU elibereaza lock-ul si altul il obtine, dureaza pana cand celelalte procesoare ajung sa cicleze pe copia locala a lock-ului
 - in tot acest timp, magistrala e saturata cu trafic de invalidare, read miss si TAS
- secventa tipica de evenimente
 - eliberare lock => invalidare copii din cache-urile altor CPU
 - invalidarile genereaza read miss pt toate CPU care cicleaza, toate citesc noua valoare a lock-ului de pe magistrala si ordinea e seriala !
 - primul CPU care obtine noua valoare a lock-ului executa TAS si obtine lock-ul
 - restul CPUs primesc noua valoare, executa TAS si esueaza
 - fiecare TAS invalideaza copiile locale din cache-uri si forteaza un read miss

TATAS cu backoff

- competitia pt. magistrala partajata seamana cu protocolul de transmisie a datelor pe Ethernet (CSMA/CD)
 - analogia nu e stricta: pe Ethernet toti transmitatorii simultani esueaza, la TAS pe MP arbitrul de magistrala garanteaza un castigator
- analogia sugereaza insa scheme de backoff pt. reducerea competitiei la magistrala a procesoarelor care cicleaza
- metoda introduce o intarziere (delay) inainte de a incerca din nou operatia TAS
- intarzierea poate fi statica sau dinamica
- intarzierea statica:
 - fiecare CPU are un slot
 - merge bine pentru multe CPU-uri
 - intarzie nejustificat un singur CPU chiar daca lock-ul e liber

TATAS cu backoff (cont.)

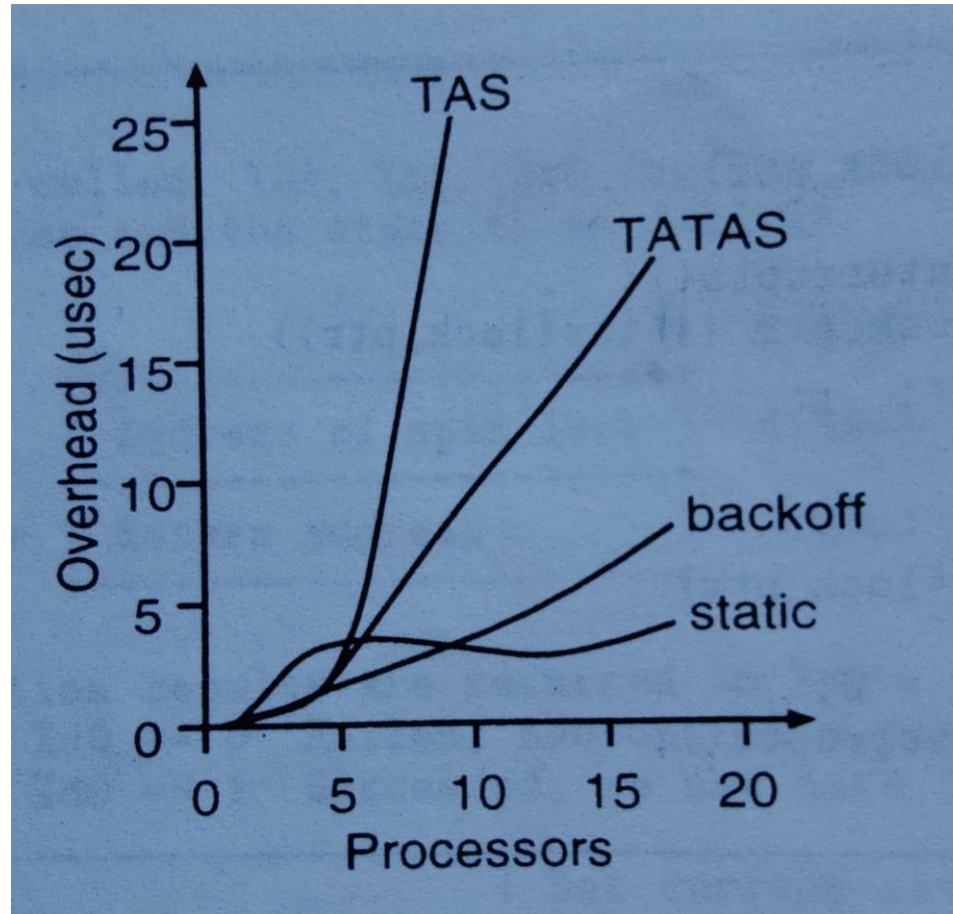
- intarzierea dinamica:
 - la inceput, toate CPU-urile aleg o intarziere mica => coliziuni
 - dupa detectarea coliziunilor, fiecare CPU mareste intarzierea
 - overhead-ul intarzierilor mici de la inceput face metoda mai costisitoare decat alocarea statica a intarzierilor
- obs: folosirea protocolului *write-update* poate imbunatati performanta TATAS prin reducerea traficului pe magistrala
 - cand un lock e eliberat, noua sa valoare poate fi transmisa prin broadcast
 - fiecare CPU care monitorizeaza magistrala isi poate astfel actualiza copia locala fara a mai genera *read miss*

Spinlock-uri TATAS cu backoff

```
void acquire(char *lock_ptr)
{
    disable_interrupts();
    while(*lock_ptr || tas(lock_ptr))
    {
        while(*lock_ptr)
            ;
        delay();
    }
}
```

```
void release(char *lock_ptr)
{
    *lock_ptr = 0;
    enable_interrupts();
}
```

Performanta TAS



Dezavantaje instructiuni atomice

- instructiunile atomice simplifica IPC pe multiprocesoare, dar au si dezavantaje
 - pot avea performanta redusa daca sunt folosite neglijent
 - daca un proces/thread pierde CPU sau e fortat sa astepte o perioada lunga (eg, page fault) in timp ce detine lock-ul, celelalte procese nu pot avansa pana cand detinatorul lock-ului revine pe CPU si elibereaza lock-ul
 - daca un proces/thread se termina anormal in timp ce detine un lock, nici un alt proces/thread nu poate intra in sectiune critica
 - inversarea prioritaticilor – cand un proces/thread cu prioritate mica detine lock-ul si impiedica un proces cu prioritate marea sa intre in sectiune critica
- solutie: sincronizare *wait-free* (*lock-free*), Herlihy 1991

Sincronizare wait-free (lock-free)

- idee centrala: se incearca executia operatiei, dar se lasa datele intr-o stare consistenta daca operatia esueaza
 - analogie cu tranzactiile din baze de date
 - model de control optimist al concurentei
- se pot folosi instructiuni HW de tipul CAS/CAS2 sau LL/SC
- model Herlihy:
 - sistem cu n procese secventiale care comunica prin memorie partajata
 - *obiect concurent* = ($\{tip+valori\}$, set operatii, specificatie secventiala)
 - *obiect concurent neblokant* – procesul care executa una dintre operatiile sale trebuie o termine intr-un nr finit de pasi
 - *obiect concurent wait-free* – fiecare proces din sistem trebuie sa termine operatia intr-un nr finit de pasi

Sincronizare wait-free (cont.)

- conditia de operare *nonblocanta* - unele procese/threaduri vor face intotdeauna progres indiferent de intarzierile sau opririle fortate ale altor procese/threaduri
- conditia de operare *wait-free* e mai puternica => toate procesele/threadurile **care nu sunt oprite** fac progres indiferent de terminarile anormale sau intarzierile altor procese/threaduri
- ambele conditii **exclud** folosirea sectiunilor critice ca metoda de implementare
 - proces/thread care se termina anormal (cu eroare) in mijlocul unei sectiuni critice va bloca pt totdeauna celelalte procese care asteapta sa intre in sectiunea critica
- **rezultat teoretic:** *este imposibil sa se construiasca implementari nebloccante sau wait-free ale tipurilor de date simple folosind operatii de citire/scriere, TAS, fetch-and-add, swap de memorie cu registre (eg, instructiunea Intel xchg)*

Semantica compare-and-swap (CAS)

- definitie

```
int compare_and_swap(int *value, int expected, int new_value)
{
    int temp = *value;

    if (*value == expected)
        *value = new_value;

    return temp;
}
```

- proprietati

- executie atomica
- intoarce valoarea originala a parametrului **value**
- seteaza variabila **value** la valoarea parametrului **new value** doar daca e indeplinita conditia ***value == expected**, adica, schimbul de valori se face doar daca e indeplinita conditia

Incrementare atomica wait-free folosind CAS

```

1      void increment(atomic_int *v)
2      {
3          int temp;
4          do {
5              temp = *v;
6              }while (temp != (compare_and_swap(v, temp, temp+1)) );
7      }

```

Scenariu de executie:

- pp initial $v = 10$
- pp. proces de prioritate mica P_l executa codul primul
- P_l pierde CPU intre liniile 5 si 6, moment in care un proces cu prioritate mare P_h incepe executia
- starea P_l este $v=10$, $temp=10$, iar P_h incheie cu succes incrementarea $\Rightarrow v = 11$
- cand P_l se reia, **cas(11,10,11)** esueaza si se re-executa linia 5 $\Rightarrow temp = v = 11$ si **cas(11, 11, 12)** reuseste $\Rightarrow v = 12$

Obs: spre deosebire de TAS care modifica operandul intotdeauna, CAS/CAS2 modifica operandul *doar daca reuseste comparatia* !

Suport HW RISC pentru sectiuni critice

- TAS, CAS specifice procesoarelor CISC, dar secvente atomice de tip *read-modify-write* nu se pot implementa pe procesoare RISC (arhitecturi load/store)
- operatii speciale RISC: Load Linked (LL) si Store Conditional (SC)
- LL incarca o variabila din memorie intr-un registru CPU si apoi verifica activ daca variabila din memorie este modificata de alte procesoare
- SC verifica daca au existat modificari ale variabilei de memorie de la ultimul LL
 - daca nu, valoarea registrului se stocheaza in memorie cu success (variabila din memorie este modificata) iar continutul registrului este setat pe 1
 - daca da, stocarea esueaza (variabila de memorie ramane nemodificata) iar valoarea registrului se seteaza pe 0

Incrementare atomica wait-free cu LL/SC

- in fapt, o forma elementara de memorie tranzactionala (zona de memorie restransa la un byte/word)
- functioneaza cf. principiilor de control optimist al concurentei din baze de date
- simuleaza executia unei tranzactii din baze de date cf principiilor ACID (Atomicity, Consistency, Isolation, Durability)

1 increment:

```

2      ll      $2, count      ; incarca valoarea counter-ului
3      addu    $3, $2, 1      ; incrementeaza valoare counter
4      sc      $3, count      ; incearca sa stocazeze noua valoare
5      beq     $3, zero, increment ; zero inseamna esec
6      j       $31            ; return din rutina
  
```

Proprietati ACID incrementare LL/SC

- *atomicitate* - modificarile (incrementarea *counter-ului*) sunt executate ca si cand operatia ar fi atomica, i.e. fie toate modificarile sunt executate, fie niciuna
- *consistenta* - datele (*counter-ul*) sunt intr-o stare consistenta cand tranzactia incepe si cand se termina, i.e. *counter-ul* e incrementat corect, chiar daca se executa mai multe tranzactii simultan, eventual intreatesut
- *izolare* - starea intermediara a tranzactiei (apelul procedurii si valorile registrelor) e invizibila altor tranzactii (altor apeluri ale aceleiasi proceduri)
 - tranzactiile concurente (apelurile concurente ale procedurii *increment*) par a fi serializate
- *durabilitate* – dupa incheierea cu succes a tranzactiei (apelul procedurii *increment*), datele sunt salvate in memoria principala RAM si modificarea e finala
 - analogia cu bazele de date se opreste aici, stocarea in RAM nu e persistenta

Comparatie CAS vs. LL/SC

- daca au aparut modificari ale counter-ului, SC va esua garantat, chiar daca valoarea initial citita de LL a fost restaurata
 - ex: problema ABA
- daca se incearca aceeasi secventa de operatii cu CAS, adica
 - se citeste valoarea *counter-ului*
 - apoi se executa CAS
 - daca vechea valoare a fost restaurata, nu se vor detecta modificarile intermediare aparute intre operatia de read si cea de CAS

=> semantica LL/SC e mai puternica decat CAS
- atat CAS cat si LL/SC se pot folosi pentru implementarea sincronizarii *wait-free*

Memorie tranzactionala

- software transactional memory, Herlihy & Moss 1993
- am vazut deja ex. LL/SC & CAS
- exploateaza ideea de tranzactii din baze de date (v. proprietati ACID) si controlul optimist al concurentei
- un thread termina modificarile operate pe o zona de memorie partajata fara a se coordona cu alte threaduri
- se inregistreaza fiecare operatie de citire/scriere intr-un log
- la finalul tranzactiei se incearca operatia de *commit*
 - daca reuseste (i.e., nici o alta tranzactie nu a apucat sa faca un *commit* reusit intre timp), tranzactia e validata si modificarile devin permanente
 - daca esueaza, tranzactia e abandonata (aborted) si se re-executa de la inceput pana reuseste (*transaction roll-back*)
- beneficiu major: grad crescut de concurenta, thread-urile nu au nevoie sa se sincronizeze prin lock-uri la accesul memoriei partajate

Suport HW pt. memorie tranzactionala

- ex. Intel TSX (Transactional Synchronization Extension), extensie ISA x86, microarhitectura Haswell (2013)
- accesibila prin intermediul a doua interfete
 - HLE (Hardware Lock Elision) – backward compatibility
 - RTM (Restricted Transactional Memory)
- HLE adauga doua prefixuri de instr. XACQUIRE & XRELEASE
 - se pot folosi doar pt anumite instructiuni care trebuie prefixate explicit cu LOCK
 - permite executia optimista a sectiunii critice sarind scrierea lock-ului, a.i. acesta apare liber pt alte threaduri
 - o tranzactie esuata determina reluarea operatiei de la instructiunea prefixata cu XACQUIRE, dar instructiunea se va reexecuta ca si cand nu a fost prefixata cu XACQUIRE

Suport HW pt. memorie tranzactionala (cont.)

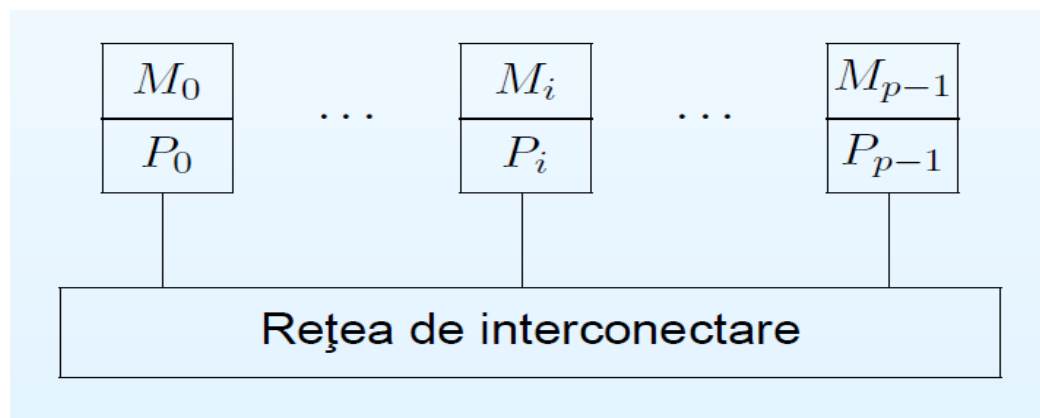
- RTM adauga la ISA trei noi instructiuni
 - XBEGIN – marcheaza inceputul zonei de memorie tranzactionala
 - XEND - marcheaza sfarsitul zonei de memorie tranzactionala
 - XABORT – abandoneaza explicit o tranzactie
 - esecul tranzactiei redirecteaza executia catre codul specificat de instr. XBEGIN, iar codul de eroare e stocat in registrul EAX
- instructiunea XTEST permite testarea starii procesorului (este sau nu in mijlocul executiei unei regiuni de memorie tranzactionala)

RTM elision pt. pthread_mutex_lock

```
void elided_lock_wrapper(lock) {  
    if(_xbegin() == _XBEGIN_STARTED) { // start tranzactie  
        if(lock e liber)  
            return; // executa sectiunea critica in tranzactie  
        _xabort(0xff); // abandoneaza tranzactia  
    }  
    obtine lock  
}  
  
void elided_unlock_wrapper(lock) {  
    if (lock e liber)  
        _xend(); // comite tranzactia  
    else  
        elibereaza lock;  
}
```

Arhitecturi NORMA

- arhitecturi MIMD cu memorie distribuita (s.n. si *sisteme distribuite*)
- fiecare procesor are memorie locala proprie
- comunicatia intre procesoare se face schimb de mesaje (*message passing*) peste o retea de interconectare
- operatii paralele la nivel de bloc
- comunicatia prin mesaje necesita algoritmi dedicati



Arhitecturi NORMA (cont.)

- topologii de retea de comunicatie
 - fixe: array (inel), grid (tor), hipercub
 - configurabile: switch, magistrala
- uzual, topologia e transparenta utilizatorului
- suport pentru comunicatie: biblioteci message passing
- avantaj: nr de procesoare foarte mare

Modelare topologii de interconectare

- se folosesc grafuri neorientate
- procesoarele sunt noduri, iar arcele linii de comunicare interprocesor
- graf neorientat $G = (V, E)$
 - V = multimea nodurilor (*vertices*)
 - E = multimea arcelor (*edges*), $E \subset V \times V$
- *ordinul* grafului = nr de noduri (echivalent, nr de procesoare)
- graf simplu = graf fara cicluri (aciclic)

Grafuri

- noduri *adiacente* (vecine): exista arc intre ele
- *gradul unui nod*: nr de arce incidente nodului (echivalent, nr de noduri adiacente nodului)
- *gradul unui graf* (Δ): gradul maxim al nodurilor din graf
- graf *regulat*: nodurile din graf au acelasi grad
- nr de arce ale unui graf regulat: $p\Delta/2$

Grafuri

- *drum (cale)* între două noduri $x, y \in V$: secvența de noduri $x_0, x_1, \dots, x_k$ a.i. $x = x_0$ și $y = x_k$ și $(x_i, x_{i+1}) \in E, \forall 0 \leq i < k$
- *drum elementar*: nodurile $x_1, \dots, x_{k-1}$ sunt distincte
- în arhitecturi NORMA, un drum este o cale de comunicație între două procesoare
- *lungimea* unui drum = nr de arce (k în notația de mai sus)
- *distanța* între două noduri $d(x, y)$ = lungimea celui mai scurt drum dintre ele
- *diametrul* grafului (D) = $\max_{i,j} d(i, j)$ unde $i, j \in V$
 - i.e., distanța dintre cele mai îndepărtate noduri din graf

Grafuri

- *ciclu*: drum elementar in care nodul initial si cel final coincid
- *ciclu hamiltonian*: ciclu care trece exact o singura data prin fiecare nod al grafului
 - lungime egala cu ordinul grafului
- graf *conex*: oricare doua noduri sunt legate prin cel putin un drum
- *arbore*: graf conex aciclic

Deziderate topologie hardware

- graf conex
- graf regulat cu grad mic: procesoare identice cu putine cai de comunicatie (usor de realizat)
- nr total de arce relativ mic (un arc revine la o conexiune fizica, electrica pe placa sau intre placi)

Inelul

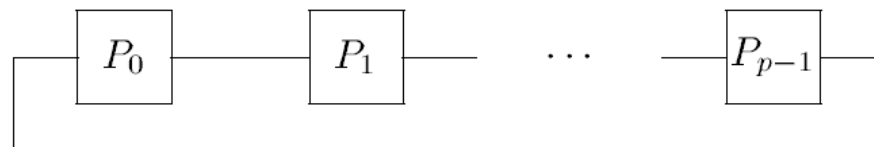
- procesorul P_i legat de P_j unde

$$j = (i - 1) \bmod p \text{ (} P_j \text{ e vecinul din stanga)}$$

sau

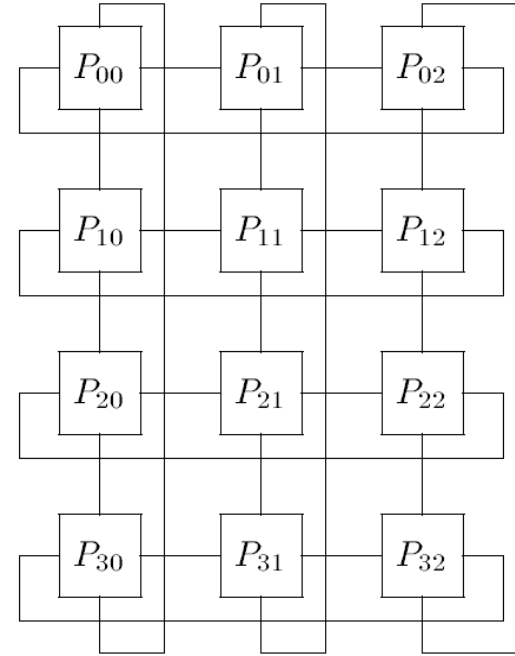
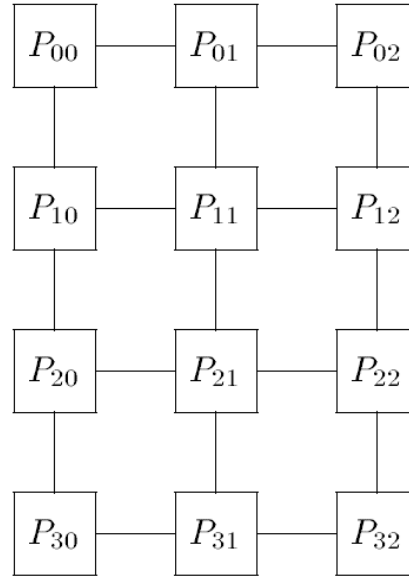
$$j = (i + 1) \bmod p \text{ (} P_j \text{ e vecinul din dreapta)}$$

- grad: $\Delta = 2$
- nr de arce = p
- diametru $D = \text{floor}(\frac{p}{2})$



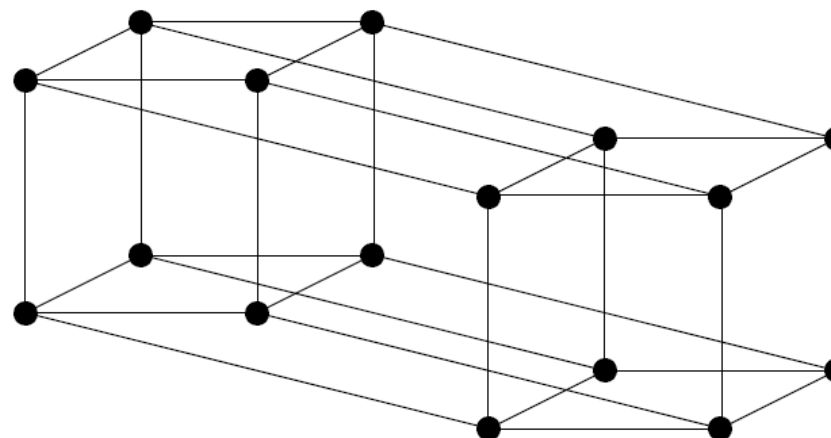
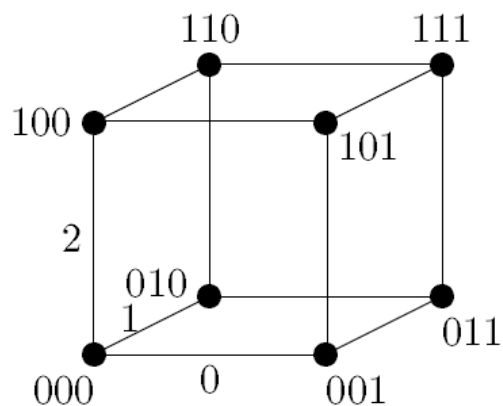
Grid (tor)

- grid patrat: $\sqrt{p} * \sqrt{p}$
- grad: $\Delta = 4$
- nr de arce = $2p$
- diametru $D = 2 * \text{floor}(\frac{p}{2})$



Hipercub

- topologie avantajoasa pentru unii algoritmi
- H_d = hipercub de dimensiune d , cu $p = 2^d$ procesoare
- fiecare procesor are d vecini
- P_i legat de P_j daca reprezentarile binare ale lui i si j difera printr-un singur bit



Hipercub (cont.)

- grad: $\Delta = d = \log(p)$
- nr de arce = $2p$
- diametru $D = p$
- $d(i,j)$ = distanta Hamming dintre codurile binare ale lui i si j
- definitie recursiva: H_d compus din doua sub-cuburi H_{d-1} unite prin arce care conecteaza nodurile din aceeasi pozitie in sub-cuburi
 - nodurile din primul sub-cub primesc 0 in cea mai semnificativa pozitie a reprezentarii lor binare, iar cele din al doilea sub-cub 1

Sumar topologii

Topologie	Δ	D
Inel	2	$O(p)$
Grid	4	$O(\sqrt{p})$
Hipercub	$\log p$	$O(\log p)$

Algoritmi paraleli

Curs 2

Vlad Olaru

vlad.olaru@fmi.unibuc.ro

Calculatoare si Tehnologia Informatiei

Universitatea din Bucuresti

Modelul SPMD

- adecvat calculatoarelor MIMD
- vom considera doar cazul MIMD NORMA (memorie distribuita)
- SPMD, Single Program Multiple Data
 - toate CPU executa acelasi program, fiecare opereaza pe propriul set de date
 - executia instructiunilor nu e sincrona (nu e sistem SIMD !), sistem NORMA
 - fiecare CPU executa propriile instructiuni in ritmul propriu

Adresa unui procesor

- program executat pe p procesoare
- $p \leq N$, unde N = nr procesoare calculator paralel
- numerotare: $0, \dots, p - 1$
- numerotare dinamica, stabilita la momentul lansarii in executie
 - un procesor isi poate afla adresa cu ajutorul unei primitive de biblioteca (sau SO)
- programele SPMD folosesc adresa de procesor ca pe o variabila
 - in particular, folosita pentru a discrimina executia diferitelor portiuni din program

Discriminarea executiei

- fie id variabila pt. adresa procesorului

- Ex:

```
1:       $id = \text{adresa CPU}$   
2:      if  $id == 4$  then  
3:           $x = 1$   
4:      else  
5:           $x = 0$ 
```

- linia 3 executata doar de catre procesorul P_4
- celelalte $p - 1$ procesoare executa linia 5
- Obs: variabila x e locala fiecarui procesor !

Variabile si indici

- variabilele unui program SPMD sunt *multiplicate*
 - fiecare CPU are propria copie locala, INACCESIBILA altor procesoare !
- regula: procesoarele nu pot modifica variabile din memoria altor procesoare
- interpretare teoretica a exemplului anterior
 - x vector cu p elemente indexat dupa id-ul de procesor
 - programul initializeaza vectorul cu zero, cu exceptia elementului 4
- in programe se folosesc indici locali
- mai rar, pt anumiti algoritmi vom folosi indici globali

Model de comunicatie prin mesaje

- message passing, model pt. calculatoare MIMD NORMA
- fiecare calculator are propria memorie, coordonarea accesului la date se face prin schimb de mesaje
- operatie de baza: procesor sursa P_s trimite mesaj M cu date din memoria locala M_s catre un procesor destinatie P_d care stocheaza datele in memoria sa M_d
- daca nu se specifica altfel, se pp ca sursa si destinatia sunt adiacente (in graful aferent topologiei de interconectare a procesoarelor)
- algoritmi paraleli se vor optimiza pentru topologia folosita

Primitive de baza pt comunicatie

- doua operatii sunt suficiente: *send*, *receive* (*recv*)
 - procesorul sursa transmite mesajul folosind *send*
 - procesorul destinatie receptioneaza mesajul folosind *receive*
- sintaxa
 - send**(*date*, *destinatie*)
 - recv**(*date*, *sursa*)
- *date* = variabila locala care stocheaza datele trimise, respectiv primite
- lungimea datelor rezulta din context
- in general, pp ca datele ocupa o zona contigua de memorie

Primitive de baza pt comunicatie (cont)

- sintaxa

send(*date*, *destinatie*)

recv(*date*, *sursa*)

- *sursa/destinatie* identifica vecinul cu care se comunica
 - uzual vom folosi adresa (id-ul de procesor)
- alternativ, se poate preciza directia in care se afla vecinul
 - inel/lant liniar: stanga/dreapta
 - grid/tor: stanga/dreapta, sus/jos (sau est/vest, nord/sud)
 - hipercub: dimensiunea pe care se comunica (nr intre 0 si $d - 1$)
 - ex: transmiterea liniei i din matricea A catre vecinul din dreapta al procesorului care executa instructiunea

send($A(i,:)$, dreapta)

Corectitudinea comunicatiei

- oricarei operatii de transmisie a unui processor trebuie sa-i corespunda o operatie de receptie a unui procesor vecin
 - i.e., **send** & **recv** apar in perechi pe ansamblul procesoarelor
- ex: pp. procesorul P_k trimite acelasi mesaj M vecinilor sai intr-un inel, P_{k-1} si P_{k+1}

```

1:      if  $id == k$  then
2:          send( $M$ , dreapta)
3:          send( $M$ , stanga)
4:      else if  $id == (k-1) \% p$  then recv( $M$ , dreapta)
5:      else if  $id == (k+1) \% p$  then recv( $M$ , stanga)

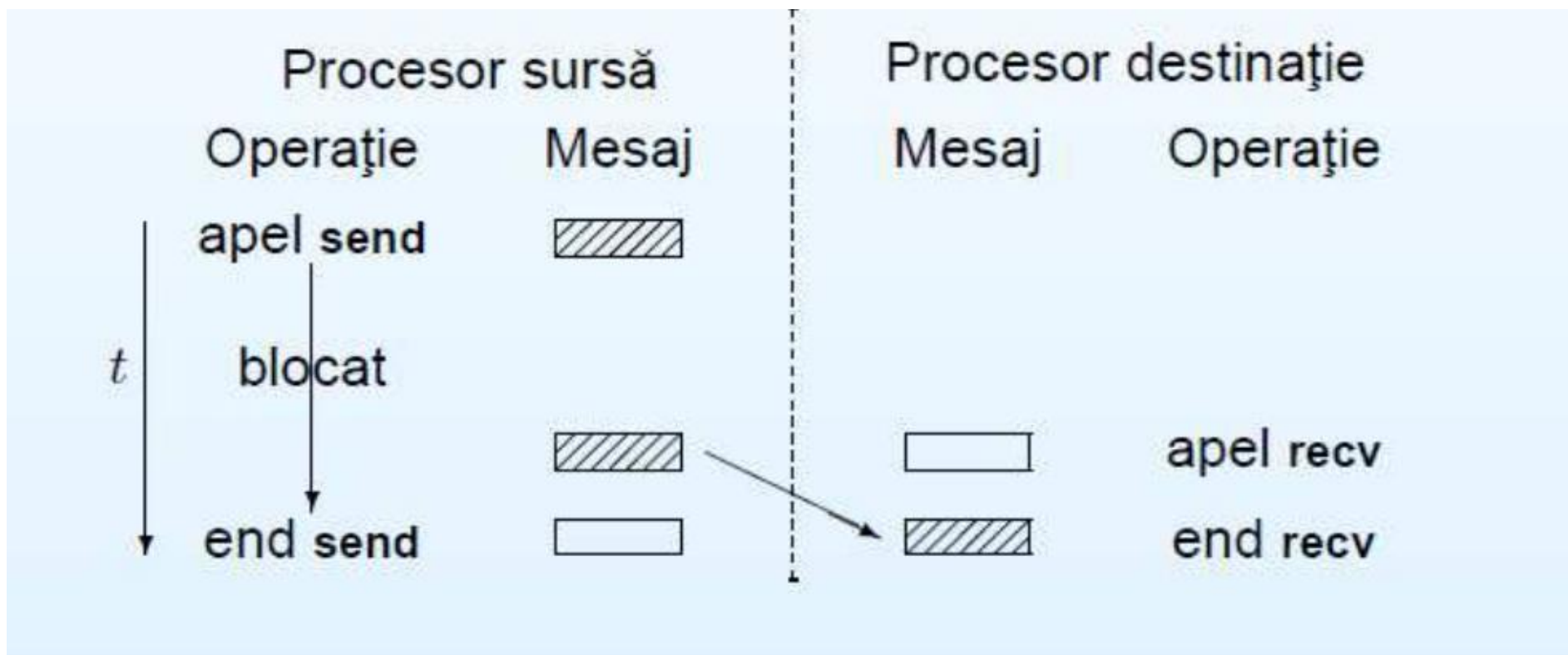
```

Sincronizarea comunicatiei

- pp. P_s trimite un mesaj catre P_d
- in general, momentul transmisiei e diferit de cel al receptiei
- transmisia mesajului devine efectiva doar dupa ce ambele procesoare au executat **send**, respectiv **recv**
- Q: ce face P_s dupa **send** pana cand P_d apeleaza **recv**?
 - doua raspunsuri posibile
 - asteapta (fara sa faca nimic) – *apel sincron, comunicatie blocanta*
 - poate apela alte operatii – *apel asincron, comunicatie ne-blocanta*

Comunicatie blocanta

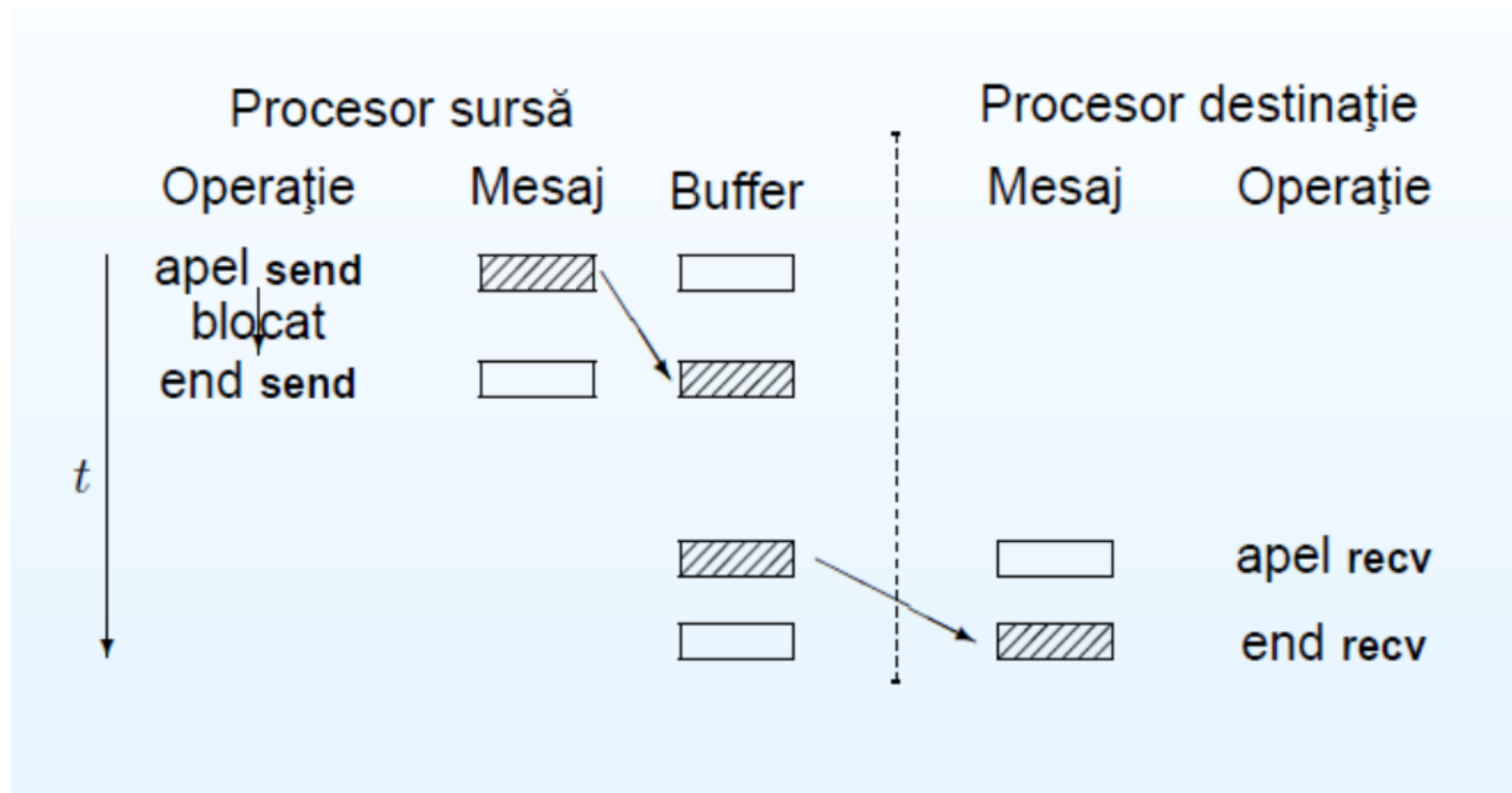
- P_d îl asteapta pe P_s fara sa faca nimic
- comunicatie blocanta, sincrona



Comunicatie blocanta prin buffer

- **send** muta mesajul intr-un buffer, apoi termina executia
- CPU sursa blocat doar pe durata copierii mesajului in buffer
- terminarea **send** nu e conditionata de apelul **recv** la destinatie
- la terminarea **send** (fie sincron, fie prin buffer), zona de memorie alocata mesajului se poate refolosi
- la terminarea **recv**, mesajul e receptionat in zona de memoria alocata in acest scop

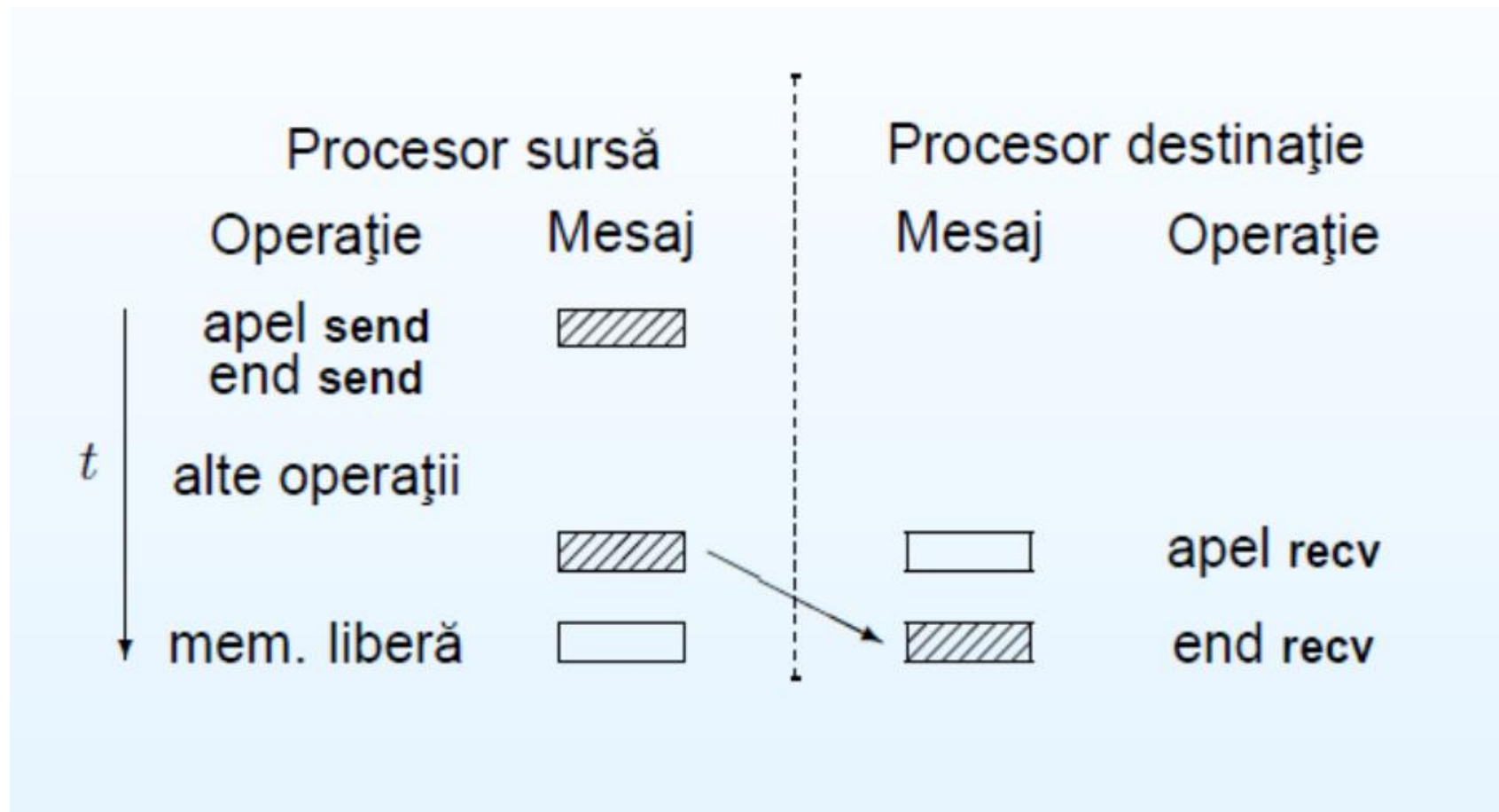
Comunicatie blocanta prin buffer



Comunicatie non-blocanta

- **send/recv** se termina *imediat* dupa apel
 - zona de memorie la **send**, respectiv zona de receptie la **recv** NU sunt libere !
- primitivele de comunicatie doar initiaza comunicatia
- transferul efectiv are loc ulterior, la nivelul sistemului de operare (sau al bibliotecii de comunicatie)
- dupa **send/recv**, procesorul poate executa *imediat* alte operatii, *in paralel sau concurent cu comunicatia* => suprapunerea calculului cu comunicatia (*computation-communication overlap*)
- exista si varianta de comunicatie non-blocanta prin buffer

Comunicatia non-blocanta



Primitive auxiliare pt. comunicatia non-blocanta

- trebuie sa existe posibilitatea de a determina cand s-a efectuat schimbul de mesaje
- primitiva **wait** asteapta terminarea comunicatiei
- scenariu tipic

1: **send**(*date*, dest)

2: executa operatii care nu implica (modifica) *date*

3: **wait** terminare **send**

4: modifica *date*

- daca **wait** se executa imediat dupa **send/recv** non-blocante, efectul este acelasi cu executia primitivelor blocante
- varianta: primitiva **test**, verifica ne-blocant terminarea comunicatiei
 - se foloseste intr-un scenariu de *polling*

Comunicatie blocanta vs ne-blocanta

- comunicatia non-blocanta faciliteaza o utilizare mai eficienta a procesoarelor, evitand timpii de asteptare
- erori de programare a comunicatiei blocante pot conduce la blocarea intregului program (*deadlock*)
- erorile de programare a comunicatiei non-blocante sunt mai subtile
 - exista posibilitatea alterarii informatiei transmise
 - eg, reutilizarea memoriei mesajului trimis, respectiv utilizarea bufferului de receptie inainte de primirea efectiva a datelor
- in general, depanarea programelor paralele e mult mai dificila decat a celor secventiale

Eroare tipica in comunicatia blocanta

- problema: fiecare procesor trimite un mesaj vecinului din dreapta pe o topologie inel
 - obs: datorita topologiei inel => fiecare procesor primeste un mesaj de la vecinul din stanga
- fie *date1* si *date2* variabilele locale folosite pentru transmisie, respectiv receptie
- program incorect in varianta blocanta sincrona

`send(date1, dreapta)`

`recv(date2, stanga)`

- fiecare procesor transmite blocant si asteapta ca procesorul din dreapta sa execute receptia, dar acesta la randul lui e blocat in propriul **send**, samd

=> deadlock !

Solutii

- solutia 1:
 - transmisie prin buffer
 - **send** se termina local cand *date1* s-a copiat in bufferul de sistem si poate incepe sa execute receptia pt *date2*
 - terminarea locala a apelului **send** se face independent de procesorul destinatie
- solutia 2:
 - o parte din procesoare incep prin a transmite, celelalte prin a receptiona

```

if id % 2 == 0 then
    send(date1, dreapta)
    recv(date2, stanga)
else
    recv(date2, stanga)
    send(date1, dreapta)

```


Solutia non-blocanta

send(*date1*, dreapta)

recv(*date2*, stanga)

wait terminare **send** si **recv**

- intre comunicatie si operatia de **wait** se pot eventual intercala alte operatii utile

Standardul MPI

- Message Passing Interface (MPI): standard de comunicatie pentru programe SPMD
- implementat atat pe calculatoare MIMD cat si pe retele de calculatoare
- implementat pe diverse sisteme de operare (Linux/Unix, Windows) si in diferite limbaje de programare (C, Fortran, python)
- avantaje
 - portabilitate, acelasi program MPI si poate compila si executa pe multe calculatoare
 - comunicatie implementata eficient pe fiecare calculator
 - testare/depanare in medii ieftine (eg, desktop-uri + LAN), urmata de rulare pe supercalculatoare dupa declararea gold standard

Generalitati MPI

- un program MPI se executa pe un grup de procesoare numit *communicator*
- communicator predefinit: MPI_COMM_WORLD, contine toate procesoarele disponibile programului
- numerotare procesoare communicator: $0, \dots, p-1$
- adresa unui procesor dintr-un communicator s.n. *rang* (*rank*)
- rangul unui procesor se afla cu functia

`int MPI_Comm_rank(MPI_Comm com, int *id)`

- numarul de procesoare ale unui communicator se afla cu functia

`int MPI_Comm_size(MPI_Comm com, int *p)`

Generalitati MPI (cont.)

- MPI_Comm, tip predefinit pt comunicatoare
- *com*, comunicatorul in care se afla procesorul care executa codul
- variabilele *p* si *id* contin dupa apel nr de procesoare, respectiv adresa procesorului apelant
- toate rutinele MPI intorc un intreg care caracterizeaza succesul executiei

Structura unui program MPI in C

- model SPMD, *seamana* cu un program secvential
- orice program MPI incepe cu **MPI_Init** (inainte sa apeleze orice alte rutine MPI)
- orice program MPI trebuie sa se termine cu **MPI_Finalize** (dupa acest apel nu se mai pot apela rutine MPI)
- **MPI_Init** primeste argumente *argc* si *argv* (semnificatia depinde de implementarea MPI)
- dincolo de aceasta restrictie, programatorul are libertate totala in folosirea rutinelor MPI cf algoritmului sau

Rutine de comunicatie

- implicit, oricare doua procesoare isi pot trimite mesaje (i.e., topologia virtuala este de graf complet)

```
MPI_Send(void *mesg, int len, MPI_Datatype type,  
          int dest, int tag, MPI_comm com);
```

- MPI_Send trimite un mesaj *mesg* de lungime *len* si tip *type* procesorului *dest* din comunicatorul *com*
- MPI are identificatori predefiniti pt tipurile uzuale de date; se pot construi tipuri complexe cu rutine specifice
- mesajul are asociata o eticheta (*tag*), care il personalizeaza
- comunicatorul *com* trebuie sa contina atat procesorul sursa cat si pe cel destinatie

Rutine de comunicatie (cont.)

```
MPI_Recv(void *mesg, int len, MPI_Datatype type, int source,  
         int tag, MPI_comm com, MPI_Status *status);
```

- MPI_Recv receptioneaza un mesaj de lungime maxima *len* de la procesorul *source* din comunicatorul *com* (mesajul poate fi eventual trunchiat)
- tipul de date poate eventual diferi la sursa si destinatie
- MPI_Recv foloseste *tag* pentru a face matching pe mesajele trimise de MPI_Send (MPI_ANY_TAG se poate folosi pentru a primi orice mesaj)
- *status* indica sursa mesajului si eticheta lui + informatie aditionala, gen nr de octeti efectiv primiti

Moduri de comunicatie MPI

- blocant/non-blocant, prin buffer sau sincron
- implementarea implicita a rutinelor MPI_Send/MPI_Recv nu e precizata
 - e posibil ca mesajele scurte sa fie transmise prin buffer iar cele lungi sincron
 - programatorul trebuie sa fie constient de acest aspect
- comunicatie non-blocanta
 - MPI_Wait asteapta terminarea transmisiei sau a receptiei
 - MPI_Test verifica daca transmisia sau receptia s-au terminat

Rutine MPI de comunicatie

Comunicație	blocantă	non-blocantă
standard	MPI_Send MPI_Recv	MPI_Isend MPI_Irecv
prin buffer	MPI_Bsend	MPI_Ibsend
sincronă	MPI_Ssend	MPI_Issend

Alte functii MPI

- comunicatie globala
 - implica toate procesoarele dintr-un comunicator
 - sincronizare, difuzare, distributie, etc
- alte operatii globale
 - calculul maximului unor valori distribuite tuturor procesoarelor dintr-un comunicator
 - calculul sumei, etc
- topologii virtuale: performanta algoritmilor paraleli depinde (si) de topologia HW de interconectare a procesoarelor
 - programatorul poate descrie virtual aceasta topologie
- procese
 - create dinamic, imbina paralelismul HW cu concurenta SW

Operatii de comunicare globala

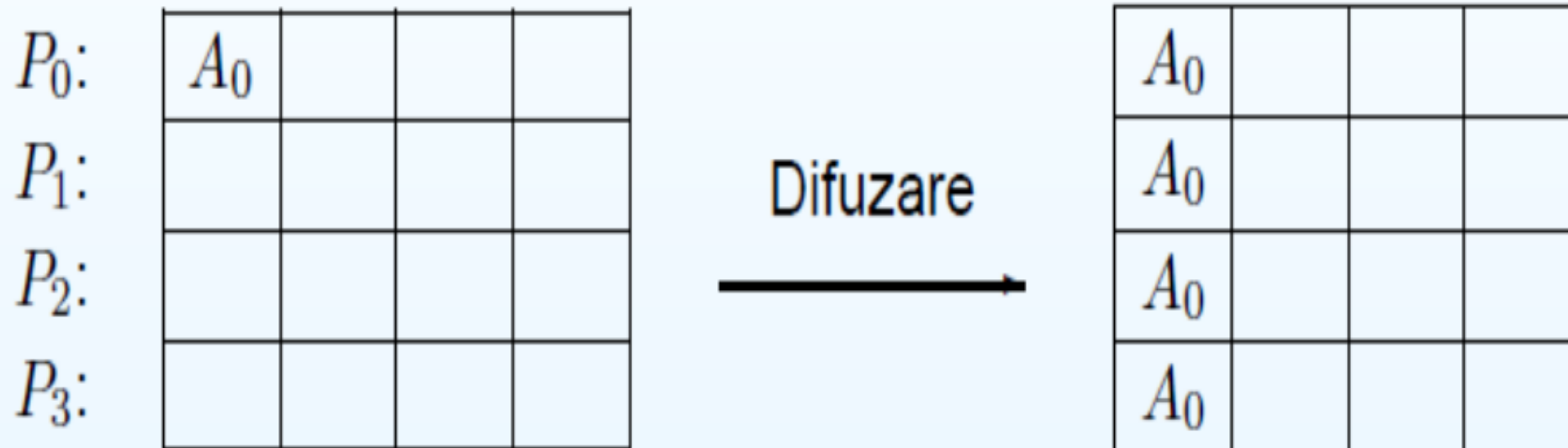
- comunicatii la care participa toate procesoarele dintr-un comunicator
 - sincronizare (bariera)
 - difuzare (broadcast, one-to-all)
 - difuzare generala (all-to-all broadcast)
 - distributie (difuzare personalizata, scattering)
 - colectare (gathering)
 - schimb complet/generalizat (total exchange, multidistributie, transpunere)

Sincronizare

- MPI_Barrier, primitiva tipica pt programele paralele
- forma implicita de comunicare, fara mesaj propriu-zis
- toate procesoarele dintr-un grup care apeleaza primitiva bariera asteapta pana cand ultimul dintre ele ajunge se execute primitiva
 - i.e. toate procesoarele asteapta sa treaca *simultan* de bariera
- dupa bariera, fiecare procesor continua individual cu executia instructiunii urmatoare

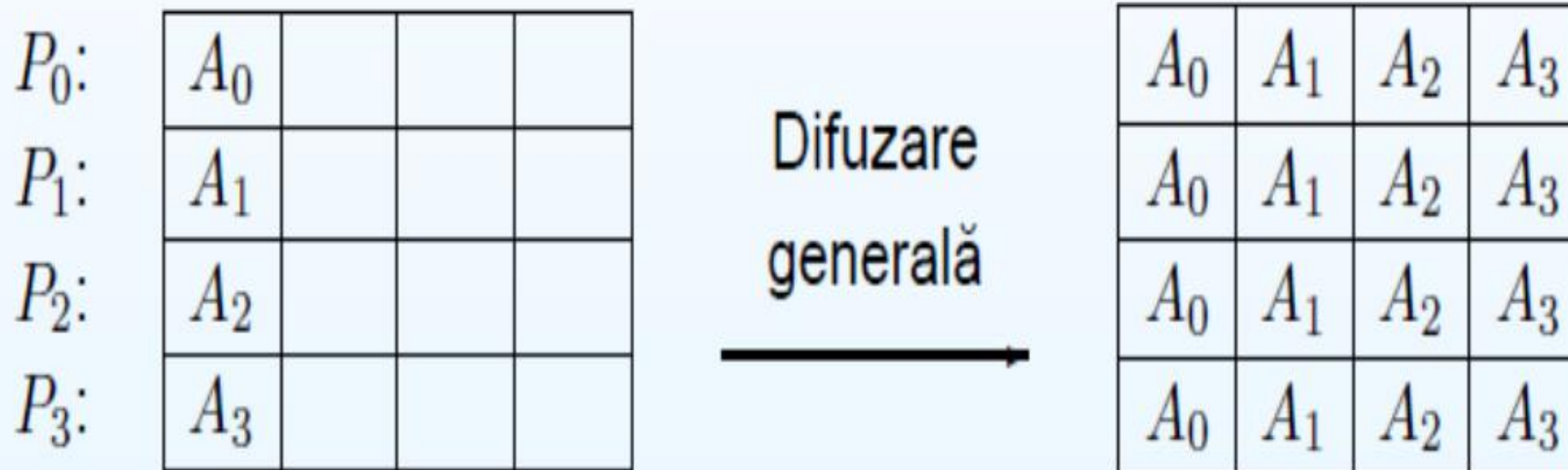
Difuzare

- un procesor trimite un singur mesaj tuturor celorlalte procesoare



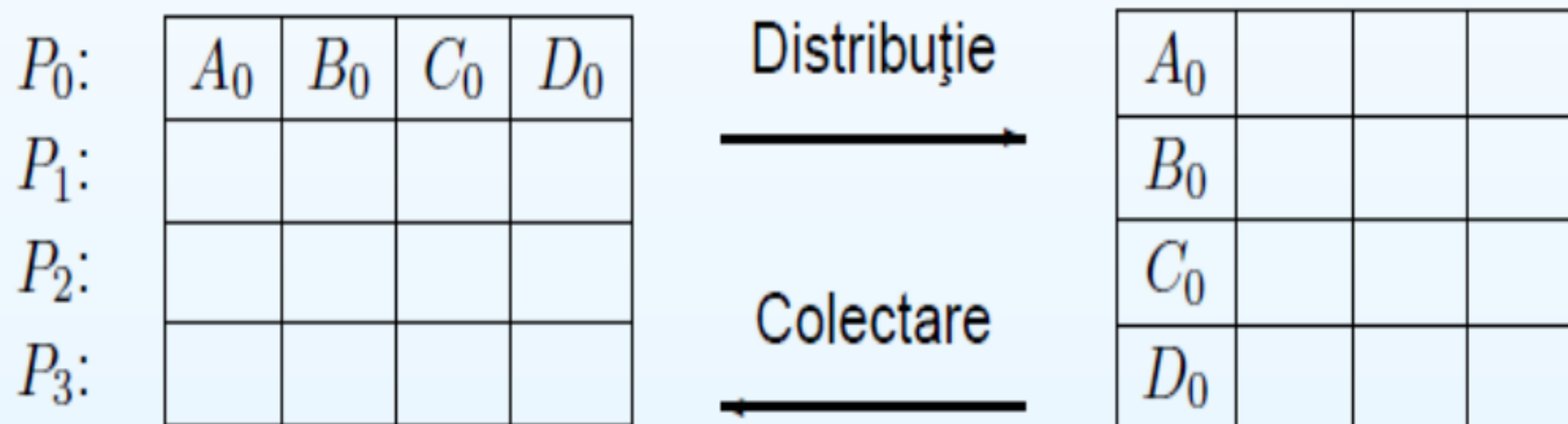
Difuzare generala

- fiecare procesor trimite acelasi mesaj tuturor celorlalte procesoare
- echivalent, fiecare procesor executa o difuzare proprie



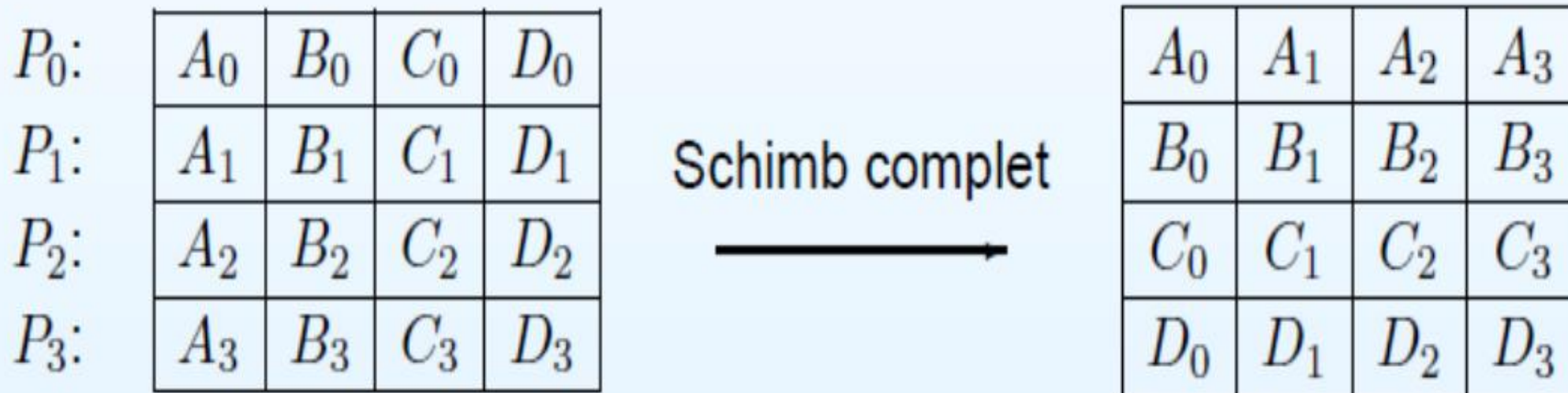
Distributie si colectare

- distributie: un procesor trimite cate un mesaj fiecarui alt procesor
- colectare: operatia inversa, un procesor primeste cate un mesaj de la toate celelalte procesoare



Schimb complet

- fiecare procesor trimite un mesaj tuturor celorlalte procesoare
- echivalent, fiecare procesor executa o distributie (sau o colectare)
- daca un procesor are initial o linie dintr-o matrice $p \times p$, in final primeste o coloana (motiv pt. care operatia se mai numeste si *transpunere*)



Exemplu send/recv

```
3 #include <mpi.h>
4 #include <stdio.h>
5 #include <stdlib.h>
6
7
8 int main(int argc, char** argv) {
9
10     MPI_Init(NULL, NULL);
11
12     // Find out rank, size
13     int rank, world_size;
14     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
15     MPI_Comm_size(MPI_COMM_WORLD, &world_size);
16
17     int number;
18
19     if (rank == 0) // procesorul 1 ignora acest bloc de instructiuni
20     {
21         number = 200;
22         MPI_Send(&number, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
23     }
24     else if (rank == 1) // procesorul 0 ignora acest bloc de instructiuni
25     {
26         MPI_Recv(&number, 1, MPI_INT, 0, 0, MPI_COMM_WORLD,
27                 MPI_STATUS_IGNORE);
28         printf("Procesor 1 primeste numarul %d de la procesorul 0\n", number);
29     }
30
31     MPI_Finalize();
32 }
```

Tipuri de date elementare

MPI datatype	C equivalent
MPI_SHORT	short int
MPI_INT	int
MPI_LONG	long int
MPI_LONG_LONG	long long int
MPI_UNSIGNED_CHAR	unsigned char
MPI_UNSIGNED_SHORT	unsigned short int
MPI_UNSIGNED	unsigned int
MPI_UNSIGNED_LONG	unsigned long int
MPI_UNSIGNED_LONG_LONG	unsigned long long int
MPI_FLOAT	float
MPI_DOUBLE	double
MPI_LONG_DOUBLE	long double
MPI_BYTE	char

Detalii MPI_Recv

- mesajul se receptioneaza cand corespund urmatoarele argumente
 - id-ul (rangul) procesorului sursa
 - tag-ul (eticheta)
 - comunicatorul (eg, MPI_COMM_WORLD)
- valori predefinite
 - MPI_ANY_TAG
 - MPI_ANY_SOURCE

Problema

- se dau doua procesoare P_0 si P_1
- implementati o comunicatie alternanta intre cele doua procesoare
 - t_0 : P_0 trimite date lui P_1
 - t_1 : P_1 trimite date lui P_0
 - t_2 : P_0 trimite date lui P_1
 - sand

Communicatie ping-pong

```
12 // Find out rank, size
13 int rank, world_size;
14 MPI_Comm_rank(MPI_COMM_WORLD, &rank);
15 MPI_Comm_size(MPI_COMM_WORLD, &world_size);
16
17 int ping_pong_count = 0;
18 int partner_rank = (rank + 1) % 2;
19
20 while (ping_pong_count < PING_PONG_LIMIT) {
21
22     if (rank == ping_pong_count % 2)
23     {
24
25         // Increment the ping pong count before you send it
26         ping_pong_count++;
27         MPI_Send(&ping_pong_count, 1, MPI_INT, partner_rank, 0, MPI_COMM_WORLD);
28         printf("%d sent and incremented ping_pong_count "
29              "%d to %d\n", rank, ping_pong_count, partner_rank);
30     } else
31     {
32         MPI_Recv(&ping_pong_count, 1, MPI_INT, partner_rank, 0,
33                MPI_COMM_WORLD, MPI_STATUS_IGNORE);
34         printf("%d received ping_pong_count %d from %d\n",
35              rank, ping_pong_count, partner_rank);
36     }
37 }
38 }
```

Deadlock

```

8 int main(int argc, char** argv) {
9
10  MPI_Init(NULL, NULL);
11
12  int world_size, rank, n=0;
13  MPI_Comm_size(MPI_COMM_WORLD, &world_size);
14  MPI_Comm_rank(MPI_COMM_WORLD, &rank);
15  MPI_Status stat;
16
17  // Varianta functionala
18
19  if (rank == 0) {
20      MPI_Send(&n, 1, MPI_INT, 1, tag, MPI_COMM_WORLD);
21      MPI_Recv(&n, 1, MPI_INT, 1, tag, MPI_COMM_WORLD, &stat);
22  }
23  else { // rank==1
24      MPI_Recv(&n, cnt, MPI_INT, 0, tag, MPI_COMM_WORLD, &stat);
25      MPI_Send(&n, cnt, MPI_INT, 0, tag, MPI_COMM_WORLD);
26  }
27
28  // Varianta deadlock
29
30  if (rank == 0) {
31      MPI_Recv(&n, 1, MPI_INT, 1, tag, MPI_COMM_WORLD, &stat);
32      MPI_Send(&n, 1, MPI_INT, 1, tag, MPI_COMM_WORLD);
33  }
34  else { // rank==1
35      MPI_Recv(&n, cnt, MPI_INT, 0, tag, MPI_COMM_WORLD, &stat);
36      MPI_Send(&n, cnt, MPI_INT, 0, tag, MPI_COMM_WORLD);
37  }
38  MPI_Finalize();
39 }

```

Comunicatie non-blocanta

- MPI_Isend (buf, count, datatype, dest, tag, comm, request)
- MPI_Irecv(buf, count, datatype, src, tag, comm, request)
- similare cu MPI_Send/MPI_Recv, apare in plus ultimul argument
- *request* este un handle care contine starea transmisiei/receptiei
 - se foloseste impreuna cu MPI_Wait

Ex. comunicatie non-blocanta (1)

```

3 #include <mpi.h>
4 #include <stdio.h>
5 #include <stdlib.h>
6
7
8 int main(int argc, char** argv) {
9
10     MPI_Request request;
11     MPI_Status status;
12     double s_buf[100], r_buf[100];
13
14     MPI_Init(NULL, NULL);
15
16     int world_size, rank, recv_count;
17     MPI_Comm_size(MPI_COMM_WORLD, &world_size);
18     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
19
20     if (rank==0){
21         MPI_Isend(s_buf, 100, MPI_DOUBLE, 1, 10, MPI_COMM_WORLD, &request);
22         MPI_Recv(r_buf, 100, MPI_DOUBLE, 1, 22, MPI_COMM_WORLD, &status);
23         MPI_Wait(&request, &status);
24     }
25     else if(rank == 1){
26         MPI_Isend(s_buf, 100, MPI_DOUBLE, 0, 22, MPI_COMM_WORLD, &request);
27         MPI_Recv(r_buf, 100, MPI_DOUBLE, 0, 10, MPI_COMM_WORLD, &status);
28         MPI_Wait(&request, &status);
29     }
30
31     MPI_Get_count(&status, MPI_DOUBLE, &recv_count);
32     printf("proc %d, source %d, tag %d, count %d\n", rank, status.MPI_SOURCE,
33           status.MPI_TAG, recv_count);
34
35     MPI_Finalize();
36 }

```


Ex. comunicatie non-blocanta (2)

```
13 int rank, world_size;
14 MPI_Request req;
15 MPI_Comm_rank(MPI_COMM_WORLD, &rank);
16 MPI_Comm_size(MPI_COMM_WORLD, &world_size);
17 int *v = calloc(20, sizeof(int));
18 int i;
19
20 if (rank == 0) // procesorul 1 ignora acest bloc de instructiuni
21 {
22     for (i=0; i<20; i++) v[i]=1;
23     MPI_Isend(v, 20, MPI_INT, 1, 0, MPI_COMM_WORLD, &req);
24 }
25 else if (rank == 1) // procesorul 0 ignora acest bloc de instructiuni
26 {
27     MPI_Irecv(v, 20, MPI_INT, 0, 0, MPI_COMM_WORLD, &req); //req);
28     for (i=0; i<20; i++) printf("%d ", v[i]);
29 }
30 MPI_Barrier(MPI_COMM_WORLD);
31
32 if (rank == 1)
33 {
34     printf("\n");
35     for (i=0; i<20; i++) printf("%d ", v[i]);
36     printf("\n");
37 }
38
39 MPI_Finalize();
40 }
```

Ex. comunicatie non-blocanta (3)

```

1 #include "mpi.h"
2 #include <stdio.h>
3
4 main(int argc, char *argv[]) {
5     int numtasks, rank, next, prev, buf[2], tag1=1, tag2=2;
6     MPI_Request reqs[4];    // required variable for non-blocking calls
7     MPI_Status stats[4];    // required variable for Waitall routine
8
9     MPI_Init(&argc,&argv);
10    MPI_Comm_size(MPI_COMM_WORLD, &numtasks);
11    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
12
13    // determine left and right neighbors
14    prev = rank-1;
15    next = rank+1;
16    if (rank == 0) prev = numtasks - 1;
17    if (rank == (numtasks - 1)) next = 0;
18
19    // post non-blocking receives and sends for neighbors
20    MPI_Irecv(&buf[0], 1, MPI_INT, prev, tag1, MPI_COMM_WORLD, &reqs[0]);
21    MPI_Irecv(&buf[1], 1, MPI_INT, next, tag2, MPI_COMM_WORLD, &reqs[1]);
22
23    MPI_Isend(&rank, 1, MPI_INT, prev, tag2, MPI_COMM_WORLD, &reqs[2]);
24    MPI_Isend(&rank, 1, MPI_INT, next, tag1, MPI_COMM_WORLD, &reqs[3]);
25
26    // do some work while sends/receives progress in background
27
28    // wait for all non-blocking operations to complete
29    MPI_Waitall(4, reqs, stats);
30
31    // continue - do more work
32
33    MPI_Finalize();
34 }

```

Comunicatie pe inel

```

3 #include <mpi.h>
4 #include <stdio.h>
5 #include <stdlib.h>
6
7
8 int main(int argc, char** argv) {
9
10     MPI_Init(NULL, NULL);
11     int world_size, rank;
12     MPI_Comm_size(MPI_COMM_WORLD, &world_size);
13     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
14
15     int token;
16     // Receive from the lower process and send to the higher process. Take care
17     // of the special case when you are the first process to prevent deadlock.
18     if (rank != 0) {
19         MPI_Recv(&token, 1, MPI_INT, rank - 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
20         printf("Procesorul %d a primit %d de la procesorul %d\n", rank, token, rank - 1);
21     } else {
22         // Set the token's value if you are process 0
23         token = -1;
24     }
25     MPI_Send(&token, 1, MPI_INT, (rank + 1) % world_size, 0, MPI_COMM_WORLD);
26     // Now process 0 can receive from the last process. This makes sure that at
27     // least one MPI_Send is initialized before all MPI_Recvs (again, to prevent
28     // deadlock)
29     if (rank == 0) {
30         MPI_Recv(&token, 1, MPI_INT, world_size - 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
31         printf("Procesorul %d a primit %d de la procesorul %d\n", rank, token, world_size - 1);
32     }
33
34     MPI_Finalize();
35 }

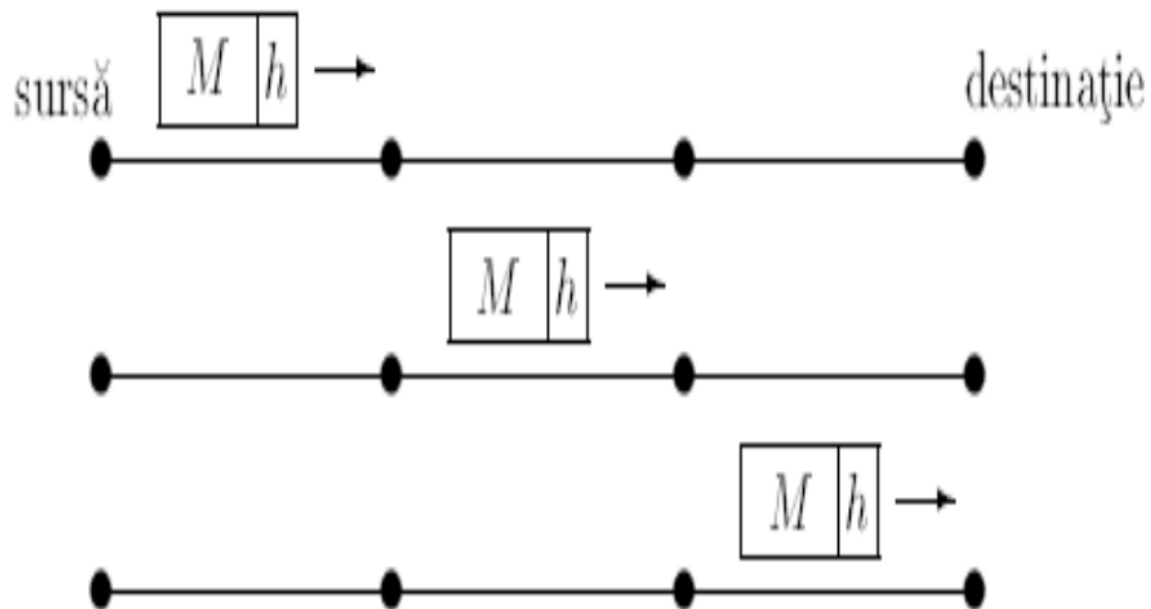
```

Algoritmi de comunicatie

- comunicatia e parte intrinseca, f. importanta, a algoritmilor paraleli
- modelele de comunicatie
 - specifica modul de transmitere al unui mesaj intre procesoare
 - estimeaza timpul necesar pentru comunicarea unui mesaj
- modele principale
 - store-and-forward
 - comutare de circuite
 - wormhole

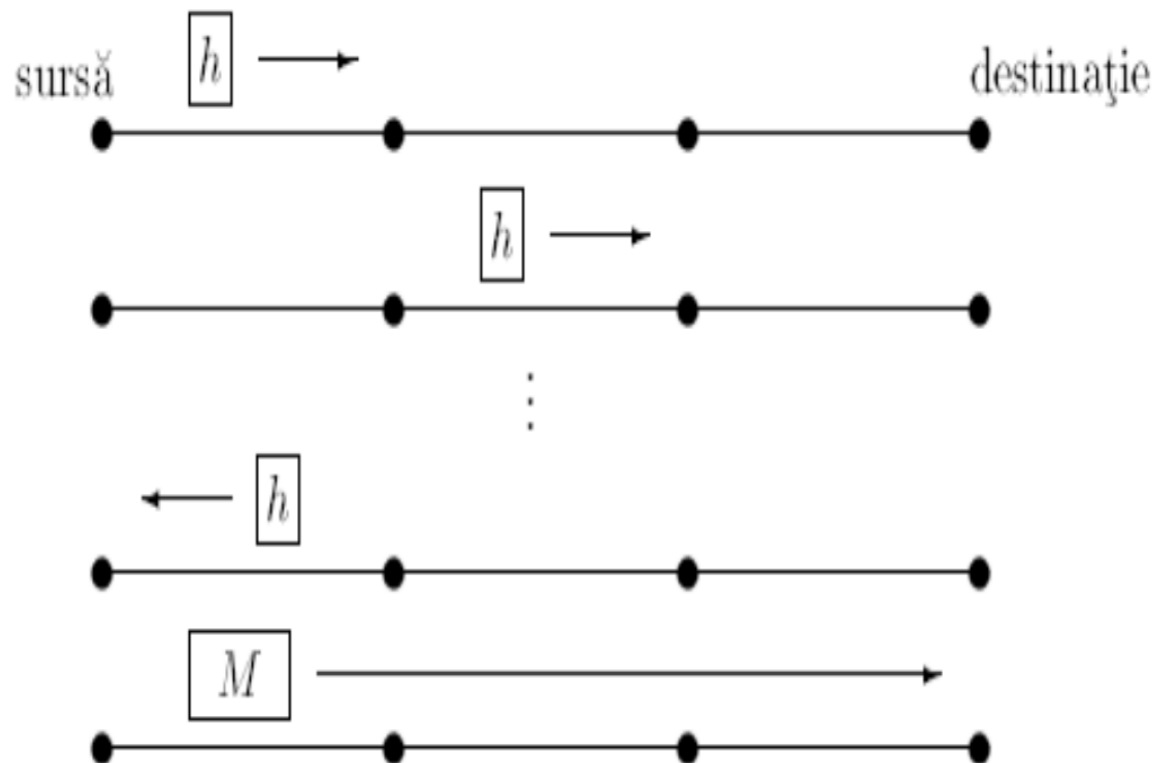
Store-and-forward

- mesajul transmis de sursă e receptat de vecin într-o zona de memorie (*store*)
- apoi e prelucrat și eventual trimis mai departe către destinație (*forward*)
- procesoarele memorează integral mesajul



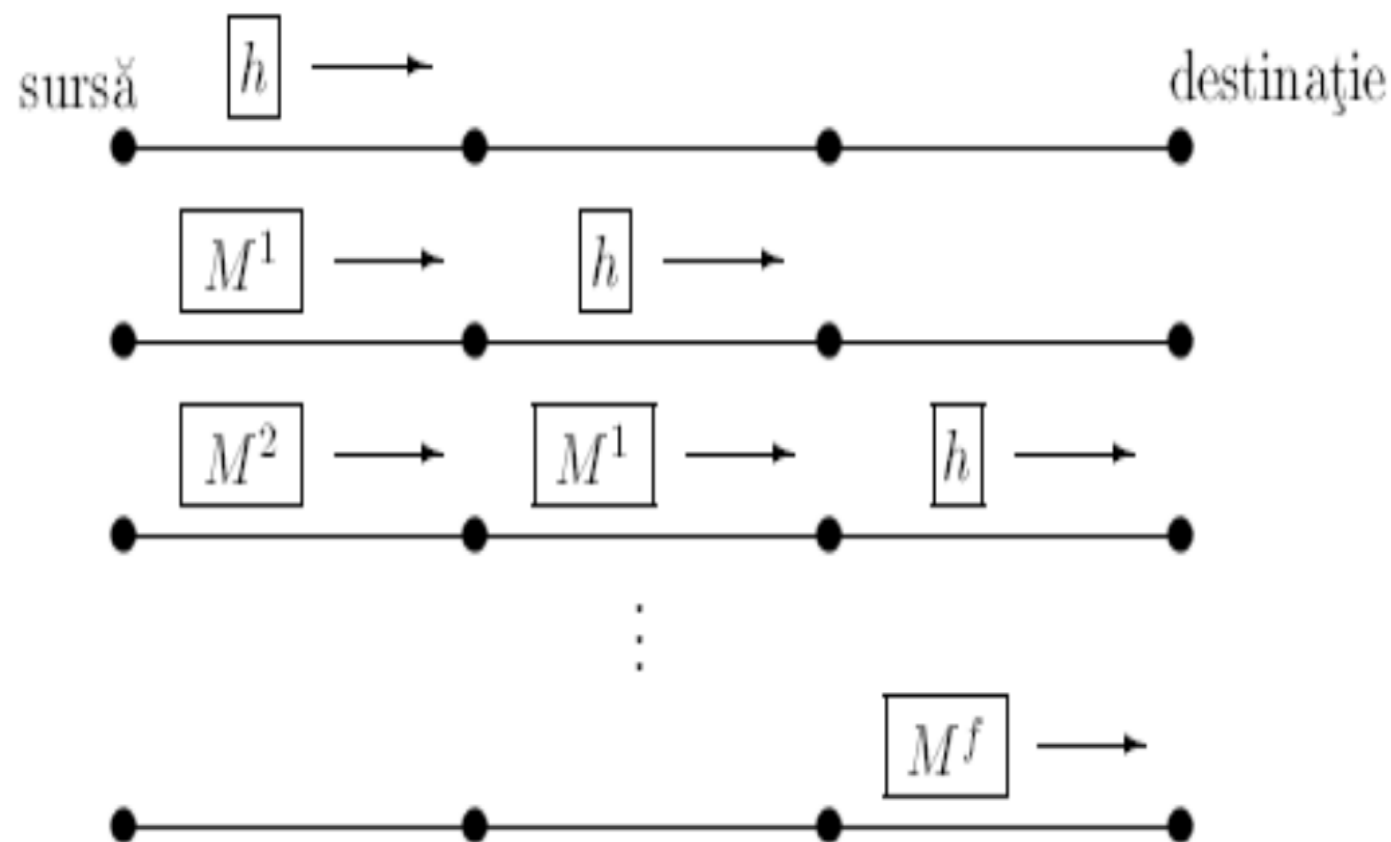
Comutare de circuite

- stabilirea unei cai fizice între sursa și destinație: antetul (header-ul) h al mesajului *rezerva* legăturile pe întreaga cale
- procesorul sursă e informat de rezervare
- mesajul este trimis direct destinatarului



Modelul wormhole

- mesajul M este impartit in pachete M^i
- dimensiunea pachetului = dimensiunea bufferului canal
- antetul avanseaza si restul pachetelor il urmeaza
- se elimina etapa de confirmare a caii spre destinatie



Modelarea timpului de comunicatie

- consideram modelul store-and-forward si un mesaj de m elemente
- modele posibile
 - durata de transmisie a unui mesaj este constanta
 - model proportional

$$t_c(m) = m\beta$$

β = durata transmisiei unui element

- model liniar

$$t_c(m) = \sigma + m\beta$$

σ = durata start-up-ului transmisiei

Operatii de comunicatie globala

- One-to-One: transmisie mesaj de la un procesor la altul (nu neaparat vecini)
- One-to-All: difuzare, transmiterea unui mesaj de la un procesor la toate celelalte
 - MPI_Bcast, MPI_Scatter
- All-to-All: difuzare generala, transmiterea unui mesaj de la fiecare procesor catre toate celelalte
 - MPI_Alltoall
- difuzare personalizata (scatter): transmiterea de la un procesor a cate unui mesaj pentru fiecare dintre celelalte procesoare
 - MPI_Scatter
- colectare (gather): operatie inversa, MPI_Gather

Algoritmi paraleli

Curs 3

Vlad Olaru

vlad.olaru@fmi.unibuc.ro

Calculatoare si Tehnologia Informatiei

Universitatea din Bucuresti

Modele formale

- varietate de modele de calcul paralel fiecare cu propriile presupuneri despre
 - puterea de calcul a procesoarelor
 - mecanismul de comunicare interprocesor
- ex presupuneri:
 - fiecare procesor poate executa instructiuni de baza (operatii aritmetice, comparatii, if-then-else)
 - fiecare instructiune ia o unitate de timp
 - exista un mecanism de comunicare interprocesor
 - cel mai adesea, operatiile de transfer de date sunt instantanee si fara costuri asociate
 - uneori pp ca folosim MIMD cu memorie partajata, alteori MIMD message-passing
 - pt schimbul de mesaje vom face pp asupra intarzierilor mesajelor in retea
- in continuare vom folosi un model simplu pt. ilustrarea unor aspecte importante ale calculului paralel
 - adecvat pt majoritatea algoritmilor sincroni despre care vom vorbi, cata vreme se ignora comunicatia interprocesor

Reprezentarea algoritmilor paraleli folosind grafuri aciclice orientate (DAG)

- grafuri aciclice orientate (DAG, directed acyclic graph) = graf orientat fara cicluri pozitive (formate din arce cu acelasi sens)

$G(V,E)$

- V este multimea nodurilor, care denota **operatiile** efectuate de algoritm
- E este multimea muchiilor, ce reprezinta **dependenta dintre date**
- $(i,j) \in E$: operatia nodului j foloseste rezultatele operatiei nodului i (nodul i este *predecesorul* lui j)
- operatia poate fi elementara (operatie aritmetica, booleana, etc) sau operatie de nivel inalt (eg, subrutina)

Terminologie DAG

- *in-degree* (i) = nr de predecesori ai nodului i
- *out-degree*(i) = nr de noduri pt care i este predecesor
- nodurile cu *in-degree* zero = noduri de intrare (*input*)
- noduri cu *out-degree* zero = noduri de iesire (*output*)
- cale pozitiva: secventa $i_0, \dots, i_K$ a. i. $(i_k, i_{k+1}) \in E$ pt $k = 0 \dots K - 1$
 - lungimea caii = K
- adancimea D a unui DAG:
 - calea pozitiva de lungime maxima din graf
 - leaga un nod input de un nod output
 - valoare finita (consecinta a lipsei ciclurilor din graf)
- obs: pp. $D \geq 1$ (adica exista cel putin o muchie/un arc)

Exemplu

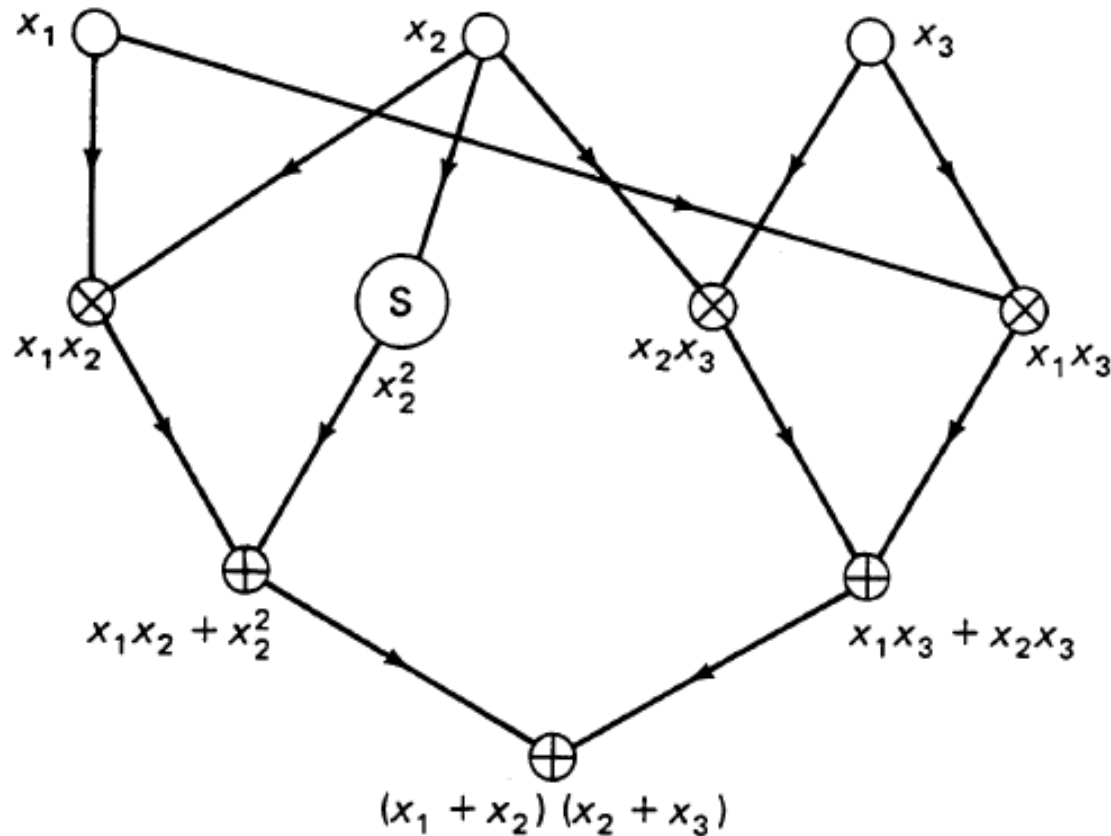


Figure 1.2.1 Representation of an algorithm for evaluating the arithmetic expression $(x_1 + x_2)(x_2 + x_3)$ by means of a DAG. The label at each node indicates the operation corresponding to that node. In particular, the label S stands for squaring.

Ipoteze

- DAG = reprezentare partiala a unui algoritm paralel
 - specifica operatiile care se executa, operanzii si constrangeri de precedenta in executia operatiilor
- 1) operatiile dureaza o singura unitate de timp (op. elementare, i.e. op. aritmetice)

$$x_i = f_i(\{ x_j \mid j \in \text{in-degree}(i) \})$$

unde f_i = operatia corespunzatoare nodului i (NB: nodul i este nod intern sau de output, nodurile de input sunt variabile externe de intrare)

- 2) pp. operatia asociata unui nod de intrare este citirea valorii, neglijabila ca durata, iar evaluarea f_i pt noduri interne si de output dureaza o unitate de timp
 - NB: in general, operatiile nodurilor pot dura mai mult de o unitate de timp
- 3) exista p procesoare disponibile si fiecare poate executa operatiile dorite

Ipoteze (cont.)

- 4) operatiile nodului i (nod intern sau de output) sunt efectuate de procesorul P_i si terminate la $t_i > 0$
- nodurilor interne i nu li se asociaza procesoare si $t_i = 0$
- 5) cel mult o operatie la un moment dat, i.e.

$$i \neq j \ \& \ t_i = t_j \Rightarrow P_i \neq P_j$$

unde i, j sunt noduri interne sau de output

- 6) $(i, j) \in E \Rightarrow t_j \geq t_i + 1$
 - operatia nodului j nu poate incepe decat dupa ce se termina operatia nodului i

Planificare

- dupa stabilirea P_i si t_i , se spune ca DAG-ul a fost planificat pentru executie paralela
- multimea $\{(i, P_i, t_i) | i \text{ nod intern sau de output}\}$ se numeste *schedule* (*program, planificare*)
- interpretare a posibilelor implementari
 - memorie partajata (shared memory) MPs
 - nodurile (procesoarele) care executa operatiile pot stoca rezultate in memoria partajata in folosul celorlalte procesoare
 - sisteme message-passing
 - un arc (i,j) reprezinta in fapt transmitia unui mesaj
 - modelul folosit nu include (neglijeaza) costul accesului la memorie, in cazul general trebuie considerat si el
 - daca $P_i = P_j$ atunci $t_j \geq t_i + 1$
 - daca $P_i \neq P_j$ atunci $t_j \geq t_i + 1 + t_{\text{acces_mem}}$
 - obs: $t_{\text{acces_mem}}$ depinde de pozitia procesoarelor P_i si P_j in retea de interconectare

Masuri de complexitate

- numarul de procesoare
- timpul de executie (*complexitate in timp*)
- numarul de mesaje transmise in timpul executiei (*complexitate de comunicatie*)

E.g.: $O(\log(n))$, $O(n^2)$, $O(n \log(n))$

Reamintim:

- $f(x) = O(g(x)) \Leftrightarrow \exists c, x_0, \text{ a.i. } f(x) \leq cg(x), \forall x \geq x_0$
- $f(x) = \Omega(g(x)) \Leftrightarrow \exists c, x_0, \text{ a.i. } f(x) \geq cg(x), \forall x \geq x_0$
- $f(x) = \Theta(g(x)) \Leftrightarrow f(x) = O(g(x)) \ \&\& \ f(x) = \Omega(g(x))$

Masuri de complexitate (cont.)

- uzual exprimate ca functii dependente de *dimensiunea* problemei
 - informal, numarul de date de intrare
 - ex: n pt. adunarea a n date scalare
- Obs: chiar daca dimensiunea problemei e constanta, e posibil ca resursele folosite sa depinda de valoarea datelor de intrare

=> pt o problema de dimensiune data, trebuie calculate resursele necesare in cel mai defavorabil caz pt. toate combinatiile posibile ale datelor de intrare
- aspecte subtile
 - e posibil ca toate procesoarele sa fi terminat calculul, fara sa o stie insa
 - in acest caz, complexitatea include si timpul necesar procesoarelor sa decida terminarea algoritmului

Complexitate in timp (DAG)

- fie $G(V,E)$ un DAG ce reprezinta un algoritm paralel
- fie $S = \{(i, P_i, t_i) \mid i \text{ nod intern sau de output}\}$ o planificare (schedule) a DAG-ului pentru p procesoare
- timpul aferent executiei planificarii este $\max_i t_i$
- **complexitatea in timp** T_p = timpul minim cheltuit pentru executia celei mai bune planificari care foloseste p procesoare

$$T_p = \min_S \max_i t_i$$

Timpul de executie minim cand avem p procesoare la dispozitie, complexitatea in timp a algoritmului descris de G

Complexitate de timp (DAG)

- T_1 = complexitatea in timp seriala (1 procesor), i.e. nr de noduri din G care nu sunt noduri input
- T_∞ = complexitatea in timp a algoritmului specificat de G , dat fiind un numar suficient de mare de procesoare (cel putin p^*)

$$T_\infty = \min_{p \geq 1} T_p$$

$$\text{i.e., } \exists p^* \text{ a.i. } T_p = T_\infty \forall p \geq p^*$$

Obs: T_p = functie necrescatoare de p , limitata inferior de 0

- in general, $T_\infty \leq T_p \leq T_1$
- T_∞ = adancimea ("depth"-ul) G ($T_\infty = D$, cf. definitiei anterioare)

Demonstratie $T_\infty = D$

$\Rightarrow$

- fie $i_0, \dots, i_k$ cea mai lunga cale (pozitiva) din $G \Rightarrow i_0$ nod de intrare si $k = D$ (cf. definitiei lui D)
- $\forall S$ planificare, $t_{i_0} = 0$ si $t_{j+1} \geq t_j + 1 \ \forall j = i_0, \dots, i_{k-1}$
 $\Rightarrow t_{i_k} \geq k = D \quad \Rightarrow T_\infty \geq D$

$\Leftarrow$

- fie procesorul P_i asignat nodului i
- $t_i = \text{nr. arce al celei mai lungi cai (pozitive) de la un nod de intrare la } i$ ($t_i = 0$ pt i nod de intrare)
- $(i, j) \in E \Rightarrow t_j \geq t_i + 1$ (adaug un arc la cea mai lunga cale catre i)
 $\Rightarrow$ planificare S valida si timpul corespunzator S este

$$\max_i t_i = D$$
 $\Rightarrow T_\infty \leq D$

Proprietati T_p

- pt. nr arbitrar de procesoare p

$$T_1 \geq T_p \geq T_\infty$$

- in general, T_p e greu de calculat
 - pp. un DAG particular si o anumita valoare a lui p
 - problema complicata de combinatorica
- totusi, acest aspect nu constituie o problema, exista limite usor de folosit

Complexitate in timp DAG – Limite

Proposition 2.1. Suppose that for some output node i , there exists a positive path from every input node to i . Furthermore, suppose that the in-degree of each node is at most 2. Then,

$$T_{\infty} \geq \log n,$$

where n is the number of input nodes.

- **oricate procesoare avem la dispozitie, nu putem scadea sub complexitatea logaritmica!**
- analogie sorting networks: sortarea paralela are lower bound $O(\log(n))$ (Ajtai, Komlos, Szemeredi 1983) vs. sortare secventiala $O(n \log(n))$

Demonstratie

- inductie dupa nr de noduri input k pt care exista o cale catre un nod j (pt un nod input pp. ca exista o cale catre el insusi)
- demonstrem ca $t_j \geq \log k$ pt. orice nod j pt. care exista o cale de la k noduri de input si pt orice schedule (planificare)
- $k = 1$, $t_j \geq 0$ adevarat pt. orice nod j
- pas inductiv:
 - pp ipoteza inductiva adevarata pt orice $k \leq m$ si consideram un nod j catre care exista cai dinspre $m+1$ noduri input
 - pt ca j nu poate avea mai mult de 2 predecesori, unul dintre ei, i , are cai de la $\text{ceiling}((m+1)/2)$ noduri input, deci

$$t_j \geq t_i + 1 \geq \log(\text{ceiling}((m+1)/2)) + 1 \geq \log(m+1)$$

Complexitate in timp - Limite

- **Q:** ce se intampla daca nr de procesoare e redus cu un anumit factor?
A: timpul de executie creste cu cel mult acel factor
- **Prop 2.2** Pt $c \in \mathbb{N}$ ($c \neq 0$) si $q = cp \Rightarrow T_p \leq cT_q$
- demonstratie:
 - fie planificarea S care necesita T_q timp pentru q procesoare
 - la fiecare pas (faza) se executa cel mult q operatii
 - pt p procesoare, faza se poate executa in max $q/p = c$ unitati de timp $\Rightarrow$ planificare cu p procesoare care se executa in cel mult cT_q

Complexitate in timp - Limite

- **Prop 2.3** Pt. orice nr de procesoare p , $T_p < T_\infty + T_1 / p$
 - demonstratie:
 - fie S schedule pt T_∞
 - fie n_t numarul de noduri i pt. care $t_i = t \in \mathbb{N}$ (i.e., nr de operatii terminate la momentul t)
 - fie un schedule S_p pentru p procesoare care in faza t executa operatiile planificate la timpul t in S
 - faza t din S_p dureaza $\text{ceiling}(n_t / p)$ unitati de timp
- $\Rightarrow T_p$ este mai mic decat suma acestor unitati de timp pentru fiecare din pasii planificarii S_p

$$T_p \leq \sum_{t=1}^{T_\infty} \text{ceiling}\left(\frac{n_t}{p}\right)$$

- dar,
 - $\text{ceiling}(n_t / p) < n_t / p + 1$ si
 - $T_1 = \text{sum}(n_t) = \text{suma tuturor operatiilor din nodurile interne si de output din G}$

$$\Rightarrow T_p \leq \sum_{t=1}^{T_\infty} \text{ceiling}\left(\frac{n_t}{p}\right) \leq \sum_{t=1}^{T_\infty} \left(\frac{n_t}{p} + 1\right) = \frac{T_1}{p} + T_\infty$$

Corolar important

Proposition 2.4. (a) If $p \geq T_1/T_\infty$, then $T_p < 2T_\infty$. More generally, if $p = \Omega(T_1/T_\infty)$, then $T_p = O(T_\infty)$.
 (b) If $p \leq T_1/T_\infty$, then

$$\frac{T_1}{p} \leq T_p < 2\frac{T_1}{p}.$$

More generally, if $p = O(T_1/T_\infty)$, then $T_p = \Theta(T_1/p)$.

- demonstratie (a)

$$p \geq T_1 / T_\infty \Rightarrow T_1 / p \leq T_\infty$$

$$\text{Cf. 2.3} \Rightarrow T_p < T_\infty + T_1 / p$$

$$\Rightarrow T_p < T_\infty + T_\infty = 2 T_\infty$$

Corolar important

Proposition 2.4. (a) If $p \geq T_1/T_\infty$, then $T_p < 2T_\infty$. More generally, if $p = \Omega(T_1/T_\infty)$, then $T_p = O(T_\infty)$.
 (b) If $p \leq T_1/T_\infty$, then

$$\frac{T_1}{p} \leq T_p < 2\frac{T_1}{p}.$$

More generally, if $p = O(T_1/T_\infty)$, then $T_p = \Theta(T_1/p)$.

- demonstratie (b)

$$p \leq T_1 / T_\infty \Rightarrow T_\infty \leq T_1 / p$$

$$\text{Cf. 2.3} \Rightarrow T_p < T_\infty + T_1 / p$$

$$\Rightarrow T_p < 2 T_1 / p$$

In plus, cf .2.2 (pt $q/p = c$ avem $T_p \leq cT_q$) si $c = p \Rightarrow T_1 \leq p T_p$

$$\Rightarrow T_1 / p \leq T_p$$

Limitari fundamentale si concluzii

- 2.4 (a) $\Rightarrow$ desi definia T_∞ pp. un numar infinit de procesoare, $p = \Omega(T_1/T_\infty)$ procesoare sunt suficiente pentru atinge T_∞ in limita unui factor constant

$$T_\infty < T_p < 2 T_\infty$$

- o planificare corespunzatoare se poate face prin simpla modificare a planificarii optimale pt. un numar nelimitat de procesoare, fara a rezolva o problema complicata de planificare
 - metodologie:
 - se dezvolta un algoritm paralel ca si cand sunt disponibile o infinitate de procesoare
 - apoi se adapteaza algoritmul pentru numarul de procesoare existente
- 2.4 (b) $\Rightarrow$ daca $p = O(T_1/T_\infty)$ procesoare sunt disponibile atunci se poate reduce timpul de rulare cu un factor proportional cu p , aproape de optimalitate in limita unui factor constant

$$T_1 / p \leq T_p \leq 2 T_1 / p$$

- **Concluzie:** cu un nr de procesoare p aproape egal cu T_1/T_∞ se pot obtine atat timp de executie optimal cat si accelerare (speed-up) optimala

Alegerea unui DAG optimal

- aceeași problema computațională poate fi calculată diferit (DAG-uri diferite)
- cautăm DAG-ul care minimizează T_p
- fie T_p^* valoarea lui T_p pentru DAG-ul optimal $\Rightarrow T_p^*$ e o măsură a complexității problemei, în timp ce T_p e complexitatea unui algoritm particular
- în general, evaluarea T_p^* e foarte dificilă

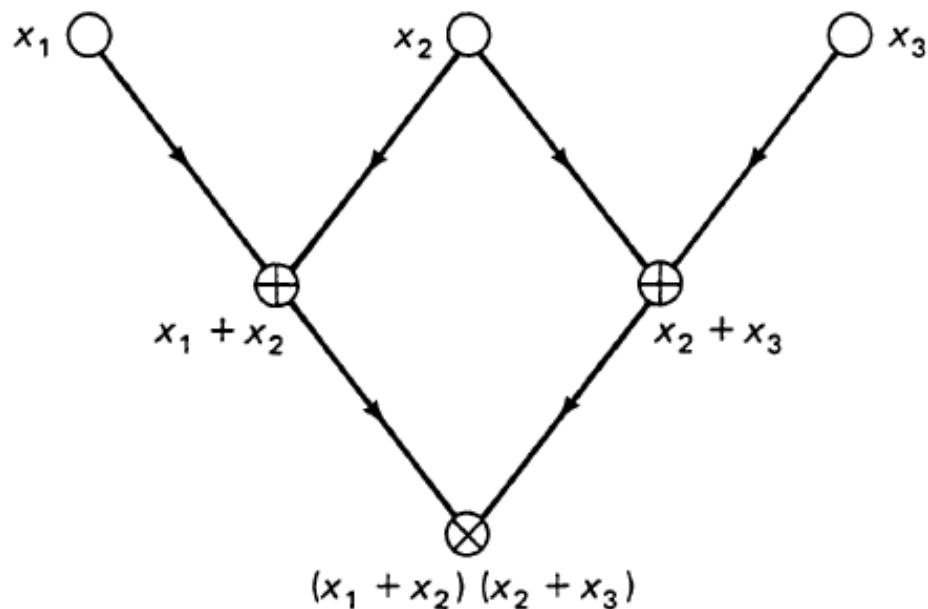


Figure 1.2.2 Another DAG representing an algorithm for evaluating the arithmetic expression $(x_1 + x_2)(x_2 + x_3)$. We have $T_1 = 3$ and $T_\infty = D = 2$. This should be contrasted with the DAG of Fig. 1.2.1 which solves the same computational problem and for which $T_1 = 7$ and $D = 3$. We conclude that the DAG given here represents a better parallel algorithm.

Comparatie calcul paralel vs. secvential

- pp alegerea unui model de calcul (ex: DAG)
- consideram problema de calcul parametrizata de n , dimensiunea problemei (nr de noduri de intrare in cazul DAG)
 - complexitatea in timp este in general dependenta de n
- pp. ca avem un algoritm paralel care foloseste p procesoare
 - p poate depinde de n
- $T_p(n)$ = timpul de rulare al algoritmului paralel
- $T^*(n)$ = timpul serial optimal (difcil de calculat, se folosesc estimari)

Acceleratia si eficienta

- **acceleratia (speed-up):**

$$S_p(n) = \frac{T^*(n)}{T_p(n)}$$

- descrie avantajul de viteza de executie al algoritmului paralel fata de cel mai bun algoritm secvential
- valoarea ideala

$$S_p(n) = p$$

- **eficienta:**

$$E_p(n) = \frac{S_p(n)}{p} = \frac{T^*(n)}{pT_p(n)}$$

- fractiunea de timp utilizata efectiv de processor
- valoarea ideala

$$E_p(n) = 1$$

- cazul ideal: disponibilitatea a p procesoare permite accelerarea calculului de p ori (niciun procesor nu e idle vreodata si nici nu executa operatii nefolositoare), *practic imposibil* !

Definitii alternative $T^*(n)$

- (1) timpul de executie al celui mai bun algoritm secvential
- (2) timpul de executie al unui algoritm secvential de benchmark
 - ex: implementare inmultire de matrici $O(n^3)$, benchmark rezonabil desi exista algoritmi cu timpi mai buni
- (3) timpul necesar unui singur procesor pentru a executa algoritmul paralel particular analizat (echivalent, un procesor simuleaza operatia paralela a p procesoare)
 - avantaj: exprima cat de bine a fost paralelizat un algoritm particular
 - dezavantaj: nu ajuta la intelegerea virtutilor algoritmului in sens absolut

Alegere $T^*(n)$ cf. celei de-a treia definitii

- pt DAG, $T^*(n) = T_1(n)$
- daca $p \leq O\left(\frac{T_1(n)}{T_\infty(n)}\right)$, cf. 2.4 (b) avem

$$T_p(n) = \Theta\left(\frac{T_1(n)}{p}\right)$$

$$E_p(n) = \frac{T_1(n)}{pT_p(n)} = \Theta(1)$$

- daca $p = \Theta\left(\frac{T_1(n)}{T_\infty(n)}\right)$, cf 2.4(a) avem

$$T_p(n) = \Theta(T_\infty(n))$$

- **Concluzie:** pentru un numar de procesoare relativ redus se obtin implementari *eficiente* (cel putin pentru modelul in care ignoram complexitatea comunicatiei)

Legea lui Amdahl

- dacă eficiența nu reprezintă o problemă majoră (cel puțin pt. DAG-uri), cf concluziei anterioare, apare întrebarea:
 - poate fi accelerat un algoritm paralel oricât de mult, odată cu creșterea lui p ?
- fie f fracțiunea de timp consumată de secțiunile *ne-paralelizabile* (*inherent sequential*) ale unui algoritm paralel, atunci

$$S_p(n) \leq \frac{1}{f + \frac{(1-f)}{p}} < \frac{1}{f}, \quad \forall p$$

Q: ce se întâmplă dacă n crește și el?

Interpretare legea lui Amdahl

- daca redefinim $s = f$, $p = 1 - f$
si $N = nr$ de procesoare
(anterior p) => definitie
Amdahl:

$$S = 1 / (s + p / N)$$

unde $s + p = 1$

- presupunere implicita: p
independent de N (i.e.,
dimensiunea problemei e fixa)
- presupunere nerealista in
practica unde dimensiunea
problemei scaleaza cu
numarul de procesoare

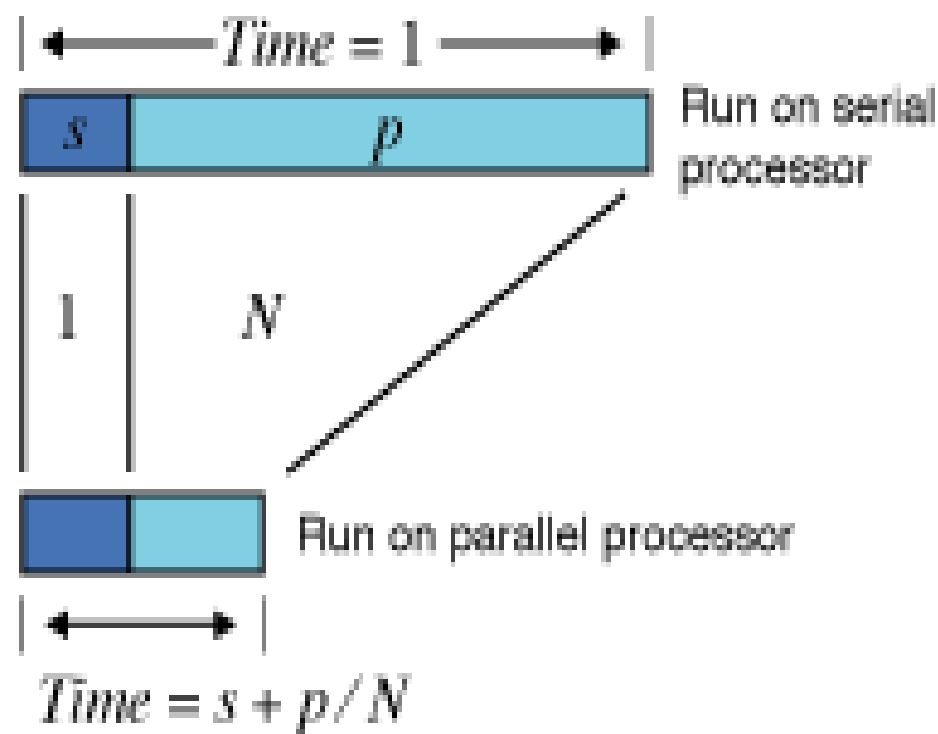


FIGURE 2a. Fixed-Size Model: $Speedup = 1 / (s + p / N)$

Legea lui Gustafson

- obs: programatorii tind sa mareasca dimensiunea problemei pe masura ce au la dispozitie un nr mai mare de procesoare
- abordare noua: pp ca timpul de rulare e constant, nu dimensiunea problemei
- acum, se considera ca p variaza liniar cu N
- obs. practica: partea seriala s (initializare vectori, incarcare program, bottleneck-uri seriale, I/O) nu creste odata cu dimensiunea problemei

Legea lui Gustafson (cont.)

- definitie noua pt speedup

$$\text{Scaled speedup} = s + pN$$

$$= s + (1 - s)N$$

$$= N + (1 - N)s$$

- daca s e constant, dependenta de N e liniara cu panta $1 - s$
- schimbare de paradigma: scaled speedup-ul este de fapt *slowdown-ul* (incetinirea) teoretic pe care il inregistreaza un program paralel atunci cand ar rula ipotetic pe o masina cu un singur procesor

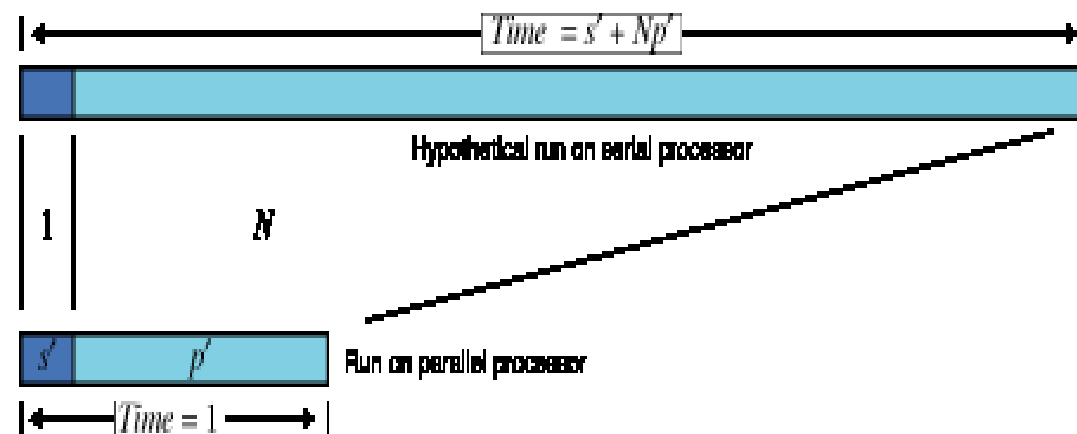


FIGURE 2b. Scaled-Speed Model: $\text{Speedup} = s + N/p$

Legea lui Gustafson, concluzii

- accentul folosirii unui nr crescut de procesoare cade pe necesitatea de a rula programe mai mari in acelasi timp daca se poate, nu pe scaderea timpului de executie al unei probleme de dimensiune fixa
- obs: nu orice problema poate fi marita arbitrar, ca atare exista si limitari ale legii lui Gustafson

Exemplu: adunare n scalari

- complexitate seriala: $T^*(n) = n - 1$
- complexitate paralela: ?

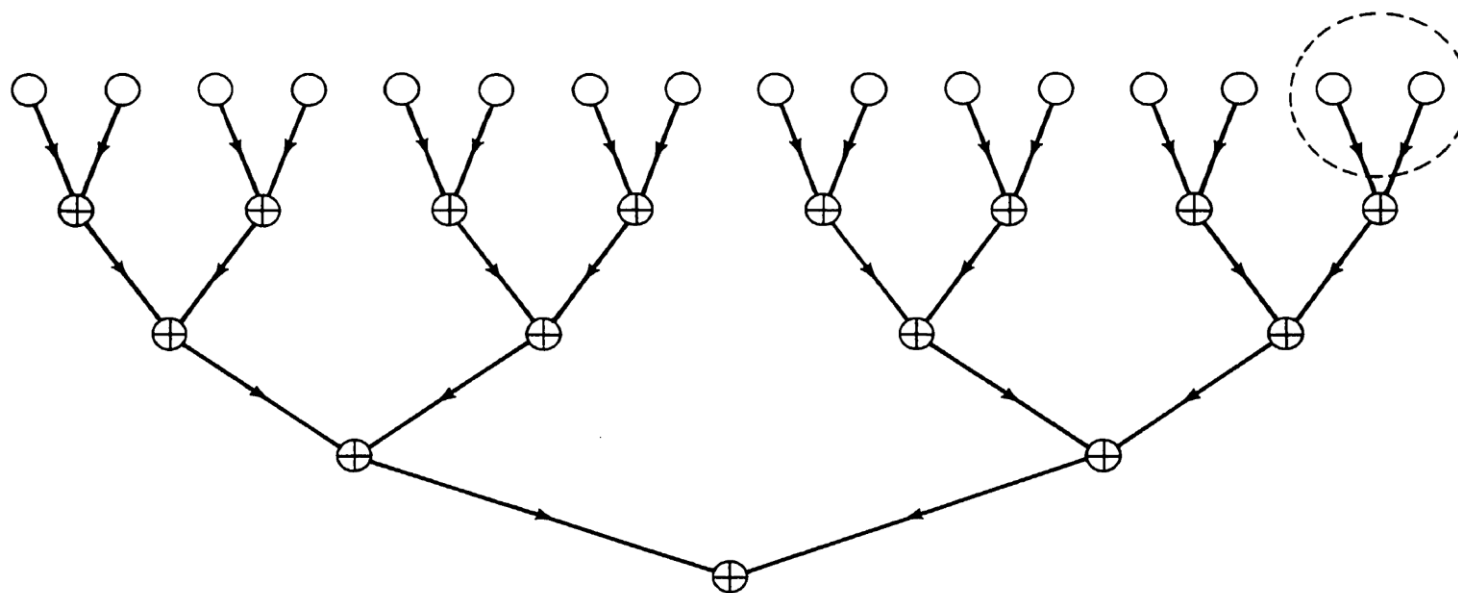


Figure 1.2.3 Parallel computation of the sum of 16 scalars. Eight processors are needed for the parallel additions at the first stage and a total of $4 = \log 16$ stages are needed. If the portion of the diagram enclosed in the dashed circle is removed, we obtain an algorithm for the parallel addition of 15 scalars. Notice that now only $7 = \lfloor 15/2 \rfloor$ processors are needed.

Adunare n scalari

- pp. n putere a lui 2
- $n/2$ perechi (1 per procesor)
- $n/2$ procesoare
- $\log(n)$ etape
- $T_{\frac{n}{2}}(n) = \log(n)$
- $E_{\frac{n}{2}}(n) = \frac{n-1}{\frac{n}{2} \log(n)} \approx \frac{2}{\log(n)}$

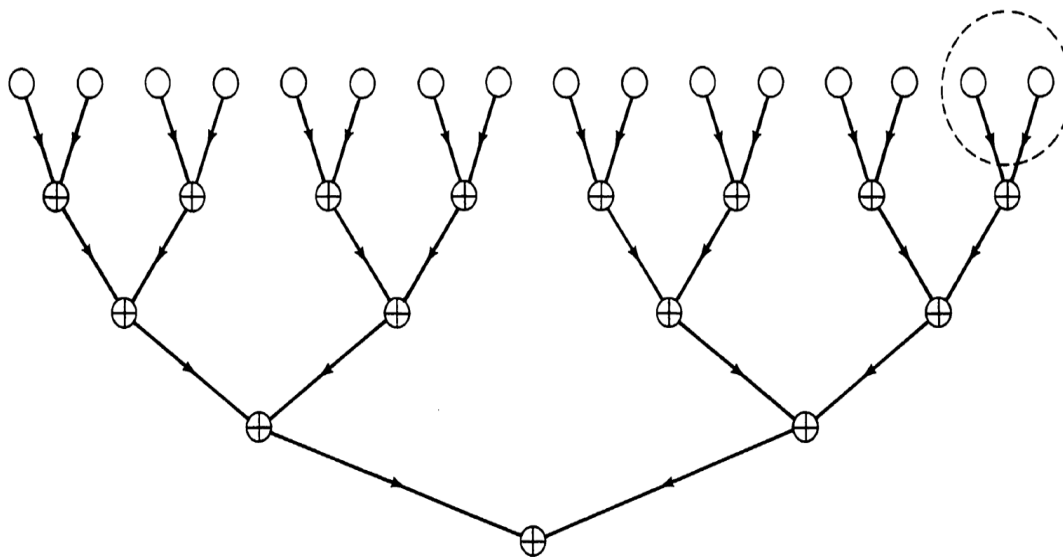


Figure 1.2.3 Parallel computation of the sum of 16 scalars. Eight processors are needed for the parallel additions at the first stage and a total of $4 = \log 16$ stages are needed. If the portion of the diagram enclosed in the dashed circle is removed, we obtain an algorithm for the parallel addition of 15 scalars. Notice that now only $7 = \lfloor 15/2 \rfloor$ processors are needed.

Adunare n scalari (alternativa)

- pp $\log(n)$ intreg, iar $\frac{n}{\log(n)}$ intreg si putere a lui 2
- impartim cele n numere in $\frac{n}{\log(n)}$ grupuri de cate $\log(n)$ numere
- folosim $\frac{n}{\log(n)}$ procesoare, procesorul i aduna numerele din grupul i
- timp etapa initiala: $\log(n) - 1$
- raman $\frac{n}{\log(n)}$ sume parțiale de adunat, folosim algoritmul anterior cu:

$$\frac{n}{2\log(n)} \text{ procesoare}$$

$\Rightarrow$ timp de rulare: $\log(n / \log n) \leq \log(n)$

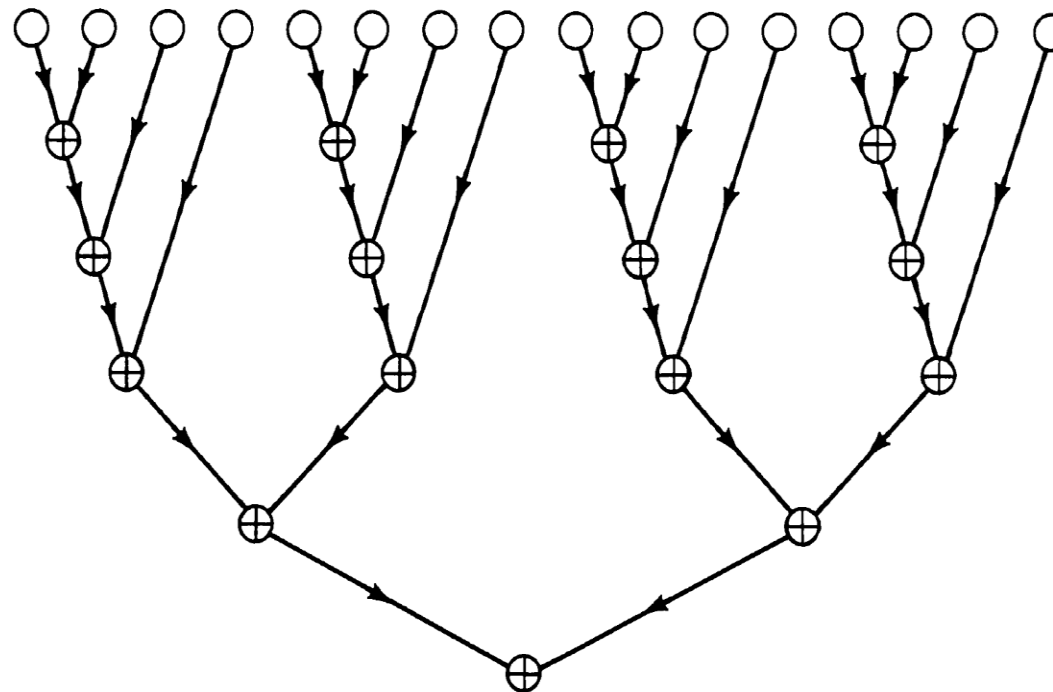


Figure 1.2.4 An alternative algorithm for the parallel addition of 16 scalars. Only four processors are used and the time requirements increase to 5 stages. Overall, however, there is an efficiency improvement over the algorithm of Fig. 1.2.3.

Concluzie algoritm alternativ

- $T_{\frac{n}{\log(n)}}(n) = 2\log(n)$
 - suma celor doua etape
 - dublu fata de primul algoritm
- $E_{\frac{n}{\log(n)}}(n) = \frac{1}{2}$, semnificativ mai bine decat $\frac{2}{\log(n)}$ (pt. $\frac{n}{2}$ procesoare)
- obs: $p = \frac{n}{\log(n)} \approx \frac{T_1(n)}{T_\infty(n)}$ procesoare
- alegerea unui nr de procesoare aproape egal cu $\frac{T_1(n)}{T_\infty(n)}$ imbunatateste eficienta, reconfirmare 2.4 (b)
- concluzie: imbunatatire substantiala a eficientei pt un sacrificiu mic de viteza

Exemplu: produs scalar

- x, y vectori n -dimensionali
- produs scalar

$$\sum_{i=1}^n x_i y_i$$

- daca se folosesc n procesoare:
 - pas 1: procesorul i calculeaza $x_i y_i$
 - apoi se foloseste algoritmul de adunare a n valori scalare

$$\Rightarrow T_n(n) = \text{ceiling}(\log(n)) + 1$$

Adunare si inmultire de matrici

- n matrici de dimensiune $m \times m$
- **adunarea** se poate face in $\lceil \log(n) \rceil$ pasi cu $m^2 \lfloor n/2 \rfloor$ procesoare
 - fiecare grup de $\lfloor n/2 \rfloor$ procesoare calculeaza o alta intrare a matricii suma
- **inmultirea** a doua matrici de dimensiuni $m \times n$ si respectiv $n \times l$
 - ml produse scalare de vectori de dimensiune n
 - se poate face in $\lceil \log(n) \rceil + 1$ pasi cu nml procesoare
 - daca $n=m=l \Rightarrow n^3$ procesoare si $T_1(n) = \Theta(n^3)$

$\Rightarrow$

$$S_n^3(n) = \frac{n^3}{\log n}$$

Puterile unei matrici

- fie A matrice $n \times n$
 - calculam A^k , unde $k \in \mathbb{N}$
 - pt. $k = 2^i$, $i \in \mathbb{N}$:
 - $A^k = A^2 \times A^2 \dots$
 - calculul dureaza $\log k = i$ pasi, la fiecare pas avand loc o inmultire de matrici $n \times n$
- $\Rightarrow$

A^k se poate calcula in timp $\log k (\text{ceiling}(\log n) + 1)$ cu n^3 procesoare

- pt $k \neq 2^i$, A^k se poate calcula in timp $\Theta(\log k \log n)$
 - *indicatie*: folositi reprezentarea binara a numarului k

$\Rightarrow A^2, \dots, A^n$ se pot calcula in $\Theta(\log^2 n)$ cu n^4 procesoare

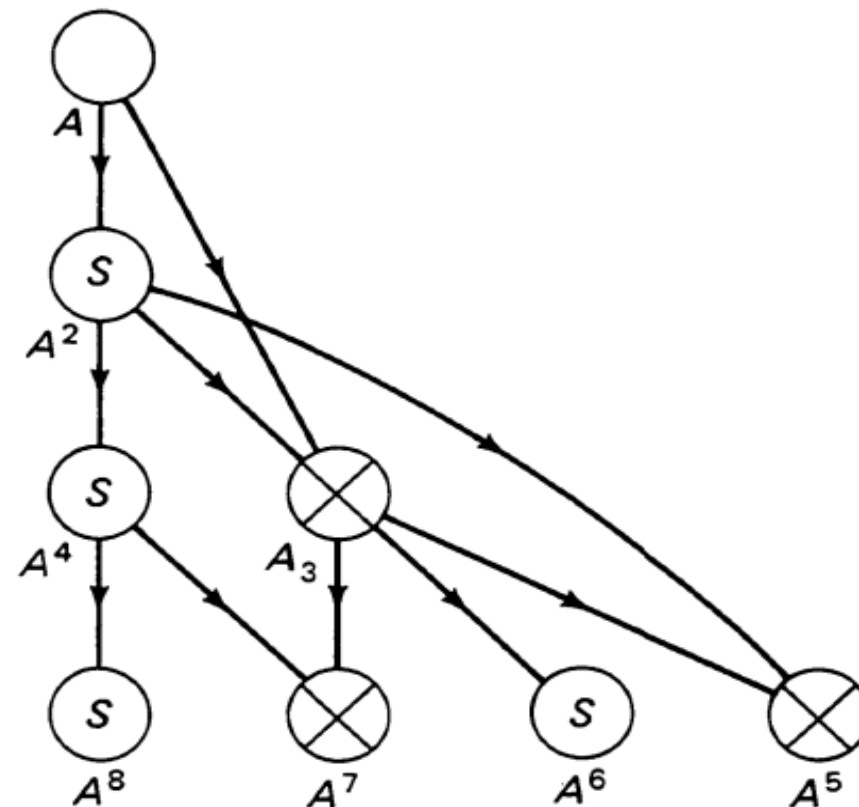
- pt fiecare A^k se foloseste un grup diferit de n^3 procesoare

Algoritm alternativ

- evita duplicarea inutila a efortului de calcul
- un nod S reprezinta ridicarea la patrat a unei matrici
- in pasul k se calculeaza puterile $2^{k-1} + 1, \dots, 2^k$ ale matricii A

$\Rightarrow$ sunt suficiente $\Theta(\log n)$ etape pt calculul $A^2, \dots, A^n$

- fiecare etapa implica cel mult $\Theta(n)$ inmultiri simultane de matrici
 - etapa se poate calcula in $\Theta(\log n)$ cu n^4 procesoare
- $\Rightarrow$ timp total de calcul $\Theta(\log^2 n)$



Analiza eficientei

- algoritmi anteriori vizeaza optimizarea timpului de rulare

=> uzual, cerere excesiva de putere de calcul (nr de procesoare) si eficienta mica

- cf. teoremei anterioare, aceiasi algoritmi se pot eficientiza daca se alege nr de procesoare $p = O(\frac{T_1(n)}{T_\infty(n)})$
- ex: inmultirea a doua matrici $n \times n$ se poate face in $\Theta(\log n)$ folosind $\Theta(n^3/\log n)$ procesoare

$$\Rightarrow E = \Theta(1)$$

Reducand si mai mult $p \Rightarrow$ timp de executie $\Theta(\frac{T_1(n)}{p})$, cf 2.4(b)

=> doua matrici $n \times n$ se pot inmulti in $\Theta(n)$ cu n^2 procesoare si in $\Theta(n^2)$ cu n procesoare

Algoritmi paraleli

Curs 4

Vlad Olaru

vlad.olaru@fmi.unibuc.ro

Calculatoare si Tehnologia Informatiei

Universitatea din Bucuresti

Metode iterative

$$f: R^n \rightarrow R^n$$
$$x(t+1) = f(x(t)) \quad t = 0, 1, \dots$$

- structura generala pt. sisteme de ecuatii, probleme de optimizare, etc
- se numesc *algoritmi iterativi* sau *metode de relaxare*
- uneori se foloseste notatia $x = f(x)$
- daca $\{x(t)\}$ converge la x^* si f continua, x^* se numeste *punct fix al lui f*
$$x^* = f(x^*)$$
- caz special, *algoritm iterativ liniar*:

$$f(x) = A x + b, \text{ unde } A \text{ e matrice patrata si } b \text{ vector}$$

Paralelizarea metodelor iterative

$$f: R^n \rightarrow R^n$$
$$x_i(t+1) = f_i(x(t)) \quad t = 0, 1, \dots$$

unde $x_i(t)$ si f_i sunt componentele i ale vectorilor x si f

- Q: este posibila paralelizarea acestui tipar de algoritmi?
- A: date fiind n procesoare, fiecare calculeaza cate un $x_i(t+1)$ si comunica rezultatul celorlalte procesoare pt iteratia urmatoare
- obs: in general, complexitatea in timp a acestor algoritmi nu e relevanta in absenta unui criteriu de terminare (valabil si pt. cazul sequential)

Metode (algoritmi) Jacobi

$$x_i(t + 1) = f_i(x_1(t), \dots, x_n(t)) \quad i = 1, \dots, n$$

- toate componentele x_i ale lui $x(t)$ sunt actualizate *simultan*
 - din nou, vom folosi o reprezentare de graf orientat aciclic (DAG), numit in acest caz *graf de dependente* (“*dependency graph*”):
 - set de noduri $N = \{1, \dots, n\}$ corespunzator componentelor lui x
 - arcul (i, j) exprima o dependenta $\Leftrightarrow f_j$ depinde de x_i
- i.e., procesorul i trebuie sa comunice procesorului j valorile lui $x_i(t)$

Exemplu graf de dependente

$$x_1(t+1) = f_1(x_1(t), x_3(t))$$

$$x_2(t+1) = f_2(x_1(t), x_2(t))$$

$$x_3(t+1) = f_3(x_2(t), x_3(t), x_4(t))$$

$$x_4(t+1) = f_4(x_2(t), x_4(t))$$

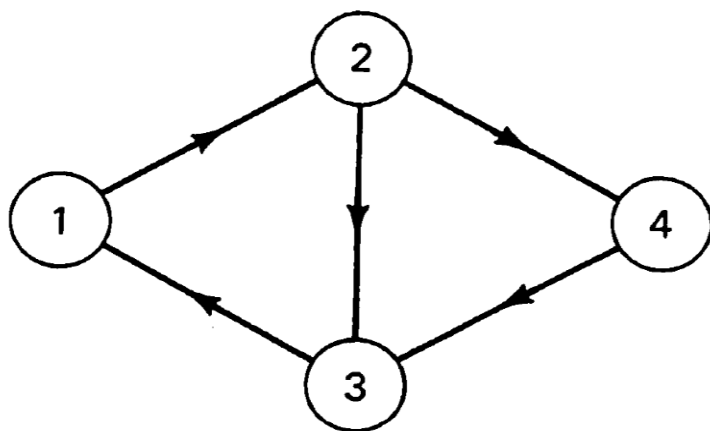


Figure 1.2.6 The dependency graph associated with an iteration of the form

$$x_1(t+1) = f_1(x_1(t), x_3(t))$$

$$x_2(t+1) = f_2(x_1(t), x_2(t))$$

$$x_3(t+1) = f_3(x_2(t), x_3(t), x_4(t))$$

$$x_4(t+1) = f_4(x_2(t), x_4(t)).$$

Desfasurarea grafului de dependente in timp

- pp. iteratia se executa pt. $t = 0, 1, \dots, T$ unde $T \in \mathbb{N}$

$$x_1(t+1) = f_1(x_1(t), x_3(t))$$

$$x_2(t+1) = f_2(x_1(t), x_2(t))$$

$$x_3(t+1) = f_3(x_2(t), x_3(t), x_4(t))$$

$$x_4(t+1) = f_4(x_2(t), x_4(t))$$

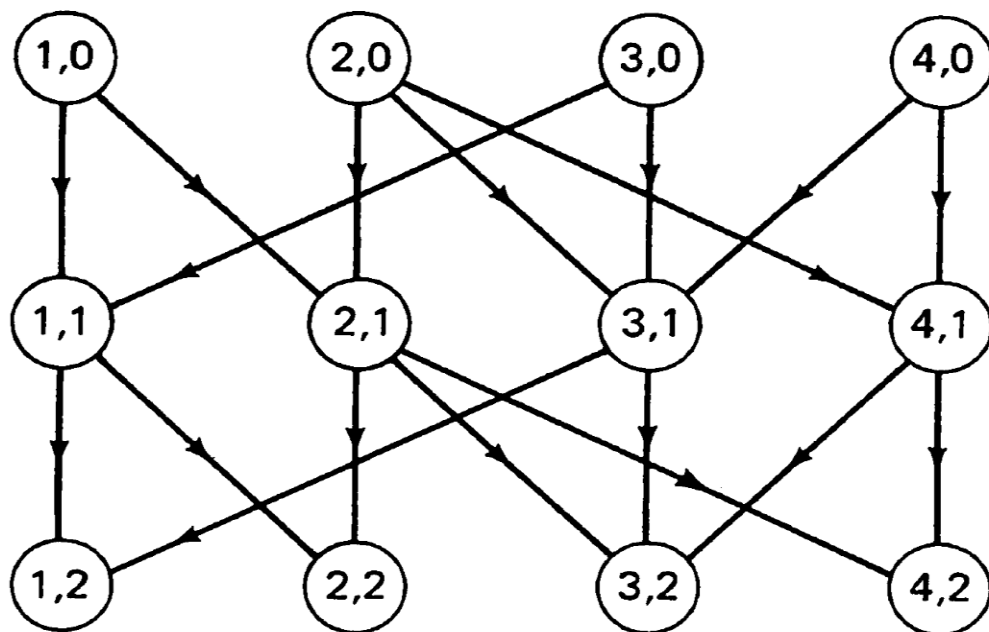


Figure 1.2.7 The DAG corresponding to two iterations when the function f has the dependency graph shown in Fig. 1.2.6. The nodes of the DAG are of the form (i, t) , where $i \in \{1, \dots, n\}$, and t is the iteration count. The arcs are of the form $((i, t), (j, t+1))$, where (i, j) is an arc of the dependency graph or $i = j$.

Metode iterative paralelizate cu componente bloc

- formulare de paralelism coarse-grain a iteratiei $x = f(x)$

$$x_j(t+1) = f_j(x(t)) \quad j = 1, \dots, p$$

unde $x = (x_1, \dots, x_j, \dots, x_p)$ si x_j vector de dimensiune n_j (*componente bloc ale lui x*) a.i.

$$\sum_{j=1}^p n_j = n$$

$$f_j: R^n \rightarrow R^{n_j}$$

$$R^n = R^{n_1} \times \dots \times R^{n_p}$$

(Obs: dependentele raman, nu se modifica decat granularitatea)

- fiecare componenta bloc x_j se asigneaza unuia dintre cele p procesoare care devine responsabil pt actualizarea componentei

=> algoritm *block-parallelized* (*bloc paralel*)

Metode iterative paralelizate cu componente bloc

- se redefineste graful de dependente $G=(N,A)$ cu
 - $N = \{1, \dots, p\}$
 - $A = \{(i,j) \mid f_j \text{ depinde de } x_i\}$
- motivatii algoritm bloc-paralel:
 - nr de procesoare redus $\Rightarrow$ plasarea mai multor x_i pe acelasi procesor
 - functiile scalare f_i pot avea operatii comune (optimizare)
 - reducerea comunicatiilor inter-procesoare (problema granularitatii)

Utilitate metode iterative

- paralelizarea unei iteratii prin asignarea actualizarilor unor procesoare diferite are sens cand actualizarea fiecărei componente x_i presupune un calcul diferit
- exemplu contrar

$$f_i(x_1(t), \dots, x_n(t)) = x_i + \sqrt{\sum_{j=1}^n x_j^2}$$

- calculul sumei de către fiecare procesor este nejustificat
 - poate fi făcut de un singur procesor care să comunice rezultatul celorlalte procesoare
 - alternativ, poate fi evaluat în paralel, colaborativ, folosind calcul prefix
- în general, calculul f_i implică puțină duplicare (sau deloc) a calculului

Metode (algoritmi) Gauss-Seidel

$$x_i(t+1) = f_i(x_1(t+1), \dots, x_{i-1}(t+1), x_i(t), \dots, x_n(t))$$

- componentele x_i se actualizeaza pe rand, folosindu-se la fiecare iteratie cea mai noua valoare a celorlalte componente

=> *algoritm Gauss-Seidel bazat pe functia f*

- pt. ca incorporeaza cea mai noua informatie, pot converge mai repede decat metodele Jacobi !
- vom discuta posibilitatea paralelizarii unei singure iteratii Gauss-Seidel, asa numita operatie de *sweep*
- la limita, o asemenea iteratie poate fi complet ne-paralelizabila
 - ex: daca toate functiile f_i depind de toate componentele x_j , doar o componenta se poate actualiza la un moment dat
- daca insa graful de dependente e rar (*sparse*), exista posibilitatea executiei in paralel a mai multor actualizari

Exemplu iteratie Gauss-Seidel cu graf de dependenta sparse

$$x_i(t+1) = f_i(x_1(t+1), \dots, x_{i-1}(t+1), x_i(t), \dots, x_n(t))$$

- componentele x_3 si x_4 pot fi actualizate in paralel !
- sunt 4 actualizari, dar adancimea grafului este $D = 3$!
- ordinea de actualizare influenteaza paralelismul

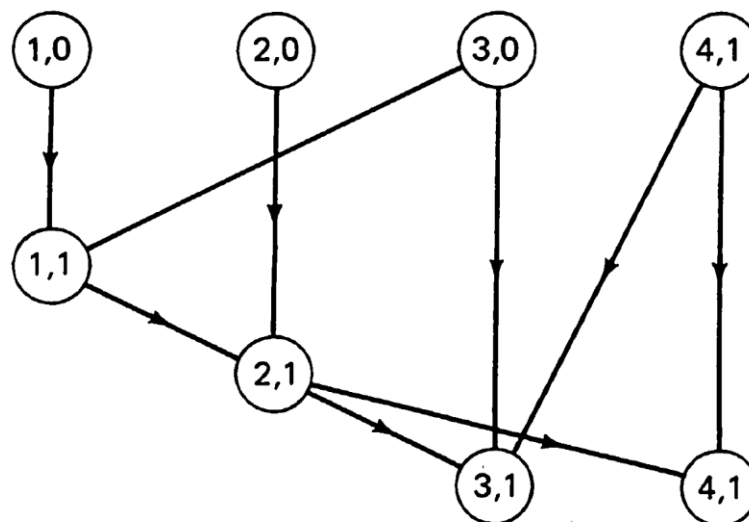


Figure 1.2.8 Illustration of the parallelization of Gauss-Seidel iterations. Let f be a function whose dependency graph is as in Fig. 1.2.6. The Gauss-Seidel algorithm based on f takes the form

$$x_1(t+1) = f_1(x_1(t), x_3(t))$$

$$x_2(t+1) = f_2(x_1(t+1), x_2(t))$$

$$x_3(t+1) = f_3(x_2(t+1), x_3(t), x_4(t))$$

$$x_4(t+1) = f_4(x_2(t+1), x_4(t)).$$

Ordinea actualizarilor in algoritmi Gauss-Seidel

- pt aceeași funcție f există mai mulți algoritmi GS datorită libertății de a schimba ordinea actualizărilor
 - ex: executăm actualizările de la coadă la capăt (de la x_n la x_1)
- ordini diferite ale actualizărilor rezultă în general în algoritmi GS diferiți, cu rezultate diferite
- există și aplicații în care un algoritm GS converge în limita unui număr mare de iterații la aceeași valoare, indiferent de ordinea actualizărilor
 - dacă viteza de convergență aferentă diferitelor ordini de actualizare nu diferă mult
=> vom alege ordinea de actualizare care permite maximum de paralelism !

Cresterea paralelismului prin schimbarea ordinii actualizarilor GS

$$x_i(t+1) = f_i(x_1(t+1), \dots, x_{i-1}(t+1), x_i(t), \dots, x_n(t))$$

- daca schimbam ordinea de actualizare si consideram:

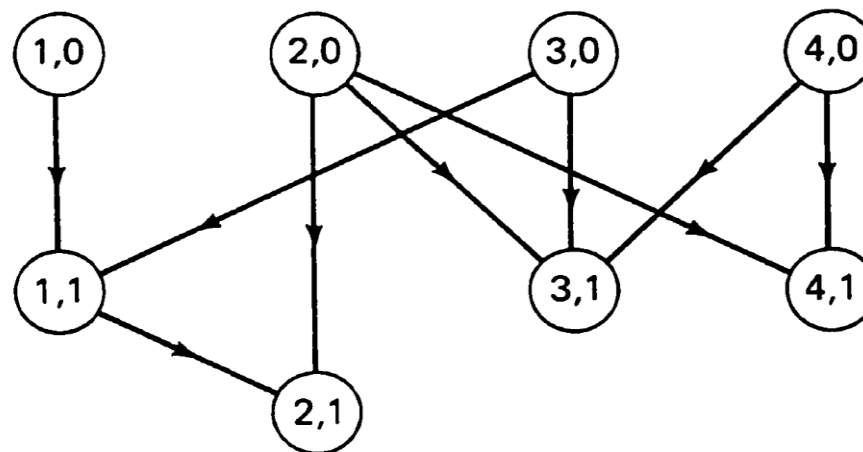
$$x_1(t+1) = f_1(x_1(t), x_3(t))$$

$$x_3(t+1) = f_3(x_2(t), x_3(t), x_4(t))$$

$$x_4(t+1) = f_4(x_2(t), x_4(t))$$

$$x_2(t+1) = f_2(x_1(t+1), x_2(t))$$

- adancimea grafului D devine 2
- se observa ca un *sweep* se poate face in paralel in doi pasi cu doua procesoare



Determinarea ordinii actualizarilor pt. paralelizarea optima a operatiei de sweep

- graf de dependenta $G = (N, A)$
- colorarea G cu K culori (fiecare nod din G are asociata o culoare) din multimea $\{1, \dots, K\}$

$$h(i) = k \quad \forall i \in N$$

- idee: variabilele colorate cu aceeasi culoare se pot actualiza in paralel
- in aceste conditii, maximizarea paralelismului revine la gasirea unei colorari optime a grafului G
- **Propozitie**
 - (i) exista o ordine de actualizare a variabilelor intr-o iteratie Gauss-Seidel care se poate efectua in K pasi paraleli
 - $\Leftrightarrow$
 - (ii) exista o colorare a grafului de dependenta G cu K culori a.i. nu exista cicluri cu toate nodurile de aceeasi culoare

Demonstratie $(i) \Rightarrow (ii)$

- pp. o ordine de actualizare a variabilelor unei iteratii care necesita K pasi paraleli
 - fie culoarea nodului i , $h(i) = k$, daca x_i se actualizeaza in pasul paralel k
 - fie un ciclu pozitiv $i_1, i_2, \dots, i_m$, cu $i_1 = i_m$
 - fie i_l un nod in ciclu ($1 \leq i_l < m$) care apare primul in ordinea actualizarilor
 - i_{l+1} se actualizeaza dupa i_l si $(i_l, i_{l+1}) \in A \Rightarrow x_{i_{l+1}}(t+1)$ *depinde de* $x_{i_l}(t+1)$
- $\Rightarrow$ cele doua noduri nu se pot actualiza simultan $\Rightarrow h(i_l) \neq h(i_{l+1})$
- $\Rightarrow$ in orice ciclu pozitiv exista doua noduri colorate diferit, Qed

Lema

- $G = (N, A)$ DAG, daca $(i,j) \in A$ exista o ordine de actualizare a nodurilor a.i. j sa apara inainte de i
- demonstratie
 - $\forall i$ nod, fie $d_i = \max$ nr de arce dintr-o cale pozitiva care porneste de la i
 - G aciclic $\Rightarrow d_i$ finit
 - ordonam nodurile in ordinea crescatoare a d_i
 - $(i,j) \in A \Rightarrow d_i > d_j \Rightarrow j$ se actualizeaza inaintea lui i

Demonstratie $(ii) \Rightarrow (i)$

- pp. o colorare h a lui G cu K culori si niciun ciclu pozitiv cu toate nodurile de aceeasi culoare
 - pt. fiecare culoare k , def G_k , subgraf al lui G cu noduri de culoare k si arcele dintre ele
 - G_k aciclic prin ipoteza $\Rightarrow$ cf lemei anterioare nodurile din G_k se pot ordona a.i. j se actualizeaza inainte de i daca $(i,j) \in A$
 - se ordoneaza G in ordinea crescatoare a culorilor
 - fie o iteratie GS cf acestei ordonari si doua noduri i si j , $i \neq j$, a.i. $h(i) = h(j) = k$
 - cazuri posibile
 - (i,j) si (j,i) nu sunt arce $\Rightarrow x_i$ si x_j se pot actualiza in paralel
 - $(i,j) \in A$ si $(j,i) \in A$ imposibil pt. ca G_k aciclic
 - $(i,j) \in A$ si (j,i) nu e arc $\Rightarrow j$ apare inainte de i in ordinea actualizarilor $\Rightarrow x_j(t+1)$ necesita doar valoarea lui $x_i(t)$ nu si $x_i(t+1)$; similar pt cazul simetric
- $\Rightarrow \forall x_i$ cu aceeasi culoare se pot actualiza in paralel

Exemplu

- doua culori sunt suficiente, eg $h(1)=h(3)=h(4)=1$ si $h(2)=2$
- toate ciclurile pozitive trec prin 2 $\Rightarrow \nexists$ ciclu pozitiv cu noduri de aceeasi culoare
- G_1 aciclic, $d_1 = 0$, $d_3 = 1$, $d_4 = 2 \Rightarrow$ ordonarea actualizarilor cf demonstratiei este (1, 3, 4, 2)

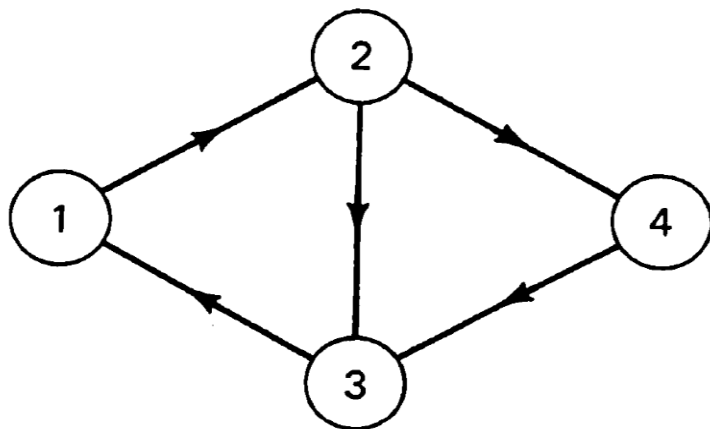


Figure 1.2.6 The dependency graph associated with an iteration of the form

$$x_1(t+1) = f_1(x_1(t), x_3(t))$$

$$x_2(t+1) = f_2(x_1(t), x_2(t))$$

$$x_3(t+1) = f_3(x_2(t), x_3(t), x_4(t))$$

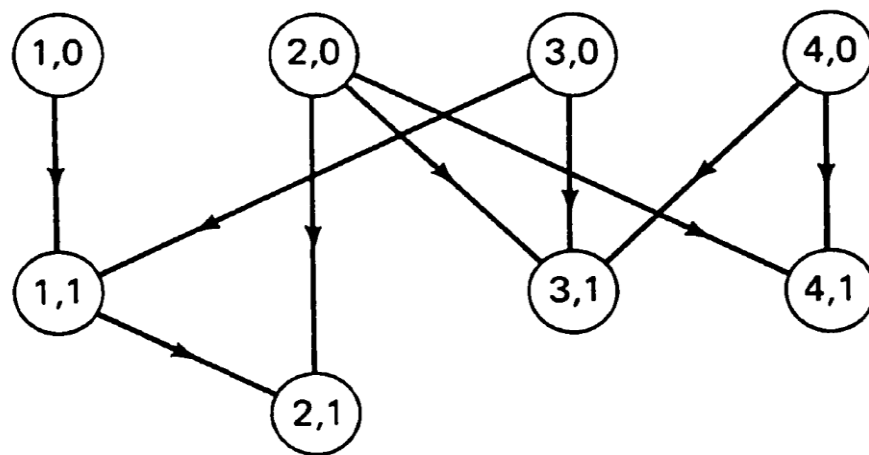
$$x_4(t+1) = f_4(x_2(t), x_4(t)).$$

Exemplu iteratie GS optima

- ordonarea actualizarilor cf demonstratiei este (1, 3, 4, 2)

=> iteratia Gauss-Seidel corespunzatoare acestei ordini este cea de mai jos

(se poate verifica vizual ca ordinea aceasta necesita nr min de pasi paraleli per sweep)



Caz simplificat de paralelizare optima a unei iteratii GS

- pp. $(i,j) \in A \Leftrightarrow (j,i) \in A$ (proprietate de simetrie). Atunci,

(i) exista o ordine de actualizare a unei iteratii GS care poate fi facuta in K pasi paraleli

$\Leftrightarrow$

(ii) exista o colorare a grafului de dependente cu cel mult K culori a.i. nodurile adiacente au culori diferite (i.e., $(i,j) \in A \Rightarrow h(i) \neq h(j)$)

- demonstratie

- suficient sa demonstram ca (ii) e echivalent cu (ii) din propozitia anterioara

“ $\Rightarrow$ ” pp $\exists$ ciclu pozitiv cu toate nodurile de aceeasi culoare $\Rightarrow \exists$ doua noduri adiacente de aceeasi culoare

“ $\Leftarrow$ ” $(i,j) \in A$ si $(j,i) \in A \Rightarrow (i,j)$ si (j,i) formeaza un ciclu pozitiv $\Rightarrow$ daca doua noduri adiacente au aceeasi culoare $\exists$ ciclu pozitiv cu toate nodurile de aceeasi culoare

Consecinte

- colorarea grafurilor cu $K > 3$ culori e NP-complete $\Rightarrow$ gasirea unei ordini a actualizarilor variabilelor care sa maximizeze paralelismul unei iteratii Gauss-Seidel e o problema dificila
- in practica, exista probleme cu structura speciala in care o colorare cu cateva culori poate fi gasita usor
- (1) graf G neorientat, daca G e arbore sunt suficiente doua culori
 - alegem un nod arbitrar din arbore
 - asociem culoarea 1 (respectiv 2) tuturor nodurilor din G care pot fi vizitate incepand din nodul respectiv traversand un nr par (respectiv impar) de arce
- (2) daca orice nod din G are cel mult D vecini, $D+1$ culori sunt suficiente
 - coloram nodurile pe rand
 - pp. primele i noduri colorate si dorim sa coloram $i+1$
 - folosim $D+1$ culori $\Rightarrow \exists$ o culoare pe care o putem folosi pt $i+1$ a.i. e colorat diferit de vecinii deja colorati ai lui $i+1$

Paralelizarea metodelor iterative

Reamintim: *Scopul paralelizarii algoritmilor iterativi este de a obtine reducere cat mai semnificativa a complexitatii in timp*

Pentru o accelerare optima avem nevoie de:

- planificator eficient
- timp de comunicatie redus

Pana acum nu am luat in calcul comunicatia in sistemele paralele!

Analiza comunicatiei

- comunicatia inter-procesor reprezinta o fractiune importanta din timpul de rulare al programelor paralele
- interpretare posibila: comunicatia are un cost de penalizare (echivalent, exista intarzieri de comunicare)

$$CP = \frac{T_{total}}{T_{comp}}$$

T_{total} este timpul total de executie al programului

T_{comp} este timpul alocat exclusiv calculului, fara comunicare (i.e., daca consideram comunicarea instantanee)

Cadrul analizei

- vom considera o retea de procesoare interconectata cu legaturi de comunicatie (MIMD fara memorie partajata)
- fiecare procesor are memorie locala si schimba informatii cu alte procesoare prin retea
- informatiile sunt structurate ca pachete de biti de lungimi diferite
- vom considera de asemenea paradigma *store-and-forward* pentru comunicarea pachetelor
- pp. ca bitii unui pachet sunt transmisi consecutiv fara intrerupere
- pp. un model de comunicare direct procesor-la-procesor

Surse de intarziere in comunicare

- timp procesare comunicatie:
 - timpul necesar pregatirii informatiei pt transmisie
 - e.g. asamblare/dezasamblare informatii in pachete, adaugarea header-elor, selectia rutei, mutarea pachetelor in bufferele de transmisie, etc
- timp de asteptare in coada:
 - pachetele asamblate pt transmisie sunt puse intr-o coada de asteptare inainte de inceperea transmisiei
 - motive:
 - planificarea accesului pachetelor la link in conditii de competitie la resurse
 - amanarea transmisiei pana cand destinatia poate primi pachetul (eg, sliding window protocols)
 - retransmisii in caz de eroare
- timp de transmisie: durata transmisiei tuturor bitilor unui pachet
- timp de propagare: durata de transfer a ultimului bit al pachetului

Calcul timp de comunicare

- oricare dintre timpii anterior pot fi neglijabili
 - informatie generata cu regularitate si resurse suficiente => timpul de asteptare in coada e neglijabil
 - distanta fizica intre sursa si destinatie este f. mica => timpul de propagare e neglijabil
- simplificari rezonabile
 - timp de procesare si propagare constant (P)
 - timp de transmisie proportional cu nr de biti (R pentru 1 bit)
- intarzierea unui pachet pe o linie de comunicatie

$$D = P + RL + Q$$

- L = lungimea in biti a pachetului
- Q = timp de asteptare in coada, in general dificil de modelat

Observatii

$$D = P + RL + Q$$

- in majoritatea retelelor moderne, $P + RL$ este semnificativ mai mare decat executia unei operatii numerice elementare (e.g. inmultire in virgula-mobila)
 - daca un algoritm foloseste comunicatia intensiv, timpul de comunicare domina timpul de executie (complexitatea timp) $\Rightarrow$ performanta redusa (echivalent, costul de penalizare CP va fi mare)
- se poate incerca **paralelizarea comunicatiei**

Mesaje

- pachetele sunt unitatea de comunicare in modelul nostru
- in general ele fac parte dintr-un mesaj care are sens doar in intregimea sa, deci intarzierea intregului mesaj devine importanta
- aceasta intarziere depinde de felul in care este impartit mesajul in pachete si modul in care se trimit pachetele individuale
- ex: un mesaj se divide in n pachete de lungimi egale
 - presupunem timpul de transmisie al unui pachet T
 - transmisia pachetelor in paralel pe n cai separate care au acelasi timp de transmisie per pachet rezulta intr-o reducere de n ori a timpului de comunicatie comparativ cu transmisia pe o singura cale
 - pp overhead-ul segmentarii in pachete, procesarea, propagarea si intarzierea in cozi neglijabile
 - ex. SCTP vs TCP (la nivel de protocol)

Transmisia mesajelor in paralel

- alta posibilitate de paralelizare a comunicatiei: *pipelining*
- mesaj impartit in n pachete de lungimi egale
- pachetele se transmit secvential pe o cale de lungime $k > 1$
- timp de transmisie de la nod la nod T (per pachet)
- timp total de transmisie a mesajului in mod pipeline ca o colectie de n pachete:

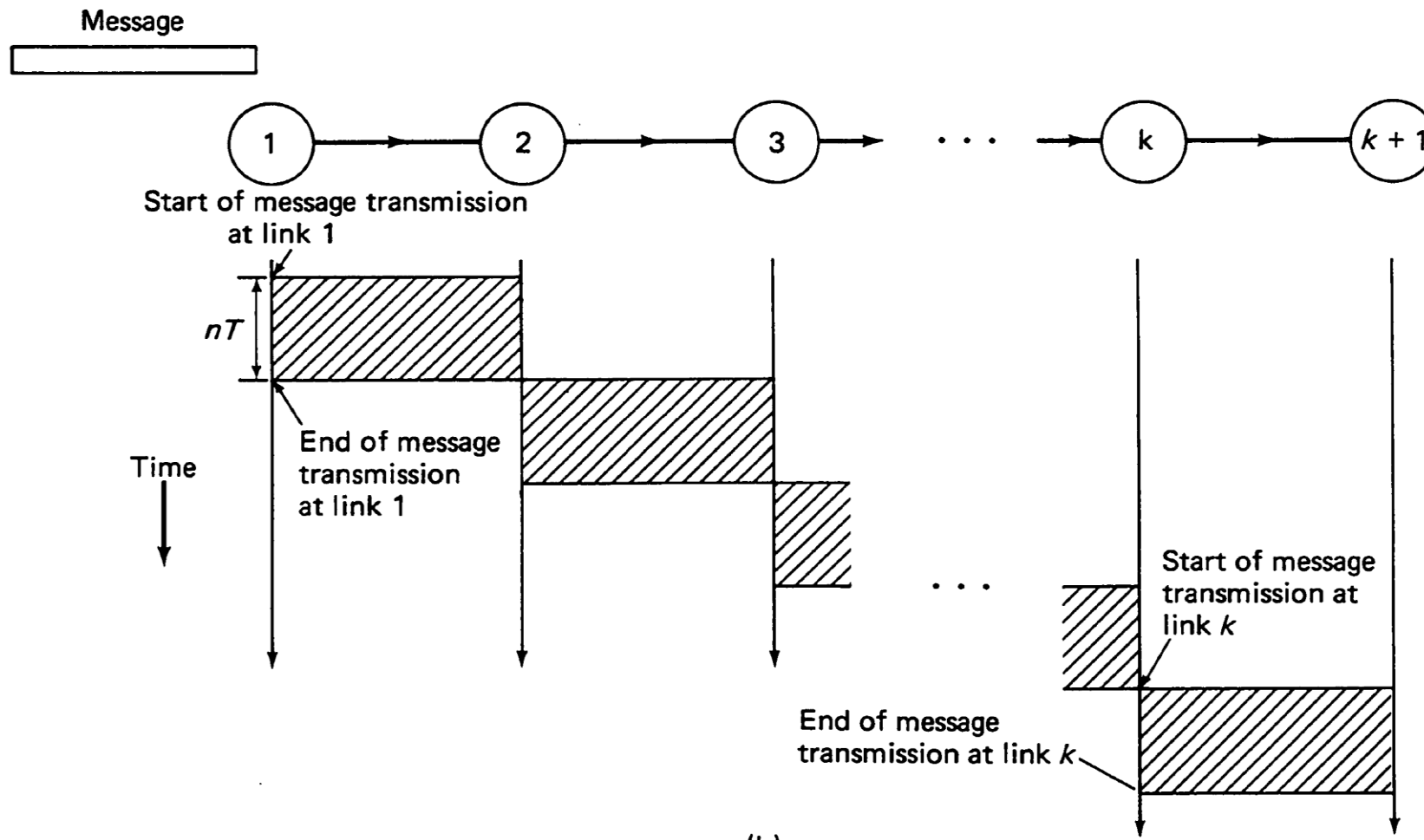
$$(n - k + 1)T$$

- timp total de transmisie mesaj ca un singur pachet de dimensiune n :

$$nkT$$

- Obs: pp neglijabile efectele overhead-ului, procesarii, propagarii si asteptarii in cozi
 - de asemenea, pp un nod primeste un pachet in intregime inainte sa trimita mai departe orice parte a lui

Transmisie mesaj ca un singur pachet



Observatii

- daca pachetele sunt f. mici $\Rightarrow$ intarzierea se poate reduce cu un factor egal cu nr de link-uri (linii de comunicatie) $\approx$ timpul de transmisie a mesajului pe un singur link

$\Rightarrow$ alternativa, *transmisia cut-through*

- un nod poate transmite altui nod o portiune a pachetului fara sa astepte primirea in intregime a pachetului
 - revine la segmentarea pachetului in multe pachete mici pt a beneficia de avantajele pipelining-ului
- pipelining-ul e aplicabil si in alte situatii, de ex peste un spanning tree (arbore de acoperire)

Factori care influenteaza intarzierile de comunicatie

- algoritmi care controleaza reseaua de comunicatie
 - controlul erorilor
 - rutarea
 - flow control
- topologia retelei de comunicatie
 - numarul, natura si locul (pozitia) liniilor de comunicare
- structura problemei si proiectarea algoritmului a.i. sa se potriveasca acestei structuri (inclusiv gradul de sincronizare necesar algoritmului)

Algoritmi paraleli

Curs 5

Vlad Olaru

vlad.olaru@fmi.unibuc.ro

Calculatoare si Tehnologia Informatiei

Universitatea din Bucuresti

Topologii de retea interprocesor

- reprezentate ca o retea de procesoare $G = (N, A)$
- presupuneri:
 - 1. comunicare simultana pe toate arcele adiacente unui nod (procesor) in ambele directii (pt graf neorientat)
 - 2. absenta intarzierilor in coada, intarzieri egale pt pachete de dimensiune egala pe toate arcele
 - 3. intarzierea unui pachet pe un arc este de 1 unitate de timp (in absenta altor precizari)
 - 4. algoritmi sunt initiati simultan pe toate nodurile (procesoarele)
- unele topologii sunt mai potrivite decat altele pt. task-uri de comunicare standard

Criterii de evaluare topologii (1)

- (1) *diametru* – distanta maxima dintre oricare perechi de noduri (nr. min de arce)
 - ex: topologie cu diametru $r \Rightarrow$ timp de transmisie pachet $O(r)$
 - (2) *conectivitate* – de arce sau de noduri, limitata superior de $\min \text{degree}(n)$
 - nr min de arce/noduri care trebuie eliminate pt. ca graful sa devina neconex
 - util pt. toleranta la defecte sau cresterea gradului de paralelism al transmisiei
 - comunicatia se pastreaza in prezenta defectelor de noduri sau arce
 - reducere timp de comunicare cu factor k
 - ex: topologie cu conectivitate $k \Rightarrow$ min k cai de comunicatie individuale intre doua noduri/procesoare, fara arce comune
 - mesaj lung impartit in pachete trimise in paralel pe k cai
- Obs: pachetele pot ajunge neordonat la destinatie $\Rightarrow$ mesajul trebuie reasamblat in ordine

Criterii de evaluare topologii (2)

- (3) *flexibilitate*

- (1) capacitatea de a rula un algoritm dezvoltat pe o topologie $G_1=(N_1,A_1)$ pe o alta topologie $G_2=(N_2, A_2)$
- pp existenta unei functii care asociaza fiecarui nod din G_1 un nod G_2 din a.i. arcele din G_1 corespund unor arce din G_2

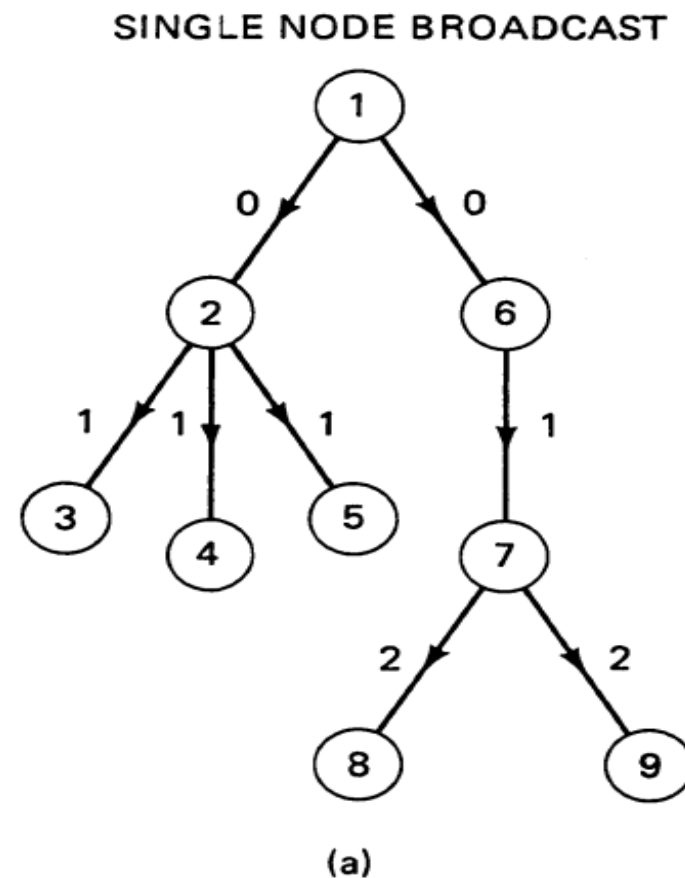
$$f: N_1 \rightarrow N_2 \text{ a.i. } f(i) \neq f(j) \text{ daca } i \neq j \text{ si } (f(i), f(j)) \in A_2 \text{ daca } (i, j) \in A_1$$

- se spune ca G_1 se *mapeaza* pe G_2
- (2) permite divizarea unui task intre procesoarele unei topologii date a.i. sa se minimizeze comunicatia
 - pp taskul divizat in subtaskuri, fiecare asociat cu un procesor din graf
 - pp anumite subtaskuri (i,j) interactioneaza, i.e. executia i sau j pp cunostinta unor valori calculate in timpul executiei j sau i
- $\Rightarrow$ subtaskurile se asigneaza unor procesoare care au link direct de comunicare
- $\Rightarrow$ graful de interactiuni intre subtask-uri G_1 se *mapeaza* pe graful de conexiuni G_2
- ex: mapare graf de dependente pe graful de interconexiune procesoare in executia unei iteratii din metode de relaxare

$$x_i(t+1) = f_i(x_1(t), \dots, x_n(t)) \quad i = 1, \dots, n$$

Criterii de evaluare topologii (3)

- (4) *intarzierea la comunicare* – importanta pt task-uri de tip broadcast, scatter/gather, etc pt. operatii de produs scalar, inmultire vector-matrici, etc
- exemple de operatii:
- **(1) difuzare uninod (single node broadcast) / multinod (multinode broadcast)**
 - trimiterea unui pachet de la un procesor la toate celelalte, respectiv executia simultana din toate nodurile a unui broadcast single node
 - ex. broadcast multinod: interatiile din metodele de relaxare studiate anterior
 - daca x_i e asociata unui procesor si f_i depinde de toate variabilele $\Rightarrow$ la sf iteratiei fiecare procesor trebuie sa trimita valoarea variabilei sale catre toate celelalte procesoare
 - solutie single node broadcast: *spanning tree* (arbore de acoperire) cu radacina in nodul sursa



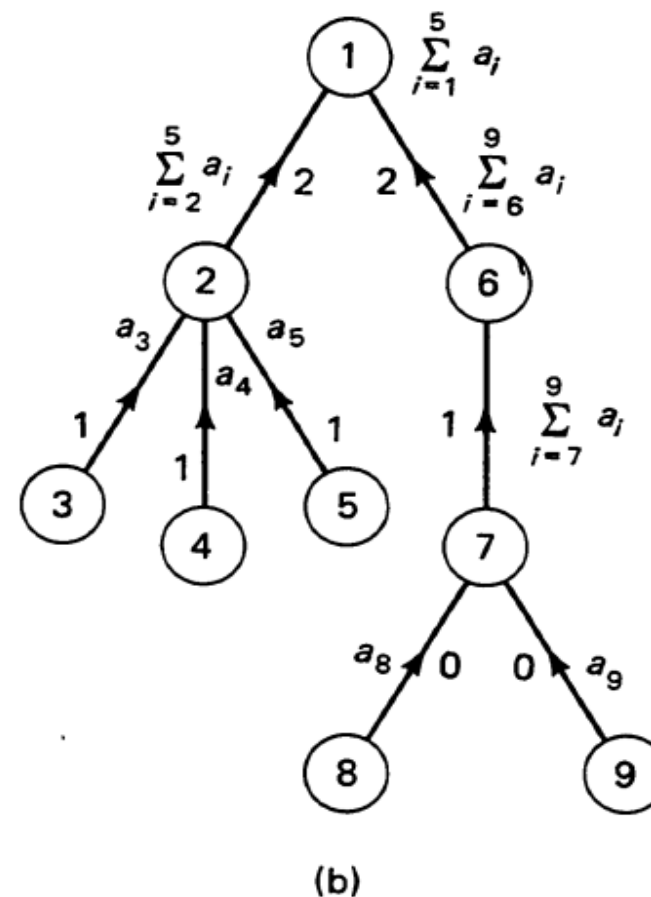
Intarzierea la broadcast

- **difuzare uninod (single node broadcast) / multinod (multinode broadcast)**
 - solutie single node broadcast: *spanning tree (arbore de acoperire)* cu radacina in nodul sursa
 - alegerea potrivita a arborelui de acoperire poate permite executia difuzarii in $O(r)$, unde r = diametrul retelei
 - optimizare broadcast mesaje lungi
 - mesajul impartit in pachete mici, trimise individual pe arcele din arborele de acoperire
 - ⇒ efect similar pipelining (cf. slide-uri curs trecut)
 - reduce timpul de broadcast de la $O(r)$ la $O((r + m - 1)/m)$, pt m pachete per mesaj, care necesita fiecare $1/m$ unitati de timp de transmisie pe link
 - solutie difuzare multinod: folosim cate un arbore de acoperire pt fiecare nod sursa
 - dificultate: arborii de acoperire pot partaja arce, la sosirea simultana a mai multor pachete la acelasi nod trebuie modelata si intarzierea de comunicare pe link
 - ⇒ nevoia de analiza specifica, pt fiecare tip de retea de interconectare

Acumulare uni- si multinod (1)

- **single node and multinode accumulation**
 - problema duala difuzarii, in sensul in care foloseste tot arbori de acoperire ca solutie
- single node accumulation
 - fiecare nod trimite un pachet catre acelasi nod
 - pp. pachetele se pot “combina” in transmisia pe un arc, intarzierea unui pachet “combinat” fiind aceeaasi cu aceea a unui pachet obisnuit
 - ex tipic: calculul prefix, acumularea sumelor partiale ale unui produs scalar

SINGLE NODE ACCUMULATION



Acumulare uni- si multinod (2)

- **single node and multinode accumulation**
 - multinode accumulation
 - acumulare uninod simultana la mai multe noduri
 - se poate demonstra ca problemele de acumulare se rezolva in acelasi timp cu cele de difuzare
 - ex: single node accumulation poate fi vazut ca un algoritm de single node broadcast rulat invers; reciproca e si ea adevarata

Alte operatii

- **schimb total (total exchange)**
 - trimite un pachet de la orice nod catre orice nod
 - diferenta fata de multinode broadcast: se trimit pachete diferite, nu acelasi pachet !
- **single node scatter**
 - trimite un pachet separat de la fiecare nod la toate celelalte noduri
- **single node gather**
 - colecteaza la un nod un pachet trimis de toate celelalte noduri

Observatii

- in multinode broadcast, fiecare nod primește un pachet diferit de la toate celelalte noduri => rezolva single node scatter
- total exchange
 - versiune multinod a doua probleme de single node scatter si single node gather
 - de asemenea, generalizare a multinode broadcast in care pachetele trimise de fiecare nod celorlalte este diferit
- concluzie: problemele de comunicare discutate formeaza o ierarhie in termeni de dificultate
 - un algoritm care rezolva o problema in ierarhie rezolva urmatoarea problema din ierarhie fara cost de timp additional
 - total exchange rezolva multinode broadcast (accumulation)
 - multinode broadcast (accumulation) rezolva single node gather (scatter)
 - single node scatter (gather) rezolva single node broadcast (accumulation)

Ierarhia complexitatii operatiilor de comunicare

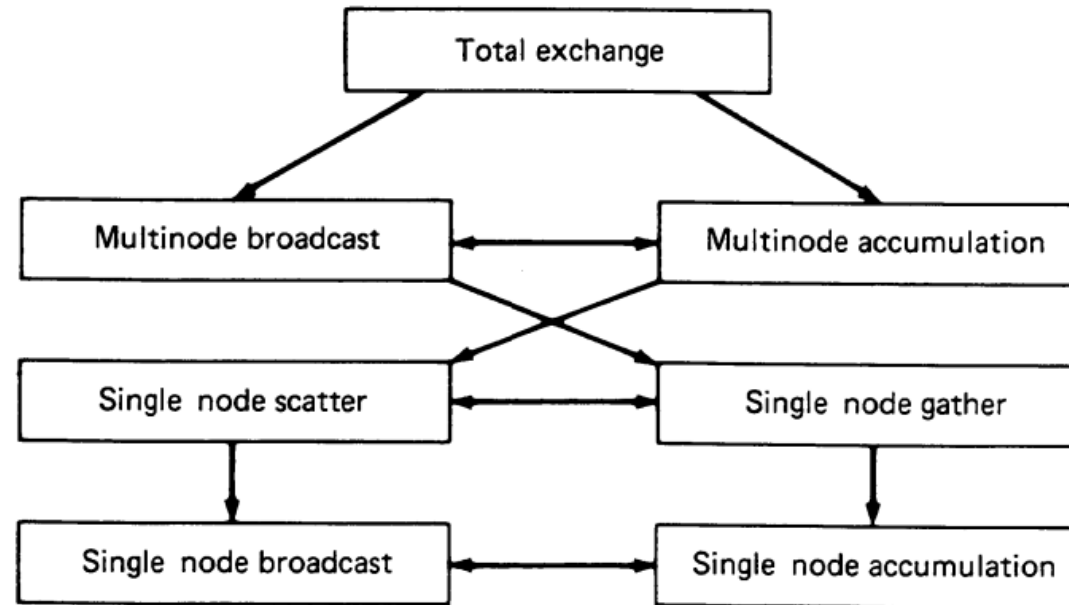
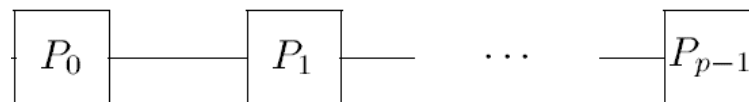


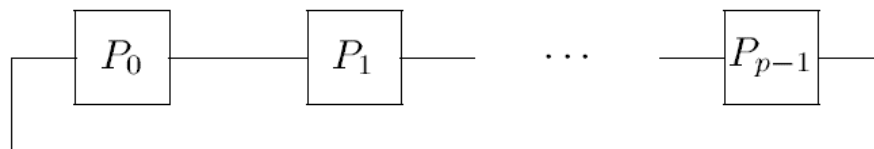
Figure 1.3.5 Hierarchy of basic communication problems in interconnection networks. A directed arc from problem A to problem B indicates that an algorithm that solves A can also solve B, and that the optimal time for solving A is not more than the optimal time for solving B. A horizontal bidirectional arc indicates a duality relation.

Topologii specifice

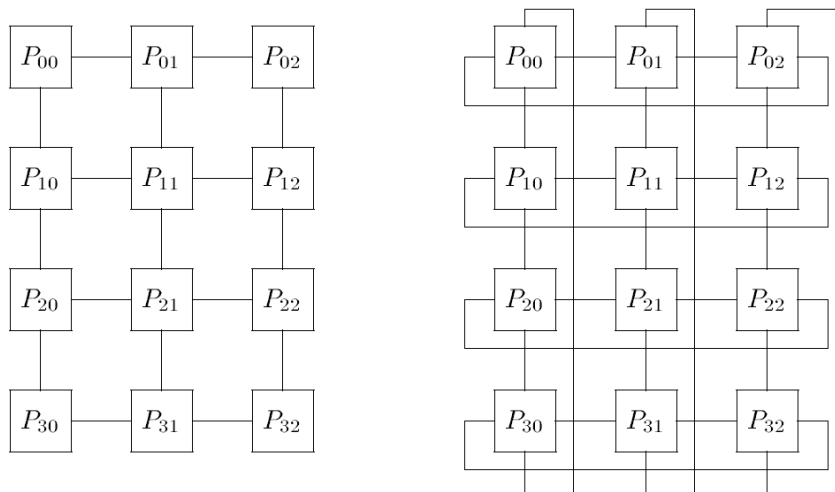
- Lant liniar



- Inel



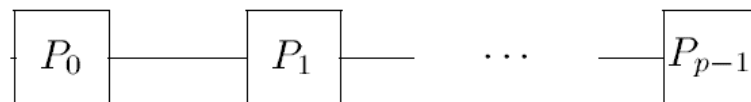
- Grid/Tor



Graf complet

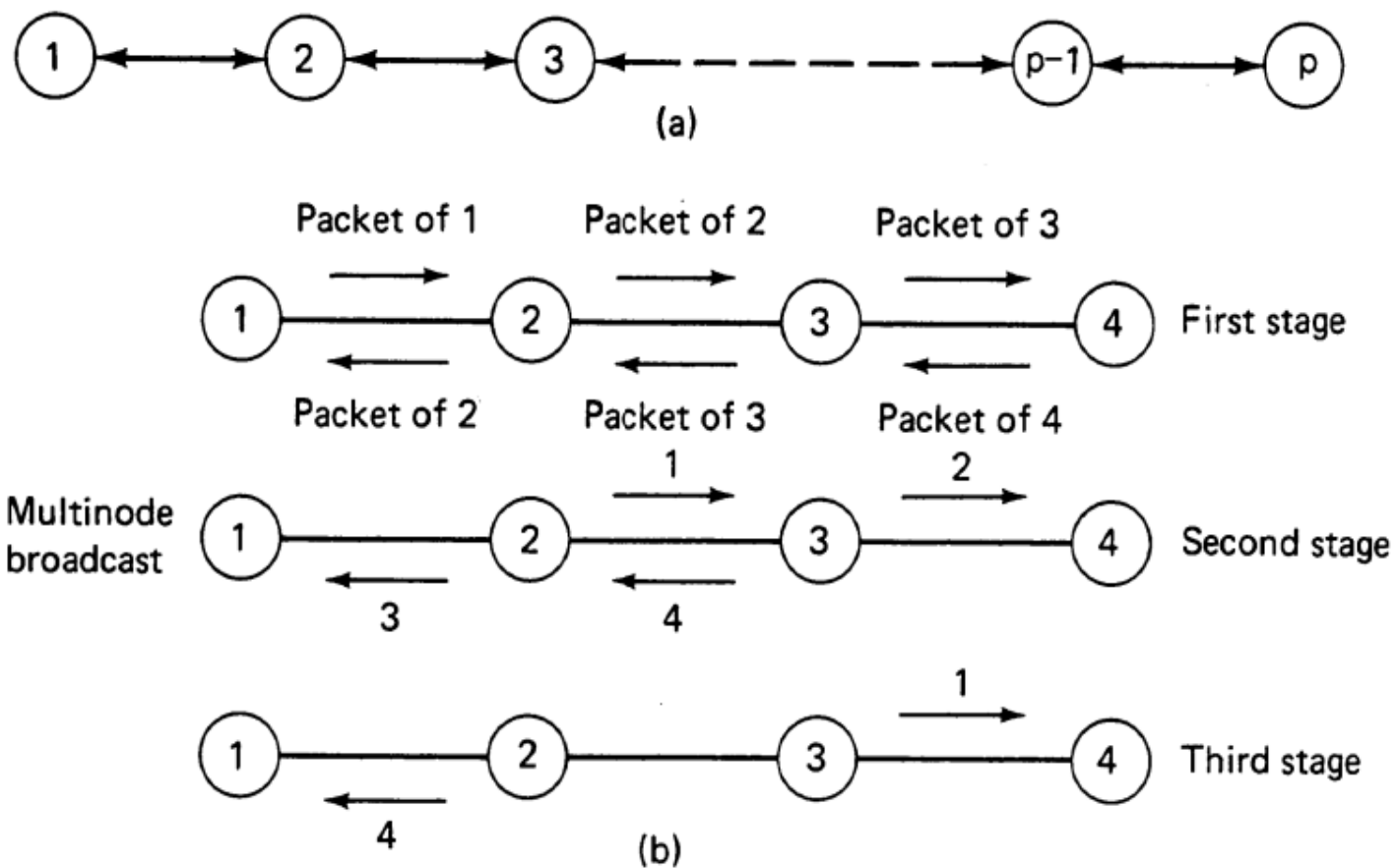
- implementare: magistrala (bus) partajata sau crossbar switch
- flexibilitate maxima
- nr mare de procesoare => crossbar switch complex si costisitor, respectiv intarzieri mari pe bus (in principal datorita asteptarilor in cozi)
- utilizate frecvent in conectarea unui nr redus de procesoare in clustere care apoi sunt interconectate ierarhic

Lant liniar (1)



- cele mai defavorabile valori pt diametru si conectivitate
- se poate mapa in mai toate topologiile despre care vom vorbi => penalizarea comunicarii nu se poate imbunatati mai mult decat la celelalte topologii
- timpul de broadcast depinde de nodul initiator, worst case $p - 1$ (diametrul)
 - obs: transmisia pachetului costa o unitate de timp
- timpul de multinode broadcast se poate optimiza tot la $p - 1$ prin folosirea in paralel a tuturor link-urilor (conexiuni full duplex):
 - algoritm in etape
 - in prima etapa, fiecare nod trimite in stanga si dreapta sa valoarea proprie
 - in fiecare etapa k , nodul i primeste pachetele nodurilor $i - k$ (pt $i > k$) si $i + k$ (pt $i + k \leq p$)
 - multicastul se termina dupa $p - 1$ etape

Multinode broadcast lant liniar



Lant liniar (2)

- timpul optimal de single node scatter
 - intre timpul optimal necesar pt single node broadcast si cel de multinode broadcast $\Rightarrow p - 1$ in cel mai defavorabil caz
 - se poate arata ca timpul optimal (si pt single node broadcast) al unui algoritm care incepe operatia in nodul k este de $\max(k - 1, p - k)$
- timpul optimal de schimb total $\Theta(p^2)$
 - link-ul $(k, k+1)$ separa lantul in doua subseturi de k si $p - k$ noduri
 - fiecare nod dintr-un subset trebuie sa trimita un pachet catre nodurile din celalalt subset
 - $\Rightarrow (k, k+1)$ transporta $k(p - k)$ pachete in fiecare directie
 - $\Rightarrow$ daca alegem cel mai defavorabil link, avem cel putin $\max_k[k(p - k)]$ pasi (unitati de timp)
 - $\Rightarrow$ prin derivare obtinem valoarea optima a lui $k = \frac{p}{2}$
 - $\Rightarrow$ ceiling $((p^2 - 1)/4)$ unitati de timp
 - alternativ, trebuie sa se rezolve secvential p probleme single node scatter pt fiecare procesor
 - fiecare problema necesita cel mult $p - 1$ pasi $\Rightarrow \Theta(p^2)$

Inel

- topologie toleranta la defecte
- daca o conexiune se defecteaza, ramane o cale de la orice procesor la alt procesor
- totusi, diametrul poate atinge valoarea $\text{ceiling}((p - 1) / 2)$
- orice operatie pe un inel se poate rezolva intre timpul corespunzator pt acea operatie pe un lant liniar cu acelasi nr de noduri si jumatatea acelui timp
- ex: broadcast, MST echilibrat

Tree (arbore)

- p procesoare, orice comunicare între două noduri folosește cel puțin $p - 1$ link-uri
- conectivitate redusă
- defectarea unui link produce două subseturi de noduri deconectate
- diametrul worst case $p - 1$ (listă, lanț liniar)
- topologia star are diametru minim, dar tot traficul trece prin nodul central
=> bottleneck
- timpul optim de single node broadcast (accumulation) și single node scatter (gather) nu depășește $p - 1$
- timpul optim de multinode broadcast este $p - 1$
- schimbul total este $O(p^2)$

Mesh/grid/tor

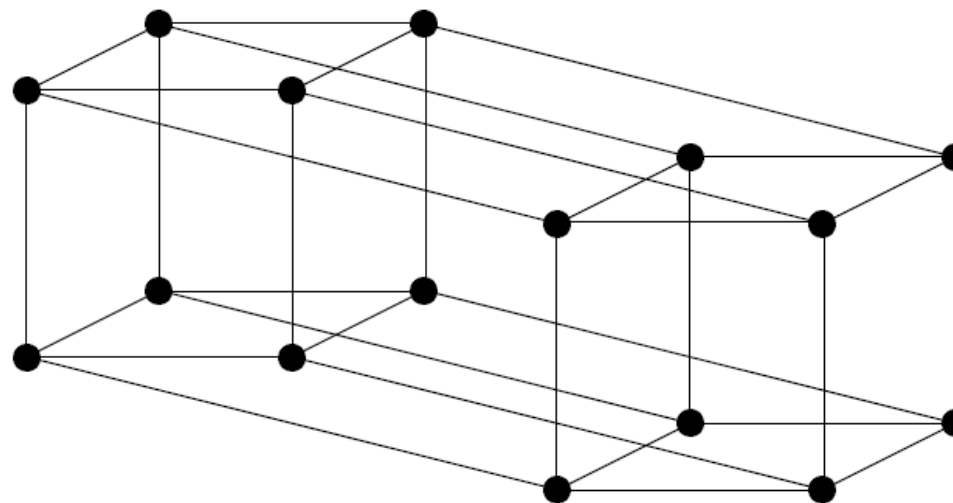
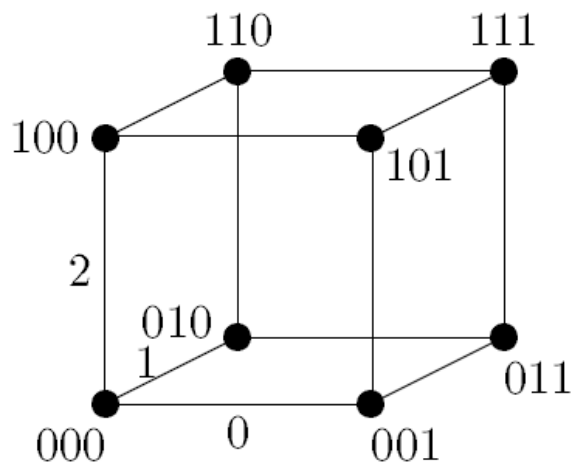
- topologie potrivita pentru probleme legate de geometria spatiului fizic
- grid d -dimensional cu n_i puncte de-a lungul dimensiunii i
 $(x_1, \dots, x_d)$, x_i ia valori de la 1 la n_i
- arc $((x_1, \dots, x_d), (x'_1, \dots, x'_d))$ a.i. $\exists i \mid |x_i - x'_i| = 1$ si $x_j \neq x'_j$ pt. $j \neq i$
- diametru $\sum_{i=1}^d (n_i - 1)$
- diametru mai mic ca la inel, dar semnificativ mai mare decat la arbore binar echilibrat cu acelasi nr de procesoare
- torul e o varianta cu diametru mai mic

Mesh/grid/tor (2)

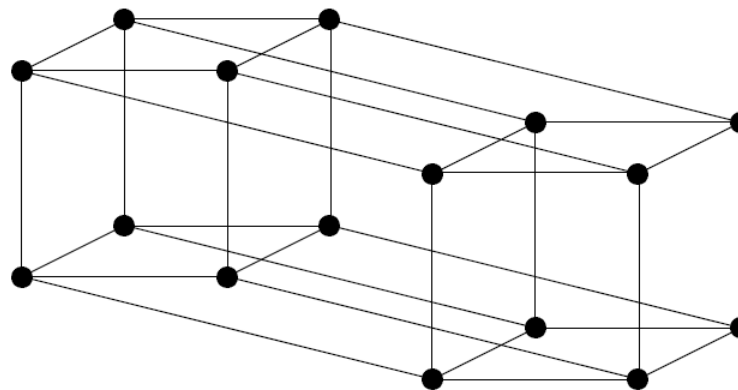
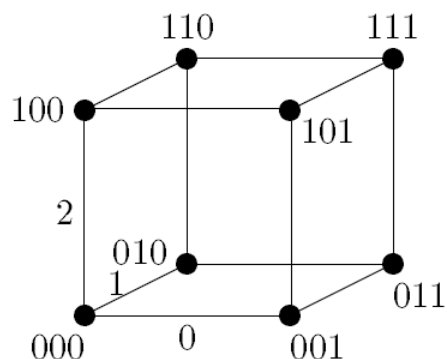
- fie un grid d -dimensional simetric (nr egal de procesoare $p^{1/d}$ pe fiecare dimensiune) cu p procesoare
- diametru $d(p^{1/d} - 1) \Rightarrow$ daca p fix $\Rightarrow$ timpul optimal de single node broadcast este $\Theta(p^{1/d})$
- maparea lant liniar pe mesh e triviala $\Rightarrow$ timpii optimali de scatter/gather sau multinode broadcast nu depasesc timpii corespunzatori de la lantul liniar $O(p)$
- *dar*, orice scatter ia $\Omega(p)$ pasi pt ca nodul transmite $p - 1$ pachete catre cei max $2d$ vecini (d fix !) $\Rightarrow$ scatter/gather optimal este $\Theta(p)$
- schimb total $\Theta(p^{(d+1)/d})$
 - pp p par si un hiperplan $d - 1$ dimensional care imparte gridul in 2 jumatati egale
 - fiecare jumatate are $p/2$ procesoare si primeste $(p/2)^2$ pachete de la procesoarele din cealalta jumatate
 - pachetele trebuie sa traverseze $p^{(d-1)/d}$ linkuri intre cele doua jumatati $\Rightarrow \Omega(p^{(d+1)/d})$ unitati de timp
 - se poate arata ca schimbul total este si $O(p^{(d+1)/d})$

Hipercub

- intr-un spatiu d -dimensional, multimea punctelor cu coordonate 0/1 pot fi vazute ca varfurile unui cub d -dimensional
- daca asociem aceste puncte intr-un graf cu procesoarele si conectam printr-un arc orice doua asemenea puncte care difera printr-un singur bit $\Rightarrow$ *hipercub* sau d -cub
- formal, un d -cub este un grid d -dimensional cu doua procesoare in fiecare dimensiune ($n_i = 2$ pt. orice i)
- fiecare nod/id procesor al unui d -cub are o reprezentare binara pe d biti



Constructie hipercub



- notam H_d hipercubul de dimensiune d , cu $p = 2^d$ procesoare
- fiecare procesor are d vecini
- P_i vecin cu P_j daca reprezentarile binare ale i si j difera printr-un singur bit (distanța Hamming 1)
- def. recursiva: H_d se obtine prin doua copii H_{d-1} in care se leaga arcele aflate in aceleasi pozitii (aceleasi reprezentari binare) in cele doua H_{d-1}
 - adresele nodurilor din H_d : se adauga un bit in pozitia cea mai semnificativa
 - 0 pentru prima copie H_{d-1}
 - 1 pentru cea doua copie H_{d-1}

Proprietati hipercub

- orice cale dintre doua noduri are un nr de arce cel putin egal cu distanta Hamming dintre reprezentarile binare ale celor doua noduri
- mai mult, exista o cale care are exact lungimea data de distanta Hamming
 - se schimba din 0->1 si invers bitii care difera in reprezentarea binara
 - ex: calea (1101)-> (0110) se construiesc astfel: (0101), (0111),(0110)
sau (1111),(1110),(0110)

=> diametru = d (sau $\log p$, pt. ca $p = 2^d$)

Mapare lant liniar pe hipercub

- hipercubul este o arhitectura flexibila
- ex: mapare lant liniar cu 2^d procesoare pe hipercub
- problema revine la constructia unei secvente de 2^d nr binare cu d biti a.i. doua numere succesive in secventa difera printr-un singur bit
- solutia: coduri Gray, in particular reflected Gray codes (RGC)
- constructia RGC similara constructiei recursive a hipercuburilor
- RGC au proprietatea ca primul si ultimul nr din secventa difera tot printr-un singur bit $\Rightarrow$ mapare inel pe hipercub

Reflected Gray codes

- data o secventa RGC de dimensiune $d - 1$

$$\{b_1, b_2, \dots, b_p\}$$

unde $p = 2^{d-1}$ si $b_1, b_2, \dots, b_p$ siruri binare

- secventa RGC de dimensiune d este

$$\{0b_1, 0b_2, \dots, 0b_p, 1b_p, \dots, 1b_2, 1b_1\}$$

- aceasta secventa mapeaza un inel de 2^d noduri pe un d -cub
- in general, se poate mapa orice inel de p procesoare, p par, pe un hipercub de dimensiune $2^{\text{ceiling}(\log p)}$
- orice ciclu intr-un hipercub are nr par de noduri => inel cu nr impar de noduri nu poate fi mapat pe hipercub
 - *indicatie*: se numara schimbarile de biti din reprezentarile nodurilor traversate in ciclu

Ex. mapare inel pe hipercub folosind RGC

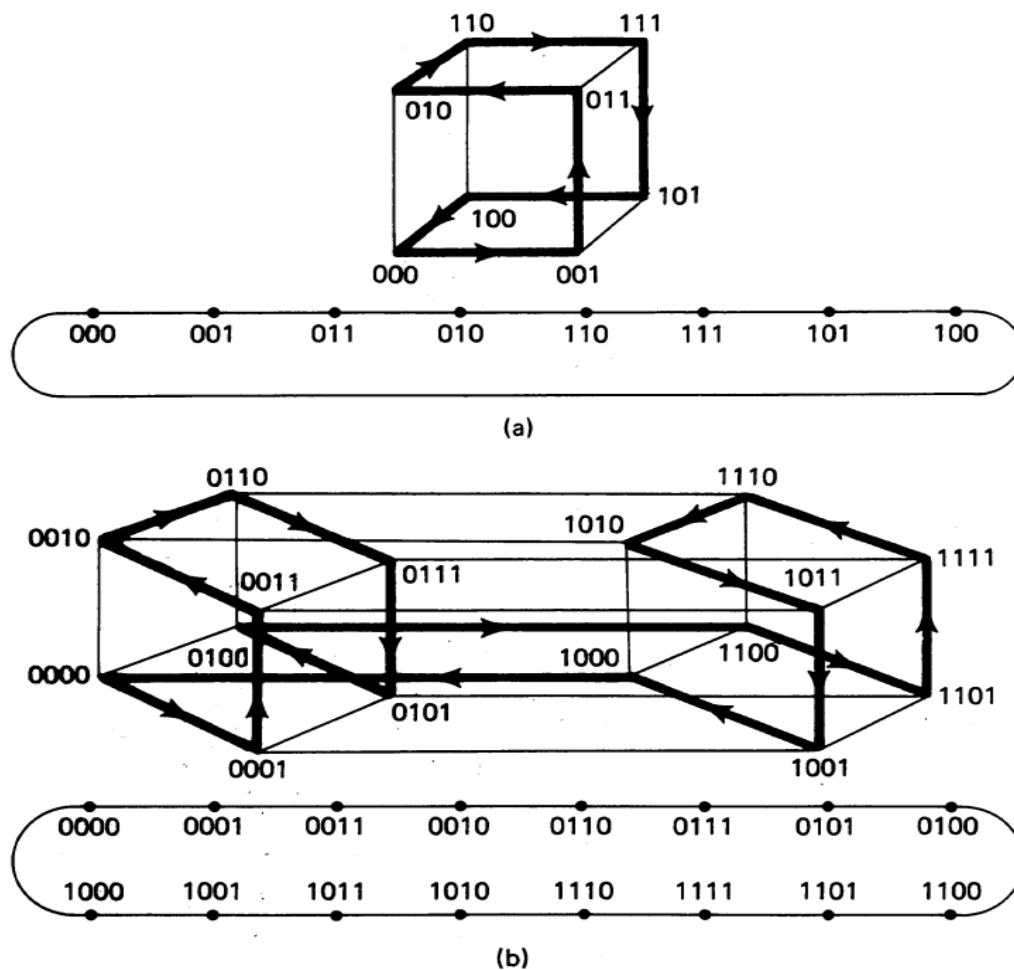


Figure 1.3.11 Reflected Gray code sequences, and the corresponding mappings of rings on (a) a 3-cube and (b) a 4-cube.

Mapare hipercub pe grid bidimensional folosind RGC

- generalizare a constructiei anterioare de RGC-uri
- fie d_a si d_b intregi pozitivi a.i. $d = d_a + d_b$
- fie $\{a_1, a_2, \dots, a_{p_a}\}$ si $\{b_1, b_2, \dots, b_{p_b}\}$ secvente d_a -bit si d_b -bit RGC

$$\text{unde } p_a = 2^{d_a} \text{ si } p_b = 2^{d_b}$$

Atunci matricea $p_a \times p_b$ de siruri de d biti

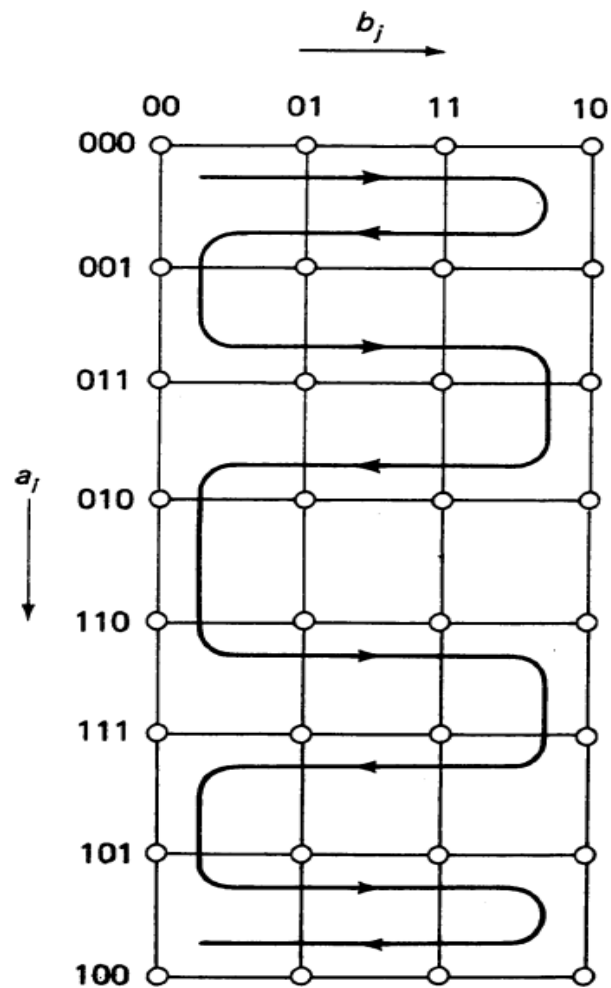
$$\{a_i, b_j \mid i=1, 2, \dots, p_a, j=1, 2, \dots, p_b\}$$

se poate folosi pentru a obtine o secventa d -bit RGC traversand liniile matricii alternand stanga-dreapta respectiv dreapta-stanga:

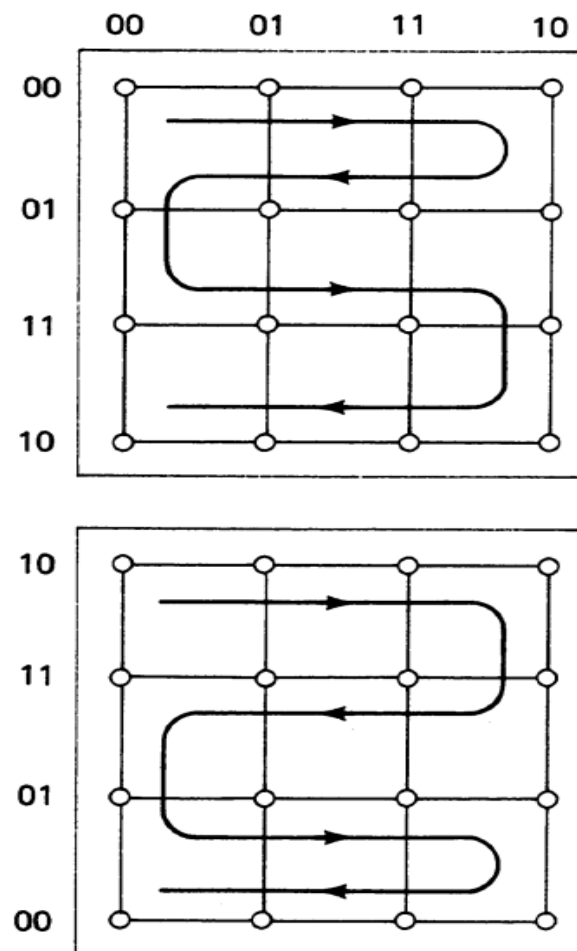
Traversare matrice pt. obtinerea d -bit RGC

$$\left[\begin{array}{cccccc} a_1 b_1 & \Rightarrow & a_1 b_2 & \Rightarrow & \dots & \Rightarrow & a_1 b_{p_b} \\ & & & & & & \Downarrow \\ a_2 b_1 & \Leftarrow & a_2 b_2 & \Leftarrow & \dots & \Leftarrow & a_2 b_{p_b} \\ \Downarrow & & & & & & \\ a_3 b_1 & \Rightarrow & a_3 b_2 & \Rightarrow & \dots & \Rightarrow & a_3 b_{p_b} \\ & & & & & & \Downarrow \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ & & & & & & \Downarrow \\ a_{p_a} b_1 & \Leftarrow & a_{p_a} b_2 & \Leftarrow & \dots & \Leftarrow & a_{p_a} b_{p_b} \end{array} \right] .$$

Ex. 5-cub mapat pe grid bidimensional



(a)



(b)

d-cub (5-cub) mapat pe grid bidimensional

- fig. *a*: nodurile unui d -cub pot fi mapate pe un grid bidimensional cu p_a (8) si p_b (4) noduri pe cele doua dimensiuni
 - elementul (i,j) din grid este nodul din d -cub cu id-ul a_i, b_j
 - pe fiecare linie din grid este mapat un d_b -cub cu p_b noduri
 - pe fiecare coloana din grid este mapat un d_a -cub cu p_a noduri
- fig. *b*: constructia inductiva a RGC de dimensiune d
- in mod similar se poate obtine o generalizare a maparii pe grid-uri multidimensionale
 - grid k -dimensional
 - n_i noduri in dimensiunea i , $i = 1, \dots k$
 - $n_i = 2^{d_i}$, $d = d_1 + \dots + d_k$

Comunicarea in hipercub H_d

- fiecare nod are d vecini $\Rightarrow$ exista cel mult d cai independente intre oricare doua noduri
- de fapt, intre orice doua noduri sunt exact d cai independente (care nu au in comun decat capetele caii)
 - daca id-urile capetelor caii difera prin n biti, atunci exista
 - n cai cu n arce intre cele doua noduri
 - $d - n$ cai cu $n + 2$ arce

$\Rightarrow$ conectivitate d -cub = d

$\Rightarrow$ comunicare simultana eficienta pe mai multe cai intre doua noduri ale unui hipercub

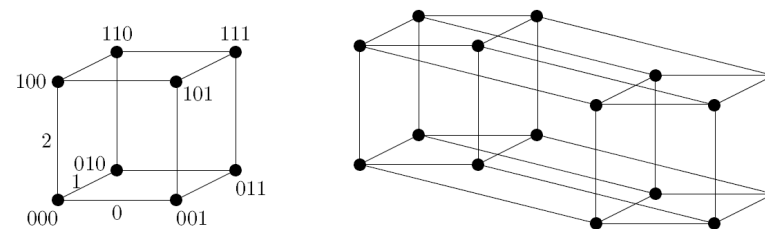
Comunicatie globala

- difuzare
- difuzare generala
- distributie
- schimb complet

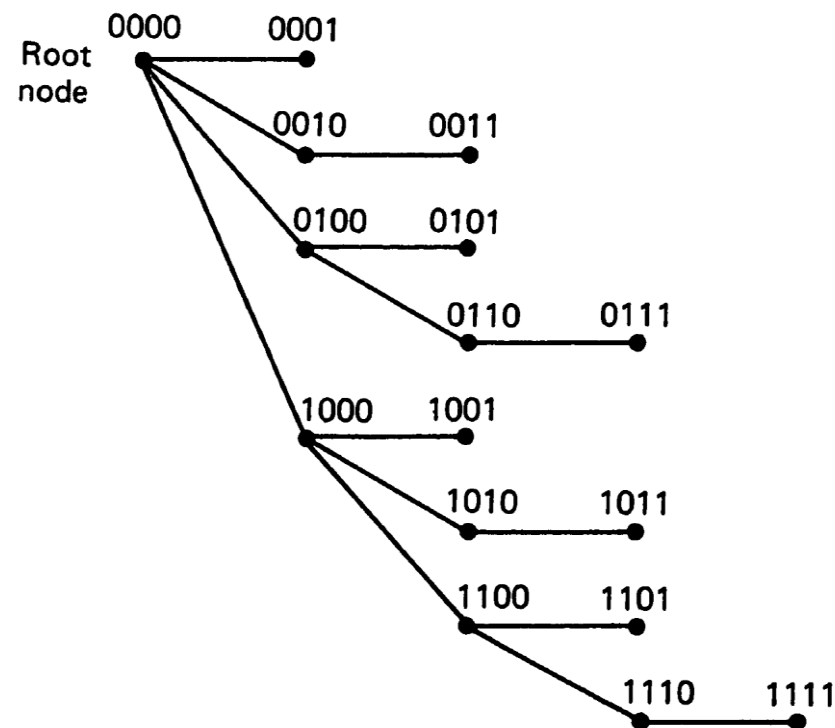
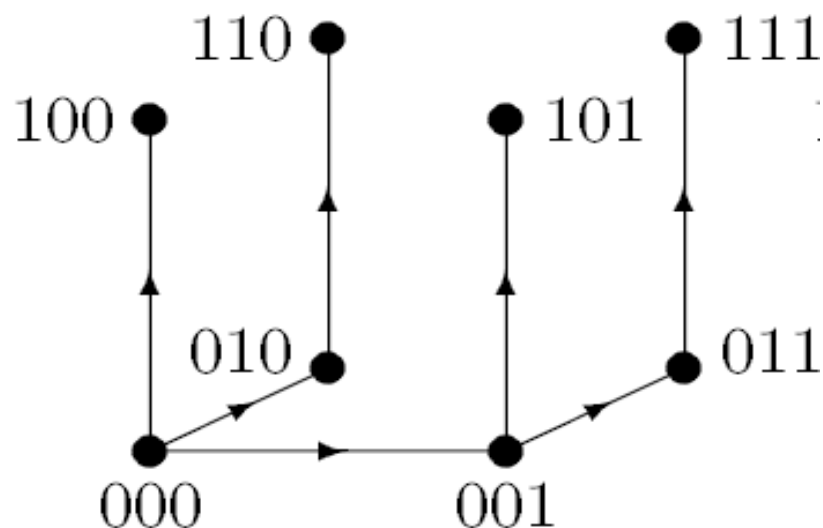
Alte proprietati hipercub H_d

- $\forall$ nod $i \in H_d$ exista un *arbore de acoperire binomial* cu radacina in i , in care exista o cale de lungime d de la i la orice nod
 - arborele se poate folosi pt single node broadcast de la radacina la orice nod in d pasi
- $\Rightarrow$ imbunatatire cu un factor de $\frac{2^d-1}{d}$ fata de un lant liniar cu 2^d procesoare
- acelasi algoritm se poate folosi pt. single node accumulation in d pasi

Difuzare hipercub



- constructia porneste din radacina (000...0)
 - constructie generala cu radacina in nodul i : aduna $i \bmod 2$ la valoarea fiecarui nod din arborele cu radacina (000...0)
- id copii nod: inverseaza unul dintre bitii zero ai parintelui care urmeaza celui mai din dreapta bit 1



Difuzare hipercub

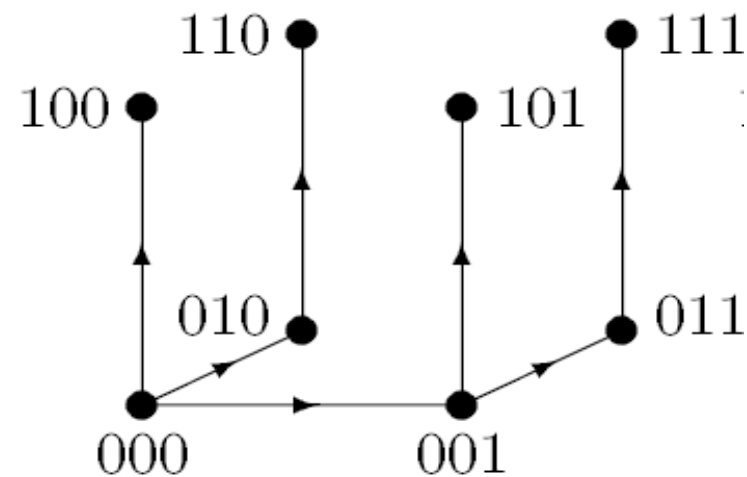
Notam:

recv(M,s) receptia mesajului M de la vecinul s

send(M,s) trimiterea mesajului M la vecinul s

Algoritm difuzare hipercub

1. $l \leftarrow$ poziția celui mai semnificativ bit de 1 din id (-1 pentru P_0)
2. **dacă** id $\neq 0$ **atunci**
 1. **recv**(M,l) {recepționează de la tată}
3. **pentru** $i = l + 1 : d - 1$
 1. **send**(M,i) {trimite tuturor fiilor}



Difuzare generala hipercub H_d

- fiecare procesor trimite un pachet catre orice alt procesor
- fiecare nod are cel mult d vecini $\Rightarrow$ primeste cel mult d pachete simultan
- fiecare pachet separat vine de la $2^d - 1$ procesoare
- $\Rightarrow$ difuzarea generala ia cel putin $\lceil (2^d - 1) / d \rceil$ pasi
- exista insa algoritmi optimali care ating aceasta valoare minima pornind de la algoritmul de broadcast (single node)

Algoritm optimal de difuzare generala

- fie un algoritm de broadcast in m pasi din nodul 0 reprezentat de o secventa de multimi de arce orientate disjuncte $A_1, \dots, A_m$
 - A_i contine arcele care transmit un pachet de la momentul $i - 1$ pana la momentul i
 - daca S_i (respectiv E_i) sunt id-urile de nodurilor de start (respectiv sfarsit) din A_i
 $\Rightarrow S_1 = \{(000\dots 0)\}$ si
 S_i e inclus in $\{(000\dots 0)\} \cup (\bigcup_{k=1}^{i-1} E_k)$
 $\forall id \neq 0$ apartine unui E_i
- setul de noduri + secventa $A_1, \dots, A_m$ alcatuiesc un arbore de acoperire al H_d
- similar, definim un algoritm de broadcast dintr-un nod oarecare t ca fiind

$$A_i(t) = \{(t \text{ xor } x, t \text{ xor } y) \mid (x,y) \text{ arc din } A_i\}$$
- obs: $(t \text{ xor } x, t \text{ xor } y)$ e link $\Leftrightarrow (x,y)$ e link (i.e., difera in acelasi bit)

Algoritm optimal de difuzare generala

- fie $r_i(x,y)$ = nr de noduri t pt. care (x,y) este arc in $A_i(t)$
- $\forall t$ ca mai sus, $\exists$ un pachet de trimis pe link-ul $(x,y) \Rightarrow$ pt. i oarecare si transmisie simultana pe toate arcele din $A_i(t)$, timpul T_i de transmisie pe aceste link-uri este

$$T_i = \max_{(x,y)} (r_i(x,y))$$

pt toate arcele (x,y) | (admitem intarzieri in cozile de asteptare)

$\Rightarrow$ timpul total al difuzarii generale este cel mult $T_1 + T_2 + \dots + T_m$

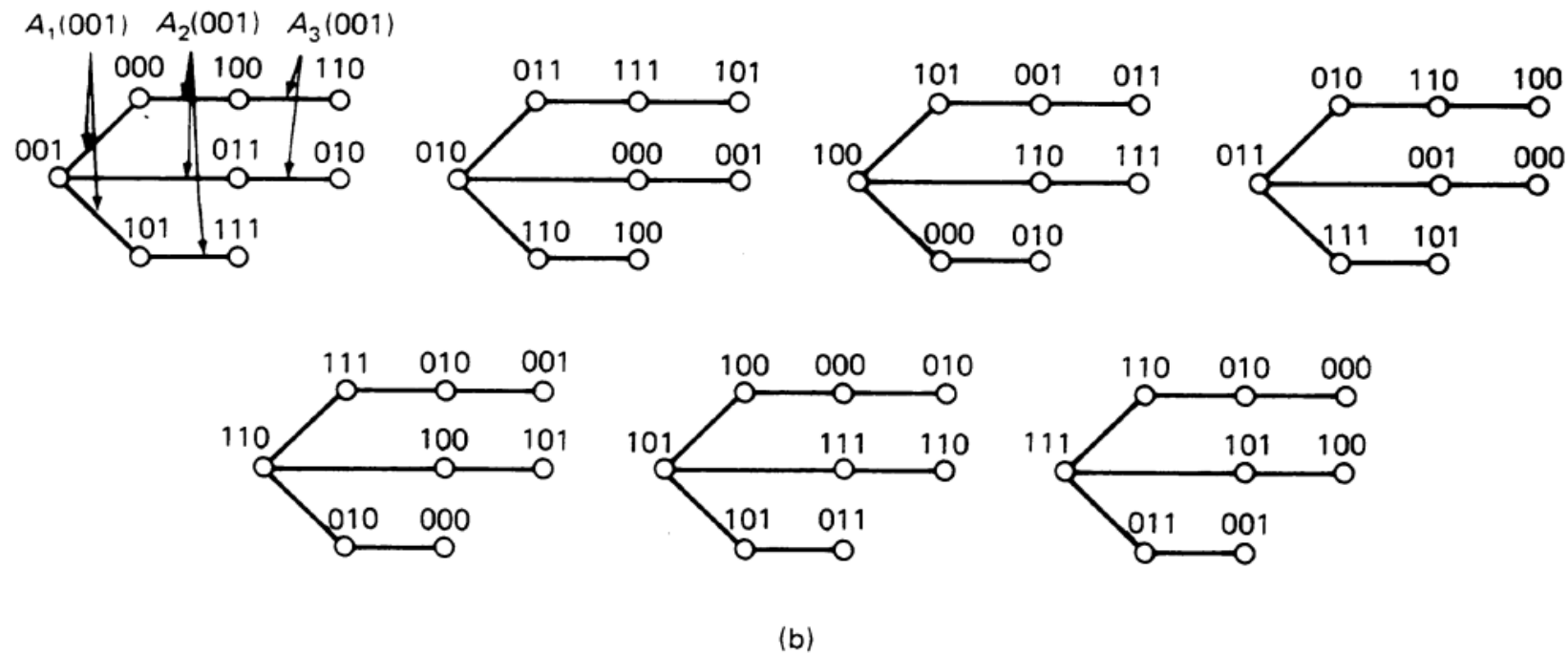
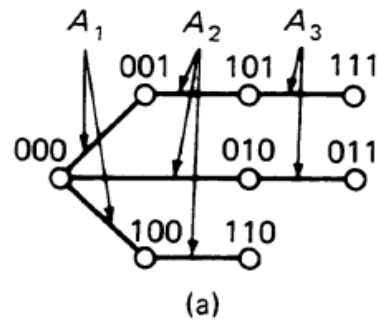
$\Rightarrow$ algoritmul optim trebuie sa selecteze secventa $A_1, \dots, A_m$ a.i. $T_1 + T_2 + \dots + T_m$ are valoare mica

si

$A_1, \dots, A_{m-1}$ au d arce, iar A_m are cel mult d arce

- *Obs:* $T_i = 1$ daca $\forall (x,y)$ si $(z,w) \in A_i$, x si y nu difera in acelasi bit ca z si w

Algoritm optimal de difuzare generala



Algoritm optimal de difuzare generala

- alegere optimala:

$$A_i \text{ a.i. } T_i = 1$$

$$\text{card}(A_i) = d \quad \forall 1 \leq i \leq m - 1$$

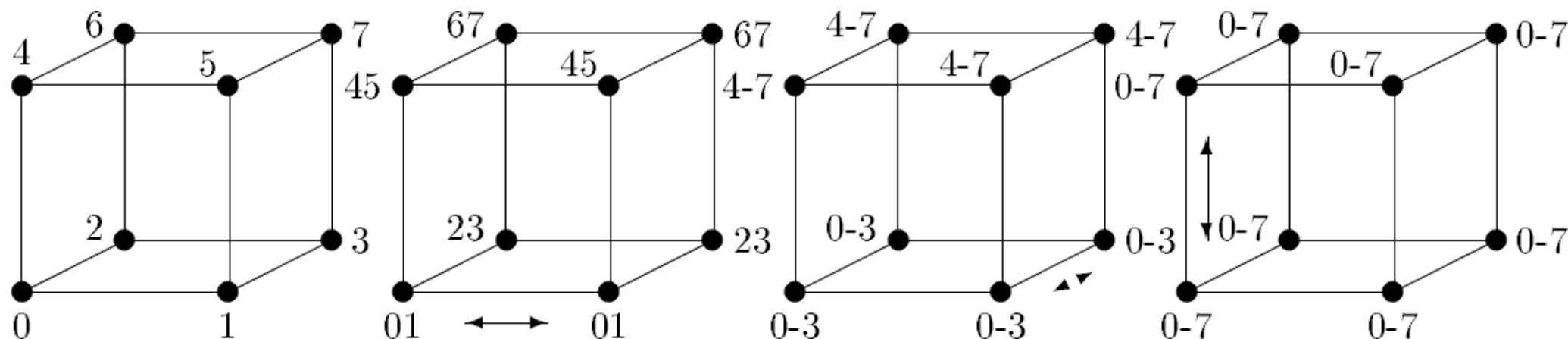
$$\text{card}(A_m) < d$$

- arborele de acoperire generat de reuniunea multimilor de arce $A_1, \dots, A_m$ are evident $2^d - 1$ arce

$$\Rightarrow T_1 + T_2 + \dots + T_m = m = \text{ceiling}((2^d - 1) / d)$$

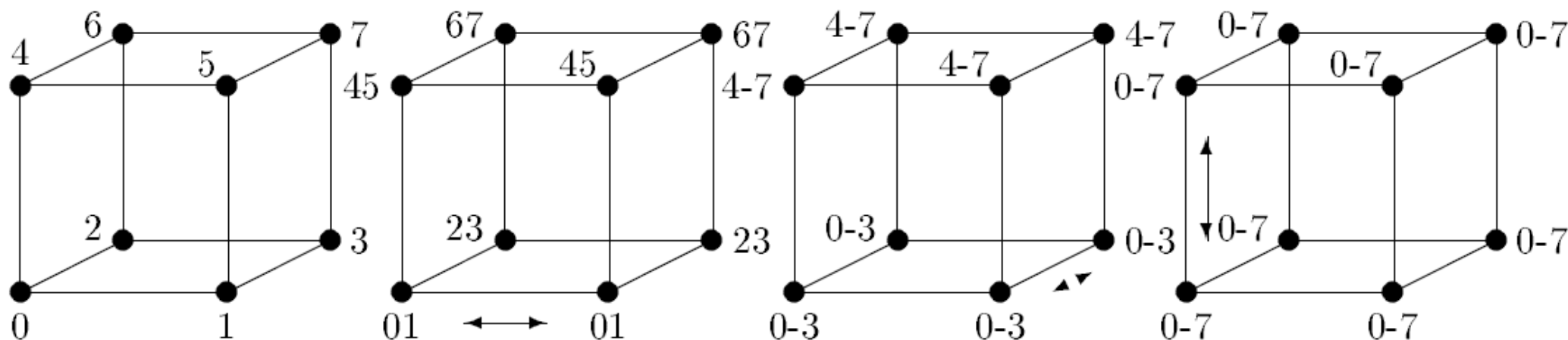
$$\Rightarrow \text{timpul optimal difuzare generala pe hipercub de dimensiune } d \text{ este chiar } \text{ceiling}((2^d - 1) / d)$$

Difuzare generala hipercub



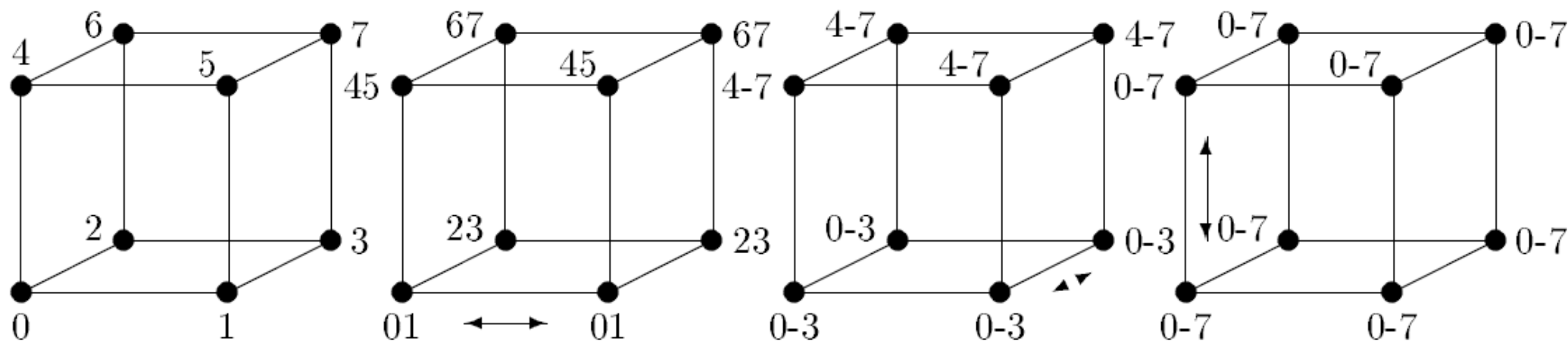
- la fiecare pas se comunica pe o dimensiune: $0, 1, \dots, d - 1$
- fiecare nod transmite vecinului sau din acea dimensiune toate mesajele
- la fiecare pas dimensiunea mesajelor se dubleaza

Difuzare generala hipercub



- dupa 2 pasi nodul 1 poseda mesajele: 0,1,2,3
- la pasul k se incheie difuzarea pe 2^{d-k+1} hipercuburi cu dimensiunile 0,1, ..., $k-1$
- se folosesc doar arcele dintr-o singura dimensiune

Difuzare generala hipercub



- se arata o paralelizare optima, cu folosirea tuturor cailor independente
- complexitate: $O\left(\frac{p}{\log(p)}\right) = O\left(\frac{p}{d}\right)$
- complexitate lant liniar: $O(p)$

Algoritmi optimali de scatter/gather, total exchange pe hipercub

- **single node scatter/gather**
 - algoritmul optimal necesita cel mult $\lceil (2^d - 1) / d \rceil$ pasi (multinode broadcast, difuzare generala)
 - $(2^d - 1)$ pachete trimise pe cele d linkuri incidente radacinii $\Rightarrow$ algoritmul optimal necesita cel putin $\lceil (2^d - 1) / d \rceil$ pasi
- **total exchange**
 - descompunem d -cubul in doua $d - 1$ subcuburi conectate prin 2^{d-1} linkuri
 $\Rightarrow (2^{d-1})^2$ pachete transmise pe linkuri
 $\Rightarrow$ cel putin 2^{d-1} unitati de timp
 - exista algoritm care atinge acest timp minim
 $\Rightarrow \Theta(2^d)$

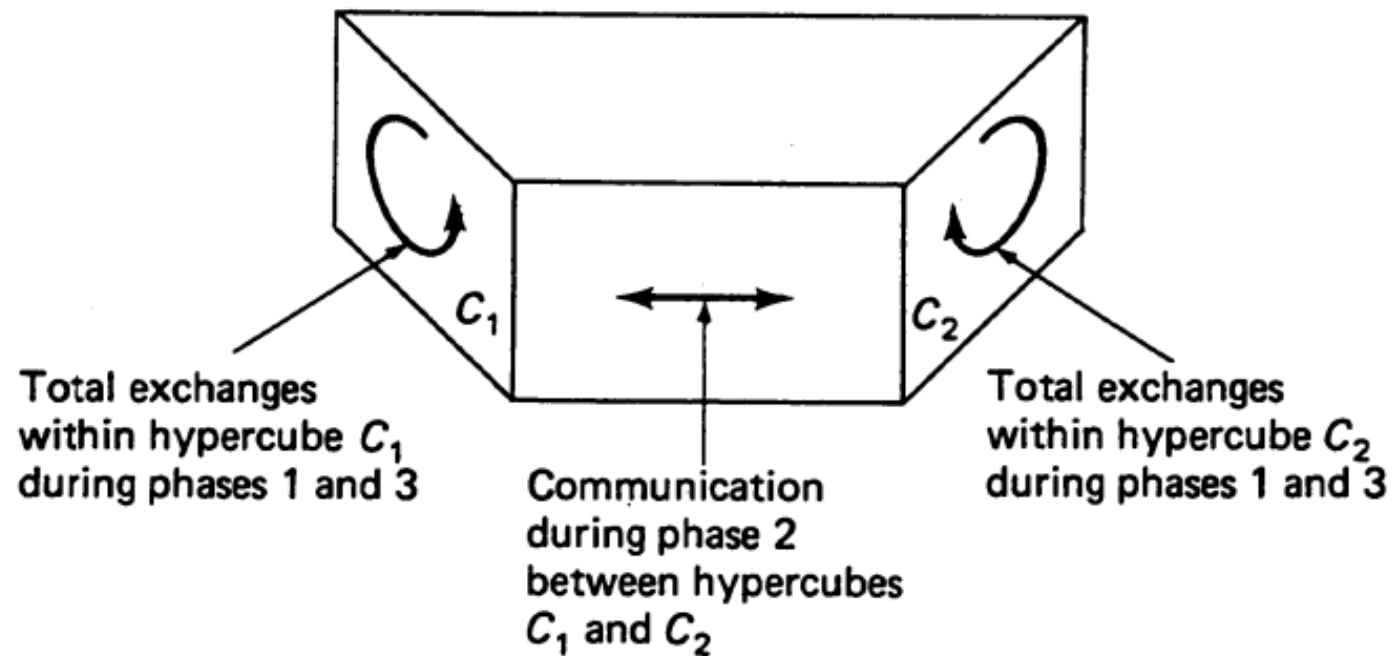
Algoritm optimal total exchange pe hipercub

- algoritm recursiv pt un d -cub care necesita timp in limita unui factor 2 de limita inferioara 2^{d-1}

$$T_d \leq 2^d - 1$$

- pt. $d = 1$, evident ($T_1 = 1$)
- pp. avem algoritm de schimb total pt d -cub cu timpul de mai sus
- construim algoritmul de total exchange pt $(d + 1) - cub$
 - descompunem hipercubul in doua d -cuburi C_1 si C_2
 - *faza 1*: schimb total in fiecare dintre cele doua d -cuburi
 - *faza 2*: fiecare nod trimite corespondentului sau din celalalt d -cub toate cele 2^d pachete destinate nodurilor din celalalt cub
 - *faza 3*: schimb total in fiecare d -cub a pachetelor primite in faza 2
 - fazele 1 & 2 ruleaza simultan

Algorithm optimal total exchange per hypercube



Analiza algoritm optimal de total exchange pe hipercub

- faza 1 dureaza T_d care din ipoteza inductiva este $< 2^d$ pt ca

$$T_d \leq 2^d - 1$$

$$\Rightarrow \text{faza 1 \& 2 dureaza } < 2^d$$

- faza 3 dureaza si ea T_d

$$\Rightarrow T_{d+1} \leq T_d + 2^d \leq 2^{d+1} - 1$$

- concluzie: schimbul total poate fi facut in $\Theta(2^d)$ pe un d -cub

Tempi comunicare ottimalea diverse topologii

Problem	Ring	Tree	Mesh	Hypercube
Single node broadcast (or single node accumulation)	$\Theta(p)$	$\Theta(\log p)$	$\Theta(p^{1/d})$	$\Theta(\log p)$
Single node scatter (or single node gather)	$\Theta(p)$	$\Theta(p)$	$\Theta(p)$	$\Theta(p/\log p)$
Multinode broadcast (or multinode accumulation)	$\Theta(p)$	$\Theta(p)$	$\Theta(p)$	$\Theta(p/\log p)$
Total exchange	$\Theta(p^2)$	$\Theta(p^2)$	$\Theta(p^{(d+1)/d})$	$\Theta(p)$

Comunicare folosind cel mult un link per node

- la orice moment de timp, orice procesor poate folosi cel mult o interconexiune incidenta
- daca $d(p)$ este gradul maxim al nodurilor oricarui tip de topologie (p = nr de procesoare/noduri) $\Rightarrow$
 - $d(p)$ = factor de slowdown vs. cazul cand nodul poate trimite simultan pe toate link-urile incidente
 - daca transmisia necesita 1 unitate de timp $\Rightarrow$ algoritmi anteriori de transmisie se pot emula in $d(p)$ unitati de timp
 - topologii cu $d(p)$ independent de p (inel, arbore binar echilibrat, grid simetric): timpul optimal identic cu algoritmi cu transmisie simultana pe toate arcele
 - pt. hipercub, se contorizeaza nr total de pachete trimise si se imparte la nr de noduri

Tempi comunicare optimala diverse topologii folosind cel mult un link per nod

Problem	Ring	Tree	Mesh	Hypercube
Single node broadcast (or single node accumulation)	$\Theta(p)$	$\Theta(\log p)$	$\Theta(p^{1/d})$	$\Theta(\log p)$
Single node scatter (or single node gather)	$\Theta(p)$	$\Theta(p)$	$\Theta(p)$	$\Theta(p)$
Multinode broadcast (or multinode accumulation)	$\Theta(p)$	$\Theta(p)$	$\Theta(p)$	$\Theta(p)$
Total exchange	$\Theta(p^2)$	$\Theta(p^2)$	$\Theta(p^{(d+1)/d})$	$\Theta(p \log p)$

Trade-off concurenta-comunicatie

- concurenta = masura a nr. de procesoare simultan active care colaboreaza la rularea unui algoritm paralel
- depinde de modalitatea de impartire a calculului global in subtaskuri si asignarea lor pe procesoare pt. executie paralela (granularitate)
- nevoia de eficienta implica o distributie relativ uniforma a taskurilor paralele pe procesoare, i.e. *load balancing*
- in general, nr de pachete schimbate pt coordonarea subtaskurilor paralele creste cu nr subtaskurilor
 - cu atat mai mult cu cat dimensiunea subtaskurilor e relativ uniforma
- deci, concurenta crescuta => nr de mesaje de sincronizare cresc => CP cresc => scaderea timpului de executie folosind tot mai multe procesoare necesita solutii pt tratarea penalizarii de comunicatie sporite
- din acest motiv, se poate limita superior dimensiunea problemelor de un anumit tip care in teorie, cel putin, s-ar putea rezolva cu un nr nelimitat de procesoare

Reducere CP prin pipelining-ul calculului cu comunicatia

- suprapunerea calcului cu comunicatia (eg, un procesor comunica rezultate partiale catre altul in timp ce alte rezultate sunt calculate)
- ex: metode iterative

$$x_i(t + 1) = f_i(x_1(t), \dots, x_p(t)) \quad i = 1, \dots, p$$

- x_i vector cu k elemente asignat procesorului i
- $n = pk$ dimensiunea problemei

Reducerea CP

- mai multe variabile asignate unui singur procesor $\Rightarrow$ variabilele deja actualizate intr-o iteratie pot fi trimise catre alte procesoare, in timp ce se asteapta pt actualizarea altora
- header-ul mesajelor inter-procesor e in general constant $\Rightarrow$ pachete mari reduc overhead-ul per bit $\Rightarrow$ transmisia mai multor variabile actualizate in acelasi mesaj reduce CP
- chiar in absenta pipelining-ului, CP se reduce daca dimensiunea k a vectorilor x_i creste
- pp. timp actualizare variabile $\Theta(nk)$ (i.e. f_i liniara fara structura sparse)
- actualizarea x_i pp. ca procesorul i trimite celorlalte procesoare cele k variabile pt. urmatoarea iteratie (sa zicem prin broadcast)
- daca se foloseste un lant liniar (array), timpul optim de comunicare pt broadcast multinod este $\Theta(n)$, daca pp. $\Theta(k)$ timpul de transmisie a k variabile pe un link

Reducerea CP (2)

- concluzie: cand creste k , penalizarea prin comunicare devine insignifianta

$$CP = T_{\text{comm}} \text{ per iteratie} / T_{\text{calcul}} \text{ per iteratie} = \Theta(1/k)$$

Obs: 1. raportul e independent de dimensiunea problemei n , depinde doar de k , dimensiunea problemei de calcul per iteratie de procesor

2. accelerarea poate creste cu dimensiunea problemei n

Cresterea accelerarii

- pp timpul serial per iteratie este $T_1 = \Theta(n^2)$ (cf ipoteza anterioara era $\Theta(nk)$, iar pt. o masina seriala $p = 1$ si $k = n$)
- speed-up folosind $k = n / p$ variabile si un lant liniar cu p procesoare

$$S_p(n) = \frac{\Theta(n^2)}{\Theta(n) + \Theta(nk)} = \Theta(p)$$

unde: $\Theta(n)$ = timp de comunicare

$\Theta(nk)$ = timp de calcul

Concluzii

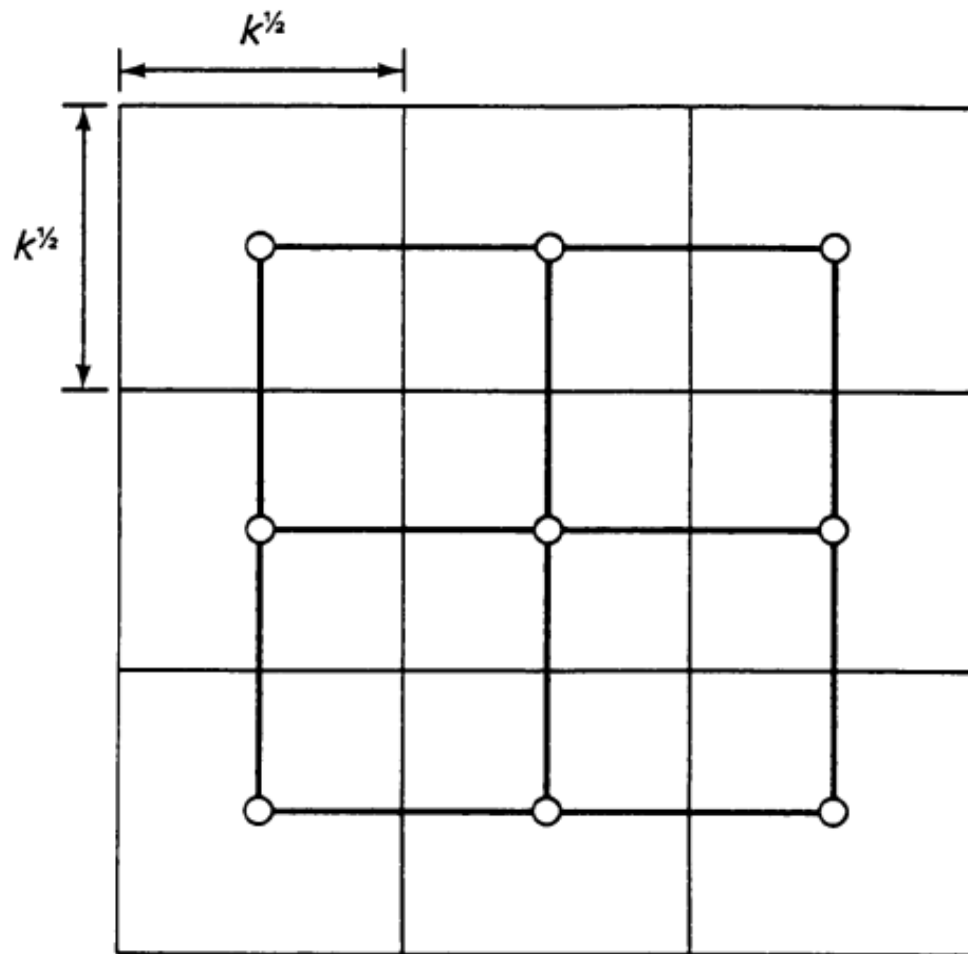
- (pt. metode iterative ca in exemplul nostru)
- penalizarea impusa de comunicare nu impiedica scalarea problemei cu nr de procesoare cand dimensiunea problemei e mare, chiar daca se foloseste cea mai putin performanta topologie de interconectare a procesoarelor
- cresterea dimensiunii n a problemei $\Rightarrow$ cresterea proportionala a nr de procesoare p a.i. nr de variabile k per procesor sa ramana relativ constant ($\Rightarrow$ comunicatie practic insignifianta)
- Obs: daca folosim un hipercub in loc de lant liniar, timpul optimal de broadcast multinod este $\Theta(pk/\log p)$, ceea ce reduce CP de la $\Theta(1/k)$ la $\Theta\left(\frac{1}{k \log p}\right)$

$\Rightarrow$ daca n creste cu un anumit factor, nr de procesoare p din hipercub poate fi crescut cu un factor si mai mare la un CP relativ nesemnificativ

$\Rightarrow S_p \text{ hipercub} > S_p \text{ lant liniar}$

Concluzii (contd.)

- analiza precedenta nu pp o structura speciala a iteratiei, ci doar ca o singura actualizare de variabile costa $\Theta(n)$ (sau $\Theta(pk)$)
- sunt multe alte cazuri cu structura speciala, in care $T_{\text{comm}} / T_{\text{calcul}}$ e mic pt valori mari ale lui k
- ex: discretizarea spatiilor fizice bidimensionale (doar variabilele vecine interactioneaza), fiecare dreptunghi contine k variabile
- nr. variabile comunicate de un procesor $\Theta(\sqrt{k})$ cu fiecare dintre vecini
- timp de comunicare per iteratie pe grid/hipercub $\Theta(\sqrt{k})$
- timp constant pt actualizarea fiecarei variabile
- timp paralel de calcul per iteratie $\Theta(k)$
- $T_{\text{comm}} / T_{\text{calcul}} = \Theta(\frac{1}{\sqrt{k}})$



Concluzii (contd.)

- dincolo de metode iterative, selectia potrivita a dimensiunii taskurilor alocate fiecarui procesor poate minimiza efectele comunicatiei
- daca dimensiunea problemei creste fara limita, si acceleratia poate creste corespunzator daca se alege algoritmul paralel potrivit
 - i.e., nu exista limita a priori impusa de cerintele de comunicatie asupra acceleratiei care se poate atinge

Algoritmi paraleli

Curs 6

Vlad Olaru

vlad.olaru@fmi.unibuc.ro

Calculatoare si Tehnologia Informatiei

Universitatea din Bucuresti

Operatii elementare de calcul matricial

- particularizam aspectele generale discutate anterior
- pastram ipotezele de baza asupra constrangerilor de comunicatie (absenta asteptarii in cozi, comunicatie simultana pe toate link-urile incidente unui nod)
- vizam calcule de tipul:

$$x(t + 1) = Ax(t) + b \quad \text{pt. } t = 0, 1, \dots$$

- produs scalar, inmultire matrice-vector, matrice-matrice pt metode Jacobi/Gauss-Seidel
- calculul minimului unui set de numere, shortest paths, dynamic programming
- caracteristica importanta: dupa iteratie trebuie stocata valoarea vectorului $x(t + 1)$ in procesoarele corespunzatoare
- pp. ca timpul initial de stocare pt A, b si $x(0)$ neglijabil, i.e. $O(1)$, daca pp ca nr de iteratii executate e mare

Operatii elementare de calcul matriceal

- **produs scalar**
- inmultire matrice-vector
- inmultire matrice-matrice
- puterea unei matrici

Produs scalar

- fie $a, b \in R^n$ doi vectori de dimensiune n
- calculam: $c = \sum_i a_i b_i$
- retea cu p procesoare ($n \geq p$); presupunem $k = \frac{n}{p}$
- P_i stocheaza (a_j, b_j) cu $j = (i - 1)k + 1, \dots, ik$
- P_i va calcula $c_i = \sum_{j=(i-1)k+1}^{ik} a_j b_j$
- se acumuleaza rezultatul final intr-un nod arbitrar al unui arbore de acoperire/spanning tree (single node accumulation)
 - ulterior, nodul poate transmite valoarea catre celelalte noduri prin single node broadcast

Produs scalar pe lant liniar

- fie $a, b \in R^n$ doi vectori de dimensiune n
- calculam: $c = \sum_i a_i b_i$
- timp executie adunare/inmultire = α
- timp transmisie produs partial = β
- alegere optimala pt. root node pe lant liniar e nodul din mijloc, la distanta maxima $\text{floor}(p/2)$ de orice alt nod (procesor)
- pe topologia de lant liniar, timpul de calcul la root este

$$O\left(k\alpha + \frac{(\alpha + \beta)p}{2}\right) = O\left(\frac{n}{p}\alpha + \frac{(\alpha + \beta)p}{2}\right)$$

Produs scalar pe lant liniar

- fie $a, b \in R^n$ doi vectori de dimensiune n
- calculam: $c = \sum_i a_i b_i$
- functia de p este convexa, deci se poate minimiza dupa p (ignoram p intreg):

$$p \approx \left(\frac{2\alpha n}{\alpha + \beta} \right)^{\frac{1}{2}}$$

$$\text{Complexitate in timp optima: } O\left(\frac{n}{p}\alpha + \frac{(\alpha+\beta)p}{2}\right) = \mathbf{o}(n^{\frac{1}{2}})$$

Obs: - p optimal e semnificativ mai mic decat $\max n$

- daca $\beta > (2n - 1)\alpha \Rightarrow p = 1$, i.e. utilizarea unui singur procesor e optimala
pt. a evita costurile mari ale comunicarii interprocesor $\Rightarrow$ trade-off concurenta
vs comunicare

Produs scalar pe topologie hipercub

- fie $a, b \in R^n$ doi vectori de dimensiune n
- calculam: $c = \sum_i a_i b_i$
- timp executie adunare/inmultire = α
- timp transmisie produs partial = β
- pe topologia hipercub cu arborele de acoperire anterior:

$$O(k\alpha + (\alpha + \beta) \log p) = O\left(\frac{n}{p}\alpha + (\alpha + \beta) \log p\right)$$

Produs scalar pe hipercub

- fie $a, b \in R^n$ doi vectori de dimensiune n
- calculam: $c = \sum_i a_i b_i$
- functia de p este convexa, deci se poate minimiza dupa p :

$$p \approx \frac{\alpha n}{\alpha + \beta}$$

Complexitate in timp optima: $O\left(\frac{n}{p}\alpha + (\alpha + \beta)\log p\right) = O(\log(n))$

- hipercubul este mai eficient: cresterea nr. proc. asigura complexitate mai buna!
- Obs: *nr optimal de procesoare e mult mai mare si timpul de rezolvare a problemei este mai mic fata de lantul liniar, ceea ce denota comunicare mai eficienta pe hipercub*
 - diferenta de performanta intre cele 2 topologii se micsoreaza daca se fac mai multe produse scalare simultan (acumulare multinod)
 - pe lantul liniar timpul de comunicare nu creste semnificativ, pe hipercub da

Operatii elementare de calcul matriceal

- produs scalar
- **inmultire matrice-vector**
- inmultire matrice-matrice
- puterea unei matrici

Inmultire matrice-vector

- fie $A \in R^{n \times n}, x \in R^n$
- calculam: $c = Ax$

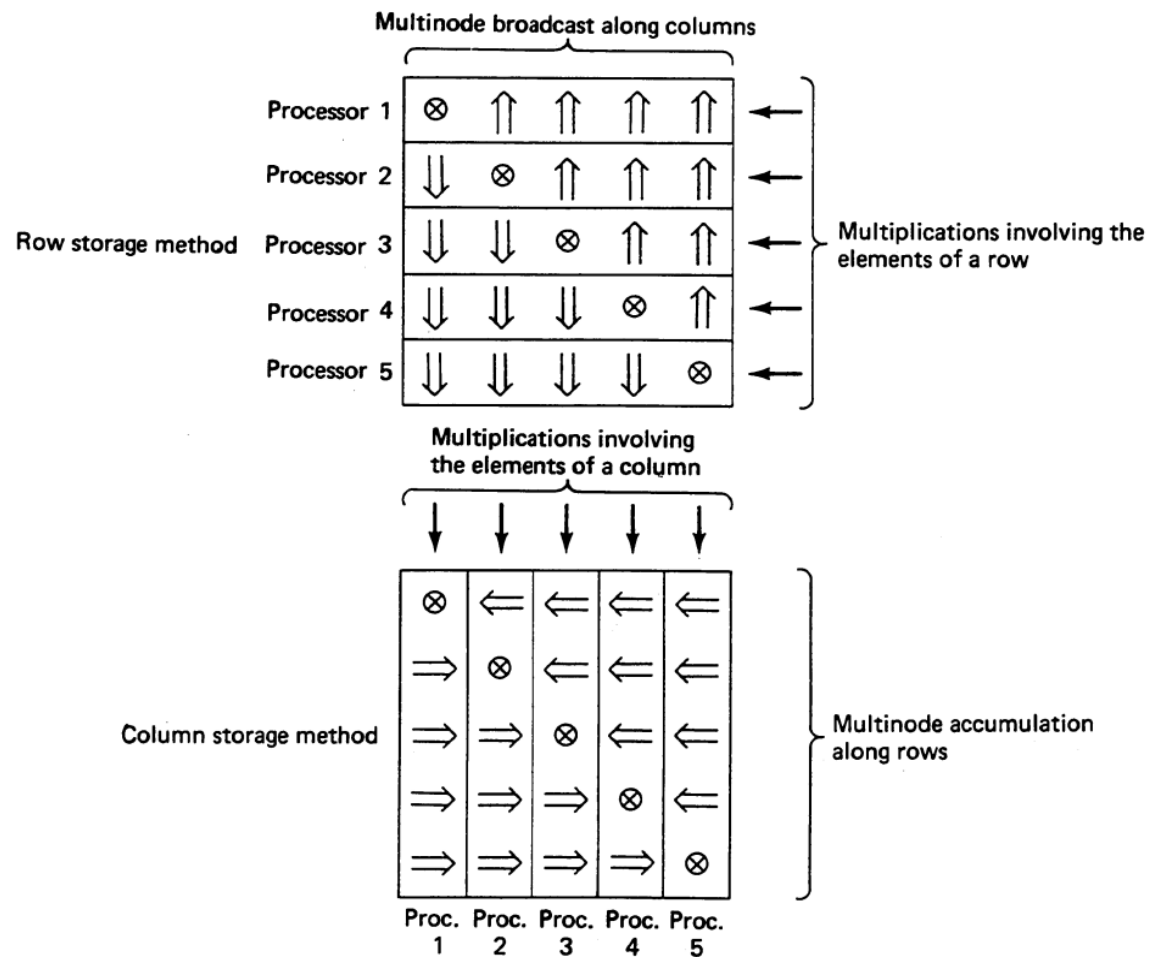
Memorare pe linii

- retea cu p procesoare ($n \geq p$); presupunem $k = \frac{n}{p}$
- P_i stocheaza x si k linii A_j cu $j = (i - 1)k + 1, \dots, ik$
- P_i va calcula k coordonate $c_i = A_i x$
- c_i sunt trimise catre toate celelalte procesoare $\Rightarrow$ o problema de difuzare generala (broadcast multinod)
- se acumuleaza rezultatul final intr-un nod arbitrar
- Obs: pt memorare pe coloane, P_i va calcula k termeni din suma care defineste fiecare coordonata a lui $Ax \Rightarrow$ acumulare multinod a acestor termeni $((i - 1)k + 1, \dots, ik)$ la nodul i

Obs. inmultire matrice-vector

- convertirea unei matrici din forma memorata pe linii in cea memorata pe coloane implica un schimb total (total exchange)
- pt matrici dense, cele doua metode necesita cam acelasi timp (v. slide urmator), indiferent de topologia de interconectare
 - ambele metode pp un nr egal de inmultiri + acumulare/broadcast multinod care necesita cam acelasi timp pe orice topologie
 - valabil si pt. matrici sparse *simetrice* !
- pt matrici sparse nesimetrice, dificil de ales, depinde de
 - structura matricii
 - topologia de interconectare a procesoarelor
- metoda orientata pe coloane stocheaza un subvector k -dimensional al lui x , metoda orientata pe linii stocheaza intregul vector x
=> stocarea matricii pe coloane e mai eficienta dpdv al consumului de memorie

Dualitete metode



Inmultire matrice-vector pe lant liniar

- fie $A \in R^{n \times n}, x \in R^n$
- calculam: $c = Ax$

Memorare pe linii

- retea cu p procesoare ($n \geq p$); presupunem $k = \frac{n}{p}$
- P_i stocheaza k linii A_j cu $j = (i-1)k + 1, \dots, ik$
- P_i va calcula $c_i = A_i x$
- timp executie adunare/inmultire = α
- timp transmisie k numere pe un link intre vecini = $\beta + k\gamma$, unde $\beta, \gamma > 0$
- pe topologia de lant liniar:

$$O(\alpha kn + (p-1)(\beta + k\gamma)) = O\left(\frac{n^2}{p}\alpha + (p-1)\left(\beta + \frac{n\gamma}{p}\right)\right)$$

Inmultire matrice-vector pe lant liniar

- fie $A \in R^{n \times n}$, $x \in R^n$
- calculam: $c = Ax$

Memorare pe linii

- retea cu p procesoare ($n \geq p$); presupunem $k = \frac{n}{p}$
- P_i stocheaza liniile A_j cu $j = (i - 1)k + 1, \dots, ik$
- P_i va calcula $c_i = A_i x$
- functia de p este convexa, deci se poate minimiza dupa p :

$$p \approx n \left(\frac{\alpha - \gamma/n}{\beta} \right)^{\frac{1}{2}}$$

Complexitate optima: $O(n)$

Obs. inmultire matrice-vector pe lant liniar

- pt. n suficient de mare

$$\frac{p}{n} \approx \left(\frac{\alpha - \gamma/n}{\beta} \right)^{\frac{1}{2}}, \text{ adica}$$

$$k = \frac{n}{p} \approx (\beta/\alpha)^{1/2} \text{ nr de linii per procesor este constant}$$

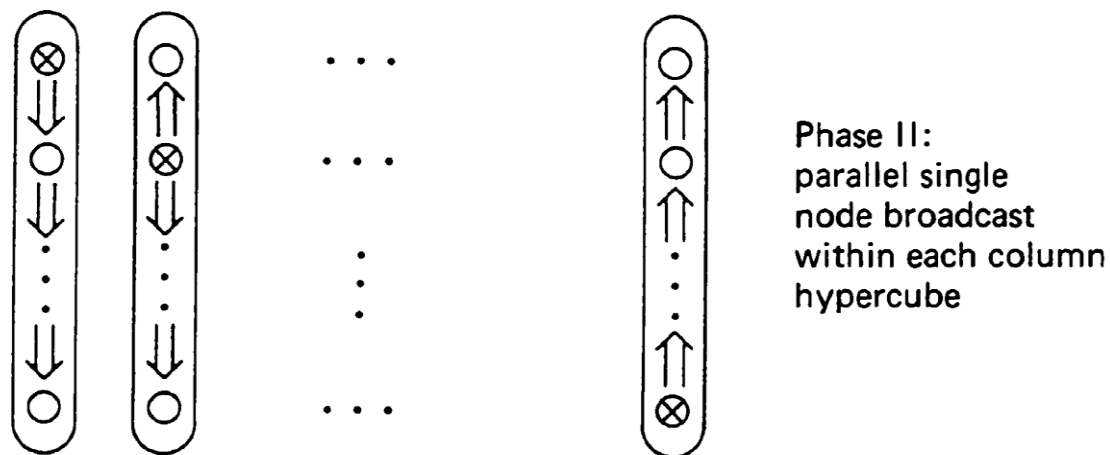
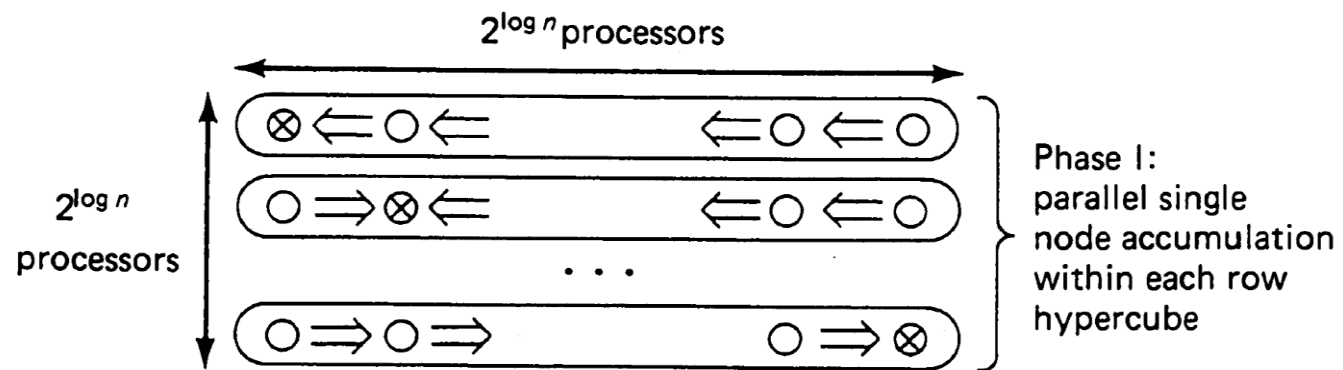
- timpul total optimal creste liniar cu n , i.e acelasi ordin de crestere ca si atunci cand neglijam costul comunicatiei !
- pt lant liniar mapat pe topologii mai performante (hipercub): timpul de rulare nu-l poate depasi pe cel de pe lantul liniar
- pe de alta parte, timpul nu poate scadea nici sub timpul solutiei care neglijeaza costurile comunicatiei (i.e., comunicatie instantanee)

=> pt. orice topologie de interes practic, produsul $\mathbf{Ax}$ folosind un nr optimal de procesoare p creste liniar cu n , **cand** $p \leq n$

Inmultire matrice-vector pe hipercub cu $p \geq n$ procesoare

- folosim tot memorare pe linii, dar $p \geq n$
- mai exact, pt. $p = n^2$ procesoare si comunicatie instantanee, timpul de executie devine $O(\log(n))$
- pe lant liniar e imposibil de atins aceasta limita pt ca produsul scalar (mai simplu decat inmultirea matrice-vector) este $O(n^{\frac{1}{2}})$
- solutie: matrice $n \times n$, mapata pe hipercub ca la grid (fiecare linie si coloana sunt noduri plasate intr-un hipercub de dimensiune n)
- initial, procesorul (i, j) , contine elementul corespunzator din A si componenta j a lui x
 - la final, contine componenta j a produsului Ax
- algoritm in doua faze, fiecare de $O(\log(n))$
 - acumulare pe fiecare linie, nodul diagonal i calculeaza coordonata i a produsului Ax
 - nodul diagonal j difuzeaza procesoarelor de pe coloana j coordonata j a produsului Ax acumulata in pasul anterior

Inmultire matrice-vector pe hipercub



Inmultire de matrici

- fie $A, B \in R^{n \times n}$
- calculam: $C = AB$

$$C_{ij} = \sum_k A_{ik} B_{kj} = A_i B^j$$

- pentru $i = 0:n-1$
 - pentru $j = 0:n-1$
 - pentru $k = 0:n-1$
 - $C_{ij} = C_{ij} + A_{ik} B_{kj}$
- Complexitate seriala: 1 procesor $\rightarrow O(n^3)$

Inmultire matrice-matrice pe grid bidimensional

- fie $A, B \in R^{n \times n}$
- **calculam:** $C = AB$
- folosim un grid de $p = n^2$ procesoare
- $A(i,j)$ si $B(i,j)$ stocate in procesorul (i,j) , iar la final se stocheaza si $C(i,j)$
- algoritm in trei faze fiecare de complexitate $O(n)$
 - global, aceeaasi complexitate ca la algoritmul care foloseste n^2 procesoare si comunicare instantanee !

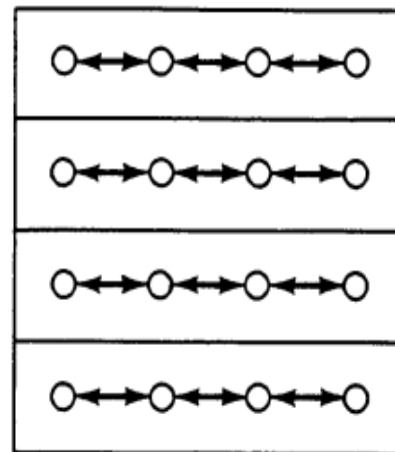
Inmultire matrice-matrice pe grid bidimensional

- Faza 1: fiecare procesor (i,j) trimite $A(i,j)$ procesoarelor de pe linia $i \Rightarrow n$ broadcast-uri multinod pe fiecare linie, adica $O(n)$
- Faza 2: fiecare procesor (i,j) trimite $B(i,j)$ procesoarelor de pe coloana $j \Rightarrow n$ broadcast-uri multinod pe fiecare coloana, adica $O(n)$
- dupa 1 & 2, procesorul (i,j) stocheaza valorile $A(i,m)$ si $B(m,j)$ si poate calcula

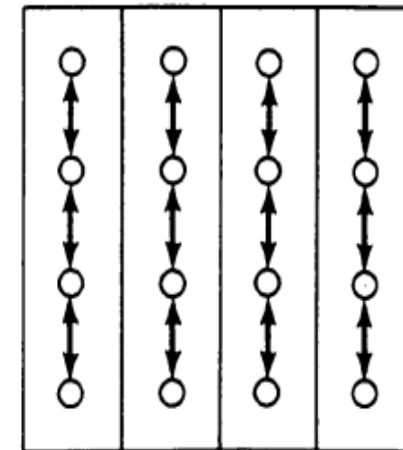
$$\sum_{m=1}^n a_{im} b_{mj} \text{ in paralel in } O(n)$$

- complexitate totala folosind n^2 procesoare:
 $O(n)$

- Obs: 1 & 2 pot rula in paralel daca e posibila comunicarea simultana pe toate link-urile/arcele incidente unui nod



Phase 1



Phase 2

Calculul puterii unei matrici

- fie $A \in R^{n \times n}$
- calculam: A^k
- folosim un grid patrat de $p = n^2$ procesoare
- $A(i,j)$ stocat in procesorul (i,j) , iar la final se stocheaza elemental (i,j) al A^k
- solutie: ridicare succesiva la patrat daca $k = 2^m$

$$A^k = A^2 \times A^2 \dots$$

- pt $k \neq 2^m$, calculam ca la cursurile trecute (calculam succesiv A^2, A^4 , si inmultim intre ele rezultatele intermediare pt a obtine urmatoarea putere a matricii)
- complexitate $O(n \log k)$

Inmultire de matrici cu n^3 procesoare

- fie $A, B \in R^{n \times n}$
 - **calculam:** $C = AB$
 - folosim $p = n^3$ procesoare
 - inmultire matrice-vector in $O(\log(n))$ pe hipercub cu n^2 procesoare
 - Obs: coloanele lui C contin produse matrice-vector de tip Ab_i , unde b_i sunt coloanele matricii B
- => aceste produse se pot calcula in $O(\log(n))$ cu n^3 procesoare

Inmultire de matrici cu n^3 procesoare

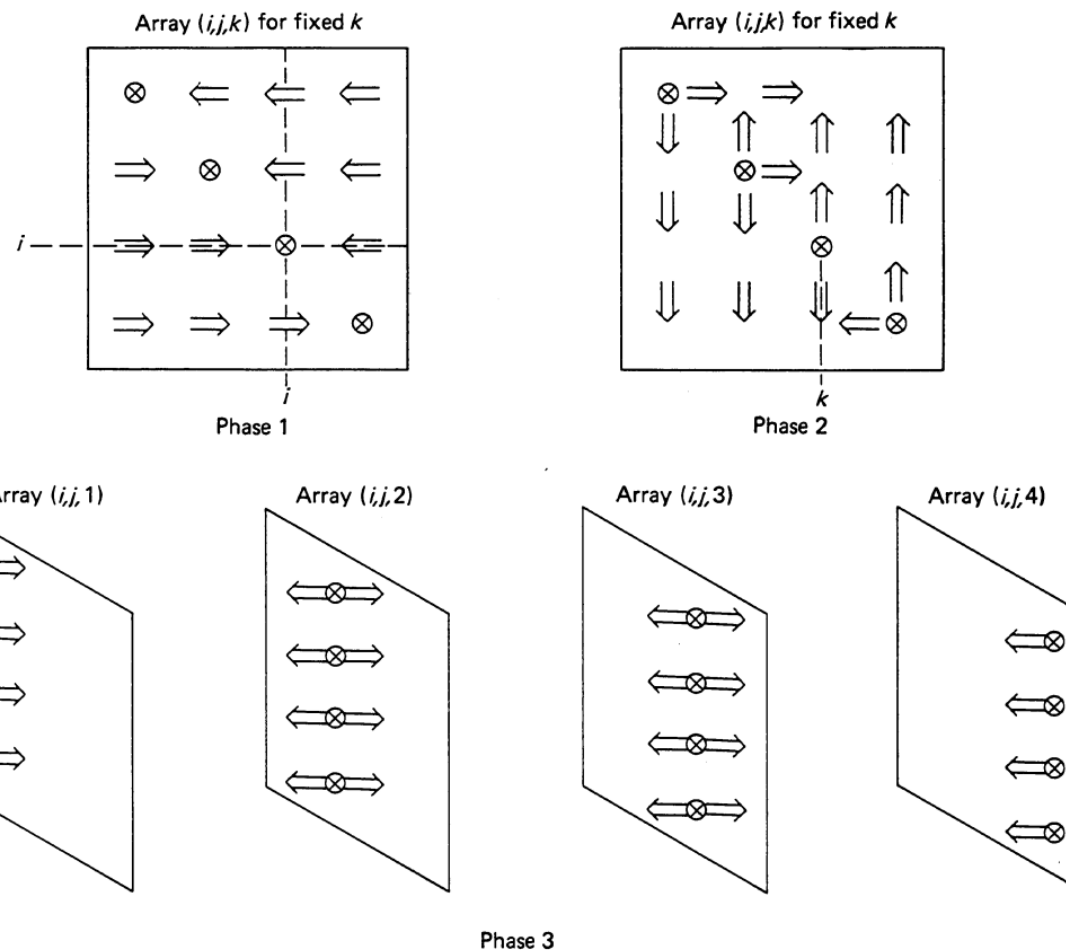
- pp. $n = 2^d$ si un hipercub de n^3 procesoare aranjate intr-un grid $n \times n \times n$
- pp procesorul (i, j, k) contine a_{ij} si b_{jk} si la final c_{ij} si c_{jk} unde $c_{ik} = \sum_{j=1}^n a_{ij}b_{jk}$
- algoritm in 3 faze de durata $O(\log(n))$ fiecare

1: c_{ik} este acumulat in procesorul (i, i, k) din gridul (i, j, k) , $j=1, \dots, n$

2: fiecare (i, i, k) trimite prin bcast c_{ik} in gridul (j, i, k) , $j=1, \dots, n$ ca si procesorului (i, k, k)

3: procesorul (i, j, j) trimite c_{ij} prin bcast in gridul (i, j, k) , $k=1, \dots, n$

- complexitate hipercub (n^3 procesoare): $O(\log(n))$



Calculul puterii unei matrici cu n^3 procesoare

- fie $A \in R^{n \times n}$
- **calculam:** A^k
- folosim un hipercub cu $p = n^3$ procesoare
- $O(\log k)$ multiplicari ale puterilor lui A
- daca folosim algoritmul anterior de inmultire de matrici $\Rightarrow O(\log n \log k)$
- Obs: aceeasi performanta ca la cursurile trecute cand consideram comunicatia instantanee !

Tempi de rulare optimala operatii matriciale

Problem	Time	Corresponding Topology
Inner Product	$O(\log n)$	Hypercube w/ $p = n$ processors
Matrix-Vector Multiplication	$O(n)$	Linear Array w/ $p = n$ processors
Matrix-Vector Multiplication	$O(\log n)$	Hypercube w/ $p = n^2$ processors
Matrix-Matrix Multiplication	$O(n)$	Mesh w/ $p = n^2$ processors
kth Power of a Matrix	$O(n \log k)$	Mesh w/ $p = n^2$ processors
Matrix-Matrix Multiplication	$O(\log n)$	Hypercube w/ $p = n^3$ processors
kth Power of a Matrix	$O((\log n)(\log k))$	Hypercube w/ $p = n^3$ processors

Concluzii operatii matriciale

- folosirea unor retele de interconectare potrivite (eg, hipercub) permite calculul unor operatii matriciale de baza in acelasi ordin de timp ca si atunci cand comunicatiile sunt instantanee
- nu inseamna ca CP e neglijabila !
- inseamna ca T_{comm} creste cu dimensiunea problemei cu o rata care nu e mai mare decat rata de crestere a T_{comp}

Problema sincronizarii in algoritmi paraleli

- coordonarea activitatilor se face adesea in *faze*
- in fiecare faza, procesorul executa calcule care depind de rezultate calculate in faze anterioare
- in general insa, nu exista sincronizare in timp a executiei procesoarelor in cadrul aceleiasi faze
- procesoarele interactioneaza in general la sfarsitul unei faze
- asemenea algoritmi se numesc *sincroni*, spre deosebire de cei *asincroni* care nu folosesc notiunea de faze si in care coordonarea procesoarelor e mult mai putin stricta
- cadrul general de discutie ramane cel al sistemelor message-passing
 - unele idei se pot adapta sistemelor cu memorie partajata

Algoritmi paraleli sincroni

- fie o secventa de operatii impartita in segmente numite *faze*
- calculele din fiecare faza sunt impartite intre procesoarele din sistemul de calcul
- in faza t , procesorul i face calcule dependente de problema, folosind si informatie primita de la alte procesoare in fazele anterioare $1, 2, \dots, t-1$
- apoi procesorul i trimite informatii catre un subset de procesoare $P(i, t)$ si procesul se repeta la faza $t+1$
- presupunere implicita: procesoarele ruleaza independent unele de altele in cadrul aceleiasi faze iar sincronizarea lor relativa e irelevanta

Exemplu de algoritm sincron

- metoda iterativa

$$x_i(t+1) = f_i(x_1(t), \dots, x_n(t)) \quad i = 1, \dots, n \quad \text{unde } x_i \text{ e actualizat de procesorul } i$$

- faza asociata cu timpul t
- procesorul i actualizeaza variabila x_i si trimite valoarea actualizata catre toate procesoarele j ale caror calcule depind de $x_i \Rightarrow$

$$P(i, t) = \{j \mid (i, j) \text{ arc in graful de dependenta}\}$$

- un asemenea algoritm distribuit s.n. algoritm *sincron*
 - echivalent dpdv matematic cu un algoritm cu ceas global
 - toate procesoarele pornesc simultan o faza
 - la sfarsitul fazei mesajele sunt receptionate simultan de catre toate procesoarele
- pt. a implementa un asemenea algoritm intr-un sistem de calcul inerent asincron e nevoie de *mecanisme de sincronizare*
 - algoritm prin care fiecare procesor poate detecta sfarsitul fiecărei faze
 - sincronizare globala vs. locala

Sincronizare globala

- fiecare procesor trebuie sa poata detecta cand s-au primit toate mesajele trimise in cadrul unei faze
 - abia apoi poate porni calculul fazei urmatoare
 - metoda simpla: timeout
 - nu exista ceas global, doar ceasuri locale procesoarelor
 - pp. se cunoaste limita $\max T_p$ a timpului de executie a unei faze pe orice procesor i si pt a trimite mesajele sale catre celelalte procesoare
 - pp. T_f = intervalul de timp in care procesoarele au inceput prima faza
- ⇒ sincronizarea se produce efectiv daca procesorul i porneste faza k dupa $k(T_f + T_p)$ unitati de timp de la pornirea primei faze
- ⇒ problema: limitele T_p si T_f pot fi fie prea conservatoare fie necunoscute

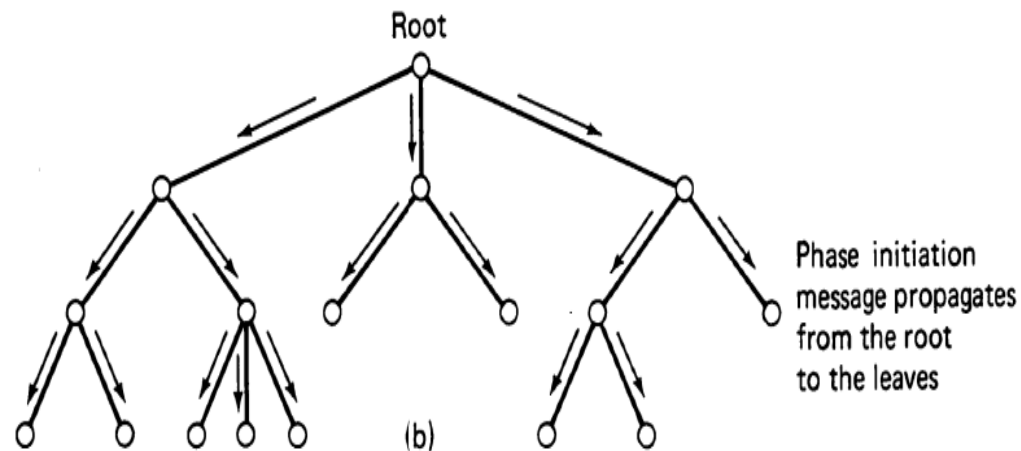
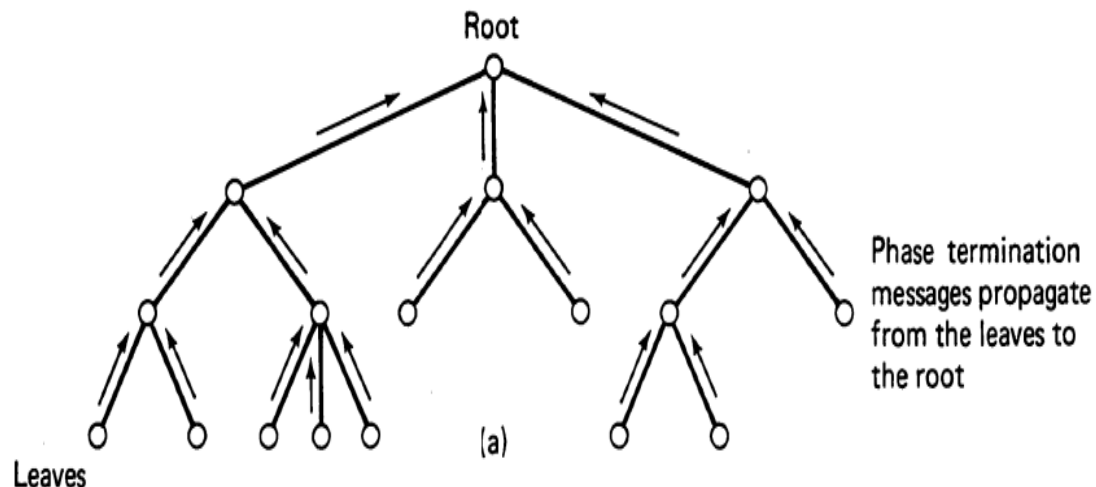
Sincronizare globala cu mesaje de terminare a fazei

- fiecare procesor trimite tuturor celorlalte procesoare un *mesaj de terminare a fazei* cand stie ca toate mesajele sale pt o anumita faza au fost receptionate
- pp implicit ca receptia fiecarui mesaj este notificata transmitatorului (ACK)
 - astfel, fiecare procesor stie ca mesajele sale au fost receptionate si poate initia mesajul de terminare a fazei
- cand un procesor a trimis mesajul de terminare a fazei si a primit toate mesajele de terminare a fazelor celorlalte procesoare initiaza noua faza
- in sisteme cu memorie partajata, acest mecanism e implementat cu variabile speciale vizibile tuturor procesoarelor: spinlocks, semafoare, monitoare, etc
 - valorile acestor variabile indica procesoarelor ca faza s-a terminat si se poate continua cu urmatoarea faza

Sincronizare globala cu mesaje de terminare a fazei in sisteme message-passing

- mecanismul se implementeaza folosind un arbore de acoperire (spanning tree) al retelei de interconectare a procesoarelor
- mesajele de terminare a fazei se strang la radacina arborelui de acoperire, pornind de la frunze
- cand radacina a primit toate mesajele, initiaza un broadcast catre toate procesoarele din arborele de acoperire cu un mesaj de incepere a noii faze

=> complexitate $\Theta(r)$, unde r este diametrul retelei



Sincronizare locala

- daca un procesor stie ce mesaje trebuie sa primeasca in fiecare faza, dupa ce le-a primit poate porni noua faza
- procesorul j incepe faza $t+1$ cand a primit toate mesajele din faza t trimise de procesoarele i din multimea $\{i \mid j \in P(i, t)\}$
- obs: procesorul nu trebuie sa stie daca mesajele trimise in faza t au fost receptionate (nici macar pt mesajele sale)
 - nu e nevoie sa se astepte receptia lor si notificarea (confirmarea) corespunzatoare
- cand fiecare procesor i cunoaste $\{i \mid j \in P(i, t)\}$, sincronizarea locala nu genereaza penalizari de comunicare mai mari decat sincronizarea globala
 - cel mai adesea, penalizarea comunicarii e mai mica
 - e.g., cand timpul de transmisie al mesajelor e comparabil cu timpul de calcul al fazei, pt. ca sincronizarea globala are nevoie de mesaje ACK
 - chiar fara mesaje ACK, diferenta dintre timpii de executie pt cele doua mecanisme de sincronizare creste cu nr de faze, datorita variabilitatii timpilor de calcul si transmisie de mesaje pt fiecare procesor in fiecare faza

Algoritmi paraleli asincroni

- penalizarea de comunicare (si implicit timpul total de executie) se poate reduce substantial folosind implementari asincrone
- dat fiind un algoritm distribuit, pt fiecare procesor exista distinct
 - timpi asociati cu executia operatiilor
 - timpi pentru transmisia de mesaje catre alte procesoare
 - timpi pentru receptia de mesaje de la alte procesoare
- intr-un algoritm sincron, toti acesti timpi pot fi considerati in sens matematic echivalenti unui algoritm cu acesti timpi fixati *a priori*
- cand acesti timpi (si ordinea lor) pot varia semnificativ intre doua rulari ale algoritmului paralel avem de-a face cu un algoritm *asincron*
- in cazul sistemelor distribuite, algoritmul paralel/distribuit poate fi vazut ca o colectie de algoritmi locali care se executa pe procesoare diferite si schimba ocazional date

Algoritmi paraleli asincroni locali

- intr-un algoritm sincron, algoritmii locali
 - fie se sincronizeaza global folosind un ceas global
 - fie asteapta momente predeterminate in timp pt ca datele sa devina disponibile
 - fara a se cunoaste dinainte timpul de executie al operatiilor locale
- intr-un algoritm asincron, algoritmii locali nu asteapta niciun fel de date sa devina disponibile, ci continua sa calculeze avand la dispozitie doar datele de la un moment dat
- caz extrem: algoritmii asincroni tolereaza schimbari in datele problemei sau in arhitectura sistemului distribuit
 - ex: retele de calculatoare, unde link-uri si noduri se pot defecta si reveni in executie
 - retelele mobile isi schimba dinamic topologia
 - chiar daca aceste schimbari nu sunt frecvente, timpi de executie mari ai algoritmilor pot determina o probabilitate neneglijabila de aparitie a schimbarilor de topologie

Algoritmi asincroni locali (cont.)

- dificultati asociate tolerantei la defecte si schimbarilor de arhitectura
 - toate nodurile trebuie notificate asupra defectelor si reparatiilor, pt ca aceste aspect influenteaza algoritmul (ex: rutare)
 - dificil pt ca informatia despre defecte este transmisa pe link-uri care se pot defecta si ele
 - algoritmi toleranti la asemenea fenomene trebuie sa fie adaptivi (sau minimal sa poata abandona executia si sa reporneasca)
 - netrivial, pt ca defecte suplimentare pot apare in timpul repornirii
 - algoritmi asincroni se pot reporni mai usor pt ca permit mai multa flexibilitate in alegerea conditiilor initiale
- Q: pot algoritmi asincroni reduce penalizarea comunicarii si pe cale de consecinta sa imbunatateasca timpii de rulare ai algoritmilor paraleli?

Metode iterative asincrone

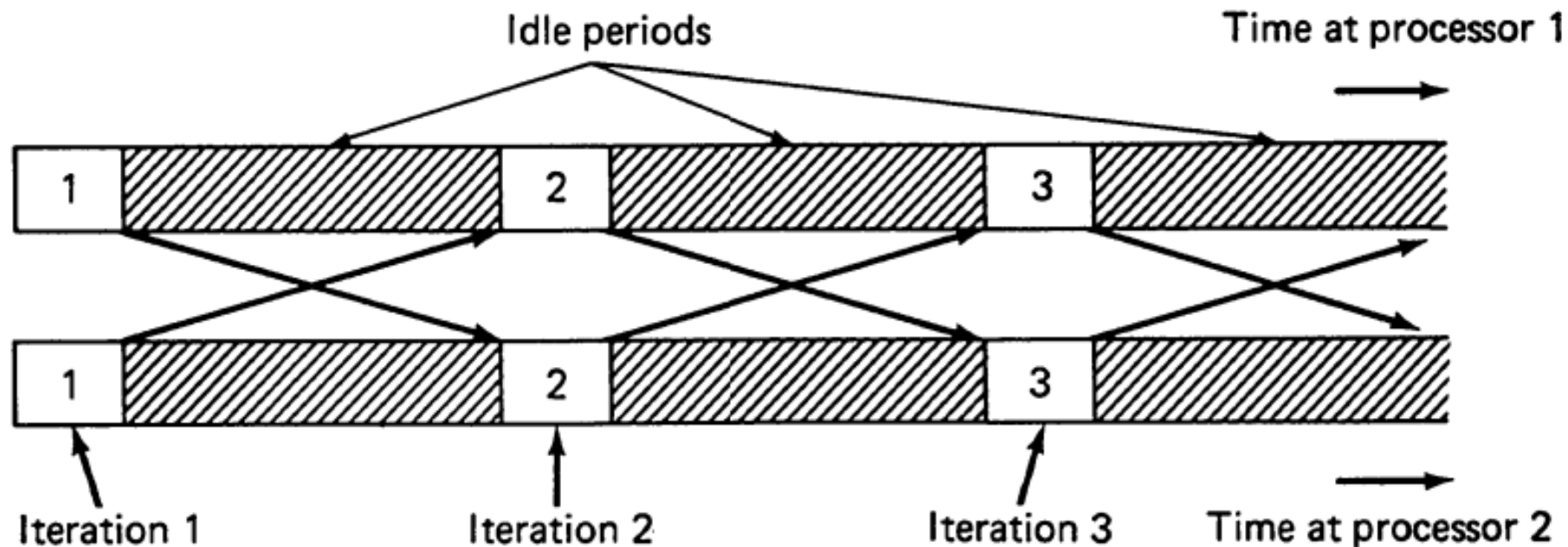
- fie un sistem cu n procesoare si o iteratie de punct fix n -dimensionala

$$x_i = f_i(x_1, x_2, \dots, x_n) \quad i = 1, 2, \dots, n$$

- procesorul i actualizeaza variabila x_i
- o implementare sincrona pp ca procesorul i executa actualizarea k doar dupa ce primeste rezultatele celor $(k - 1)$ actualizari de la procesoarele din graful de dependente
- aceasta ineficienta inerenta solutiei alese se mai numeste si *penalizare de sincronizare*
- depinde de doi factori: viteza canalelor de comunicatie folosite si respectiv a procesoarelor implicate in calcul

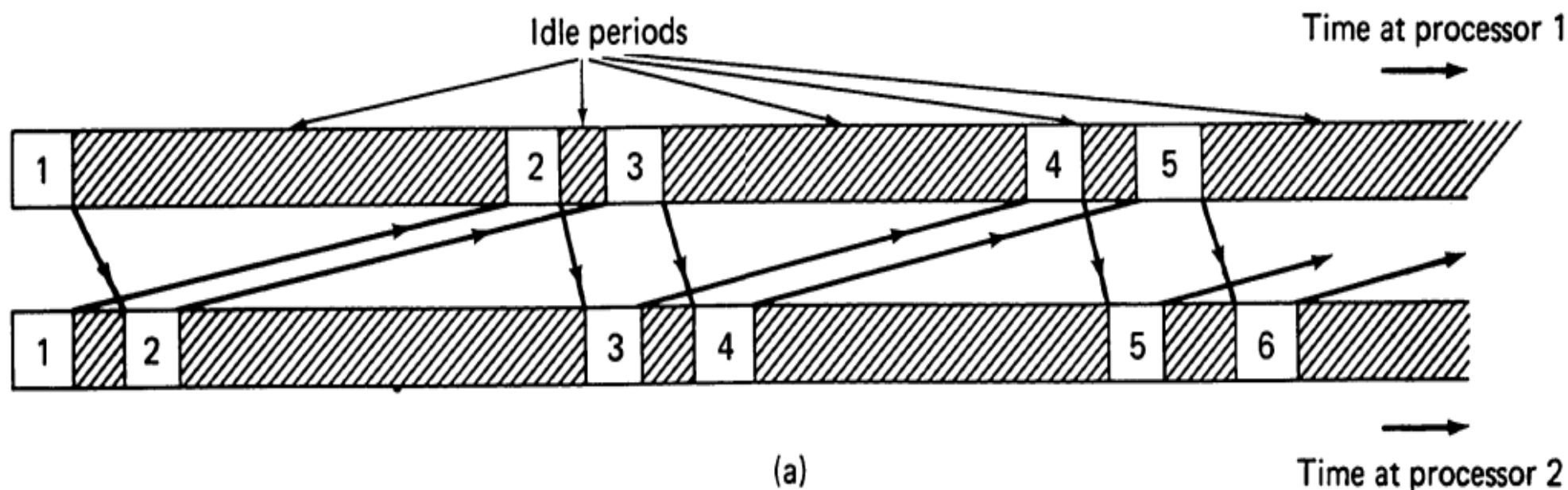
Influenta vitezei canalelor de comunicatie si a procesoarelor

- cand procesorul i actualizeaza variabila x_i trebuie sa astepte mesajele celorlalte procesoare



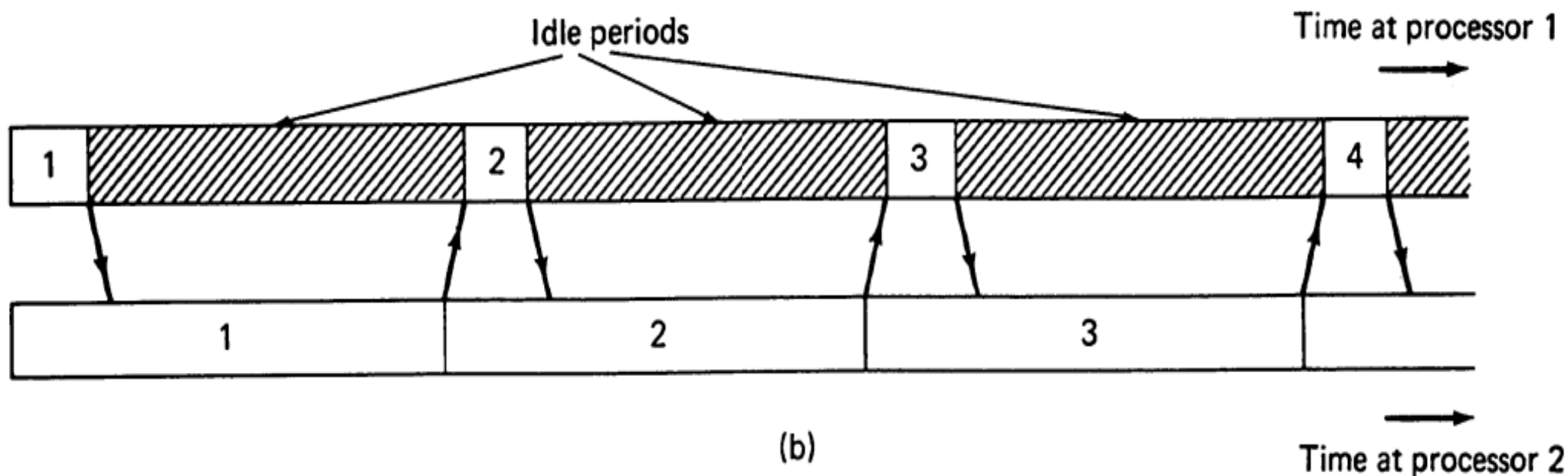
Influenta vitezei canalelor de comunicatie si a procesoarelor

- un canal de comunicatie lent poate incetini progresul intregului algoritm



Influenta vitezei canalelor de comunicatie si a procesoarelor

- procesoarele rapide sau cele care au incarcare usoara per iteratie trebuie sa astepte terminarea iteratiilor de pe procesoarele lente sau incarcate
- astfel, progresul algoritmului este dictat de cel mai lent procesor



Penalizarea de sincronizare

- cand e influentata de prezenta canalelor lente de comunicatie, contribuie la penalizarea comunicarii CP
- cand e influentata de vitezele de executie diferite ale procesoarelor, contribuie la o pierdere de eficienta prin dezechilibrul incarcarii procesoarelor (*poor load balancing*), chiar si cand comunicatiile sunt instantanee

Varianta asincrona a algoritmului

- procesorul i executa actualizarea k folosind *unele* dintre rezultatele actualizarilor anterioare, *nu neaparat cele mai recente* !
 - ex: procesorul i excuta actualizarea k folosind rezultatul actualizarii $(k + 10)$ a procesorului j in vreme ce acesta executa actualizarea $(k + 100)$ folosind rezultatul actualizarii $(k - 10)$ a procesorului i
- vitezele de executie si comunicatie pot diferi la procesoare diferite iar intarzierile de comunicare pot fi substantiale
 - mai mult, ele pot evolua impredictibil pe masura ce algoritmul progresa
- nici macar ordinea mesajelor pe link-urile de comunicatie nu e garantata
 - un procesor poate folosi actualizarea k de la un alt procesor la un moment dat si rezultatele actualizarii $(k - 10)$ ale aceluasi procesor la un moment ulterior
- si conditiile initiale ale unui procesor pot fi flexibile
 - variabilele x_i de pe procesoare diferite pot avea valori diferite pt acelasi i la momentul inceperii algoritmului

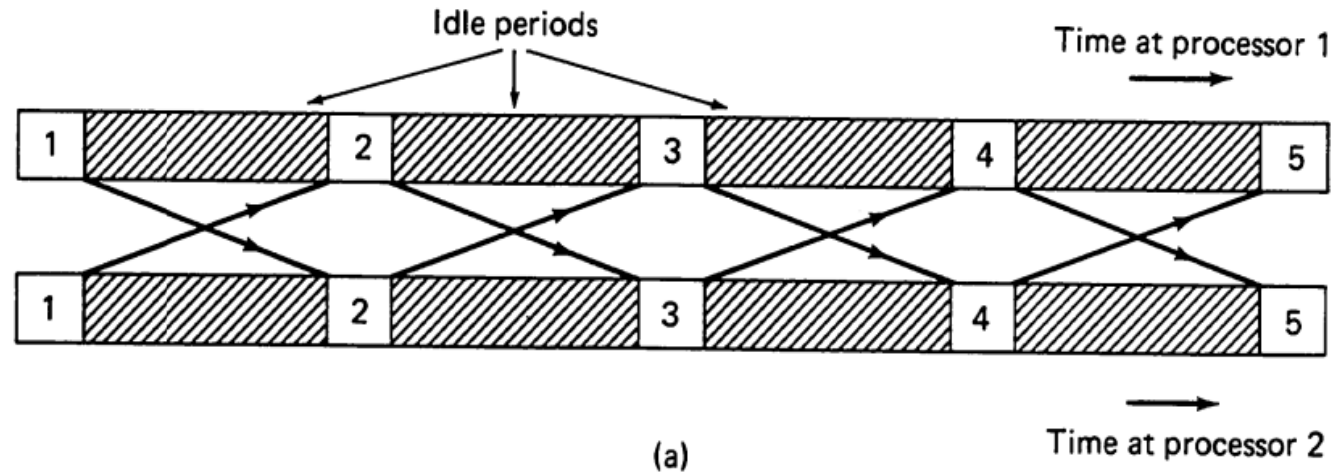
Consecinte algoritm asincron

- algoritmi asincroni pot reduce penalizarea de sincronizare indusa de
 - procesoare rapide care asteapta procesoarele lente sa livreze actualizari
 - canale de comunicatie lente in a livra mesaje
- procesoarele rapide pot executa mai multe iteratii fara sa astepte cele mai recente rezultate ale celorlalte procesoare
- pericol: iteratiile bazate pe informatie veche pot fi ineficiente si pot afecta convergenta
- cand functioneaza, algoritmi asincroni reduc penalizarea de sincronizare si genereaza solutii mai rapide
- in schimb, creste potential volumul de comunicatie comparativ cu solutiile sincrone
- dezavantaj: afecteaza convergenta pe care solutiile sincrone ar putea-o avea, fiind nevoie sa se limiteze intarzierile de comunicatie pt a garanta convergenta
- in orice caz, analiza algoritmilor sincroni este mult mai dificila decat cea a algoritmilor sincroni

Comparatie algoritm asincron vs sincron

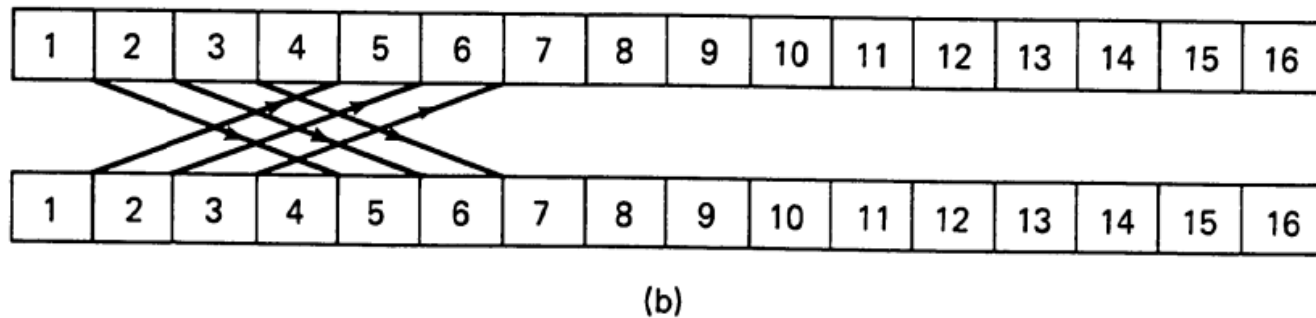
(a) algoritm sincron

- sagetile indica momentul primirii mesajelor, zonele hasurate indica procesor idle, numerele corespund perioadelor de actualizari
- perioadele de asteptare sunt de trei ori mai mari decat cele de calcul in exemplul nostru



(b) algoritm asincron

- procesoarele pot executa mai multe iteratii pe unitatea de timp (de 3 ori mai multe in exemplul nostru)
- nu e nevoie sa se astepte receptia mesajelor



Avantajele algoritmilor asincroni pt metode iterative

- Jacobi

$$x_i(t+1) = f_i(x_1(t), \dots, x_n(t)) \quad i = 1, \dots, n$$

- potrivita pt implementare paralela sincrona

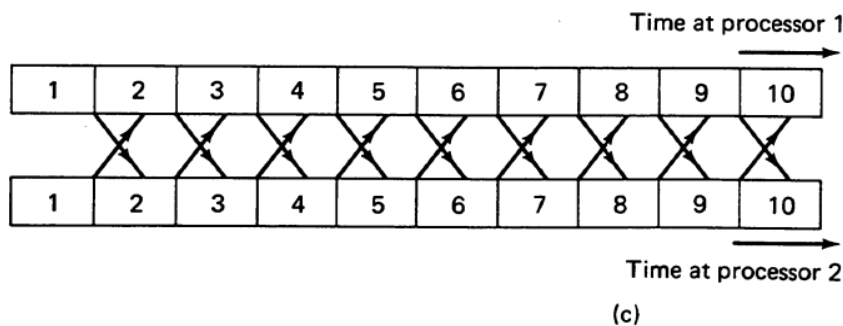
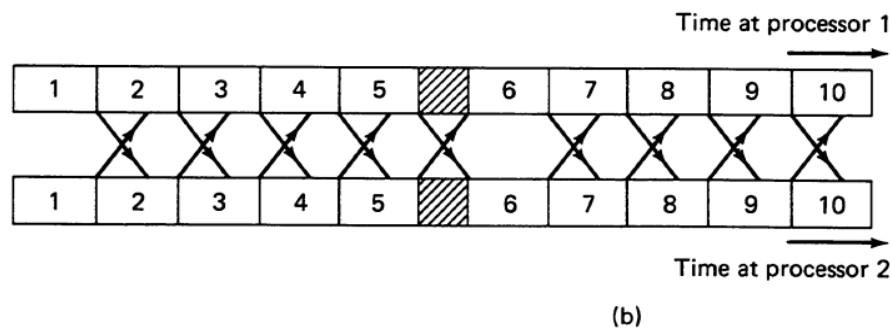
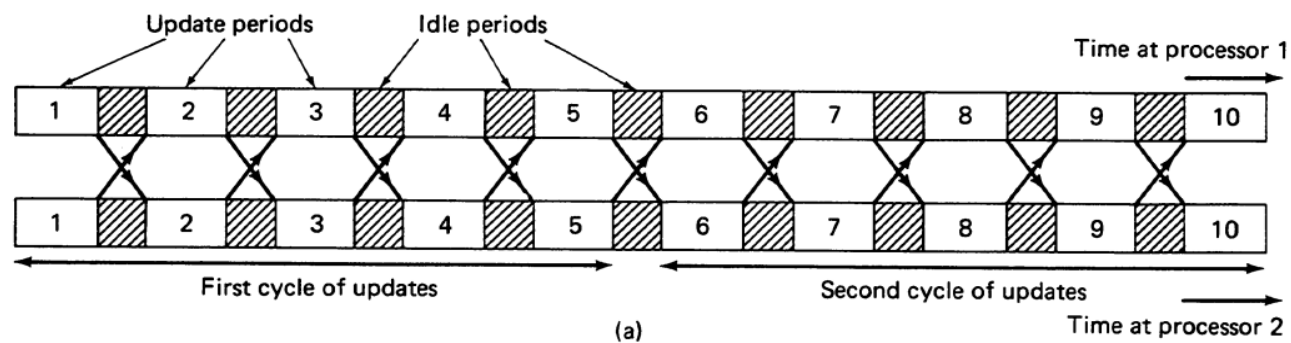
- Gauss-Seidel

$$x_i(t+1) = f_i(x_1(t+1), \dots, x_{i-1}(t+1), x_i(t), \dots, x_n(t))$$

- inerent secventiala (se poate paraleliza pt grafuri de dependenta sparse)
- in general, Gauss-Seidel converge mai rapid decat Jacobi
- in general, convergenta se poate accelera prin incorporarea valorilor actualizate ale variabilelor in valori urmatoare pt alte variabile cat mai devreme cu putinta
 - intr-un algoritm asincron exista potentialul de a se putea incorpora valorile actualizate in calcul mai repede decat intr-un algoritm sincron
- deci este posibil ca un algoritm asincron sa foloseasca avantajele de convergenta ale metodei Gauss-Seidel, fara a sacrifice potentialul de paralelism a metodei Jacobi

Gauss-Seidel asincron vs Jacobi

- se poate gândi o implementare asincronă Gauss-Seidel mai rapidă care să păstreze potențialul de paralelism din Jacobi

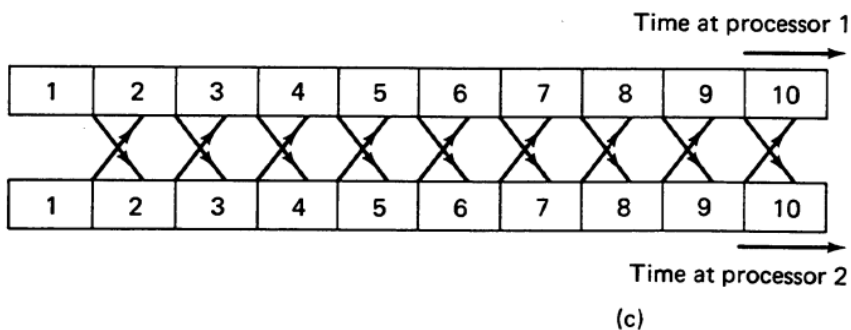
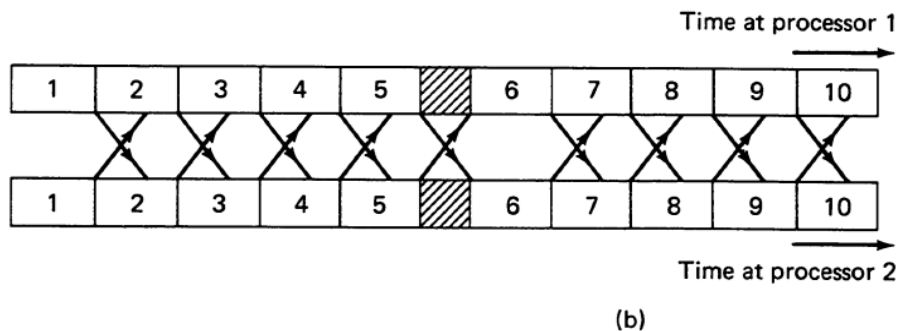
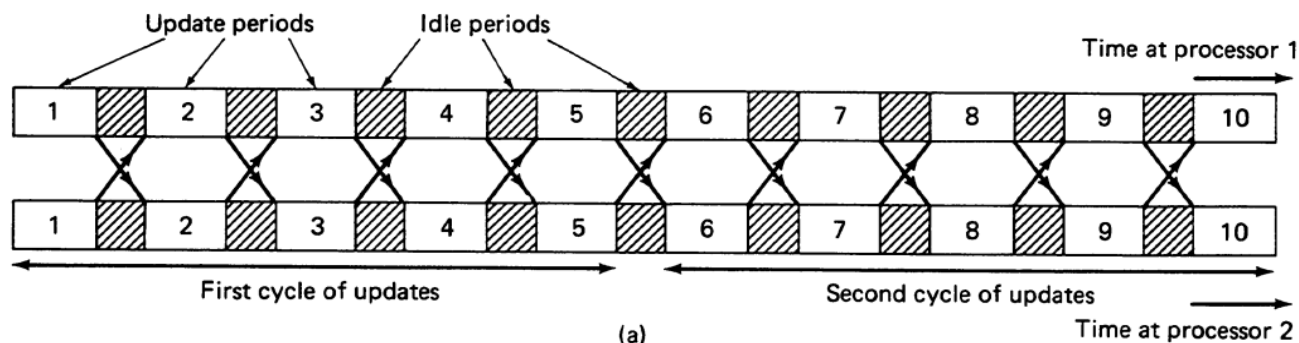


Gauss-Seidel asincron vs Jacobi

- 2 procesoare care actualizeaza 5 variabile fiecare

(a) algoritm sincron care incearca sa incorporeaze informatia noua cat mai repede posibil

Penalizarea de sincronizare este mare.

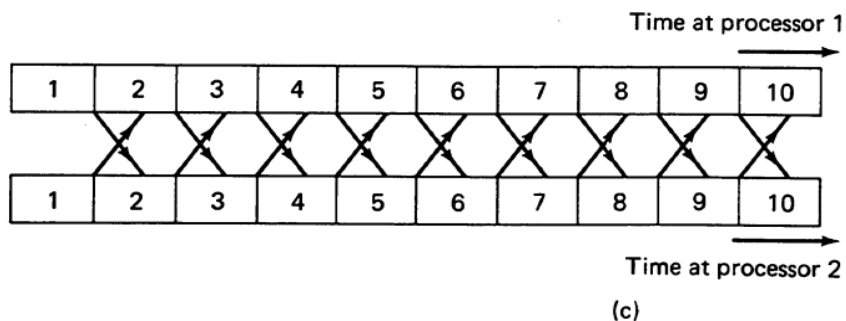
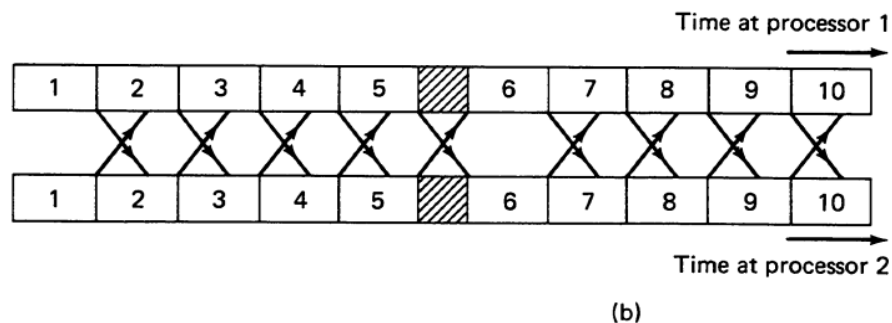
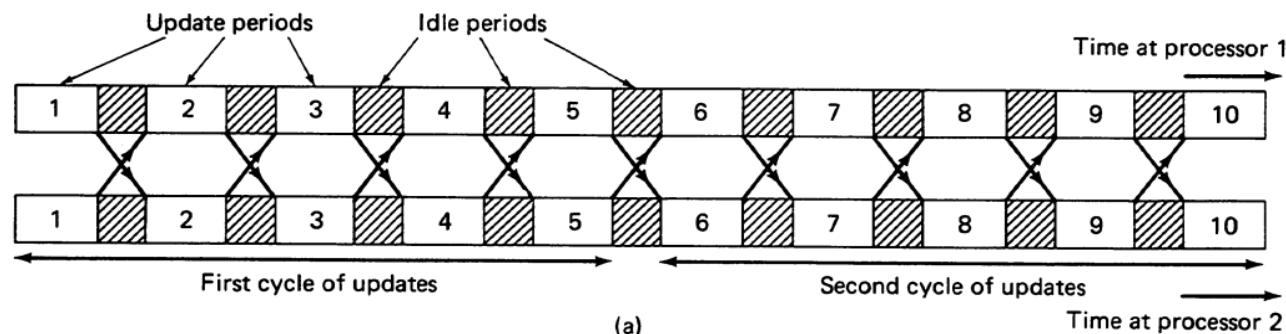


Gauss-Seidel asincron vs Jacobi

(b) algoritm sincron care reduce penalizarea de sincronizare prin pipelining-ul calculului cu comunicatia

- actualizarile per procesor se iau in considerare la fiecare 5 pasi

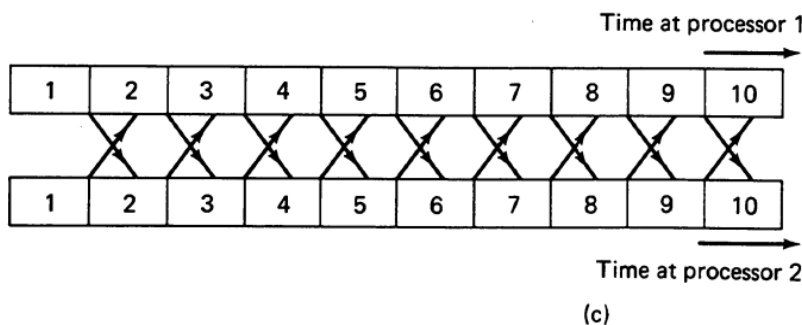
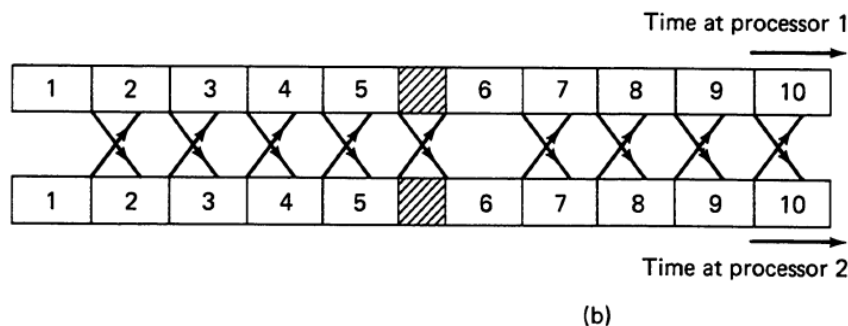
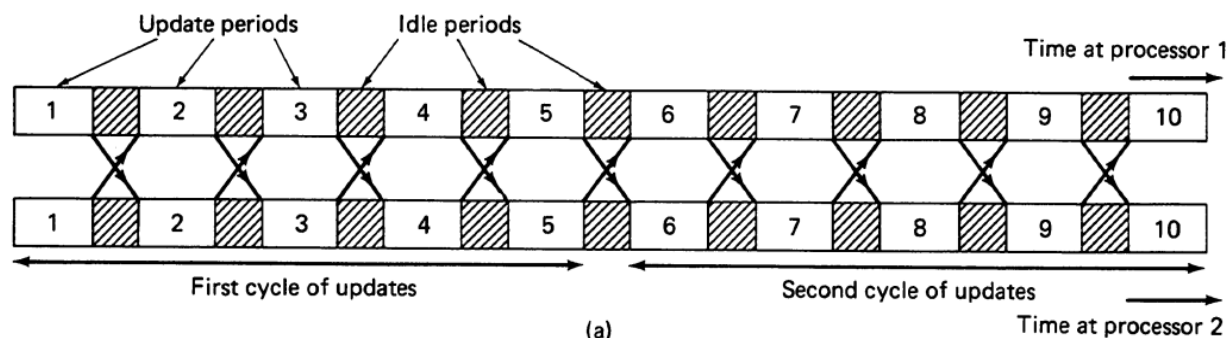
=> metoda Jacobi-like



Gauss-Seidel asincron vs Jacobi

(c) algoritm asincron, fara penalizare de sincronizare; rezultatul fiecărei actualizari e luat in calcul de alt procesor dupa o intarziere de o actualizare

- in acest exemplu toate actualizarile necesita acelasi timp
- penalizarile sunt datorate exclusiv comunicatiei
- Obs: timpi variabili de actualizare a variabilelor pot creste suplimentar penalizarea de sincronizare



Algoritmi paraleli

Curs 7

Vlad Olaru

vlad.olaru@fmi.unibuc.ro

Calculatoare si Tehnologia Informatiei

Universitatea din Bucuresti

Sisteme de ecuatii liniare

Fie matricea $A \in \mathbb{R}^{n \times n}$, aflată soluția sistemului:

$$Ax = b; x = \begin{bmatrix} x_1 \\ \dots \\ x_n \end{bmatrix}$$

Ipoteze de lucru: model de calcul sincron, comunicare instantanee

- **Metode directe:** eliminare gaussiană, factorizare LU etc.
 - afla o ***soluție exactă*** într-un nr. finit de operații
 - în general, complexitate $O(n^3)$
 - recomandate pentru dimensiuni mici
 - li se aplică conceptele de complexitate și eficiență discutate

Sisteme de ecuatii liniare

Fie matricea $A \in R^{n \times n}$, aflatii solutia sistemului:

$$Ax = b; x = \begin{bmatrix} x_1 \\ \dots \\ x_n \end{bmatrix}$$

- **Metode iterative:** Jacobi, Gauss-Seidel
 - nu obtin o solutie exacta in timp finit
 - converg asimptotic catre o solutie exacta
 - in general, dupa un nr mic de pasi (iteratii) ating o solutie cu precizie acceptabila, ceea ce le face preferabile metodelor directe (in special cand n e mare)
 - pt matrici sparse, au cerinte de stocare mai mici decat metodele directe

Sisteme de ecuatii liniare

Fie matricea $A \in \mathbb{R}^{n \times n}$, aflată soluția sistemului:

$$Ax = b; x = \begin{bmatrix} x_1 \\ \dots \\ x_n \end{bmatrix}$$

- **Metode iterative:** Jacobi, Gauss-Seidel
 - pt ca pot rula la infinit, nu li se aplica notiunile de complexitate/eficienta
 - o masura mai potrivita este viteza de convergenta a solutiei
 - convergenta e in general *geometrica* (viteza de convergenta e cea a unei *progresii geometrice*)
 secventa de vectori $\{x(t)\}$ a.i. $\|x(t) - x^*\| \leq c\rho^t$
 unde x^* este solutia $Ax = b$, iar $c > 0$ si $0 < \rho < 1$ sunt constante
 - cu cat ρ este mai mic, cu atat convergenta e mai rapida

Sisteme de ecuatii liniare cu structura speciala

- sisteme triunghiulare

- superior: $A_{ij} = 0, i > j$

$$A_{11}x_1 + A_{12}x_2 + \dots + A_{1n}x_n = b_1$$

$$A_{22}x_2 + \dots + A_{2n}x_n = b_2$$

...

$$A_{nn}x_n = b_n$$

- inferior: $A_{ij} = 0, i < j$

$$A_{11}x_1 = b_1 \Rightarrow x_1 = \frac{b_1}{A_{11}}$$

$$A_{21}x_1 + A_{22}x_2 = b_2 \Rightarrow x_2 = \frac{1}{A_{22}}(b_2 - A_{21}x_1)$$

...

$$A_{n1}x_1 + A_{n2}x_2 + \dots + A_{nn}x_n = b_n \Rightarrow x_n = \frac{1}{A_{nn}}(b_n - A_{n1}x_1 - A_{n2}x_2 - \dots - A_{nn-1}x_{n-1})$$

Matrici triunghiulare inferioare

- pp A invertibila, vrem sa calculam A^{-1}
- pp $A_{ii} = 1$ si vom generaliza ulterior
- daca $A = I - L$ unde $L_{ij} = 0$ pt. $i \leq j \Rightarrow L^n = 0$
- mai mult

$$A^{-1} = (I + L + L^2 + \dots + L^{n-1})$$

$$\text{pt. ca } (I + L + L^2 + \dots + L^{n-1})(I - L) = I - L^n = I$$

- algoritm
 - calculam in paralel $L^2, \dots, L^{n-1}$
 - adunam rezultatele
 - complexitate $O(\log^2 n)$ folosind n^4 procesoare

Matrici triunghiulare inferioare

- algoritm mai eficient

$$A^{-1} = (I + L^{2^{\lceil \log n \rceil - 1}})(I + L^{2^{\lceil \log n \rceil - 2}}) \cdots (I + L^4)(I + L^2)(I + L).$$

pt. ca efectuand inmultirile succesiv ramane $(I + L + L^2 + \dots + L^{n-1})$

- algoritm:
 - calculam in paralel $L^2, L^4, \dots, L^{2^{\log n - 1}}$
 - adunam matricea identitate la fiecare matrice de mai sus $O(1)$
 - calculam inmultirile (sunt $O(\log n)$ inmultiri)
 - complexitate $O(\log^2 n)$ folosind n^3 procesoare (fara cost de comunicatie)

Matrici triunghiulare cu elemente diagonale diferite de 1

- fie D matrice diagonala a.i. elementele ei sunt elementele de pe diagonala matricii A
- atunci $D^{-1}A$ e matrice triunghiulara cu elemente unitare pe diagonala
- algoritm
 - transformam A in $D^{-1}A$ $O(1)$ cu n^2 procesoare
 - inversam $D^{-1}A$ pt a obtine $A^{-1}D$ $O(\log^2 n)$ cu n^3 procesoare
 - inmultim rezultatul din pasul anterior cu D^{-1} la dreapta pt. a obtine A^{-1} $O(1)$ cu n^2 procesoare
- complexitate: $O(\log^2 n)$ cu n^3 procesoare

Metoda divide & conquer

- partitionam

$$A = \begin{pmatrix} A_1 & 0 \\ A_2 & A_3 \end{pmatrix} \text{ unde } A_1 \text{ are dimensiunea } \text{ceil}(\frac{n}{2}) \times \text{ceil}(\frac{n}{2})$$

iar A_1 si A_3 sunt triunghiular inferioare

- in aceste conditii

$$A^{-1} = \begin{bmatrix} A_1^{-1} & 0 \\ -A_3^{-1}A_2A_1^{-1} & A_3^{-1} \end{bmatrix}$$

- algoritm (pt $n > 1$)

- inversam in paralel A_1 si A_3

- inmultim $A_3^{-1}A_2$

$O(\log n)$ cu n^3 procesoare

- inmultim rezultatul pasului anterior cu A_1^{-1}

$O(\log n)$ cu n^3 procesoare

Complexitate metoda divide & conquer

- $T(n)$ = timp inversare matrice $n \times n$

$$\Rightarrow T(n) = T\left(\text{ceil}\left(\frac{n}{2}\right)\right) + O(\log n)$$

$$\Rightarrow T(n) = O(\log^2 n) \text{ folosind } n^3 \text{ procesoare}$$

- demonstratie: divide & conquer in $O(\log n)$ pasi

$$T(n) - T\left(\text{ceil}\left(\frac{n}{2}\right)\right) = O(\log n)$$

$$T\left(\text{ceil}\left(\frac{n}{2}\right)\right) - T\left(\text{ceil}\left(\frac{n}{4}\right)\right) = O(\log \text{ceil}(\frac{n}{2}))$$

...

Problemele metodelor bazate pe inversare de matrici

- exista solutii mai simple fara sa fie nevoie sa aflam inversa matricii?
- daca luam in calcul costul comunicatiei, complexitatea creste mult peste $O(\log^2 n)$
- daca tinem cont si de nr cubic de procesoare => solutii interesante teoretic, dar nepractice

Eliminare gaussiana (inferior triunghiular), “back substitution”

- ecuatia i din sistemul triunghiular inferior:

$$A_{i1}x_1 + A_{i2}x_2 + \cdots + A_{ii}x_i = b_i$$

- pp. folosirea a n procesoare
- pp ca la inceputul fazei i valorile $\{x_1, \dots, x_{i-1}\}$ si expresiile

$$A_{j1}x_1 + A_{j2}x_2 + \cdots + A_{ji-1}x_{i-1}$$

sunt cunoscute pt. fiecare $j \geq i$ din ec. precedente

=> procesorul i calculeaza

$$x_i = \frac{1}{A_{ii}} (b_i - A_{i1}x_1 - \cdots - A_{ii-1}x_{i-1})$$

cf. formulei de mai sus

Eliminare gaussiana (inferior triunghiular), “back substitution”

- apoi, fiecare procesor j , unde $j \geq i + 1$, calculeaza

$$A_{j1}x_1 + A_{j2}x_2 + \dots + A_{ji}x_i$$

adunand $A_{ji}x_i$ la expresia anterior cunoscuta

$$A_{j1}x_1 + A_{j2}x_2 + \dots + A_{ji-1}x_{i-1}$$

- algoritmul se opreste la sf fazei n , cand toate variabilele $x_1, x_2, \dots, x_n$ sunt calculate
- complexitate:

- calculul x_i necesita $O(1) \Rightarrow$ complexitate $O(n)$ cu n procesoare si comunicatie instantanee

Analiza comparativa

- eficienta

$$E(n) = \frac{T^*(n)}{pT_p(n)}$$

unde p e nr de procesoare iar n dimensiunea problemei

- obs: $Ax = b$ necesita cel putin n^2 pasi (A are n^2 elemente)
- implementarea seriala back substitution este $O(n^2)$

$$\Rightarrow T^*(n) = \Theta(n^2)$$

$\Rightarrow$ eficienta algoritmilor bazati pe inversare de matrici este $O(\frac{1}{n \log^2 n})$ iar pt back substitution este $\Theta(1)$

$\Rightarrow$ back substitution e mai lent, dar mai eficient !

Celelalte metode necesita un nr disproportionat de mare de procesoare fata de imbunatatirea pe care o produc asupra accelerarii.

Back substitution pt inversarea de matrici triunghiulare

- $AA^{-1} = I \Rightarrow x^i$, coloana i a matricii A^{-1} satisface
$$Ax^i = e^i$$

unde e^i este al i -lea vector unitate

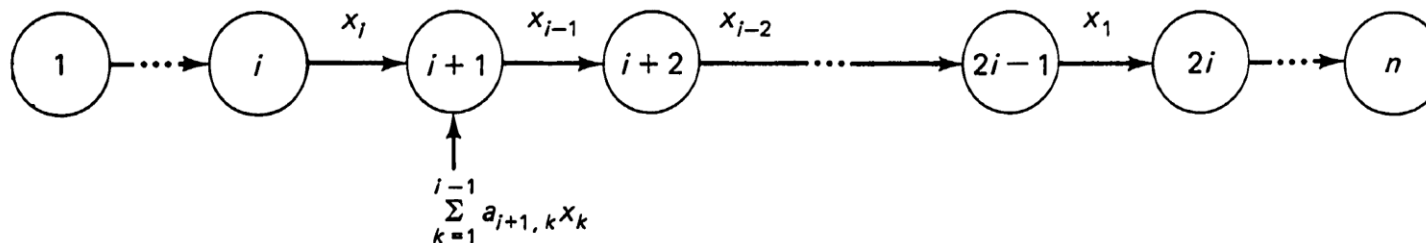
- algoritm: se rezolva n sisteme $Ax^i = e^i$
- complexitate
 - $O(n)$ folosind n^2 procesoare pt rezolvarea celor n probleme in paralel
 - $O(n^2)$ folosind n procesoare pt rezolvarea celor n probleme una dupa alta

Implementarea back substitution pe topologii particulare

- consideram un lant liniar cu n procesoare
- procesorul i primeste linia i a matricii A si componenta i a vectorului b
- pipelining-ul calculului cu computatia permite un timp de executie de $O(n)$
=> costul comunicatiei poate cel mult afecta timpul de executie in limita unui factor constant
- daca acest factor e mare (comparabil cu n), se poate mari granularitatea prin asignarea mai multor linii unui nr mai mic de procesoare
- pe hipercub penalizarea comunicatie va fi mai mica, asa cum am vazut la maparea lanturilor liniare pe hipercuburi

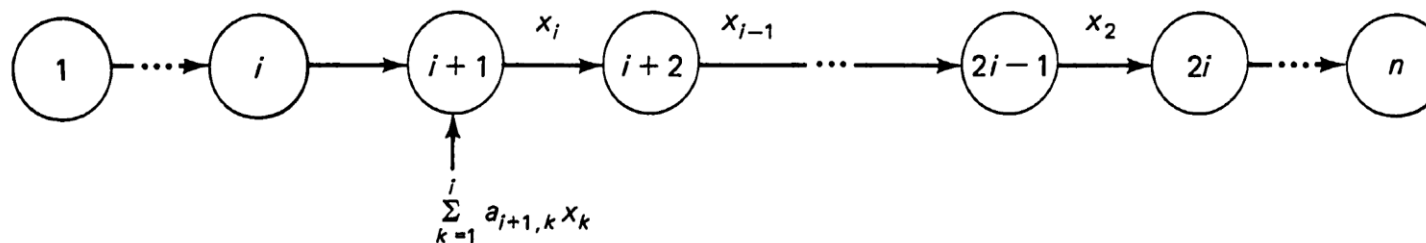
Back substitution pe lant liniar

(a) procesorul $i + 1$ a primit x_i ,
 $\dots, x_{i-1}$
 si a evaluat suma partiala



(a)

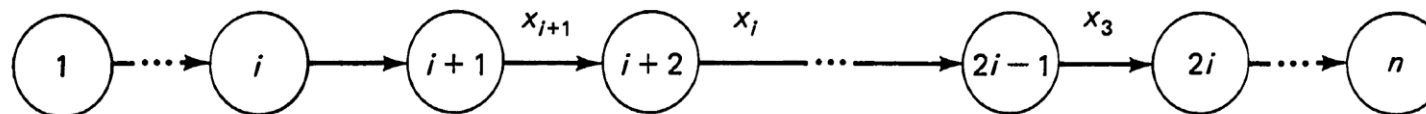
(b) cand x_i a fost primit de catre
 procesorul $i + 1$ este trimis la
 dreapta (procesorul $i + 2$) si
 noua suma partiala este
 calculata



(b)

Apoi se calculeaza x_{i+1}

(c) x_{i+1} se transmite procesorului
 $i + 2$



(c)

Calculul x_i si x_{i+1} este $O(1)$ iar
 timpul total al algoritmului este
 proportional cu n

Sisteme tridiagonale

$$A = \begin{bmatrix} 1 & -1 & 0 \\ -2 & 2 & 1 \\ 0 & -2 & 3 \end{bmatrix}; g = [A_{11} \cdots A_{nn}]; h = [A_{12} \cdots A_{n-1n}];$$

$$f = [A_{21} \cdots A_{nn-1}];$$

- Sisteme tridiagonale $Ax = b$

- $A_{ij} = 0, |i - j| > 1$

$$\begin{aligned} g_1 x_1 + h_1 x_2 &= b_1 \\ f_i x_{i-1} + g_i x_i + h_i x_{i+1} &= b_i, \quad i = 2, 3, \dots, n-1 \\ f_n x_{n-1} + g_n x_n &= b_n \end{aligned}$$

unde g_i sunt elementele de pe diagonala matricii A , iar f_i (respectiv h_i) sunt elementele lui A de sub (respectiv de deasupra) diagonalei

Sisteme tridiagonale rezolvate prin metoda reducerii par-impare

Idee: daca $g_i \neq 0$ putem rezolva x_i pentru fiecare i impar si substituim expresia lui x_i in restul ecuatiilor $\Rightarrow$ un sistem de ecuatii x_i cu i par

$\Rightarrow$ sistem tridiagonal cu jumătate din variabile

$\Rightarrow$ apoi aplicam procedura recursiv

Conventie: $x_0 = x_{n+1} = 0$ pt a putea calcula x_i ca mai jos si pt

$i = 1$ respectiv $i = n$

$$x_i = \frac{1}{g_i} (b_i - f_i x_{i-1} - h_i x_{i+1})$$

Deci

$$x_{i-1} = \frac{1}{g_{i-1}} (b_{i-1} - f_{i-1} x_{i-2} - h_{i-1} x_i)$$

$$x_{i+1} = \frac{1}{g_{i+1}} (b_{i+1} - f_{i+1} x_i - h_{i+1} x_{i+2})$$

Sisteme tridiagonale rezolvate prin metoda reducerii par-impare

$$\begin{aligned} g_1 x_1 + h_1 x_2 &= b_1 \\ f_i x_{i-1} + g_i x_i + h_i x_{i+1} &= b_i, \quad i = 2, 3, \dots, n-1 \\ f_n x_{n-1} + g_n x_n &= b_n \end{aligned}$$

Consideram $x_0 = x_{n+1} = 0$

$$x_i = \frac{1}{g_i} (b_i - f_i x_{i-1} - h_i x_{i+1})$$

Deci

$$\begin{aligned} x_{i-1} &= \frac{1}{g_{i-1}} (b_{i-1} - f_{i-1} x_{i-2} - h_{i-1} x_i) \\ x_{i+1} &= \frac{1}{g_{i+1}} (b_{i+1} - f_{i+1} x_i - h_{i+1} x_{i+2}) \end{aligned}$$

Inlocuim in a doua ecuatie de mai sus:

$$\frac{f_i}{g_{i-1}} (b_{i-1} - f_{i-1} x_{i-2} - h_{i-1} x_i) + g_i x_i + \frac{h_i}{g_{i+1}} (b_{i+1} - f_{i+1} x_i - h_{i+1} x_{i+2}) = b_i$$

Sisteme tridiagonale rezolvate prin metoda reducerii par-impare

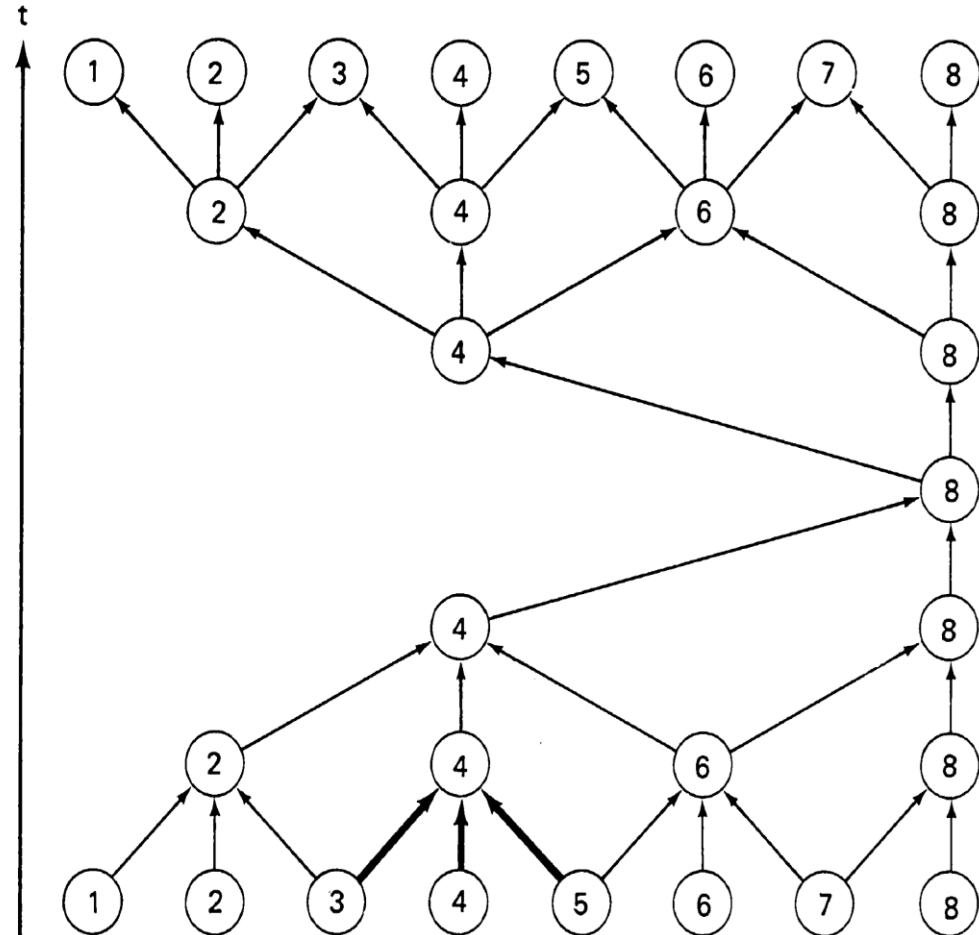
Prin simplificare rezulta:

$$\begin{aligned}
 - \left(\frac{f_i f_{i-1}}{g_{i-1}} \right) x_{i-2} + \left(g_i - \frac{h_{i-1} f_i}{g_{i-1}} - \frac{h_i f_{i+1}}{g_{i+1}} \right) x_i - \left(\frac{h_i h_{i+1}}{g_{i+1}} \right) x_{i+2} \\
 = b_i - \frac{f_i}{g_{i-1}} b_{i-1} - \frac{h_i}{g_{i+1}} b_{i+1}.
 \end{aligned}$$

- obținem un alt sistem tridiagonal cu $n/2$ variabile: $x_2, \dots, x_{2 * \text{floor}(\frac{n}{2})}$!
- prin eliminari succesive înjumătățim recursiv dimensiunea până obținem un sistem cu o singură variabilă care se rezolvă direct
- apoi, se parcurge procesul în sens invers pt a calcula valorile variabilelor eliminate

Vizualizarea metodei reducerii par-impare

- $n = 8$
- prima faza: eliminare variabile x_1, x_3, x_5, x_7
- se obtin valorile x_2, x_4, x_6, x_8
- faza a doua: x_2 si x_6 sunt eliminate
- apoi x_4 e eliminat si ramane o singura ecuatie cu o singura necunoscuta x_8
- dupa evaluarea lui x_8 , celelalte variabile se evalueaza in sens invers
- ex: odata ce s-au evaluat x_2, x_4, x_6 , se obtin imediat si valorile lui x_1, x_3, x_5, x_7
- obs: arcele din diagrama indica dependenta de date (ex: arcele ingrosate reprezinta coeficientii f_i, g_i, h_i pt $i = 3, 4, 5$)



Complexitatea metodei reducerii par-impare pt sisteme tridiagonale

- la fiecare etapa, nr variabilelor se reduce aproxmativ la jumătate
- după $O(\log n)$ pasi ramane doar o variabila de evaluat
- in fiecare etapa, e nevoie sa se calculeze coeficientii sistemului redus $\Rightarrow 4$ pasi cu $O(n)$ procesoare
- similar pentru etapa de evaluare in sens invers
- per total, complexitatea este de $O(\log n)$ cu $O(n)$ procesoare
- Obs: nr total de operatii este $O(n)$ pt. ca in fiecare etapa se injumatatesc calculele

$\Rightarrow$ exista un algoritm secvential de complexitate $O(n)$ (optimal, de fapt, pt ca sunt necesari cel putin n pasi pt a citi datele)

$\Rightarrow T^*(n) = \Theta(n)$ iar eficienta este $\Theta(1/\log n)$

Metoda reducerii par-impair pe lant linar

- n procesoare, fiecare procesor i stocheaza linia i din matrice si componenta i din vectorul b si va calcula variabila x_i
- evaluarea fazei pare se face in $O(1)$ pt. ca procesorul i are nevoie doar de coeficientii procesoarelor vecine
- dupa k reduceri, sistemul contine doar variabile x_i unde i este un multiplu intreg de $2^k \Rightarrow$ procesorul $i2^k$ trebuie sa comunice cu procesorul $(i+1)2^k$
- in ultima etapa, costul comunicarii este $\Omega(n) \Rightarrow$ complexitate $\Omega(n)$, la fel ca si algoritmul secvential !
- Obs: se putea infera si din faptul ca valoarea lui x_1 depinde de date stocate pe procesoare la distanta $\Theta(n)$

Metoda reducerii par-impair pe lant linear cu cresterea granularitatii

- p procesoare ($p < n$), fiecare procesor i stocheaza linia n/p linii din matrice, componentele corespunzatoare din vectorul b si va calcula n/p variabile
- in primele $N = \text{floor} \left(\log \left(\frac{n}{p} \right) \right)$ etape calculele sunt locale $\Rightarrow O \left(\frac{n}{p} \right)$ operatii aritmetice si f. putina comunicatie
- dupa N etape $\Rightarrow$ un sistem tridiagonal cu $O(p)$ variabile
 - se rezolva trimitand toti coeficientii catre un singur procesor
 - procesorul rezolva sistemul si trimite rezultatele prin broadcast inapoi catre cele p procesoare $\Rightarrow$ complexitate $O(p)$ pt calcule si $O(p)$ pt comunicatie
- deci, timpul total este $O \left(\frac{n}{p} \right) + O(p)$
- optimizand dupa p , valoarea optima este $p = \Theta(\sqrt{n}) \Rightarrow$ timp de executie optimal este $O(\sqrt{n})$
 - superior timpului serial $O(n)$ dar inferior valorii pt comunicatie instantanee $O(\log n)$

Metoda reducerii par-impare pe hipercub

- n procesoare
- mapam un lant liniar pe hipercub folosind coduri Gray reflectate (RGC)
- cand folosim maparea RGC, procesoare aflate la distanta logica 2^k in lant se mapeaza la distanta fizica 2 in hipercub

=> toate perechile de procesoare $(i2^k, (i+1)2^k)$ pot comunica simultan in 2 pasi

⇒ fiecare etapa din reducerea par-impair consuma $O(1)$ pt comunicatie

⇒ in total, complexitatea ramane cea teoretica de $O(\log n)$ cand se folosesc n procesoare

Obs: in practica, e posibil ca totusi operatiile de comunicare sa fie dominante comparativ cu calculul (chiar daca sunt $O(1)$)

=> trebuie crescuta granularitatea pt a scadea penalizarea comunicatiei

Metode paralele directe pentru sisteme de ecuatii liniare

- A matrice $n \times n$
- aplica o serie de transformari sistemului de ecuatii liniare a.i. matricea A sa devina triunghiulara si sa aplica *back substitution*
- aceste transformari constau intr-o serie de inmultiri la stanga cu matrici a.i. sistemul devine

$$(M^{(k)}M^{(k-1)} \dots M^{(1)}A)x = (M^{(k)}M^{(k-1)} \dots M^{(1)}b)$$

- matricile M se aleg a.i.
 - inmultirile cu o matrice arbitrara (initial A) sa fie rapide computational
 - produsul final $M^{(k)}M^{(k-1)} \dots M^{(1)}A$ sa fie o matrice triunghiulara
- consecinta: sistemul se poate rezolva in $O(n)$ cu n procesoare
- bonus: $A^{-1} = (M^{(k)}M^{(k-1)} \dots M^{(1)}A)^{-1} (M^{(k)}M^{(k-1)} \dots M^{(1)})$
 - pt. ca $(M^{(k)}M^{(k-1)} \dots M^{(1)}A)$ e triunghiulara, se poate inversa folosind back substitution in $O(n)$ folosind n^2 procesoare sau in $O(n^2)$ folosind n procesoare

Eliminare Gaussiana (cazul general)

- principiu: variabila x_i este exprimata ca o functie de $x_{i+1}, \dots, x_n$ si e eliminata din sistem
=> dupa $n - 1$ pasi, ramane o singura ecuatie in x_n

- definitie recursiva a algoritmului

$$C^{(0)} = A$$

$$C^{(i)} = M^{(i)} \dots M^{(1)} A$$

- pp. $C^{(i-1)}$ calculata pt. $1 \leq i \leq n - 1$ si elementele subdiagonale ale coloanei j sunt zero pt. $j < i$
- aratam cum sa construim $M^{(i)}$ a.i.

$$C^{(i)} = M^{(i)} C^{(i-1)}$$

$$C^{(i-1)} = \begin{bmatrix} * & \dots & \dots & \dots & \dots & * \\ 0 & \ddots & & & & \vdots \\ \vdots & \ddots & \ddots & & & \\ & & 0 & * & \dots & * \\ \vdots & & \vdots & \vdots & & \\ 0 & \dots & 0 & * & \dots & * \end{bmatrix} \begin{matrix} \\ \\ \\ i \\ \\ i \end{matrix}$$

Eliminare Gaussiana fara pivotare

- pp. “pivotul” $C^{(i-1)}_{ii} \neq 0$
- fie $M^{(i)} = I - N^{(i)}$ unde elementele nenule ale lui $N^{(i)}$ sunt

$$N^{(i)}_{ji} = C^{(i-1)}_{ji} / C^{(i-1)}_{ii} \quad j > i$$

$$N^{(i)} = \begin{bmatrix} 0 & \dots & \dots & \dots & \dots & \dots & 0 \\ 0 & \ddots & & & & & \vdots \\ \vdots & \ddots & \ddots & & & & \\ & & 0 & 0 & \dots & \dots & 0 \\ & & \vdots & * & 0 & \dots & 0 \\ \vdots & & \vdots & \vdots & \vdots & & \vdots \\ 0 & \dots & 0 & * & 0 & \dots & 0 \end{bmatrix}_i$$

i

Eliminare Gaussiana fara pivotare

- pp. “pivotul” $C^{(i-1)}_{ii} \neq 0$
- fie $M^{(i)} = I - N^{(i)}$ unde elementele nenule ale lui $N^{(i)}$ sunt

$$N^{(i)}_{ji} = C^{(i-1)}_{ji} / C^{(i-1)}_{ii} \quad j > i$$

- atunci, $C^{(i)} = (I - N^{(i)}) C^{(i-1)}$ pt. ca
 - primele $i - 1$ coloane ale lui $N^{(i)}$ sunt zero $\Rightarrow$ primele $i - 1$ coloane ale lui $C^{(i)}$ sunt la fel cu cele ale lui $C^{(i-1)}$ (adica zero)
 - elementele subdiagonale ale lui $C^{(i)}$ din coloana i , $C^{(i)}_{ji}$ cu $j > i$ sunt

$$C^{(i)}_{ji} = C^{(i-1)}_{ji} - \frac{C^{(i-1)}_{ji}}{C^{(i-1)}_{ii}} C^{(i-1)}_{ii} = 0$$

- deci $C^{(i)}$ e de aceeași formă cu $C^{(i-1)}$ (în particular, superior triunghiulară)
- Obs: metoda esuează când pivotul e zero; chiar pt valori mici ale pivotului pot exista probleme numerice

$\Rightarrow$ metoda se folosește în special pt. matrici A simetrice (pt care $C^{(i-1)}_{ii}$ nu e niciodată zero dacă A e inversabilă)

Eliminare Gaussiana cu pivotare

- pp. schimbarea a doua linii (sau coloane) ale matricii $C^{(i-1)}$ a.i. elementul diagonal de pe pozitia i este nenul si preferabil mare
- metoda uzuala, pivotarea pe linii
 - calculeaza $C^{(i-1)}$
 - gaseste k a.i. $|C^{(i-1)}_{ki}| = \max_{j \geq i} |C^{(i-1)}_{ji}|$
 - schimba liniile k si i intre ele
 - apoi calculeaza $C^{(i)}$ ca mai inainte

Comparatie sisteme tridiagonale – eliminare Gaussiana

- metoda reducerii par-impare e un caz particular al eliminarii gaussiene
- se elimina variabile prin inlocuirea lor cu functii dependente de variabilele care raman
- structura speciala tridiagonala permite eliminarea a jumătate din variabile într-un singur pas
- variabilele nu sunt eliminate in ordine (intai se elimina variabilele impare)
=> e vorba de o eliminare Gaussiana pe o permutare a matricii A care muta liniile impare la inceputul matricii
 - consecinta: variabilele impare sunt primele eliminate
- reducerea par-impara e echivalenta cu eliminarea gaussiana fara pivotare pt. ca elimina variabilele intr-o anumita ordine
 - daca e nevoie de operatie de pivotare, metoda reducerii par-impare nu mai e aplicabila (pivotarea afecteaza structura tridiagonala)

Implementarea paralela a eliminarii Gaussiene fara pivotare

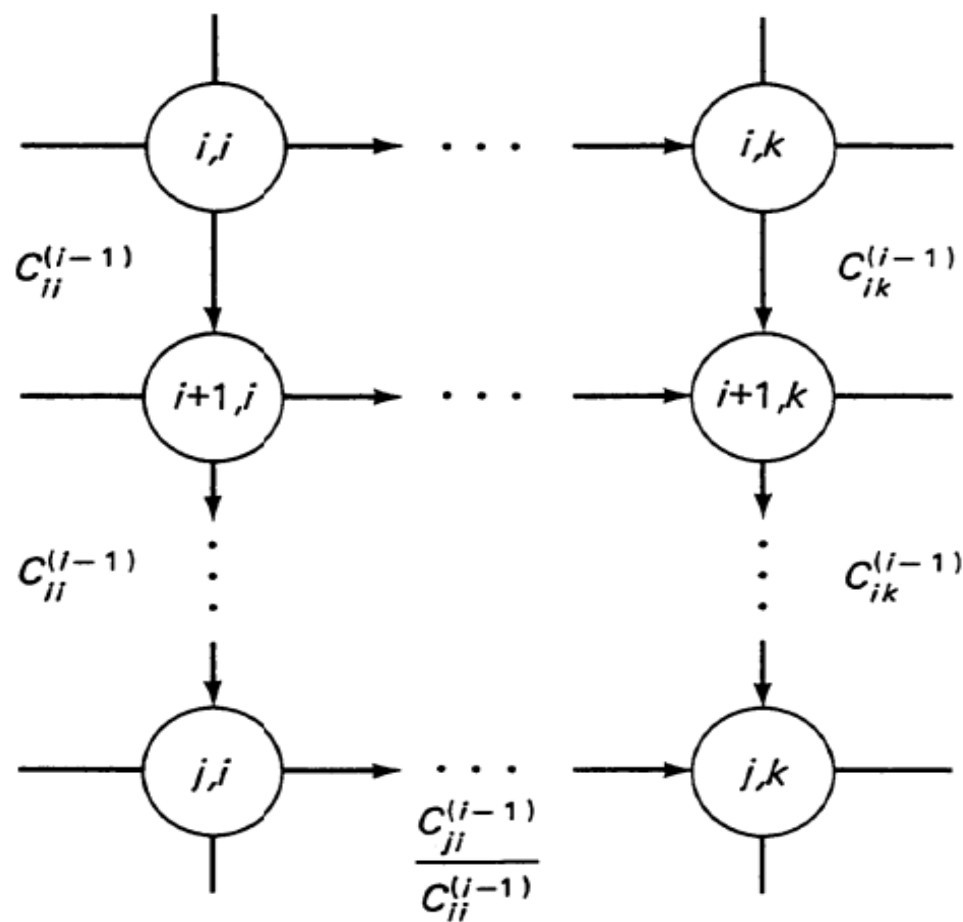
- ignoram costul comunicatiei

$$C_{jk}^{(i)} = C_{jk}^{(i-1)} - \sum_{\ell=1}^n N_{j\ell}^{(i)} C_{\ell k}^{(i-1)} = C_{jk}^{(i-1)} - N_{ji}^{(i)} C_{ik}^{(i-1)} = C_{jk}^{(i-1)} - \frac{C_{ji}^{(i-1)}}{C_{ii}^{(i-1)}} C_{ik}^{(i-1)}.$$

- folosind n^2 procesoare, toate elementele $C^{(i)}$ se pot calcula in $O(1)$ (o inmultire, o impartire si o scadere)
- deci $C^{(n-1)}$ se poate calcula in cca $3n$ pasi folosind n^2 procesoare
- cu n procesoare $\Rightarrow O(n^2)$
- eficienta: $\Theta(1)$ pt benchmark serial $T^*(n) = \Theta(n^3)$ si n^2 sau n procesoare

Eliminarea Gaussiana fara pivotare pe grid

- pp. grid patrat cu n^2 procesoare $\Rightarrow O(n)$
- fiecare procesor se asociaza cu un element al matricilor manipulate
- datele se transmit ca in figura alaturata
- o etapa are nevoie de $\Theta(n)$ pasi $\Rightarrow$ daca noua etapa porneste abia dupa ce s-a incheiat etapa anterioara $\Rightarrow \Theta(n^2)$ pasi
- cu interleaving comunicatie/calcul $\Rightarrow \Theta(n)$ pasi



Eliminarea Gaussiana fara pivotare pe grid

- pp. grid patrat cu n^2 procesoare $\Rightarrow O(n)$
- fiecare procesor se asociaza cu un element al matricilor manipulate
- pt a mentine timpul de executie liniar, e nevoie de pipelining-ul comunicatiei cu calculul
 - calculele fazei $i + 1$ incep inainte ca toate mesajele trimise in faza i sa fie primite
 - astfel, timpul de comunicatie este de acelasi ordine de marime cu timpul de calcul
 - altfel, timpul de executie e patratic
 - daca timpul de necesar unei singure comunicatii e semnificativ mai mare decat timpul necesar unei singure operatii de calcul, e nevoie de cresterea granularitatii prin folosirea unui nr mai mic de procesoare

Eliminarea Gaussiana fara pivotare pe hipercub si lant linier

- pp. hipercub cu $\theta(n^2)$ procesoare $\Rightarrow O(n)$ pt ca grid-ul se poate mapa pe hipercub
- un lant linier cu n procesoare poate simula un grid $n \times n$ cu o reducere de $O(n)$ in termeni de viteza $\Rightarrow$ eliminarea Gaussiana se poate executa pe un lant linier in $O(n^2)$

Finalul metodei de rezolvare bazata pe eliminare Gaussiana

- presupune rezolvarea unui sistem triunghiular
- daca folosim back substitution, metoda seriala e $O(n^2)$ => neglijabil fata de $O(n^3)$ cat necesita eliminarea Gaussiana seriala
- in paralel insa, cand se folosesc n^2 procesoare, back substitution necesita $O(n)$, comparabil cu eliminarea Gaussiana => e importanta o implementare paralela eficienta pt back substitution

Summary of Gaussian Elimination

TABLE 2.1 Bounds on the timing of Gaussian elimination for several architectures. The upper bounds are the same as those obtained for the same number of processors, if communication is assumed instantaneous.

Number of Processors	Architecture	Without Pivoting	With Pivoting
n^2	Hypercube	$O(n)$	$O(n \log n)$
n^2	Mesh	$O(n)$	$\Omega(n^{4/3})$
$n^2 / \log n$	Hypercube	$O(n \log n)$	$O(n \log n)$
n	Hypercube	$O(n^2)$	$O(n^2)$
n	Linear Array	$O(n^2)$	$O(n^2)$

Sisteme de ecuatii liniare

-Metode iterative-

Sisteme de ecuatii liniare

Fie matricea $A \in R^{n \times n}$, aflatii solutia sistemului:

$$Ax = b; \quad x = \begin{bmatrix} x_1 \\ \dots \\ x_n \end{bmatrix}$$

- **Metode directe:** Eliminare gaussiana, Factorizare LU etc.
 - afla o **solutie exacta** intr-un nr. finit de operatii
 - in general, complexitate $O(n^3)$; recomandate pentru dim. mici
- **Metode iterative: Jacobi, Gauss-Seidel**
 - afla o solutie inexacta, de precizie acceptabila (converg asimptotic catre una exacta)
 - complexitate = numar de iteratii (din rata de convergenta)
 - ex: $\|x^{k+1} - x^*\| \leq \frac{1}{2} \|x^k - x^*\| \Rightarrow \|x^k - x^*\| \leq \left(\frac{1}{2}\right)^k \|x^0 - x^*\|$
 - $\|x^k - x^*\| \leq \epsilon \Rightarrow k \geq O\left(\log\left(\frac{\|x^0 - x^*\|}{\epsilon}\right)\right)$

Sisteme de ecuatii liniare

$$Ax = b$$

Este echivalent cu

$$x_1 = -\frac{1}{A_{11}}(A_{12}x_2 + \cdots + A_{1n}x_n - b_1)$$

$$x_2 = -\frac{1}{A_{22}}(A_{21}x_1 + \cdots + A_{2n}x_n - b_2)$$

...

$$x_n = -\frac{1}{A_{nn}}(A_{n1}x_1 + \cdots + A_{nn-1}x_{n-1} - b_n)$$

Sisteme de ecuatii liniare

- daca $A_{ii} \neq 0$ si $\forall j, j \neq i, x_j$ cunoscut $\Rightarrow x_i$
- daca se cunosc insa doar valori aproximative (estimate) ale componentelor x_j , atunci se poate calcula o *valoare estimata* a lui x_i
- calculul valorilor estimate se poate face simultan pt toate componentele lui x
- rezultatul este un algoritm iterativ Jacobi
 - se porneste cu o valoare initiala $x(0) \in R^n$
 - se evalueaza $x(t), t = 1, 2, \dots$ cf iteratiei
$$x_i(t+1) = -\frac{1}{A_{ii}} \left(\sum_{j \neq i} A_{ij} x_j(t) - b_i \right)$$
 - daca secventa converge la o valoare, atunci acea valoare e solutia sistemului de ecuatii liniare
 - Obs: e perfect posibil ca secventa sa nu converga

Algoritmul Jacobi

$$Ax = b$$

Consideram un sir iterativ $x(0), x(1), \dots \in R^n$, daca $x(t) \rightarrow x^*, t \rightarrow \infty \Rightarrow x^*$ e solutia sistemului de ecuatii liniare

La pasul $t + 1$:

$$x_1(t + 1) = -\frac{1}{A_{11}}(A_{12}x_2(t) + \dots + A_{1n}x_n(t) - b_1)$$

$$x_2(t + 1) = -\frac{1}{A_{22}}(A_{21}x_1(t) + \dots + A_{2n}x_n(t) - b_2)$$

...

$$x_n(t + 1) = -\frac{1}{A_{nn}}(A_{n1}x_1(t) + \dots + A_{nn-1}x_{n-1}(t) - b_n)$$

Algoritmul Jacobi pt iteratie convergenta

Fie sistemul:
$$\begin{cases} 2x_1 - x_2 = 0 \\ -x_1 + 2x_2 = 0 \end{cases}$$

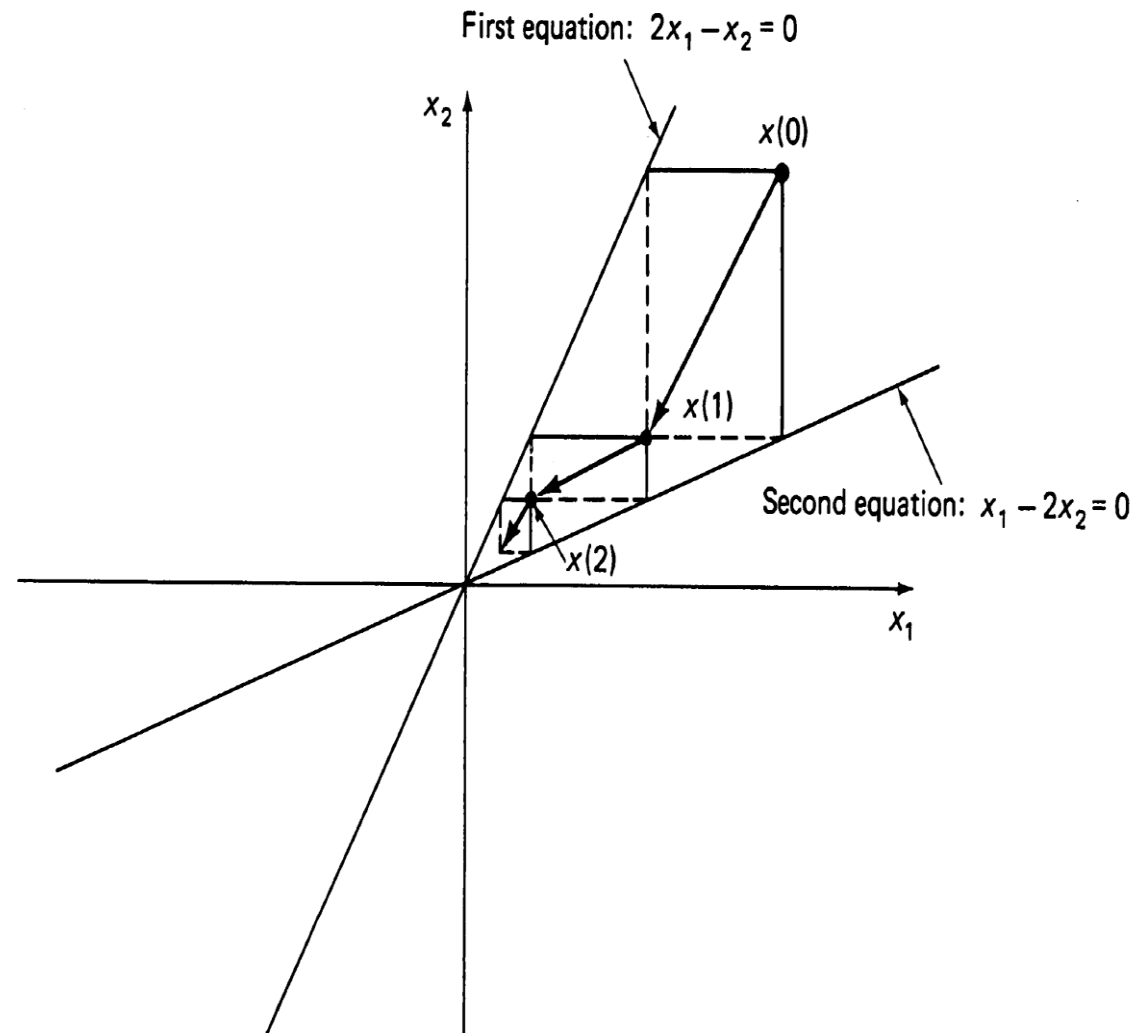
$$\begin{cases} x_1 = \frac{x_2}{2} \\ x_2 = \frac{x_1}{2} \end{cases}$$
 Pornind din $x(0) = \begin{bmatrix} 4 \\ 4 \end{bmatrix}$, iteram:

$$x_1(1) = \frac{x_2(0)}{2} = 2$$

$$x_2(1) = \frac{x_1(0)}{2} = 2$$

$$x_1(t) = \frac{x_2(t-1)}{2} = x_2(0) \left(\frac{1}{2}\right)^t \rightarrow 0$$

$$x_2(t) = \frac{x_1(t-1)}{2} = x_1(0) \left(\frac{1}{2}\right)^t \rightarrow 0$$



Algoritmul Jacobi pt iteratie divergenta

Fie sistemul:
$$\begin{cases} -x_1 + 2x_2 = 0 \\ 2x_1 - x_2 = 0 \end{cases}$$

Matriceal:
$$\begin{bmatrix} -1 & 2 \\ 2 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Pornind din $x(0) = \begin{bmatrix} 4 \\ 4 \end{bmatrix}$, iteram:

$$x_1(1) = 2x_2(0) = 8$$

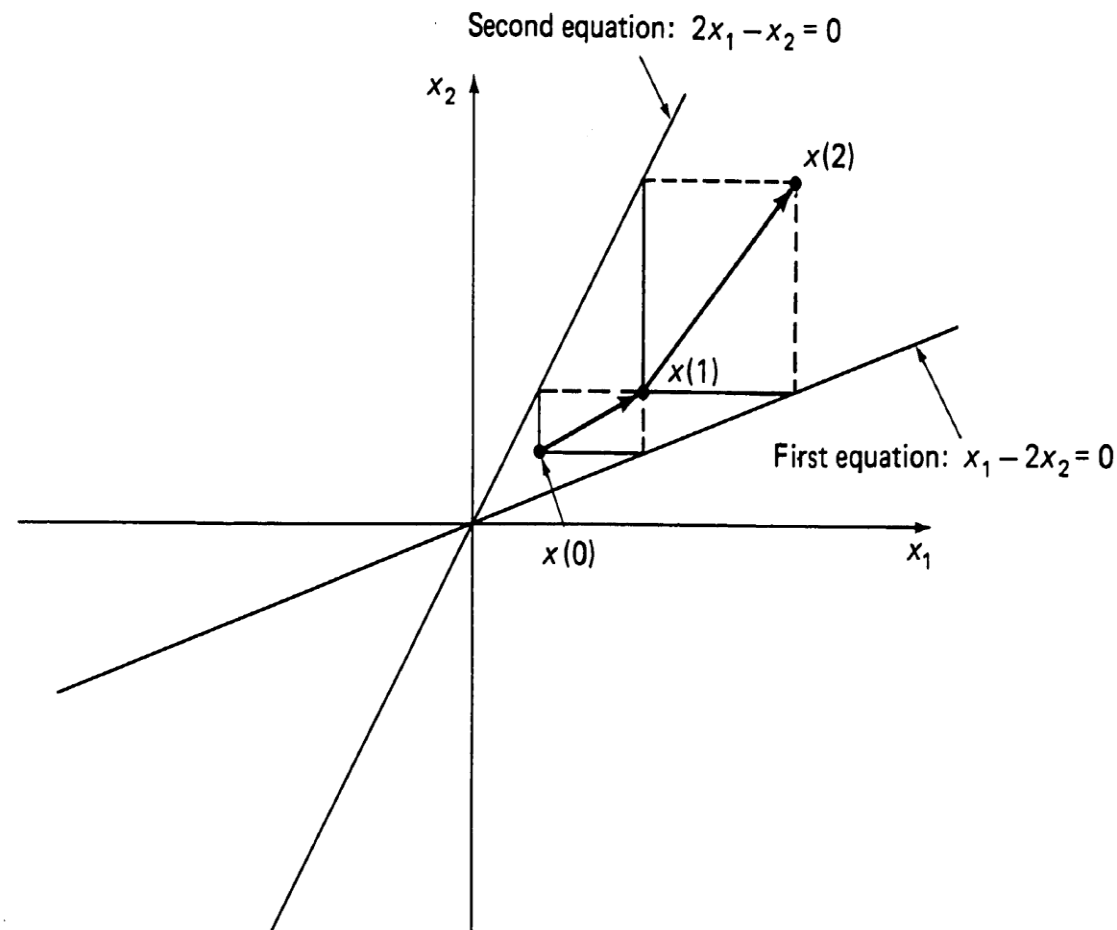
$$x_2(1) = 2x_1(0) = 8$$

$$x_1(2) = 2x_2(1) = 16$$

$$x_2(2) = 2x_1(1) = 16$$

$$x_1(t) = 2x_2(t-1) = 2^t x_2(0)$$

$$x_2(t) = 2x_1(t-1) = 2^t x_1(0)$$



Algoritmul Jacobi

- O matrice A este **diagonal dominanta** (pe linii) daca:

$$\sum_{j \neq i} |A_{ij}| < |A_{ii}|$$

- Exemplu: $\begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}$ este diagonal dominanta, $\begin{bmatrix} -1 & 2 \\ 2 & -1 \end{bmatrix}$ nu este!

Teorema: Daca A este diagonal dominanta, atunci sirul generat de algoritmul Jacobi:

$$x_i(t + 1) = -\frac{1}{A_{ii}} \left(\sum_{j \neq i} A_{ij} x_j(t) - b_i \right)$$

converge.

Algoritmul Jacobi

$$Ax = b$$

Este echivalent cu

$$x_i = -\frac{1}{A_{ii}} \left(\sum_{j \neq i} A_{ij} x_j - b_i \right)$$

Idea algoritmului Jacobi:

$$x_i(t+1) = -\frac{1}{A_{ii}} \left(\sum_{j \neq i} A_{ij} x_j(t) - b_i \right)$$

Observati ca la pasul $t+1$ fiecare $x_i(t+1)$ poate fi evaluat complet paralel!

Este necesar ca fiecare procesor sa aiba $x(t)$!

Algoritmul Jacobi paralel

- am discutat deja paralelizarea metodelor iterative
- cea mai mare parte a calculului revine la inmultiri de matrici cu vectori
- ne concentram pe sisteme distribuite (message passing)
- ipoteze de lucru
 - pp n procesoare disponibile, procesorul i calculeaza $x_i(t)$ la fiecare iteratie t
 - procesorul i stocheaza linia i a matricii A
- pt a calcula $x_i(t + 1)$ procesorul i are nevoie de valorile $x_j(t)$ calculate de procesoarele j la iteratia t pt care $A_{ij} \neq 0$

Algoritmul Jacobi paralel

$$x_i(t+1) = -\frac{1}{A_{ii}} \left(\sum_{j \neq i} A_{ij} x_j(t) - b_i \right)$$

Obs: la pasul $t+1$ fiecare $x_i(t+1)$ poate fi calculat complet paralel!

La fiecare iteratie t :

- P_i calculeaza $x_i(t)$, produs scalar $O(n)$ (pp. A densa, fara structura speciala, sparse)
- difuzare generala (broadcast multinod): P_i difuzeaza $x_i(t)$
 - $O(n)$ pe lant liniar sau grid, $O(\frac{n}{\log n})$ pe hipercub
- procentul de timp consumat in comunicatie scade cu valoarea lui n pt hipercub si ramane constant pt lant liniar
 - daca acest factor constant e mare, crestem granularitatea (mai multe linii stocate pe mai putine procesoare)

Algoritmul paralel Jacobi pt matrici sparse

- matricea A e sparse
- structura rara a matricii e descrisa de un graf $G = (V, E)$ unde

$$E = \{(i, j) | i \neq j \text{ si } A_{ji} \neq 0 \text{ a. i. procesorul } i \text{ comunica cu } j\}$$

- acesta e un graf de dependente asa cum am vazut anterior
- implementari paralele eficiente implica maparea eficienta a acestui graf de dependente pe topologii speciale a.i.
 - toate comunicatiile au loc intre procesoare vecine in topologie
 - penalizarea de comunicare e minimala

Algoritmul Gauss-Seidel

$$Ax = b$$

sau echivalent

$$x_i = -\frac{1}{A_{ii}} \left(\sum_{j \neq i} A_{ij} x_j - b_i \right)$$

- daca o iteratie Jacobi

$$x_i(t+1) = -\frac{1}{A_{ii}} \left(\sum_{j \neq i} A_{ij} x_j(t) - b_i \right)$$

se executa secvential, la momentul evaluarii $x_i(t+1)$ exista deja cateva valori estimate noi $x_j(t+1)$ pt $j < i$

Idee: folosirea acestor noi valori estimate x_j pt $j < i$ in estimarea lui x_i

Algoritmul Gauss-Seidel (2)

Idee:

- se porneste cu o valoare initiala $x(0) \in R^n$
- se evalueaza $x(t), t = 1, 2, \dots$ cf iteratiei

$$x_i(t+1) = -\frac{1}{A_{ii}} \left(\sum_{j<i} A_{ij}x_j(t+1) + \sum_{j>i} A_{ij}x_j(t) - b_i \right)$$

Obs:

- $x_i(t+1)$ depinde de $x_j(t+1)$ pt. $j < i$
- se poate paraleliza doar in cazuri in care matricea A are structura speciala (e.g. A rara)

Algoritmul Gauss-Seidel (3)

$$Ax = b$$

Este echivalent cu

$$x_1 = -\frac{1}{A_{11}}(A_{12}x_2 + A_{13}x_3 \cdots + A_{1n}x_n - b_1)$$

$$x_2 = -\frac{1}{A_{22}}(A_{21}x_1 + A_{23}x_3 \cdots + A_{2n}x_n - b_2)$$

$$x_3 = -\frac{1}{A_{33}}(A_{31}x_1 + A_{32}x_2 \cdots + A_{3n}x_n - b_3)$$

...

$$x_n = -\frac{1}{A_{nn}}(A_{n1}x_1 + A_{n2}x_2 \cdots + A_{nn-1}x_{n-1} - b_n)$$

Algoritmul Gauss-Seidel (4)

$$Ax = b$$

Este echivalent cu

$$x_1(t+1) = -\frac{1}{A_{11}}(A_{12}x_2(t) + A_{13}x_3(t) \cdots + A_{1n}x_n(t) - b_1)$$

$$x_2(t+1) = -\frac{1}{A_{22}}(A_{21}x_1(t+1) + A_{23}x_3(t) \cdots + A_{2n}x_n(t) - b_2)$$

$$x_3(t+1) = -\frac{1}{A_{33}}(A_{31}x_1(t+1) + A_{32}x_2(t+1) \cdots + A_{3n}x_n(t) - b_3)$$

...

$$x_n(t+1) = -\frac{1}{A_{nn}}(A_{n1}x_1(t+1) + A_{n2}x_2(t+1) \cdots + A_{nn-1}x_{n-1}(t+1) - b_n)$$

Algoritmul Gauss-Seidel pt iteratie convergenta

Fie sistemul $\begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$

$$x_1(t+1) = \frac{x_2(t)}{2}$$

$$x_2(t+1) = \frac{x_1(t+1)}{2}$$

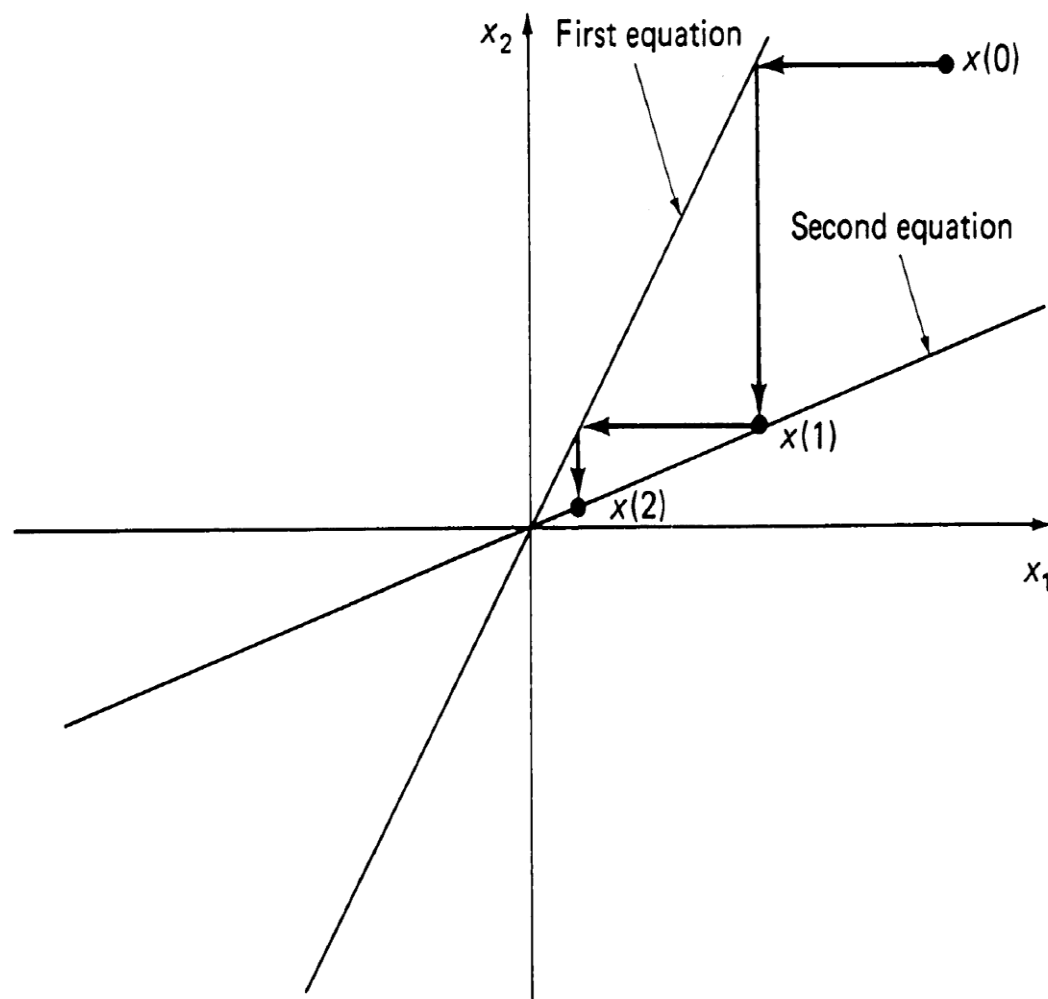
Pornind din $x(0) = \begin{bmatrix} 4 \\ 4 \end{bmatrix}$, iteram:

$$x_1(1) = \frac{x_2(0)}{2} = 2$$

$$x_2(1) = \frac{x_1(1)}{2} = 1$$

$$x_1(2) = \frac{x_2(1)}{2} = \frac{1}{2}$$

$$x_2(2) = \frac{x_1(2)}{2} = \frac{1}{4}$$



Algoritmul Gauss-Seidel pt iteratie divergenta

Fie sistemul $\begin{bmatrix} -1 & 2 \\ 2 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$

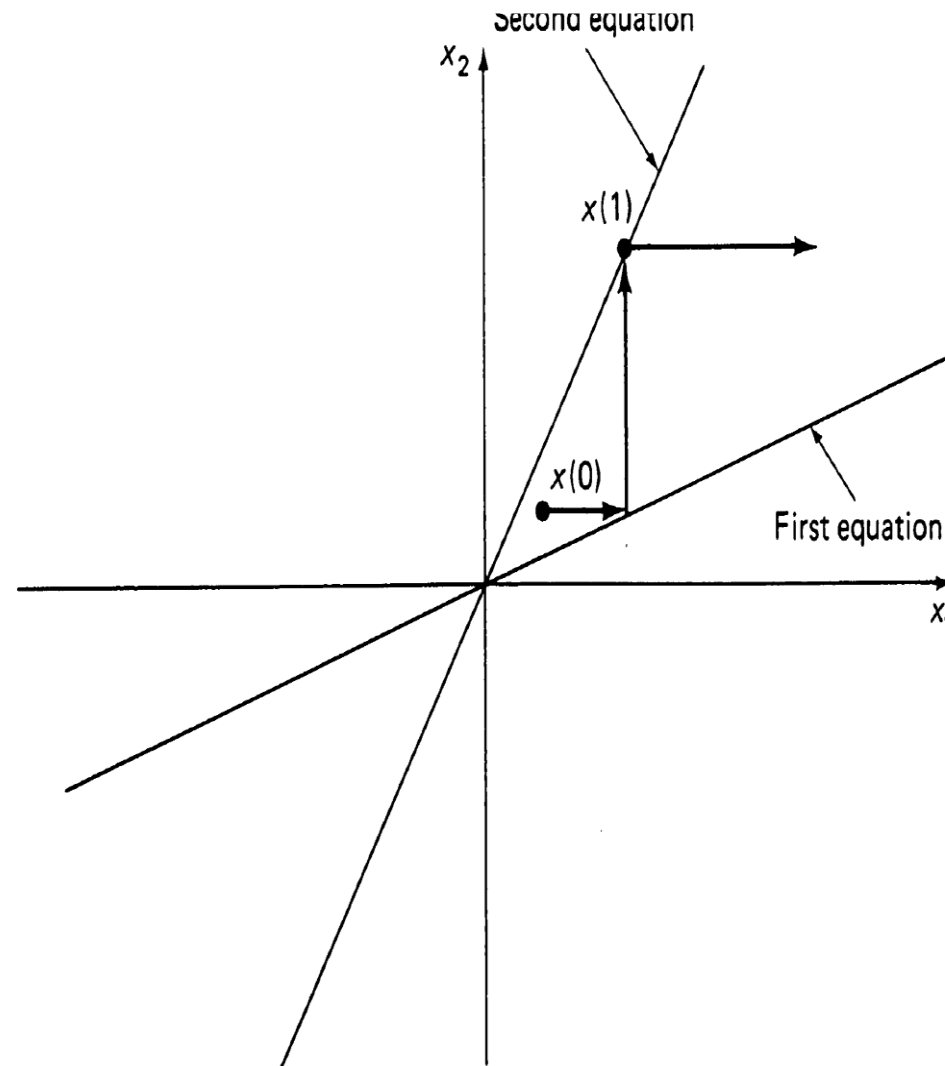
Pornind din $x(0) = \begin{bmatrix} 4 \\ 4 \end{bmatrix}$, iteram:

$$x_1(1) = 2x_2(0) = 8$$

$$x_2(1) = 2x_1(1) = 16$$

$$x_1(2) = 2x_2(1) = 32$$

$$x_2(2) = 2x_1(2) = 64$$



Convergenta algoritmului Gauss-Seidel

Teorema: Daca A este diagonal dominanta, atunci sirul generat de algoritmul Gauss-Seidel

$$x_i(t + 1) = -\frac{1}{A_{ii}} \left(\sum_{j < i} A_{ij} x_j(t + 1) + \sum_{j > i} A_{ij} x_j(t) - b_i \right)$$

converge.

Paralelizarea algoritmului Gauss-Seidel

$$x_i(t+1) = -\frac{1}{A_{ii}} \left(\sum_{j<i} A_{ij}x_j(t+1) + \sum_{j>i} A_{ij}x_j(t) - b_i \right)$$

- A matrice complet densa (*i.e.*, $A_{ij} \neq 0 \forall i, j$)
 - cand procesorul i calculeaza $x_i(t+1)$ are nevoie de $x_j(t+1)$ pt $\forall j < i$
 - algoritm inherent secvential, nu se pot calcula in paralel componente x_i diferite
- A matrice rara
 - se pot folosi scheme de colorare a grafurilor
 - reminder: exista o ordine de evaluare a variabilelor in k pasi paraleli $\Leftrightarrow$ exista o colorare optimala cu k culori a grafului de dependenta

Algoritmul Gauss-Seidel paralel

$$x_i(t+1) = -\frac{1}{A_{ii}} \left(\sum_{j<i} A_{ij}x_j(t+1) + \sum_{j>i} A_{ij}x_j(t) - b_i \right)$$

La pasul t :

- P_1 calculeaza $x_1(t)$, produs scalar $O(n)$
- difuzare: P_1 difuzeaza pe $x_1(t)$ catre $P_2, \dots, P_n$
- ...
- P_i primeste $x_{i-1}(t)$
- P_i calculeaza $x_i(t)$, produs scalar $O(n)$
- difuzare: P_i difuzeaza pe $x_i(t)$ catre $P_{i+1}, \dots, P_n$

Terminarea algoritmilor iterativi

- conditie tipica de terminare

$$\|Ax(t) - b\| < \varepsilon$$

unde ε e o valoare suficient de mica

- pt matrici dense overhead-ul utilizarii normei nu e semnificativ

- ex: norma infinit

- la fiecare iteratie t , fiecare procesor evalueaza

$$\max_i |[Ax]_i - b_i|$$

peste indicii i ai componentelor asignate acelui procesor

- aceste valori se compara folosind un arbore de acoperire (spanning tree)

=> fiecare procesor propaga catre root maximul dintre valorile primite si cea calculate local

=> detectarea terminarii are nevoie doar de acumulare intr-un nod (penalizare mica fata de broadcastul/acumularea multinod executate in fiecare iteratie)

Terminarea algoritmilor iterativi pt matrici sparse

- A matrice sparse si variabile asiguate procesoarelor a.i. fiecare iteratie nu necesita decat comunicare cu cel mai apropiat vecin in graful de dependente
 - timp de comunicare per iteratie proportional cu nr de variabile asiguate fiecarui procesor
 - timpul de comunicare necesar testarii terminarii algoritmului proportional cu diametrul retelei de interconectare a procesoarelor (topologiei)
- ⇒ daca diametrul nu e comparabil (sau mai mic) decat nr variabilelor per procesor, testarea terminarii se face din cand in cand

Alternativ, testarea terminarii se poate face in paralel cu executia iteratiilor
⇒ $Ax = b$ va fi invecitata la momentul luarii deciziei de terminare

- in practica, rareori acest lucru are consecinte adverse

Algoritmi paraleli

Curs 8

Vlad Olaru

vlad.olaru@fmi.unibuc.ro

Calculatoare si Tehnologia Informatiei

Universitatea din Bucuresti

Algoritmi asincroni
-Metode iterative-

Algoritmul Jacobi sincron

$$Ax = b$$

Este echivalent cu

$$x_1 = -\frac{1}{A_{11}}(A_{12}x_2 + \cdots + A_{1n}x_n - b_1)$$

$$x_2 = -\frac{1}{A_{22}}(A_{21}x_1 + \cdots + A_{2n}x_n - b_2)$$

...

$$x_n = -\frac{1}{A_{nn}}(A_{n1}x_1 + \cdots + A_{nn-1}x_{n-1} - b_n)$$

Algoritmul Jacobi sincron

$$Ax = b$$

La pasul $t + 1$:

$$x_1(t + 1) = -\frac{1}{A_{11}}(A_{12}x_2(t) + \cdots + A_{1n}x_n(t) - b_1)$$

$$x_2(t + 1) = -\frac{1}{A_{22}}(A_{21}x_1(t) + \cdots + A_{2n}x_n(t) - b_2)$$

...

$$x_n(t + 1) = -\frac{1}{A_{nn}}(A_{n1}x_1(t) + \cdots + A_{nn-1}x_{n-1}(t) - b_n)$$

Algoritmul Jacobi sincron

Ideea algoritmului Jacobi sincron:

$$x_i(t+1) = -\frac{1}{A_{ii}} \left(\sum_{j \neq i} A_{ij} x_j(t) - b_i \right)$$

Echivalent: $[x(t+1) = D^{-1}(b - Rx(t))]$

Sincronizare:

Pentru calculul $x_i(t+1)$, P_i asteapta $x_j(t)$ de la P_j , unde $j = 1, \dots, p, j \neq i$

Este necesar ca fiecare procesor sa aiba $x(t)$!

Algoritmul Jacobi sincron

- O matrice A este **diagonal dominanta** (pe linii) daca:

$$\sum_{j \neq i} |A_{ij}| < |A_{ii}|$$

- Exemplu: $\begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}$ este diagonal dominanta, $\begin{bmatrix} -1 & 2 \\ 2 & -1 \end{bmatrix}$ nu este!

Teorema: Daca A este diagonal dominanta, atunci sirul generat de algoritmul Jacobi:

$$x_i(t+1) = -\frac{1}{A_{ii}} \left(\sum_{j \neq i} A_{ij} x_j(t) - b_i \right)$$

converge.

Algoritmi asincroni

- dorim paralelizarea unui algoritm pe o arhitectura in care **fiecare procesor ruleaza un (sub)algoritm local**.
- algoritmi locali nu sunt constransi sa astepte la momente predeterminate mesaje predeterminate
- procesoarele ruleaza la viteze diferite => unele executa un nr mai mare de iteratii
- unele procesoare comunica mai des decat altele
- intarzierile mesajelor pot fi substatale si sunt impredictibile
- mesajele pot fi primite in alta ordine decat cea in care au fost trimise

Avantaje algoritmi asincroni

- reducerea penalizarilor de sincronizare

⇒ castig de viteza comparativ cu algoritmi sincroni

- acest castig poate fi diminuat de o complexitate sporita a comunicarii
- flexibilitate sporita a implementarii
- toleranta mai mare pentru schimbari ale datelor problemei

Dezavantaje algoritmi asincroni

- conditiile de validitate sunt in general mai stringente decat cele ale algoritmilor sincroni
- detectia terminarii algoritmului e indeobste mai dificila
- comportament particular fata de timpi de comunicare si calcul cu durata arbitrara
 - *algoritmi total asincroni (s.n. si haotici)* – tolereaza intarzieri de comunicatie/calcul oricat de mari
 - *algoritmi partial asincroni* – ruleaza corect (converg) doar in conditiile in care intarzierile sunt limitate
 - => mecanismele de convergenta (si analiza lor) sunt fundamental diferite

Modelarea algoritmilor total asincroni

Consideram:

$$X = X_1 \times X_2 \times \dots \times X_n$$

si

$$x \in X \text{ a.i. } x = (x_1, x_2, \dots, x_n) \text{ si } x_i \in X_i \forall i = 1, \dots, n$$

Problema: fie functia $f: X \rightarrow X, f(x) = (f_1(x), f_2(x), \dots, f_n(x))$ a.i.

$$f_i: X \rightarrow X_i \text{ si } x_i = f_i(x), \quad \forall i = 1, \dots, n$$

Sa se determine punctul fix al lui f , adica $x^* \in X$ a.i. $x^* = f(x^*)$ sau

$$x_i^* = f_i(x_i^*)$$

Notatii:

$x_i(t)$ = valoarea componentei x_i la momentul t .

$T = \{0, 1, 2, \dots\}$ setul timpilor la care una sau mai multe componente x_i ale lui x sunt actualizate de un procesor sau altul din sistemul distribuit

T^i = setul momentelor de timp la care este actualizat x_i ; $T^1 = \{0, 2, 5, 6, \dots\}$

Modelarea algoritmilor total asincroni

Pp procesorul P_i nu are neaparat access la cele mai noi valori ale componentelor lui x pentru actualizarea lui x_i :

$$x_i(t+1) = f_i \left(x_1 \left(\tau_1^i(t) \right), x_2 \left(\tau_2^i(t) \right), \dots, x_n \left(\tau_n^i(t) \right) \right), \forall t \in T^i$$

unde:

$$\begin{aligned} \tau_j^i(t) &= \text{momentele de timp ale actualizarilor a.i.} \\ 0 &\leq \tau_j^i(t) \leq t, \quad \forall t \in T. \end{aligned}$$

De asemenea, $x_i(t+1) = x_i(t), \quad \forall t \notin T^i$

Modelarea algoritmilor total asincroni - observatii

- T poate fi vazut si ca un set de indecsi al momentelor de timp la care se petrec actualizarile
- multimile T^i (si secventele de timpi fizici asociate) nu sunt necesare tuturor procesoarelor pt ca nu sunt folosite in iteratie => nu e nevoie de ceas global sau de sincronizarea ceasurilor locale ale procesoarelor
- diferenta $t - \tau_j^i(t)$ reprezinta intarzierea de comunicatie intre timpul curent si timpul la care componenta j este disponibila pentru actualizarea $x_i(t)$

Model conceptual util

- P_i se trezeste spontan la momentul $t \in T^i$
- P_i primeste printr-un mechanism oarecare valorile

$$x_1\left(\tau_1^i(t)\right), x_2\left(\tau_2^i(t)\right), \dots, x_n\left(\tau_n^i(t)\right)$$

- P_i actualizeaza x_i
- actualizarea se face fara cunostinta valorilor $t, \tau_1^i(t), \dots, \tau_n^i(t)$ sau a oricarui element din $T^j, j = 1, \dots, n$
- obs: Jacobi, Gauss-Seidel (si variantele bloc iterative) sunt cazuri speciale ale acestui tip de iteratie

Model explicitat

- P_i stocheaza local $x^i(t) = (x_1^i(t), x_2^i(t), \dots, x_n^i(t))$, pe baza caruia actualizeaza $x_i^i(t)$ la $t \in T^i$ prin relatia:

$$x_i^i(t+1) = f_i(x^i(t))$$

- ocazional (la momente nespecificate), P_i transmite x_i^i celorlalte procesoare
- cand P_j primeste (dupa o intarziere oarecare, nespecificata) noul x_i^i , il memoreaza in componenta i local stocata a lui x_i^j ; la acest moment avem

$$x_i^j(t) = x_i^i(\tau_i^j(t)) \quad \text{iar } (t - \tau_i^j(t)) \text{ este intarzierea de comunicare}$$

De asemenea, in mod natural $\tau_i^i(t) = t$

- Obs: - in general, vectorii locali $x^i(t)$ au valori diferite pe procesoare diferite la acelasi moment de timp t !
- nu exista nici o presupunere asupra ordinii transmiterii mesajelor
- procesoarele nu pot determina daca valorile actualizarilor primite sunt mai vechi decat valorile stocate in memoria locala
- **Scop final:** $x(t) = (x_1^1(t), x_2^2(t), \dots, x_n^n(t))$ converge catre solutia $x = f(x)$.

Exemplu algoritm total asincroni

Determinati x astfel incat

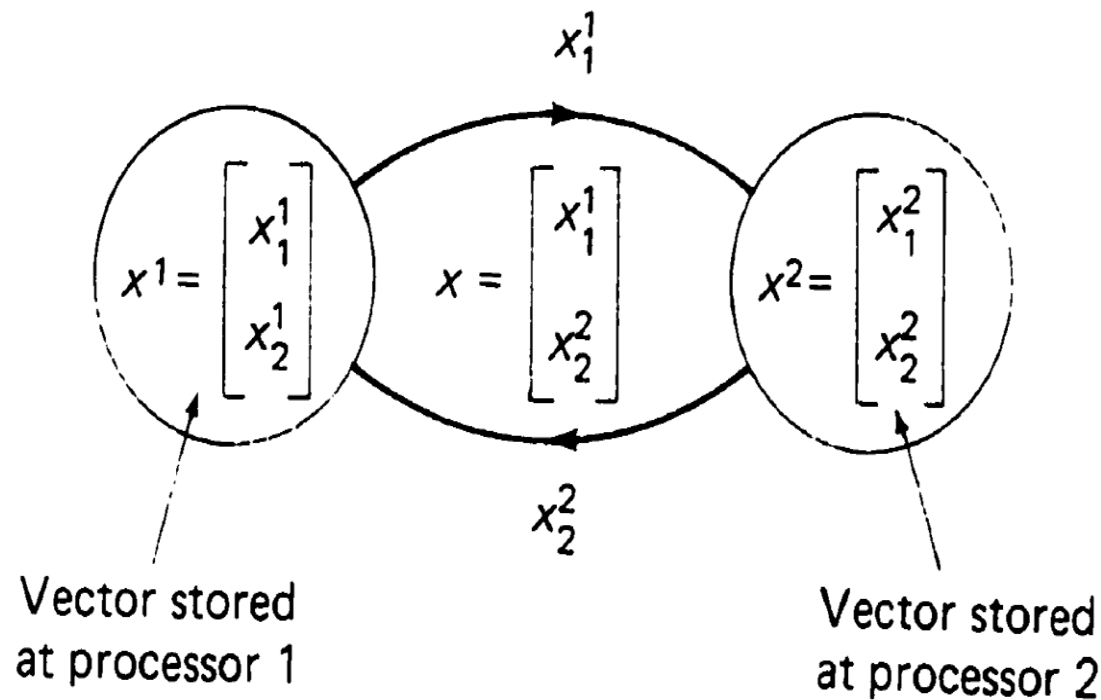
$$x_1 = 0.5x_1 + 0.5x_2$$

$$x_2 = 0.2x_1 + 0.7x_2$$

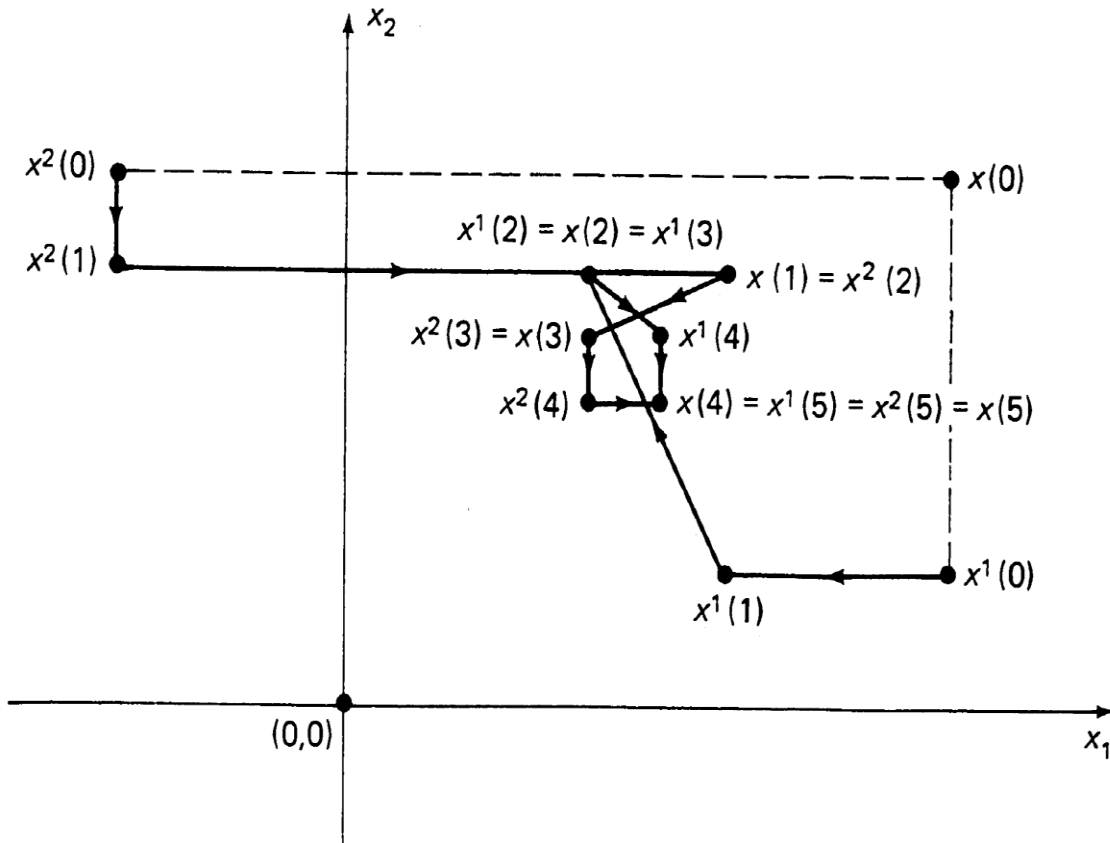
2 procesoare P_1 si P_2 :

P_1 stocheaza $x^1 = (x_1^1, x_2^1)$ si actualizeaza x_1^1 la momentele $t \in T^1$

P_2 stocheaza $x^2 = (x_1^2, x_2^2)$ si actualizeaza x_2^2 la momentele $t \in T^2$



Exemplu algoritmi total asincroni



- t=0:** Processor 1 updates x_1^1 and transmits it to processor 2, where it is received at a time between $t = 1$ and $t = 2$. Processor 2 updates x_2^2 and transmits it to processor 1, where it is received at a time between $t = 1$ and $t = 2$.
- t=1:** Processor 1 updates x_1^1 and transmits it to processor 2, where it is received at a time between $t = 2$ and $t = 3$. [Processor 2 does not update X_2^2 but X_1^1 will change at some time $t \in (1, 2)$ because of the reception of $X_1^1(1)$; for this reason, $x^2(1) \neq x^2(2)$ as shown in the figure.]
- t=2:** Processor 2, having received $x_1^1(1)$, updates x_2^2 and transmits it to processor 1, where it is received at a time between $t = 3$ and $t = 4$. [Processor 1 does not update X_1^1 and does not receive a value of X_2^2 from Processor 2 at any time $t \in (2, 3)$; for this reason, $X^1(2) = X^1(3)$ as shown in the figure.]
- t=3:** Processor 1, having received $x_2^2(1)$ [at some time $t \in (1, 2)$], updates x_1^1 and transmits it to processor 2, where it is received at a time between $t = 4$ and $t = 5$. Processor 2, having received $x_1^1(2)$, updates x_2^2 and transmits it to processor 1, where it is received at a time between $t = 4$ and $t = 5$.
- t=4:** Processor 1 has already received $x_2^2(3)$; no updating takes place.
- t=5:** Processor 1 has already received $x_2^2(4)$ and processor 2 has already received $x_1^1(4)$.

Teorema de convergenta asincrona

- Fie secventa de multimi nevide $\{X(k)\}$ a.i.

$$\dots \subset X(k+1) \subset X(k) \subset \dots \subset X$$

si

(a) *conditia de convergenta sincrona*

$$f(x) \in X(k+1) \forall k \text{ si } x \in X(k).$$

In plus, daca $\{y^k\}$ e o secventa a.i. $y^k \in X(k) \forall k \Rightarrow$ daca y punct limita al $\{y^k\}$, y punct fix al lui f

si

(b) *box condition*

$$\forall k \text{ exista } X_i(k) \subset X_i \text{ a.i. } X(k) = X_1(k) \times X_2(k) \times \dots \times X_n(k)$$

Atunci, daca $x(0) = (x_1(0), x_2(0), \dots, x_n(0)) \in X(0)$

si

$$\forall x^* \text{ a.i. } \{x(t)\} \rightarrow x^*$$

$$\Rightarrow f(x^*) = x^*$$

Iteratii asincrone pt. sisteme de ecuatii liniare

- fie functia

$$f(x) = Ax + b \text{ unde } A \in R^{n \times n} \text{ si } b \in R^n$$

- vrem sa gasim solutia x^* a.i.

$$x^* = Ax^* + b$$

folosind o iteratie asincrona definita astfel

$$x_i(t+1) = \sum_{j=1}^n A_{ij} x_j(\tau_j^i(t)) + b_i, \quad t \in T^i$$

$$x_i(t+1) = x_i(t), \quad t \notin T^i$$

Conditie de convergenta iteratie

• **Propozitie:** Fie $A \in R^{n \times n}$ a.i. $I - A$ inversabila. Atunci

(i) $\rho(|A|) < 1$ unde $\rho(A) = \max_{1 \leq i \leq n} |\lambda_i(A)|$ e *raza spectrala* a matricii A

$\Leftrightarrow$

(ii) $\forall x(0)$ *conditie initiala*, $\forall b \in R^n$ si

$\forall T^i$ a.i. fiecare T^i e infinita, $\forall \tau_j^i(t)$ a.i. $t - 2 \leq \tau_j^i(t) \leq t$, $\forall t$

secventa $\{x(t)\}$ converge la $(I - A)^{-1}b$

Concluzie: “intarzieri” $t - \tau_j^i(t)$ de doar doua unitati de timp pot induce divergenta cand $\rho(|A|) \geq 1$

Algoritmi iterativi partial asincroni

- dorim paralelizarea unui algoritm pe o arhitectura in care **fiecare procesor ruleaza un (sub)algoritm local**.
- am retinut ca nu toti algoritmii total asincroni converg
- pt. anumiti algoritmi asincroni exista posibilitatea de a “redeveni” convergenti daca se impun anumite constrangeri asupra gradului de asincronism
- pp. ca exista o constanta B (numita masura asincronismului) a.i.
 - (a) fiecare procesor executa o actualizare cel putin o data pe durata unui interval de timp de durata B
 - (b) informatia folosita de orice procesor nu este mai veche de B unitati de timp

Un algoritm asincron care satisface (a) si (b) s.n. *partial asincron*

Algoritmi iterativi partial asincroni

- analiza convergentei algoritmilor partial asincroni nu se poate reduce la un singur rezultat general de tipul *teoremei de convergenta asincrona* pt. algoritmi total asincroni
- exista doua tipuri de algoritmi partial asincroni
 - (a) algoritmi care nu converg total asincron, dar executia lor partial asincrona converge pt. orice valoare finita a lui B
 - (b) algoritmi care converg partial asincron pt valori suficient de mici ale lui B , iar pt valori mari ale lui B diverg

Modelarea algoritmilor partial asincroni

Consideram: $X = X_1 \times X_2 \times \dots \times X_n$ unde X_i sunt submultimi de spatii Euclidiene si

$$x \in X \text{ a.i. } x = (x_1, x_2, \dots, x_n) \text{ si } x_i \in X_i \forall i = 1, \dots, n$$

Problema: fie functia $f: X \rightarrow X, f(x) = (f_1(x), f_2(x), \dots, f_n(x))$ a.i.

$$f_i: X \rightarrow X_i \text{ si } x_i = f_i(x), \quad \forall i = 1, \dots, n$$

Executia iteratiei de mai sus e complet determinata de:

(a) Conditiiile initiale $x(t) \in X \forall t \leq 0$

(b) T^i = multimea momentelor de timp la care este actualizata componenta x_i

(c) variabilele $\tau_j^i(t) \forall i, j$ si $t \in T^i$ care:

- reprezinta durata de timp cu care informatia folosita la actualizarea x_i este invecchita

- satisfac conditia $0 \leq \tau_j^i(t) \leq t$

Ecuatii algoritmi partial asincroni

$$\forall t \geq 0$$

$$x_i(t+1) = f_i \left(x_1 \left(\tau_1^i(t) \right), x_2 \left(\tau_2^i(t) \right), \dots, x_n \left(\tau_n^i(t) \right) \right), \text{ daca } t \in T^i$$

$$x_i(t+1) = x_i(t), \quad \forall t \notin T^i$$

- o alegere particulara a multimilor T^i si a valorilor variabilelor $\tau_j^i(t)$ s.n. *scenariu*
- Obs: - pt. orice scenariu fixat, valoarea lui $x(t)$ pt. $t > 0$ e determinata unic de conditiile initiale
- pt scenarii diferite, valorile pot diferi

Presupuneri fundamentale asincronism partial

$\exists B \in \mathbb{N}$ a.i.

(a) $\forall i$ si $t \geq 0$ cel putin un element al multimii $\{t, t + 1, \dots, t + B - 1\}$ apartine T^i

(b) $t - B < \tau_j^i(t) \leq t, \forall i, j$ si $\forall t \geq 0, t \in T^i$

(c) $\tau_i^i(t) = t \forall i$ si $t \in T^i$

Obs: variabila t nu e accesibila procesoarelor

- t poate fi vazut ca o variabila de timp masurata de un observator extern
- t nu corespunde in mod necesar timpului real, poate fi doar o variabila care indexeaza timpul evenimentelor de interes (eg, actualizarea variabilelor)

Exemplu algoritm partial asincron care converge pt orice valoare a lui B

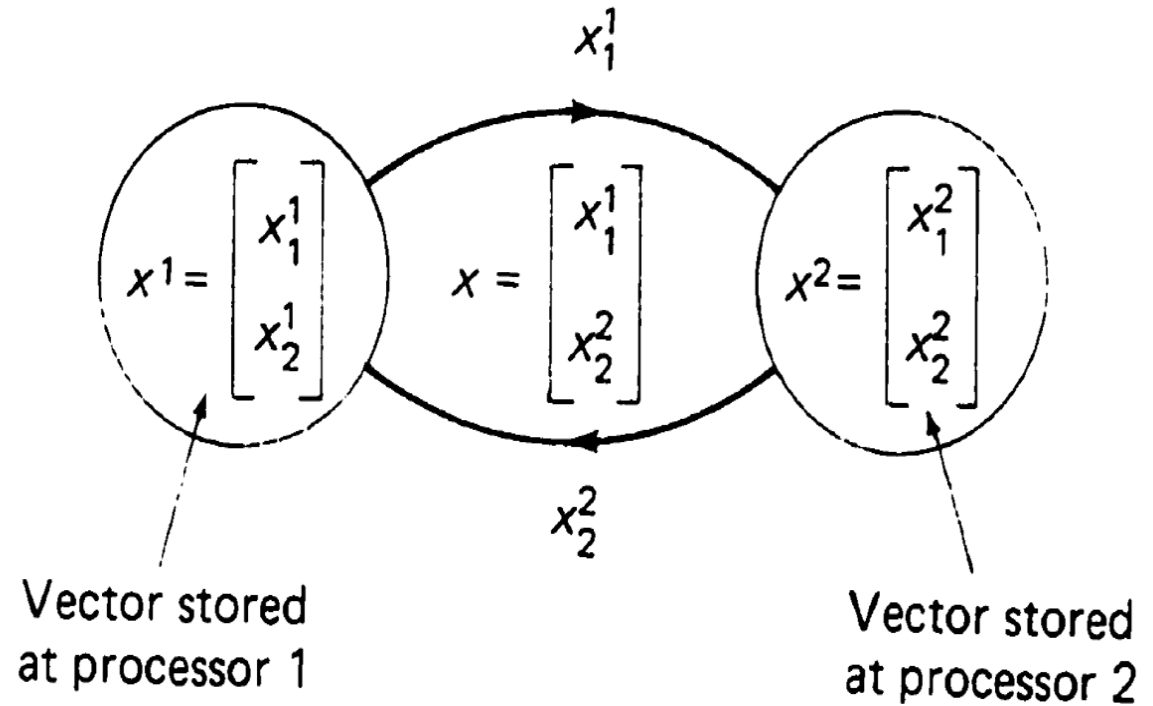
Fie iteratia $x = Ax$

$$\begin{aligned}x_1 &= \frac{1}{2}x_1 + \frac{1}{2}x_2 \\x_2 &= \frac{1}{2}x_1 + \frac{1}{2}x_2\end{aligned}$$

2 procesoare P_1 si P_2 :

P_1 stocheaza $x^1 = (x_1^1, x_2^1)$ si actualizeaza x_1^1 la
momentele $t \in T^1$

P_2 stocheaza $x^2 = (x_1^2, x_2^2)$ si actualizeaza x_2^2 la
momentele $t \in T^2$



Exemplu algoritm partial asincron care converge pt orice valoare a lui B

- Obs: multimea X^* a punctelor fixe ale lui $f(x) = Ax$ este multimea vectorilor $(x_1, x_2) \in R^n$ a. i. $x_1 = x_2$
- sincron, iteratia $x(t + 1) = Ax(t)$, ajunge dupa 1 pas la solutie !

$$x(1) = \begin{bmatrix} \frac{1}{2}(x_1(0) + x_2(0)) \\ \frac{1}{2}(x_1(0) + x_2(0)) \end{bmatrix}$$

Exemplu algoritm partial asincron care converge pt orice valoare a lui B

Scenariu:

- 2 procesoare P_1, P_2 actualizeaza variabila corespunzatoare la fiecare pas
- procesoarele comunica variabila lor la anumite momente $\{t_1, t_2, \dots\}$
- transmiterea/folosirea informatiei comunicate se face instant

=> iteratia este

$$\begin{aligned}x_1(t+1) &= \frac{x_1(t)}{2} + \frac{x_2(t_k)}{2}, & t_k \leq t < t_{k+1} \\x_2(t+1) &= \frac{x_1(t_k)}{2} + \frac{x_2(t)}{2}, & t_k \leq t < t_{k+1}\end{aligned}$$

Exemplu algoritm partial asincron care converge pt orice valoare a lui B

$$x_1(t+1) = \frac{x_1(t)}{2} + \frac{x_2(t_k)}{2}, \quad t_k \leq t < t_{k+1}$$

- intre t_k si t_{k+1} , P_1 mentine $x_2(t_k)$ constant si executa iteratie de mai sus de $t_{k+1} - t_k$ ori:

$$x_1(t_k + 1) = \frac{1}{2}x_1(t_k) + \frac{1}{2}x_2(t_k)$$

$$x_1(t_k + 2) = \frac{1}{2}x_1(t_k + 1) + \frac{1}{2}x_2(t_k)$$

$$= \frac{1}{2} \left[\frac{1}{2}x_1(t_k) + \frac{1}{2}x_2(t_k) \right] + \frac{1}{2}x_2(t_k) = \frac{1}{4}x_1(t_k) + \left[\frac{1}{2} + \frac{1}{4} \right]x_2(t_k)$$

Exemplu algoritm partial asincron care converge pt orice valoare a lui B

$$x_1(t_k + 2) = \frac{1}{2}x_1(t_k + 1) + \frac{1}{2}x_2(t_k)$$

$$= \frac{1}{2} \left[\frac{1}{2}x_1(t_k) + \frac{1}{2}x_2(t_k) \right] + \frac{1}{2}x_2(t_k) = \frac{1}{4}x_1(t_k) + \left[\frac{1}{2} + \frac{1}{4} \right]x_2(t_k)$$

$$x_1(t_k + i) = \left(\frac{1}{2} \right)^i x_1(t_k) + \left[\frac{1}{2} + \frac{1}{4} + \cdots + \left(\frac{1}{2} \right)^i \right] x_2(t_k)$$

Algoritmi asincroni - exemplu

$$x_1(t+1) = \frac{x_1(t)}{2} + \frac{x_2(t_k)}{2}, \quad t_k \leq t < t_{k+1}$$

- între t_k și t_{k+1} , P_1 menține $x_2(t_k)$ constant și execută iteratia de mai sus de $t_{k+1} - t_k$ ori:

$$x_1(t_{k+1}) = \left(\frac{1}{2}\right)^{t_{k+1}-t_k} x_1(t_k) + \left(1 - \left(\frac{1}{2}\right)^{t_{k+1}-t_k}\right) x_2(t_k)$$

Exemplu algoritm partial asincron care converge pt orice valoare a lui B

$$x_1(t_{k+1}) = \left(\frac{1}{2}\right)^{t_{k+1}-t_k} x_1(t_k) + \left(1 - \left(\frac{1}{2}\right)^{t_{k+1}-t_k}\right) x_2(t_k)$$

$$x_2(t_{k+1}) = \left(\frac{1}{2}\right)^{t_{k+1}-t_k} x_2(t_k) + \left(1 - \left(\frac{1}{2}\right)^{t_{k+1}-t_k}\right) x_1(t_k)$$

Exemplu algoritm partial asincron care converge pt orice valoare a lui B

$$x_1(t_{k+1}) = \left(\frac{1}{2}\right)^{t_{k+1}-t_k} x_1(t_k) + \left(1 - \left(\frac{1}{2}\right)^{t_{k+1}-t_k}\right) x_2(t_k)$$

$$x_2(t_{k+1}) = \left(\frac{1}{2}\right)^{t_{k+1}-t_k} x_2(t_k) + \left(1 - \left(\frac{1}{2}\right)^{t_{k+1}-t_k}\right) x_1(t_k)$$

Deci:

$$\begin{aligned} |x_2(t_{k+1}) - x_1(t_{k+1})| &\leq (1 - \epsilon_k) |x_2(t_k) - x_1(t_k)| \\ &\leq \prod_k (1 - \epsilon_k) |x_2(0) - x_1(0)|, \end{aligned}$$

$$\text{unde } \epsilon_k = 2 \left(\frac{1}{2}\right)^{t_{k+1}-t_k}$$

Exemplu algoritm partial asincron care converge pt orice valoare a lui B

Deci:

$$|x_2(t_{k+1}) - x_1(t_{k+1})| \leq (1 - \epsilon_k) |x_2(t_k) - x_1(t_k)|$$

$$\leq \prod_k (1 - \epsilon_k) |x_2(0) - x_1(0)|$$

Convergenta catre un vector din X^* e garantata doar daca

$$\prod_k (1 - \epsilon_k) = 0$$

Dar, daca alegem diferentele $t_{k+1} - t_k$ suficient de mari a.i. $\epsilon_k < k^{-2}$

$$\Rightarrow \prod_k (1 - k^{-2}) > 0$$

$\Rightarrow \mathbf{x} = \mathbf{Ax}$ nu converge total asincron !

Exemplu algoritm partial asincron care converge pt orice valoare a lui B

Daca impunem presupunerea de asincronism partial din scenariu, avem

$$t_{k+1} - t_k \leq B$$

$$\Rightarrow \epsilon_k > \left(\frac{1}{2}\right)^B \Rightarrow \prod_k (1 - \epsilon_k) = 0$$

$\Rightarrow |x_2(t_{k+1}) - x_1(t_{k+1})|$ converge la zero

$\Rightarrow$ secventa $\{x(t_k)\}$ converge la X^*

$\Rightarrow$ conditia de convergenta e indeplinita indiferent de valoarea lui B
(presupunerea din scenariu e ca B e numar intreg pozitiv)

$\Rightarrow$ **algitm partial asincron convergent, fara a fi total convergent !**

Consecinte privitoare la structura iteratiilor partial asincrone

Daca avem pt $\forall t \geq 0$

$$x_i(t+1) = f_i \left(x_1 \left(\tau_1^i(t) \right), x_2 \left(\tau_2^i(t) \right), \dots, x_n \left(\tau_n^i(t) \right) \right), \text{ pt } t \in T^i$$

$$x_i(t+1) = x_i(t), \quad \forall t \notin T^i$$

si $\exists B \in \mathbb{N}$ a.i.

(a) $\forall i$ si $t \geq 0$ cel putin un element al multimii $\{t, t+1, \dots, t+B-1\}$ apartine T^i

(b) $t-B < \tau_j^i(t) \leq t, \quad \forall i, j$ si $\forall t \geq 0, t \in T^i$

(c) $\tau_j^i(t) = t \quad \forall i$ si $t \in T^i$

$\Rightarrow x(t+1)$ depinde doar de $x(t), x(t-1), \dots, x(t-B+1)$ si nu de informatie anterioara $x(\tau)$, cu $\tau \leq t-B$!

Reformulare

- daca informatia veche este eliminata din sistem dupa maximum B unitati de timp, putem introduce vectorul

$$z(t) = (x(t), x(t-1), \dots, x(t-B+1))$$

care sumarizeaza informatia utila (ne-eliminata inca)

- pt. orice scenariu, $x(t+1)$ se poate determina din $z(t)$
- pe cale inductiva, pt orice scenariu fixat si orice intreg pozitiv s , valoarea $z(t+s)$ este unic determinata de $z(t)$
- de asemenea, daca f e continua, pt orice scenariu fixat si orice intreg pozitiv s , $z(t+s)$ este o functie continua de $z(t)$

Convergenta generalizata algoritmi partial asincroni

- fie $X^* = \{x \in X \mid x = f(x)\}$ multimea punctelor fixe ale lui f
- Z produsul cartezian al B copii ale lui X
- $Z^* = \{(x^*, x^*, \dots, x^*) \mid x^* \in X^*\} \Rightarrow$ daca $z(t) = z^* \in Z^*$ atunci

$$z(t + s) = z^* \quad \forall s > 0 \text{ si orice scenariu}$$

- daca definim si o distanta $d: Z \rightarrow [0, \infty)$ care masoara progresul $z(t)$ catre Z^* putem defini urmatorul rezultat valabil pt algoritmi partial asincroni definiti de ecuatiile

$$x_i(t + 1) = f_i \left(x_1 \left(\tau_1^i(t) \right), x_2 \left(\tau_2^i(t) \right), \dots, x_n \left(\tau_n^i(t) \right) \right), \text{ pt } t \in T^i$$

$$x_i(t + 1) = x_i(t), \quad \forall t \notin T^i$$

Teorema Lyapunov

- ipoteze: f continua si presupunerile asincronismului partial sunt indeplinite
- Pp. $\exists t^* \in N$ si $d: Z \rightarrow [0, \infty)$ functie continua cu urmatoarele proprietati

(a) $\forall z(0) \notin Z^*$ si orice scenariu, avem $d(z(t^*)) < d(z(0))$

(b) $\forall z(0) \notin Z$ si $\forall t \geq 0$ si orice scenariu, avem

$$d(z(t+1)) \leq d(z(t))$$

$\Rightarrow z^* \in Z^*$ pt. $\forall z^* \in Z$ punct limita al secventei $\{z(t)\}$

Obs: - rezultat general, nu vizeaza viteza de convergenta

- prea general pt a avea implicatii practice utile

- se poate demonstra insa ca pentru algoritmi iterativi liniari convergenta este geometrica

Algoritmi paraleli – concluzii

- arhitecturi paralele (taxonomia Flynn) si modele de programare paralela (SPMD)
- topologii interconectare procesoare (inel, grid, hipercub), complexitatea comunicarii, mapari inter-topologii
- masuri de performanta (speed-up, eficienta) si limitari (Amdahl, Gustafson)
- analiza algoritmi paraleli: raportul optimal comunicare/calcul, incarcarea echilibrata a procesoarelor, penalizarea sincronizarii, detectia terminarii
- clase generale de algoritmi: sincroni vs asincroni